TEORIA DELLA COMPUTABILITÀ

MACCHINE DI TURING, GRAMMATICHE, AUTOMI E LINGUAGGI DI PROGRAMMAZIONE

MATTEO BELENCHIA
FLAVIO CORRADINI

CAMERINO
BOZZA v. 24-05-2023

Indice

1 Introduzione **9**
1.1 Algoritmi . . . 13
1.1.1 Notazione per algoritmi . . . 16
1.2 Insiemi . . . 17
1.3 Funzioni . . . 19
1.4 Stringhe e linguaggi . . . 20
1.5 Esercizi . . . 25

2 La computabilità secondo Turing **29**
2.1 La Macchina di Turing . . . 29
2.1.1 Rappresentazione grafica . . . 35
2.1.2 Calcolabilità di funzioni . . . 36
2.1.3 Decidibilità di linguaggi . . . 37
2.1.4 Macchine di Turing con più nastri . . . 40
2.1.5 Macchine di Turing non deterministiche . . . 43
2.2 Gödelizzazione . . . 45
2.2.1 Codifica di stringhe . . . 45
2.2.2 Codifica di naturali . . . 48
2.2.3 Codifica di Macchine di Turing . . . 50
2.3 La tesi di Church-Turing . . . 52
2.4 La macchina di Turing universale . . . 54
2.5 Alcune funzioni calcolabili . . . 55
2.6 Esercizi . . . 57

3 Problemi non risolvibili algoritmicamente **63**
3.1 Il problema dell'arresto . . . 64
3.2 Insiemi decidibili, semidecidibili e non semidecidibili . . . 67
3.2.1 Il teorema di Post . . . 68
3.2.2 L'insieme K e il suo complemento . . . 69
3.2.3 La tecnica della coda di colomba . . . 69
3.3 Diagonalizzazione . . . 71
3.4 Riduzione . . . 73
3.5 Altri problemi non decidibili . . . 76
3.6 Il teorema di Kleene . . . 78
3.7 Il teorema di Rice . . . 80
3.8 Il problema di decisione . . . 82
3.9 Esercizi . . . 85

4 Grammatiche e Automi **93**
4.1 Grammatiche generative 94
4.1.1 La gerarchia di Chomsky 97
4.1.2 Le lambda produzioni 100
4.1.3 Alberi di derivazione 105
4.2 Automi di riconoscimento 106
4.2.1 Automi a stati finiti 108
4.2.2 Automi a pila 118
4.2.3 Automi lineari limitati 124
4.3 Esercizi 127

5 Proprietà dei linguaggi formali **131**
5.1 Linguaggi regolari 132
5.1.1 Pumping lemma per i linguaggi regolari 132
5.1.2 Proprietà di chiusura 135
5.1.3 Proprietà decidibili e indecidibili 136
5.1.4 Complessità dell'analisi sintattica 138
5.1.5 Espressioni regolari 138
5.2 Linguaggi liberi dal contesto 144
5.2.1 Pumping lemma per i linguaggi liberi dal contesto 145
5.2.2 Proprietà di chiusura 149
5.2.3 Proprietà decidibili e indecidibili 150
5.2.4 Complessità dell'analisi sintattica 151
5.3 Linguaggi dipendenti dal contesto 152
5.3.1 Proprietà di chiusura 153
5.3.2 Proprietà decidibili e indecidibili 154
5.3.3 Complessità dell'analisi sintattica 154
5.4 Linguaggi a struttura di frase 154
5.4.1 Proprietà di chiusura 156
5.4.2 Proprietà decidibili e indecidibili 156
5.4.3 Complessità dell'analisi sintattica 157
5.5 Sommario dei risultati 157
5.6 Esercizi 157

6 Linguaggi di programmazione **161**
6.1 Il linguaggio WHILE 161
6.1.1 Sintassi del linguaggio WHILE 161
6.1.2 Semantica del linguaggio WHILE 163
6.1.3 Funzioni WHILE-calcolabili 165
6.2 Estensioni del linguaggio WHILE 167
6.2.1 Assegnamento di variabili 167
6.2.2 Assegnamento di costanti 168
6.2.3 Operazioni aritmetiche 168
6.2.4 Operatori di confronto 169
6.2.5 Operatori Booleani 170
6.2.6 Cicli con condizioni Booleane 170
6.2.7 Esecuzione condizionale 171
6.2.8 Numeri interi e razionali 171
6.2.9 Input e output 171
6.3 Equivalenza con le macchine di Turing 172

6.4 II teorema di Kleene per i programmi 174
6.5 Esercizi 176

Prefazione

Questo libro si propone di adeguare i contenuti della Parte A del libro *Teoria della calcolabilità e complessità* [6] alle necessità del corso di Fondamenti di Informatica così come viene somministrato oggigiorno nel corso di Laurea di Triennale in Informatica all'Università di Camerino. La trattazione quindi si è arricchita in alcuni argomenti, mentre altri sono stati modificati, sfoltiti oppure rimossi.

L'argomento centrale del libro è la teoria della calcolabilità (o computabilità), cioè lo studio di quali problemi sono risolvibili algoritmicamente e quali no. A questo scopo si è scelto di focalizzarsi su tre particolari caratterizzazioni della calcolabilità:

- L'approccio computazionale delle macchine di Turing,
- L'approccio generativo delle grammatiche a struttura di frase (insieme a quello riconoscitivo dei corrispondenti automi),
- L'approccio imperativo dei linguaggi di programmazione.

La teoria della computabilità è quindi presentata attraverso tutti questi differenti punti di vista, mostrando come essi siano tutti equivalenti tra di loro e corroborano ulteriormente la Tesi di Church-Turing.

In confronto al precedente testo menzionato all'inizio, le differenze più sostanziali sono:

- Nuovi esercizi di fine capitolo, la maggior parte dei quali provenienti da prove di esame passate, corredati di soluzioni.
- Un capitolo introduttivo che riassume la nomenclatura e altre nozioni base che vengono assunte come note nel resto della trattazione.
- Una presentazione formale della tecnica di diagonalizzazione e riduzione per funzioni e insiemi. Cenni sul teorema S_n^m.
- Sono state aggiunte sezioni per automi ed espressioni regolari.
- Vengono mostrate le dimostrazioni di equivalenza tra grammatiche regolari, espressioni regolari e automi a stati finiti, di fatto mostrando anche i rispettivi algoritmi di conversione tra queste rappresentazioni dei linguaggi regolari.
- Il Pumping Lemma per i linguaggi regolari è enunciato in maniera più familiare rispetto alla letteratura, e la sua dimostrazione ora si basa sugli automi a stati finiti deterministici invece che le grammatiche regolari.

- É stato rimosso il capitolo sulle funzioni primitive ricorsive.
- La dimostrazione di equivalenza tra il linguaggio WHILE e le macchine di Turing è stata riscritta, dato che rimandava al capitolo sulle primitive ricorsive.

Capitolo 1

Introduzione

Il nostro libro si interessa anzitutto del tema della *computabilità*: parola forse impegnativa e difficile, e pur tuttavia ovviamente collegata all'informatica, capace di richiamare l'idea del *computer* e del *calcolatore*. Vale allora la pena di spiegarne subito la rilevanza, anche per introdurre in maniera adeguata i temi del libro.

Nell'accezione originaria e comune, *computare*, o *calcolare*, significa far di conto con i numeri usuali, e cioè gli interi $0, \pm 1, \pm 2, \ldots$: sommarli, moltiplicarli, semmai sottrarli e dividerli. Per chi ha qualche maggior dimestichezza con un po' di matematica, l'arte di far di conto si può estendere a operazioni più sofisticate e a contesti più ampi (limiti, derivate, integrali). Ma, in realtà, tutti i problemi che si prestano a formalizzazioni tramite modelli matematici possono essere "*computati*". Già René Descartes (Cartesio), secoli fa, all'inizio del Seicento, vagheggiava la possibilità di tradurre in termini matematici ogni problema, e dunque di risolverlo tramite equazioni e computazioni; applicava poi l'intuizione al campo geometrico, identificando, ad esempio, punti del piano con coppie ordinate di numeri (reali), rette del piano con equazioni di primo grado in due incognite, e così via. Pochi anni dopo, nel 1666, Gottfried Wilhelm Leibniz esortava appassionatamente "*Calculemus*" (calcoliamo!), invitando a usare conti e computazioni anche per risolvere oggettivamente gli stessi conflitti umani e condurre a termine le controversie trasferendole "*dai ragionamenti complicati ai calcoli semplici, dai vocaboli di significato incerto e vago a caratteri determinati*". Del resto lo stesso Leibniz provvedeva a chiarire di intendere per calcolo "*qualunque notazione che rappresenti il ragionamento, quand'anche non avesse alcun rapporto con i numeri*".

A queste applicazioni computazionali, rivolte a orizzonti sempre più larghi, si accompagnava il tentativo di costruire macchine calcolatrici capaci di aiutare, simulare e, talora, sostituire, la mente umana nel suo lavoro. Proprio al Seicento risalgono vari esempi di meccanismi automatici per fare di conto e svolgere almeno le tradizionali operazioni di somma, prodotto, sottrazione e divisione. Possiamo così ricordare che proprio Leibniz aveva concepito nel 1671 una *ruota* adatta a sviluppare questo genere di computazione, preceduto comunque in questo da Blaise Pascal e da Wilhelm Schickard che, rispettivamente nel 1643 e nel 1623, avevano inventato analoghi procedimenti meccanici. Nei secoli successivi ci furono anche casi di macchine capaci di svolgere computazioni fuori dal solo ambito numerico. Si parla, ad esempio, di un automa di Kempelen ca-

pace di giocare a scacchi (1768) e di anticipare in questo senso il moderno *Deep Blue* (il *computer IBM* capace di sconfiggere nel 1997 anche il campione del mondo degli scacchi): ma, nel caso di Kempelen, non fu mai chiaro quale fosse il segreto programma della macchina e ci fu chi fondatamente dubitò trattarsi soltanto di un imbroglio ben riuscito (i lettori appassionati di libri polizieschi potranno trovare ampia discussione dell'argomento nel giallo "*L'Automa*" di J. Dickson Carr). Nell'Ottocento, tuttavia, Charles Babbage concepì teoricamente un meccanismo automatico, da lui chiamato *macchina analitica* (*analytical engine*), virtualmente capace di adattarsi non solo a numeri e a mosse di scacchi, ma a ogni possibile contesto, dunque una sorta di *hardware* universale, che Ada Lovelace Byron si preoccupò in quegli stessi anni di corredare di quel che oggi chiameremmo il *software*.

A questi progressi tecnici dovevano del resto accompagnarsi corrispondenti approfondimenti teorici, a proposito di una questione che diventa fondamentale non appena si affronta il tema del rapporto uomo-macchina: si tratta infatti di

- identificare un linguaggio astratto appropriato, con simboli opportuni, in cui formulare i problemi (matematici e non) da risolvere, per poterli poi proporre alla macchina che deve computarli,
- individuare, ancora, le leggi che la macchina deve seguire per svolgere i suoi calcoli.

Entrambe le questioni sono tutto men che trascurabili. Oggi, ad esempio, siamo abituati a usare le cifre $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$ per rappresentare i numeri interi, e i simboli $+, \cdot, -, \ldots$ per indicare le loro usuali operazioni. Ma la genesi di questi simboli non è stato processo banale ed ha richiesto talora il progresso di secoli; del resto, gli antichi Greci e Romani non utilizzavano ancora questi caratteri, ma altri ben più complicati. Ancora nel Rinascimento, o nello stesso Seicento, queste notazioni si affacciavano timidamente. Anzi, proprio a Leibniz si attribuisce l'idea di un linguaggio universale ("*lingua characteristica*" nella denominazione latina da lui adoperata) con cui formulare le comunicazioni scientifiche tra uomini di idiomi diversi, e dunque anche tra uomo e macchina.

Sempre Leibniz propugnò l'idea di un calcolo della ragione ("*calculus ratiocinator*" in latino) atto a individuare le leggi fondamentali di sviluppo del pensiero (e delle computazioni), utile dunque anche nella programmazione di nuove macchine pensanti. Come si vede, siamo qui agli albori di quelle che oggi si chiamano *intelligenza artificiale* e *deduzione automatica*, della capacità, cioè, di sviluppare calcoli e pensieri in modo meccanico, dunque delegabile ai "*computers*". Del resto, l'idea di uno studio dei procedimenti del pensiero (quello che in termini ufficiali si chiama Logica) è ben precedente Leibniz, e risale almeno ad Aristotele e al 300 AEC. Il *calculus ratiocinator* fu comunque ampiamente sviluppato già nell'Ottocento da George Boole, che formulò una sorta di teoria algebrica del pensiero, oggi comunemente conosciuta proprio come *algebra Booleana*.

Come si vede, il tema della *computabilità* si collega a svariatissimi argomenti come

- le macchine calcolatrici per attuare le computazioni,
- i linguaggi e le regole con cui favorire la loro collaborazione,

e altri ancora. Tra l'altro, è interessante osservare come queste riflessioni anticipino largamente la nascita dei moderni *computers* e si sviluppino nei secoli precedenti. Infatti i primi *calcolatori* (come oggi comunemente li intendiamo) risalgono a tempi relativamente recenti, al periodo della seconda guerra mondiale e agli anni immediatamente successivi se è vero, come è vero, che il primo computer elettronico, l'*ENIAC* (architettato da Von Neumann) è del 1946.

La computabilità che vogliamo qui trattare si interessa ovviamente a tutte queste problematiche, ma vuole anche sottolinearne un aspetto ulteriore. In effetti, il suo ideale punto di partenza e, se vogliamo, l'anno stesso di nascita della moderna Informatica Teorica (se pure ha senso istituire un'anagrafe delle correnti scientifiche) si colloca in una data intermedia, che segue tutti i germi computazionali che abbiamo descritto, ma precede di una decina di anni la nascita di *ENIAC*. Infatti, la data che attira il nostro interesse è il 1936. Nel prossimo paragrafo tenteremo di spiegare il perché.

Consideriamo la Logica Matematica, e in particolare la logica del primo ordine con gli usuali connettivi $\wedge, \vee, \neg, \ldots$, quantificatori $\forall, \exists$, variabili $x, y, z, \ldots$ e predicati. Possiamo quindi costruire delle proposizioni come:

$$\forall x.P(x) \implies P(x)$$

$$\forall x.P(x) \vee \forall x \neg P(x)$$

$$\exists x.P(x) \implies \forall y P(y)$$

$$\exists x.P(x) \wedge \neg P(x)$$

Notiamo che alcune di queste proposizioni (la prima e la terza) sono *valide*, ossia universalmente vere qualsiasi sia il significato assegnato alle variabili e ai predicati.

- $\forall x.P(x) \implies P(x)$ è valido perché avremo o il caso $\top \implies \top$ oppure $\bot \implies \bot$, che sono entrambi veri.

- $\exists x.P(x) \implies \forall y P(y)$ è valido perché anche qui abbiamo due casi: nel caso P sia vera per ogni possibile elemento, fissiamo un x qualsiasi e l'implicazione si riduce nuovamente a un caso $\top \implies \top$. Se invece P non è valida, basta scegliere un x per il quale $P(x)$ non è vera e per il principio *ex falso quodlibet* l'implicazione è nuovamente verificata.

Invece le altre due proposizioni non sono valide:

- $\forall x.P(x) \vee \forall x \neg P(x)$ non è valida, perché possiamo definire una interpretazione in cui la proposizione non è vera. Difatti, la proposizione specificherebbe che il predicato P o è vero per ogni elemento, oppure non è vero per nessun elemento. Una interpretazione in cui questa affermazione non è valida, per esempio, è sui numeri naturali dove per $P(x)$ intendiamo il predicato *x è pari*. Chiaramente non tutti i numeri naturali sono pari, ma esistono numeri naturali pari, quindi la proposizione è falsa.

- $\exists x.P(x) \wedge \neg P(x)$ non è valida, e in effetti è una proposizione che è sempre falsa, perché avremo sempre una congiunzione tra un termine falso e un termine vero.

Ci si potrebbe chiedere se fosse possibile distinguere quali proposizioni siano valide e quali no. Si tratta di un problema tutt'altro che banale, tanto è che il grande matematico David Hilbert lo propose nel 1928 al Congresso Internazionale dei Matematici, che successivamente divenne noto come *Entscheidungsproblem* (il Problema di Decisione). Hilbert si chiese se esistesse un metodo definito che potesse essere applicato a qualsiasi proposizione matematica e che fosse in grado di garantire una decisione esatta riguardo se la proposizione fosse universalmente vera oppure no.

Problema 1.1 (Entscheidungsproblem)**.** Determinare un procedimento capace di stabilire, per ogni proposizione della logica del primo ordine, se essa sia valida o meno, ovvero se essa sia universalmente vera in ogni possibile interpretazione oppure no.

Dunque ci viene richiesto un algoritmo che abbia

- INPUT: una proposizione della logica del primo ordine;
- OUTPUT: SÍ/NO, a seconda se la proposizione sia valida oppure no.

Di fronte alla difficoltà di determinare l'algoritmo richiesto, si poteva reagire in due modi:

- pensare che i tempi (e le conoscenze matematiche) non erano ancora così maturi da permetterne la soluzione, e che ulteriori progressi scientifici erano richiesti;
- dubitare che l'algoritmo potesse davvero trovarsi e che nessuno, nel 1930 e negli anni successivi, fosse effettivamente in grado di concepirlo.

La prima reazione sembra più ragionevole e realistica; la seconda, più pessimista, aveva comunque all'epoca fondate motivazioni. Infatti altre questioni di matematica e logica condividevano la stessa situazione del Problema di Decisione: una soluzione che tardava a venire, ostacolata da formidabili complicazioni tecniche e teoriche. Ad esempio, era questo il caso di un altro problema di matematica che lo stesso Hilbert aveva sollevato 27 anni prima, nel 1900, nella sua famosa lista di 23 problemi matematici che a suo avviso meritavano attenzione: il cosiddetto Decimo Problema di Hilbert. Questo problema chiede di saper riconoscere, per ogni polinomio a coefficienti interi, se esso ha o no soluzioni intere.

Del resto, a favore della visione più "pessimista" contribuiva in quegli anni '30 la fresca dimostrazione (proprio del 1931) di due famosissimi risultati di Logica Matematica — i Teoremi di Incompletezza di Kurt Gödel — che stabilivano in qualche modo l'incapacità umana di cogliere i reali fondamenti dei numeri interi con le loro usuali operazioni $+, \cdot$, e conseguentemente l'impossibilità di delegare completamente la ricerca matematica alle macchine, secondo le idee accennate nel paragrafo precedente.

Non ci attardiamo comunque su queste pur affascinanti connessioni e torniamo al nostro problema di decisione, e alla possibilità di una risposta negativa del tipo: "l'algoritmo richiesto non esiste". Ora, per convincere che un algoritmo per l'Entscheidungsproblem c'è, basta proporlo: stabilirne le istruzioni e

verificarne la correttezza. Invece, per escluderne l'esistenza, non ci basta formulare qualche tentativo più o meno maldestro e poi denunciarne il fallimento. Dobbiamo infatti stabilire una volta per tutte che nessuno, nel 1930 od oggi o in futuro, saprà mai escogitare un procedimento di calcolo. Ma allora dobbiamo anticipatamente convenire una qualche definizione generale di

- ciò che è computabile (il problema)

e, parallelamente, di

- chi computa (l'algoritmo che affronta il problema e la macchina che lo svolge),
- come si computa (le regole che algoritmo e macchina devono rispettare);

dimostrare conseguentemente che l' Entscheidungsproblem, o il Decimo Problema di Hilbert non rientrano in questa classe di questioni "calcolabili" (se davvero questa è la loro sorte). Così la questione si allarga dai contesti particolari dei numeri interi o delle proposizioni logiche, e va a introdurre temi basilari di informatica:

- che cosa è un "calcolatore"?
- quali sono i problemi che i calcolatori sanno (o non sanno) risolvere?

Fu nel 1936 che queste domande ebbero una prima risposta grazie a personaggi illustri degli albori dell'informatica moderna, come Turing, Church, Kleene, Gödel e altri.

1.1 Algoritmi

Il grande scrittore di fantascienza Isaac Asimov fissò nei suoi racconti le tre leggi fondamentali che dovevano regolare il comportamento dei robot nei loro rapporti con l'Uomo, ispirandole a principi inderogabili di rispetto, obbedienza e collaborazione. Il nostro obiettivo qui è per certi versi analogo. Come già detto, vogliamo infatti descrivere in modo rigoroso come avviene una computazione e precisare:

- anzitutto chi computa (l'algoritmo che la svolge e la macchina su cui si svolge),
- ma anche e soprattutto come si computa (le regole che algoritmo e macchina devono rispettare nel loro lavoro);

formulare in conclusione una specie di decalogo del bravo algoritmo e del bravo calcolatore. Dunque il nostro contesto richiama in qualche senso le leggi dei robot androidi di Asimov. Ma per ottenere il nostro obiettivo seguiamo, più che i romanzi di fantascienza, le riflessioni che gli autorevoli scienziati citati alla fine dello scorso paragrafo, e tra essi segnatamente Alan Turing, svolsero intorno al 1936.

Notiamo anzitutto che, per calcolare un dato problema, serve anzitutto un alfabeto appropriato con cui formalizzarne i contenuti (una lingua characteristica, per dirla alla Leibniz): una sequenza finita di simboli che descrivano i

termini della questione. Questo alfabeto può essere semplicemente quello della lingua italiana, o magari quello della lingua scientifica dominante (l'inglese), ma può anche talora allargarsi in ragione del contesto. Ad esempio, se trattiamo i polinomi a coefficienti interi, come nel caso del Decimo Problema di Hilbert, è bene che prevediamo anche i simboli 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, $+, \cdot, -$. Altri ambiti ci potranno poi suggerire alfabeti ancor più specialistici e mirati.

Concordato dunque il nostro alfabeto, possiamo formulare in modo adeguato il nostro problema e trasmetterlo all'algoritmo e alla macchina calcolatrice per la loro computazione.

A questo punto dobbiamo finalmente pensare a concordare con precisione a quali modelli di algoritmo e di macchina intendiamo ricorrere. In questo possiamo fare momentaneo riferimento alla nostra esperienza personale e a un'idea intuitiva di algoritmo e di calcolatore, per immaginare quali regole di buon comportamento ci aspettiamo che essi rispettino durante il loro lavoro. Semmai, nei casi più delicati e incerti, proviamo a riferirci al modo di agire della mente umana come possibile termine di paragone. Possiamo allora convenire quanto segue.

1. Un algoritmo è di lunghezza finita.
2. Esiste un agente di calcolo (la macchina calcolatrice, appunto) che sviluppa la computazione eseguendo le istruzioni dell'algoritmo.
3. L'agente di calcolo ha a disposizione una memoria, dove vengono immagazzinati i risultati intermedi del calcolo.
4. Il calcolo avviene per passi discreti.
5. Il calcolo non è probabilistico.

I punti 1-3 hanno un'ovvia interpretazione. Il punto 4 afferma che il calcolo non avviene mediante dispositivi analogici, ma si svolge ordinatamente, un passo dietro l'altro. Il punto 5 afferma che il calcolo non è soggetto a fattori casuali, ma si svolge in modo assolutamente preciso e rigoroso (o anche, come in genere si dice, "deterministico"). Altre caratteristiche degli algoritmi sono le seguenti.

6. Non deve esserci alcun limite finito alla lunghezza dei dati di ingresso.
7. Allo stesso modo, non deve esserci alcun limite alla quantità di memoria disponibile.

II punto 6 ci dice che l'input del problema può essere arbitrariamente lungo. La condizione è assolutamente ragionevole: ad esempio, un algoritmo di somma tra gli interi si deve applicare a ogni possibile addendo, quindi a ogni numero intero, comunque grande. Anche il punto 7 è meritevole di qualche chiarimento. Asserisce la possibilità di accedere a una memoria potenzialmente illimitata. Per sostenere la plausibilità di questa assunzione, proviamo a pensare che cosa accade in caso contrario, se ammettiamo di limitare preliminarmente a un livello uniforme prefissato le potenzialità di memoria. In queste condizioni può capitare che algoritmi anche elementari, costruiti per eseguire semplici computazioni, si trovino talora nell'impossibilità di completarle. Ad esempio, la funzione che a ogni intero associa il suo quadrato $f(x) = x^2$ non è più calcolabile, perché lo

spazio di memoria necessario per computare il quadrato di x dipende ovviamente da x; e dunque, per x molto grande, si trova a infrangere i limiti eventualmente prefissati. Così il punto 7 risulta del tutto plausibile.

Anche le seguenti osservazioni sono essenziali per cogliere la natura del calcolo.

8. Deve esserci un limite finito alla complessità delle istruzioni eseguibili dal dispositivo.

9. Il numero di passi della computazione è finito ma non limitato.

10. Sono ammesse computazioni senza fine.

Il punto 8 ribadisce, insieme al punto 1, l'intrinseca finitezza del dispositivo di calcolo. Ci deve essere una limitazione finita non solo per il numero delle istruzioni di un algoritmo (come il punto 1 sostiene), ma anche per la complessità delle singole istruzioni; in altre parole, solo una porzione finita della memoria dell'agente di calcolo deve essere occupata da queste istruzioni iniziali al momento in cui la computazione su un dato input si avvia. La condizione non contraddice la condizione 7, che riguarda i successivi passi del calcolo e afferma la possibilità di accoglierne senza limite le informazioni nella restante porzione di memoria. D'altra parte il punto 8 si può intuitivamente giustificare proprio in riferimento alle caratteristiche della mente umana, considerandone l'intrinseca limitatezza (ma anche le possibilità potenzialmente illimitate di crescita).

Il punto 9, poi, ci dice che non è possibile stabilire a priori un limite massimo per numero dei passi richiesti per eseguire un generico algoritmo su un certo input.

Il punto 10, infine, fa riferimento all'eventualità di dover affrontare problemi senza soluzione, come quelli accennati nei precedenti paragrafi: in questi casi una computazione è destinata a prolungarsi indefinitamente senza riuscire a produrre risposte soddisfacenti.

Queste sono le condizioni che possiamo ragionevolmente richiedere a un qualunque sistema di computazione (che includa sia il programma di calcolo che la macchina che lo esegue). Sono dieci, e quindi completano il nostro decalogo nel senso stretto del termine. Sono anche sufficienti a individuare il nostro obiettivo. Infatti, proprio a partire da queste riflessioni, Alan Turing affrontò nel 1936 il problema di definire rigorosamente che cosa possa intendersi per *computabile*, ponendo un semplicissimo modello di calcolatore ante litteram (la *macchina di Turing*, appunto) e sostenendo che computabile è esattamente quanto una macchina di Turing riesce a computare; dimostrò conseguentemente l'impossibilità risolvere, ad esempio, l'Entscheidungsproblem di Hilbert, perché non ci sono macchine di Turing che lo soddisfino e, di conseguenza, neppure algoritmi di alcun genere con questa capacità. Tra parentesi, potrà essere interessante anticipare che qualche anno dopo, nel 1970, anche il Decimo Problema di Hilbert ottenne la risposta negativa.

In realtà, nei capitoli che verranno, avremo modo di presentare, descrivere e discutere vari modelli di computazione, anzitutto quello di Turing, ma anche altri, classici come quello delle grammatiche o degli automi, e altri relativamente più recenti, collegati ai linguaggi di programmazione. Avremo modo di confrontare questi approcci e di mostrarne la sostanziale equivalenza. Incontreremo anche (come già anticipato nelle scorse righe) casi di funzioni che non

si possono calcolare e di problemi che non si possono risolvere (perché estranei a qualunque trattamento mediante i nostri modelli di calcolo). La teoria della computabilità si interessa principalmente a tutte queste tematiche; in questo senso precorre e inaugura, come già accennato, la moderna Informatica Teorica; concorre ancora sostanzialmente al suo sviluppo.

1.1.1 Notazione per algoritmi

Anche se non ci sono precise linee guida per la notazione da usare per la definizione di algoritmi, in questa sezione descriviamo alcune convenzioni formali che useremo nel resto libro. Naturalmente queste convenzioni servono solamente a rendere più consistente la stesura di algoritmi in questo testo, e in nessun caso sono da intendersi come nulla più che una decisione stilistica degli autori.

Scriviamo gli algoritmi in linguaggio naturale, ma prestando particolare cura alla rappresentazione di cinque aspetti fondamentali: input, output, sequenzializzazione, alternativa e iterazione.

- Un algoritmo è delimitato da una istruzione `begin` che ne contrassegna l'inizio, e una istruzione `end` che ne contrassegna la fine.
- I passaggi dell'algoritmo sono eseguiti dall'alto verso il basso e da sinistra verso destra, o in altre parole seguendo l'usuale ordine stabilito dal numero di riga e le regole di lettura nella lingua italiana. I comandi possono essere messi in sequenza, delimitati da un `begin` e da un `end`, tramite l'uso di punti e virgola `;`.
- L'istruzione `skip` non ha alcun effetto.
- Utilizziamo lettere minuscole $x, y, z, \ldots$ per riferisci ai dati su cui opera l'algoritmo.
- Leggiamo i dati di ingresso dell'algoritmo scrivendo `input(`x`)`, `input(`y`)`, `input(`z`)`, dove $x, y, z, \ldots$ sono simboli di variabile su cui vengono salvati i valori di input dell'algoritmo.
- In maniera analoga, scriviamo `output(`x`)`, `output(`y`)`, `output(`z`)`, dove $x, y, z, \ldots$ sono simboli di variabile o valori, per denotare che l'algoritmo produce come dati in uscita i valori referenziati da quelle variabili o corrispondenti a quei valori.
- Per riferirsi a risultati parziali all'interno dell'algoritmo, scriviamo x `= e` con x un simbolo di variabile e `e` un' espressione che denoti un valore, per specificare che la variabile x ora contiene il valore indicato da `e`.
- Utilizziamo le parole chiave `True` e `False` per riferirci a speciali valori che sono, rispettivamente, la costante vera Booleana e la costante falsa Booleana.
- Per eseguire una serie di comandi `s` o un'altra `s'` dell'algoritmo a seconda di una certa condizione `c`, scriviamo `if c then s else s'`.
- Per iterare una serie di comandi `s` a seconda che una certa condizione `c` sia soddisfatta o meno, scriviamo `while c do s`. Un comando di questo

tipo prima verifica `c`, se la condizione è soddisfatta esegue `s` e poi ripete nuovamente questo procedimento (in questo caso, diciamo che il comando sta "ciclando"), fino a che la condizione `c` non risulta falsa. In tal caso il comando termina la sua esecuzione e diciamo che è uscito dal ciclo. Se invece `c` è falsa dall'inizio, l'istruzione `while c do s` viene interpretata come una istruzione `skip`.

- Per far divergere l'algoritmo, possiamo utilizzare l'istruzione `while true do skip`.
- Infine, per interrompere l'esecuzione dell'algoritmo in qualsiasi momento, usiamo la parola chiave `stop`. Altrimenti l'algoritmo termina quando non c'è una prossimo passo da eseguire.

Mostriamo un esempio di algoritmo seguendo queste linee guida.

Esempio 1.1. Scriviamo un algoritmo che trovi il massimo comune divisore tra due numeri naturali x e y.

```
Begin
Input(x);
Input(y);
While y ≠ 0 do
    Begin
    Sia z = x mod y;
    x = y;
    y = z
    End
Output(x)
End
```

1.2 Insiemi

Un *insieme* è una collezione di elementi, che possono essere oggetti di qualsiasi tipo: numeri, ma anche simboli, parole, o perfino altri insiemi. Tipicamente denotiamo gli insiemi con lettere maiuscole, come A, B, X, Y mentre utilizziamo lettere minuscole per i suoi elementi. Usiamo la notazione $x \in X$ per denotare che un certo oggetto x appartiene a un insieme X, o in altre parole che x è un membro di X, mentre scriviamo $y \notin X$ per indicare invece che un certo oggetto y non appartiene a X. Diciamo che un insieme è *finito* se ha un numero finito di elementi, altrimenti diciamo che l'insieme è *infinito*.

Un insieme è definito da quali elementi ne fanno parte. Possiamo definire un insieme in vari modi, ma per i nostri scopi ci limitiamo a presentarne due: la definizione *estensionale* e la definizione *intensionale*. La definizione estensionale di un insieme consiste nella enumerazione esplicita dei suoi elementi elencando all'interno di parentesi graffe tutti i membri dell'insieme. Quindi per esempio possiamo scrivere $A = \{\text{mela}, \text{pera}, \text{arancia}\}$ per definire estensionalmente un

insieme composto dagli oggetti *mela*, *pera* e *arancia*. In alternativa, la definizione intensionale di un insieme consiste nello specificare quali sono le proprietà che tutti i membri dell'insieme devono possedere. Per esempio possiamo definire l'insieme dei numeri naturali pari minori di dieci come segue:

$$P_{10} = \{n \mid n \in \mathbb{N}, n \leq 10 \text{ e } n \text{ è pari}\}$$

dove la barra verticale | viene letta come "tale che" e alla sua destra segue l'elenco delle proprietà che gli elementi n dell'insieme devono soddisfare. La corrispondente definizione estensionale sarebbe l'insieme $P_{10} = \{0, 2, 4, 6, 8, 10\}$.

È importante evidenziare che un insieme è definito solamente da quali elementi ne fanno parte: questo significa che l'ordine degli elementi o la loro molteplicità (cioè, quante volte uno stesso elemento si ripete) non hanno alcuna importanza. Quindi possiamo dire che gli insiemi $\{a, b, c\}$, $\{b, c, a\}$ e $\{b, b, c, a\}$ sono equivalenti e rappresentano la stessa collezione di oggetti.

Possiamo descrivere anche insiemi con un numero infinito di elementi: per esempio l'insieme dei numeri pari. Possiamo descrivere questo insieme intensionalmente come

$$P = \{n \mid n \in \mathbb{N}, \text{ e } n \text{ è pari}\}$$

ed estensionalmente utilizzando una ellissi:

$$P = \{0, 2, 4, 6, \ldots\}$$

Come ulteriore esempio di definizione estensionale di un insieme infinito, consideriamo un altro insieme molto importante per la teoria della calcolabilità, l'insieme dei numeri naturali $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$.

Il numero di oggetti che compongono un insieme viene detto *cardinalità*, che rappresentiamo racchiudendo il nome dell'insieme tra due barre verticali: quindi nel caso dell'insieme A abbiamo $|A| = 3$ e nel caso di P_{10} abbiamo $|P_{10}| = 6$. Nel caso di insiemi infiniti, ci accontentiamo di dire che anche la loro cardinalità è infinita. Un insieme che non contiene nessun elemento viene detto *insieme vuoto*, ed è tipicamente denotato dal simbolo $\varnothing$. Naturalmente la cardinalità dell'insieme vuoto è 0, $|\varnothing| = 0$.

Diciamo che un insieme A è un *sottoinsieme* di un insieme B se tutti gli elementi di A sono anche elementi di B, e lo scriviamo come $A \subseteq B$ oppure $B \supseteq A$. In questi casi possiamo anche dire che A è incluso o contenuto in B, oppure che B include o contiene A. Possiamo quindi dire che due insiemi sono *uguali* se si contengono a vicenda, ovvero se $A \subseteq B$ e $B \subseteq A$. Infine denotiamo con $A \subset B$ o equivalentemente $B \supset A$ il caso in cui A è un sottoinsieme di B ma $A \neq B$. In questo caso si dice che A è un *sottoinsieme proprio* di B, o che A è incluso o contenuto strettamente in B.

Esempio 1.2. Consideriamo $A = \{1, 3, 5\}$, $B = \{1, 3, 5, 7, 9\}$ e $C = \{1, 3, 5\}$. Valgono le seguenti relazioni:

- $A \subseteq A, A \subseteq B, A \subset B$ e $A \subseteq C$;
- $B \subseteq B$;
- $C \subseteq C, C \subseteq A, C \subseteq B$ e $C \subset B$.

Inoltre abbiamo $A = C$ perché i due insiemi si contengono a vicenda.

Tipiche operazioni tra insiemi sono unione, intersezione, differenza, prodotto e complemento:

- l'*unione* $A \cup B$ di A e B è l'insieme di tutti gli elementi che sono membri di A, B o entrambi;
- l'*intersezione* $A \cap B$ di A e B è l'insieme di tutti gli elementi che sono membri sia di A che di B;
- la *differenza* $A \setminus B$ (anche denotata come $A - B$) tra A e B è l'insieme degli elementi che sono membri di A ma non di B;
- il *prodotto cartesiano* $A \times B$ tra A e B è l'insieme delle coppie ordinate (a, b) dove $a \in A$ e $b \in B$;

Per quanto riguarda il complemento dobbiamo considerare un insieme di riferimento, che nel nostro caso sarà tipicamente $\mathbb{N}$. Quindi

- il *complemento* A^c, anche denotato come $\bar{A}$, $\mathbb{N} \setminus A$ o $\mathbb{N} - A$, è l'insieme degli elementi di $\mathbb{N}$ che non appartengono ad A.

Esempio 1.3. Siano $A = \{1, 3, 5\}$ e $B = \{2, 3, 4\}$ due insiemi di numeri naturali. Allora:

- $A \cup B = \{1, 2, 3, 4, 5\}$
- $A \cap B = \{3\}$
- $A \setminus B = \{1, 5\}$
- $A \times B = \{(1, 2), (1, 3), (1, 4), (3, 2), (3, 3), (3, 4), (5, 2), (5, 3), (5, 4)\}$
- $\bar{A} = \{0, 2, 4, 6, 7, 8, 9, \ldots\}$

1.3 Funzioni

In matematica il concetto di *funzione* viene tipicamente presentato come un particolare tipo di relazione tra due insiemi, ovvero una relazione f tra un insieme X detto *dominio di definizione*, o anche *dominio*, e un insieme Y detto *dominio di arrivo*, o *codominio*. Questa relazione f è tale che per ogni elemento x del dominio esiste un unico elemento $f(x)$ appartenente al codominio. In informatica, si rende tipicamente necessario gestire anche casi in cui una funzione, per certi input, non è definita, ovvero casi in cui il dominio di definizione della funzione è un sottoinsieme dell'insieme sul quale consideriamo gli input della funzione, che chiamiamo *dominio di partenza* della funzione[1].

Per ovviare a questa necessità, distinguiamo il concetto di funzione *totale* da quello di funzione *parziale*: una funzione da un dominio di partenza X a un codominio Y si dice parziale se il suo dominio di definizione è un sottoinsieme di X, mentre invece, nel caso che X e il dominio di definizione coincidano, diciamo che la funzione è totale. Quindi tutte le funzioni sono parziali, le funzioni totali sono anche parziali (ogni insieme è un sottoinsieme di sé stesso), mentre ci sono

[1] Notazione di V.A. Zorich in Mathematical Analysis I.

funzioni che sono parziali ma non totali, quando prese in considerazione su un dominio di partenza che includa strettamente il loro dominio di definizione.

Nel resto di questo libro ci limiteremo a considerare i numeri naturali, quindi diremo che una funzione è totale se e solo se è definita per ogni possibile combinazione dei suoi input presi tra numeri naturali. Quindi quando scriviamo $f : \mathbb{N}^n \to \mathbb{N}$ stiamo denotando una funzione in n argomenti, o n-aria, il cui dominio di definizione è un sottoinsieme del dominio di partenza $\mathbb{N}^n$. Per esprimere che una funzione f non è definita su un certo valore di input x scriveremo $f(x) \uparrow$, e analogamente, possiamo scrivere $f(x) \downarrow$ per esprimere invece che x appartiene al dominio di definizione della funzione. Per quanto riguarda il codominio, anche questo tipicamente sarà l'insieme dei numeri naturali. Per questo motivo, è più interessante considerare l' *immagine* di una funzione, cioè il sottoinsieme del codominio che consiste di tutti gli elementi che la funzione può effettivamente generare. Indicheremo il dominio di una funzione f con $Dom(f)$, mentre denoteremo la sua immagine con $Im(f)$.

Esempio 1.4. Un esempio di funzione parziale è la seguente funzione $f : \mathbb{N} \to \mathbb{N}$:

$$f(x) = \begin{cases} 1 & \text{se } x > 10 \\ \uparrow & \text{altrimenti} \end{cases}$$

che chiaramente non è definita sul sottoinsieme $\{0, 1, \ldots, 10\}$ dei numeri naturali. In questo caso $Dom(f) = \{n \in \mathbb{N} \mid n > 10\}$ mentre $Im(f) = \{1\}$, dato che 1 è l'unico possibile valore che questa funzione può generare.
Altri esempi di funzioni parziali, quando prese in considerazione sui numeri naturali, sono il logaritmo $\log(x)$ (che non è definito per $x = 0$) e la divisione intera $\div(x, y)$ (che non è definita per $y = 0$).

Concludiamo questa sezione con la definizione di alcune funzioni di uso comune in teoria della calcolabilità:

- La *funzione identità* $id : \mathbb{N} \to \mathbb{N}$, $id(x) = x$ è una funzione totale che restituisce il proprio input senza modifiche.
- La *funzione indefinita* $\emptyset : \mathbb{N} \to \mathbb{N}$, $\emptyset(x) \uparrow$ è una funzione parziale che non è definita su nessun input (quindi sia il dominio che l'immagine sono insiemi vuoti).
- Per ogni numero naturale k, la *funzione costante* $c_k : \mathbb{N} \to \mathbb{N}$, $c_k(x) = k$ è una funzione totale che restituisce sempre k per ogni possibile input. Per esempio $c_1(x) = 1$ è la funzione costante 1 che restituisce sempre 1. Talvolta denoteremo queste funzioni omettendo l'argomento x e le parentesi, per esempio scrivendo $c_1 = 1$.

1.4 Stringhe e linguaggi

L'abilità nel rappresentare l'informazione è cruciale nel processo di calcolo dell'informazione stessa. Le società umane hanno creato linguaggi parlati per comunicare a un livello elementare, e hanno sviluppato il metodo della scrittura

per raggiungere un livello ben più sofisticato. La lingua italiana, ad esempio, nella sua forma parlata si basa su un insieme finito di suoni elementari, le parole sono definite in termini di sequenze finite di tali suoni, le frasi vengono derivate da sequenze finite di parole, i discorsi sono ottenuti da sequenze di frasi, e così via. L'italiano scritto usa un insieme finito di simboli come insieme di primitive, le parole sono definite da sequenze finite di simboli, i periodi vengono derivati da sequenze finite di parole, i paragrafi da sequenze finite di periodi, e così via. Approcci simili sono stati sviluppati anche in altri contesti; per esempio, un numero naturale può essere rappresentato come una sequenza finita di cifre decimali $0, 1, 2, 3, 4, 5, 6, 7, 8, 9$; ma anche alternativamente in forma binaria cioè come sequenza finita di bit $0, 1$, dove

$$0, 1, 2, 3, 4, 5, \ldots, 10, 11, 12, \ldots$$

diventano rispettivamente

$$0, 1, 10, 11, 100, 101, \ldots, 1010, 1011, 1100, \ldots$$

Ci sono del resto altre possibili rappresentazioni dei naturali, rispetto a basi ancora diverse; ad esempio un naturale si può esprimere in modo molto rozzo con l'uso del solo simbolo 1, come sequenza finita di 1: in questo caso

$$0, 1, 2, 3, \ldots$$

diventano

$$1, 11, 111, 1111, \ldots$$

e così via.

Ovviamente ci si aspetta che i processi di calcolo trattino l'informazione anche in ambiti più generali: infatti essi manipolano non solo numeri, ma anche grafi, programmi, e molti altri tipi di entità. Comunque questi oggetti possono essere tutti descritti come sequenze finite di lettere su opportuni alfabeti; dunque possiamo assumere in astratto che i processi di calcolo considerino stringhe di *simboli* scelti in un insieme finito e non vuoto (che viene in genere chiamato *alfabeto*). Una sequenza finita di simboli di un dato alfabeto A viene chiamata *stringa* o *parola* su A. La stringa formata dalla sequenza di simboli $a_1, a_2, \ldots, a_n$ viene usualmente denotata $a_1 a_2 \ldots a_n$. Ammettiamo anche l'eventualità di una stringa costituita da nessun simbolo, chiamata *stringa vuota* e denotata dalla lettera greca λ. La stringa vuota è sempre generabile da ogni alfabeto.

Esempio 1.5. Esempi di alfabeti sono l'alfabeto latino $S = \{a, b, c, \ldots, x, y, z\}$, le cifre decimali $D = \{0, 1, 2, \ldots, 9\}$ ma anche insiemi arbitrari di simboli come per esempio l'insieme $I = \{\xi, \psi, 0, 1\}$.
La sequenza di lettere *bca* è una stringa sull'alfabeto S, così come la sequenza di cifre 010 è una stringa sull'alfabeto D (e anche su I). La sequenza di simboli $a55$ invece non è una stringa su S (perché il simbolo 5 non appartiene all'alfabeto S) né su D (perché l'alfabeto dei decimali non contiene il simbolo a). Inoltre la sequenza di zero simboli λ è una stringa su S, D e I.

La lunghezza di una stringa α su un certo alfabeto A, denotata come $l(\alpha)$, è il numero di simboli che la compongono. Quindi se $\alpha = a_1 a_2 \dots a_n$ allora $l(\alpha) = n$.

Esempio 1.6. La lunghezza di una stringa abc è 3, cioè $l(abc) = 3$. Analogamente $l(b) = 1$ e $l(informatica) = 11$. Inoltre si ha che $l(\lambda) = 0$.

Date due stringhe α e β, possiamo ottenere una nuova stringa γ ottenuta dalla *concatenazione* di α con β: la stringa che otteniamo consiste nella sequenza di simboli di α seguita dalla sequenza di simboli di β. Utilizziamo la notazione α^i con $i \in \mathbb{N}$ per rappresentare la stringa ottenuta dalla concatenazione di i copie di α: quindi $\alpha^2 = \alpha\alpha, \alpha^3 = \alpha\alpha\alpha$. Consideriamo poi i casi particolari $\alpha^1 = \alpha$ e $\alpha^0 = \lambda$. Naturalmente è anche possibile concatenare i simboli stessi piuttosto che le stringhe: è sufficiente considerare stringhe composte da un solo simbolo.

Esempio 1.7. La concatenazione di abb con bca è $abbbca$, mentre la concatenazione di bca con abb è $bcaabb$. Inoltre $\lambda\alpha = \alpha\lambda = \alpha$.
La notazione $(abb)^3$ corrisponde alla stringa $abbabbabb$, mentre con abb^3 si intende la stringa $abbbb$, ovvero l'esponente si riferisce solamente al simbolo alla sua immediata sinistra, considerandolo come una stringa di lunghezza 1. Così la notazione $a^2b^3c^1$ corrisponde alla stringa $aabbbc$, mentre $a^3b^0c^2$ corrisponde a $aaacc$.

Diciamo che una stringa α è una *sottostringa* di una stringa β se $\beta = \gamma\alpha\rho$, per una qualche scelta di stringhe γ e ρ. Una sottostringa α di una stringa β si chiama *prefisso* di β se $\beta = \alpha\rho$ per qualche stringa ρ (dunque per $\gamma = \lambda$); α è un prefisso *proprio* se $\rho \neq \lambda$. Una sottostringa α di una stringa β è un *suffisso* di β se $\beta = \gamma\alpha$ per qualche stringa γ; α è un suffisso *proprio* se $\gamma \neq \lambda$.

Esempio 1.8. Data la stringa $abbc$, sono sottostringhe $\lambda, a, b, c, ab, bb, bc, abb, bbc, abbc$. Di queste, λ, a, ab, abb sono prefissi propri, mentre λ, c, bc, bbc sono suffissi propri. Infine $abbc$ è sia un suffisso che prefisso di sé stessa.

Se $\alpha = a_1 a_2 \dots a_n$, allora denotiamo con α^{rev} la sua *inversa*, ovvero la stringa $a_n \dots a_2 a_1$.

Esempio 1.9. Data la stringa $abbc$, la sua inversa è $cbba$. Naturalmente per stringhe α composte da un solo simbolo si ha che $\alpha = \alpha^{rev}$, e $\lambda = \lambda^{rev}$.

L'insieme di tutte le possibili stringhe su un dato alfabeto A viene indicato con $A^\star$, mentre con A^+ indichiamo l'insieme $A^\star$ senza la stringa vuota λ, $A^+ = A^\star \setminus \{\lambda\}$.

Un alfabeto A, come insieme finito di simboli, si può sempre ordinare in più

modi possibili. Sarà spesso conveniente nel seguito di questo libro fissare un tale ordinamento totale $<$ e, per $A = \{a_1, \ldots, a_n\}$, assumere ad esempio

$$a_1 < a_2 < \ldots < a_n$$

In realtà è bene concordare sin d'ora un qualche ordinamento anche per le parole su A. Infatti in Scienza dell'Informazione si affrontano comunemente tecniche di ricerca sequenziale e binaria, *insertion sort*, *quick sort* e *merge sort* che trattano stringhe e si riferiscono a un loro prefissato ordinamento. Una strategia frequentemente usata è quella *lessicografica*, definita nel modo che segue.

Definizione 1.1. Sia A un alfabeto (ordinato da qualche relazione $<$) e siano α e β stringhe in $A^\star$. Si dice che α è *lessicograficamente minore* di β, $\alpha < \beta$, o equivalentemente β *lessicograficamente maggiore* di α, $\beta > \alpha$, se vale uno dei due casi:

1. α è prefisso proprio di β.
2. α ha un prefisso γa, β ha un prefisso γb, per lo stesso $\gamma \in A^\star$, con $a, b \in A$ e $a < b$ nell'ordinamento di A.

Esempio 1.10. Se S è l'alfabeto latino, ordinato ponendo $a < b < c < \ldots$ l'ordine lessicografico in $S^\star$ è quello comune dei vocabolari; quindi $abb < abba$ perché il primo è un prefisso proprio del secondo e $aba < abb$ perché per $\gamma = ab$ abbiamo che $a < b$. In particolare abbiamo anche che $ab < z$, perché per $\gamma = \lambda$ abbiamo che $a < z$.
Invece prendendo l'alfabeto $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ delle cifre decimali dove si fissa, come di consueto, $0 < 1 < 2 < \ldots < 9$. L'ordine che ne deriva sui naturali, intesi come stringhe in $A^\star$ è il loro ordine abituale: $16 < 160$ perché il primo è un prefisso proprio del secondo e $151 < 153$ perché per $\gamma = 15$ abbiamo che $1 < 3$.

Consideriamo ancora i numeri naturali $\mathbb{N}$, intesi come parole sull'alfabeto $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ rispetto alla loro tradizionale rappresentazione in base 10. È ben noto che un numero n è primo se $n > 2$ e gli unici divisori di n sono 1 e n, e che ogni $n > 2$ si decompone in modo unico nel prodotto di fattori primi. Nascono così due classici problemi: entrambi hanno per input il nostro $n > 2$,

- il primo chiede se n è primo o no,
- il secondo di decomporre n nei suoi fattori primi.

Allora la prima domanda intende riconoscere tra le stringhe su D quelle che corrispondono ai numeri primi; la seconda vuole calcolare, per ogni n, quelle stringhe che rappresentano numeri primi che dividono n. Nel primo caso si vuole individuare un sottoinsieme di $D^\star$, nel secondo computare una funzione che da parole su D genera nuove sequenze di parole su D.

Introduciamo ora il concetto di *linguaggio*.

Definizione 1.2. Sia A un alfabeto. Un *linguaggio*, o *linguaggio formale*, o anche *problema* è un sottoinsieme L di $A^\star$.

Il primo termine, linguaggio, fa riferimento alla sua natura di insieme di parole, come ogni lingua tradizionale; il secondo, problema, alla questione computazionale che L ovviamente propone, e cioè riconoscere le stringhe su A che stanno in L ed escludere le altre.

Esempio 1.11. L'insieme dei numeri primi è un linguaggio su $D = \{0, 1, 2, \ldots, 9\}$ (e su qualunque alfabeto per i naturali). L'insieme delle parole della lingua italiana è un linguaggio sull'alfabeto S delle lettere latine. L'insieme dei numeri binari palindromi è un linguaggio sull'alfabeto $B = \{0, 1\}$ delle cifre binarie.

Non dimentichiamo che, accanto al "problema" di riconoscere un linguaggio, c'è anche quello di computare funzioni su stringhe su un alfabeto A, come il caso della decomposizione in fattori primi ci ha sopra illustrato.

Dunque i linguaggi sono particolari insiemi, e si possono applicare a loro proposito le usuali operazioni insiemistiche: per L, L' linguaggi rispettivamente sugli alfabeti A, A',

- l'*unione* $L \cup L'$ di L e L' è il linguaggio su $A \cup A'$ composto dalle stringhe che stanno in L o L';
- l'*intersezione* $L \cap L'$ di L e L' è il linguaggio su $A \cap A'$ composto dalle stringhe che stanno sia in L che L';
- la *differenza* $L - L'$ di L e L' è il linguaggio su A composto dalle stringhe di L che non stanno in L';
- il *complemento* $A^\star - L$, anche denotato come L^c o $\bar{L}$, è il linguaggio su A formato dalle stringhe di $A^\star$ che non sono in L.

Tra i linguaggi su un alfabeto A c'è anche il linguaggio vuoto $\varnothing$, quello privo di stringhe, da non confondersi con $\{\lambda\}$, cioè con il linguaggio formato dall'unica stringa vuota λ. Ci sono poi altre operazioni tra linguaggi che non corrispondono a nozioni insiemistiche generali ma derivano dal contesto particolare di stringhe e alfabeti:

- la *composizione* LL' di L e L' è il linguaggio su $A \cup A'$ formato dalle concatenazioni ww' con $w \in L$ e $w' \in L'$;
- le *potenze* L^n di L si definiscono per ogni $n \in \mathbb{N}$ ponendo $L^0 = \{\lambda\}$ e $L^{n+1} = L^n L$; dunque per ogni $n > 0$, $L^n = \{w_1 w_2 \ldots w_n \mid w_1, w_2, \ldots, w_n \in L\}$ è la composizione di n copie di L;
- la *chiusura di Kleene* $L^\star$ è l'unione di tutte le potenze L^n di L al variare di n tra i numeri naturali; se invece ci limitiamo a considerare solamente gli $n > 0$ otteniamo la *chiusura positiva* L^+. Naturalmente $L^\star = L^+ \cup \{\lambda\}$.

1.5 Esercizi

Esercizio 1.1
Definire in maniera intensionale ed estensionale i seguenti insiemi:

1. L'insieme dei numeri naturali dispari.
2. L'insieme dei numeri primi pari.
3. L'insieme delle cifre binarie.
4. L'insieme dei divisori di 0.

Esercizio 1.2
Eseguire le seguenti operazioni tra insiemi (il complemento si considera sempre rispetto ai numeri naturali):

1. $(\{0,1,2,3\} \cup \{1,2\}) \setminus \{2\}$
2. $\{n \mid n \in \mathbb{N}, n > 0\} \cap \{0\}$
3. $(\{1,2,3\} \setminus \{0,3\}) \times \{1\}$
4. $\overline{\{2\}} \cap \overline{\{0,1\}}$

Esercizio 1.3
Si definisca, per ognuna delle seguenti funzioni, il dominio e l'immagine, e si dica se la funzione è totale o parziale. (Dove viene usato il simbolo di frazione si intende l'operazione di divisione tra naturali).

$$f_1(x) = x + 1 \qquad f_2(x,y) = \frac{x}{y}$$

$$f_3(x) = \begin{cases} 1 & \text{se } x = 0 \\ 0 & \text{se } x = 1 \\ \uparrow & \text{altrimenti} \end{cases} \qquad f_4(x) = \frac{1}{0}$$

$$f_5(x) = 2x \qquad f_6(x) = \frac{x}{x}$$

$$f_7(x,y) = \begin{cases} 1 & \text{se } y = 0 \\ y & \text{se } y = 1 \\ x + y & \text{altrimenti} \end{cases} \qquad f_8(x) = \begin{cases} 0 & \text{se } x = 0 \\ \uparrow & \text{altrimenti} \end{cases}$$

Esercizio 1.4
Definire i seguenti linguaggi:

1. Il linguaggio di tutte le possibili stringhe di zeri e di uni.
2. Il linguaggio di tutte le parole palindrome sull'alfabeto latino S.
3. Il linguaggio di tutti i numeri che finiscono per 9 sull'alfabeto delle dieci cifre decimali D.
4. Il linguaggio di tutte le stringhe che iniziano per a o finiscono per 0 sull'alfabeto $I = \{a, b, 0, 1\}$.

Esercizio 1.5
Dire, per ognuna delle seguenti coppie di stringhe e linguaggi, se la stringa appartiene o meno al linguaggio e perché.

1. $aba \in L_1 = \{a^n b^m \mid n, m > 0\}$?
2. $\lambda \in L_2 = \{a^n b^m c^n \mid n \leq 2, m \geq 0\}$?
3. $0111110 \in L_3 = \{00\alpha \mid \alpha \in \{0,1\}^\star\} \cup \{\alpha 110 \mid \alpha \in \{0,1\}^\star\}$?
4. $bbcbba \in L_4 = \{a^n b^m c^k b^m a^k \mid n \geq 0, m \geq 1, k \geq 0\}$?

Soluzioni

Soluzione 1.1
Gli insiemi sono definibili come segue, dove con I_e denotiamo la definizione estensionale e con I_i quella intensionale. Nelle definizioni intensionali abbiamo evitato di usare termini autoreferenziali.

1. $I_i = \{n \mid n \in \mathbb{N}, n \bmod 2 = 1\},\ I_e = \{1, 3, 5, 7, \ldots\}$
2. $I_i = \{n \mid n \in \mathbb{N}, n \bmod 2 = 0, \forall a, b \in \mathbb{N}(x = ab \implies a = 1 \vee b = 1)\},\ I_e = \{2\}$
3. $I_i = \{n \mid n \in \mathbb{N}, 0 \leq n \leq 1\},\ I_e = \{0, 1\}$
4. $I_i = \{n \mid n \in \mathbb{N}, n > 0, \exists k(n \cdot k = 0)\},\ I_e = \{1, 2, 3, 4, \ldots\}$

Soluzione 1.2

1. $(\{0,1,2,3\} \cup \{1,2\}) \setminus \{2\} = \{0,1,2,3\} \setminus \{2\} = \{0,1,3\}$
2. $\{n \mid n \in \mathbb{N}, n > 0\} \cap \{0\} = \varnothing$
3. $(\{1,2,3\} \setminus \{0,3\}) \times \{1\} = \{1,2\} \times \{1\} = \{(1,1),(2,1)\}$
4. $\overline{\{2\}} \cap \overline{\{0,1\}} = \{n \mid n \in \mathbb{N}, n \neq 2\} \cap \{n \mid n \in \mathbb{N}, n \geq 2\} = \{n \mid n \in \mathbb{N}, n \geq 3\}$

Soluzione 1.3

f_1: Il dominio è $\mathbb{N}$, l'immagine è $\mathbb{N} \setminus \{0\}$. La funzione è totale.

f_2: Il dominio è $\mathbb{N} \times (\mathbb{N} \setminus \{0\})$, l'immagine è $\mathbb{N}$. La funzione è parziale.

f_3: Il dominio è $\{0,1\}$, l'immagine è $\{0,1\}$. La funzione è parziale.

f_4: Il dominio è $\varnothing$, l'immagine è $\varnothing$. La funzione è parziale.

f_5: Il dominio è $\mathbb{N}$, l'immagine è l'insieme dei numeri naturali pari. La funzione è totale.

f_6: Il dominio è $\mathbb{N} \setminus \{0\}$, l'immagine è $\{1\}$. La funzione è parziale.

f_7: Il dominio è $\mathbb{N} \times \mathbb{N}$, l'immagine è $\mathbb{N} \setminus \{0\}$. La funzione è totale.

f_8: Il dominio è $\{0\}$, l'immagine è $\{0\}$. La funzione è parziale.

Soluzione 1.4

I linguaggi sono definibili come segue:

1. $L_1 = \{0,1\}^+$, poiché non consideriamo la stringa vuota come una "stringa di zeri e di uni".
2. $L_2 = \{\alpha\beta\alpha^{rev} \mid \alpha \in S^\star, \beta \in S \cup \{\lambda\}\}$.
3. $L_3 = \{\alpha 9 \mid \alpha \in D^\star\} \setminus \{0\alpha \mid \alpha \in D^\star\}$, dove il secondo insieme si può sottrarre al primo se si volesse escludere i numeri che iniziano per 0, come 0054.
4. $L_4 = \{a\alpha \mid \alpha \in I^\star\} \cup \{\alpha 0 \mid \alpha \in I^\star\}$.

Soluzione 1.5

1. No, $aba \notin L_1$, perché il linguaggio include solamente stringhe che consistono in una sequenza di a seguita da una sequenza di b, quindi dopo i simboli b non sono ammessi altri simboli a.
2. Sì, $\lambda \in L_2$, perché il linguaggio ammette sia $n = 0$ che $m = 0$, quindi $\lambda = a^0b^0c^0$ è ammessa nel linguaggio.
3. Sì, $0111110 \in L_3$, perché la stringa si può scrivere come $\alpha = 0111$ concatenata al suffisso 110.
4. Sì, $bbcbba \in L_4$, perché $bbcbba = a^0b^2c^1b^2a^1$ quindi con $n = 0, m = 2, k = 1$ la stringa appartiene al linguaggio. Inoltre a differenza del punto (1) la stringa con gli esponenti proposti rispetta anche l'ordine dei simboli stabilito dal linguaggio (prima una sequenza, possibilmente vuota, di a, poi di b, c, b e infine di nuovo a).

Capitolo 2

La computabilità secondo Turing

Un processo di calcolo è finalizzato a elaborare informazioni. Può essere semplice quanto una stima della durata di un percorso tra due città, e complicato quanto una previsione meteorologica. Lo studio della *calcolabilità* si prefigge l'obiettivo fornire l'intuizione degli elementi e delle proprietà salienti che caratterizzano il processo. Tale intuizione può essere usata per predire la complessità della computazione, per scegliere in modo consapevole un tipo di calcolo piuttosto che un altro, e per sviluppare strumenti che facilitino il progetto del processo stesso.

Lo studio della computabilità rivela addirittura che ci sono problemi che non possano avere risposta; inoltre, ci mostra che tra i problemi che invece possono essere risolti, ce ne sono alcuni che richiedono risorse così ingenti (ad esempio, un tempo di calcolo dell'ordine del milione di anni) da scoraggiare ogni tentativo di soluzione pratica. Lo studio della computabilità riesce a identificare queste situazioni; permette così di determinare con opportuni strumenti i casi "positivi" in cui la soluzione si ottiene, magari anche a costi accessibili.

In questo libro ci concentreremo in realtà solo sul primo aspetto, ci dedicheremo cioè a distinguere quali problemi si possono risolvere, e quali no. Presenteremo tre modelli astratti di calcolo che permettono questa identificazione: nell'ordine, *macchine di Turing*, *grammatiche* e *programmi*. Non faremo invece alcun riferimento al tema della complessità computazionale e non ci interesseremo dunque dei costi dei problemi che hanno soluzione: per questo aspetto rimandiamo ad altri testi sull'argomento.

In particolare, in questo capitolo verrà presentato il formalismo delle Macchine di Turing: dapprima sarà definita la versione più semplice, si passerà poi a varianti più complesse, utili per approfondire aspetti particolari del concetto di calcolo. Verranno poi presentate le nozioni di calcolabilità e di decidibilità indotte dal modello della macchina di Turing.

2.1 La Macchina di Turing

La *Macchina di Turing*, qui abbreviata *MdT*, prende il nome dal matematico inglese Alan Turing, che la introdusse nel 1936 precedendo di almeno un decennio

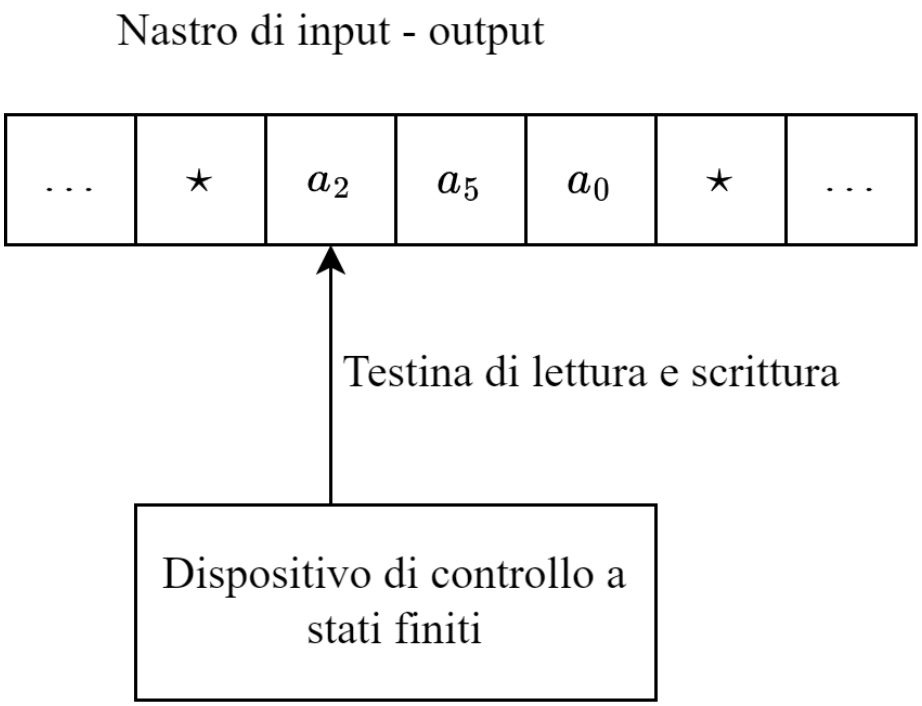


Figura 2.1: Schema di una macchina di Turing.

l'era del computer.

È il primo dei modelli di calcolo che vogliamo trattare. L'idea di Turing è infatti quella d'immaginare un semplice meccanismo astratto che riassuma e simuli tutte le potenzialità computazionali dell'uomo comune. Così Turing prende a esplicito modello l' "*impiegato diligente*", che svolge con ordine e cura gli incarichi assegnatigli, ma non fa niente di più: all'ora stabilita timbra il cartellino e torna a casa. Vediamo come la MdT traduce questo comportamento.

Da un punto di vista fisico la MdT può essere pensata come composta da una unità di controllo a stati finiti, un nastro di lunghezza infinita e una testina di lettura e scrittura, che permette la comunicazione tra controllo e nastro. Il nastro è infinito e suddiviso in *celle* (anche chiamate *quadri*). Ogni cella è in grado di memorizzare un simbolo di un certo alfabeto $A = \{a_0, a_1, \ldots, a_n\}$, oppure un simbolo "bianco" $\star$ che denota l'assenza di scrittura in una cella. Il nastro contiene solo un numero finito di simboli di A; tutte le altre celle contengono comunque il simbolo bianco $\star$. La testina di lettura e scrittura permette all'unità di controllo di leggere e scrivere un simbolo per volta dal nastro, quindi a ogni istante la testina può indicare una sola cella del nastro (e leggere o scrivere il simbolo che la riguarda). L'unità di controllo, oltre ad avere gli organi meccanici per lo spostamento del nastro e della testina, contiene il programma secondo cui verrà eseguito il calcolo e mantiene lo stato della macchina. L'insieme di possibili stati della macchina è un insieme finito $Q = \{q_0, q_1, \ldots, q_m\}$. Una rappresentazione schematica della macchina di Turing è mostrata in Figura 2.1.

Se vogliamo riprendere il paragone con l'impiegato diligente, A rappresenta l'insieme di lettere con cui egli legge la domanda di partenza e svolge i suoi calcoli successivi; gli stati di Q corrispondono invece alle possibilità che l'impiegato ha di scrivere la stessa lettera in più modi distinti, sottolineandola, o evidenziandola, o colorandola per darle maggiore risalto.

La computazione della MdT avviene per passi discreti. Si concorda che, all'avvio di ogni sua computazione, la macchina si trovi in uno stato iniziale prefissato q_0. A ogni passo l'unità di controllo prende atto dello stato in cui si trova e del simbolo contenuto nella cella che la testina indica, e di conseguenza esegue le operazioni sotto elencate:

1. rivede il suo stato;
2. scrive un simbolo nella cella indicata dalla testina, sostituendo il simbolo esistente (ricordiamo che tra i simboli ammessi c'è anche $\star$);
3. sposta la testina di una posizione a sinistra o a destra.

Il nuovo stato assunto dall'unità di controllo, il simbolo da scrivere sulla cella indicata dalla testina e lo spostamento della testina a sinistra o a destra sono determinati dal programma della MdT, che stabilisce il comportamento della macchina stessa. Il programma di una MdT può quindi essere pensato come un insieme di quintuple della forma (q, a, q', a', x), dove q indica lo stato dell'unità di controllo, a il simbolo nella cella indicata dalla testina mentre q', a', x specificano l'azione che la MdT deve intraprendere: in dettaglio q' rappresenta il nuovo stato dell'unità di controllo, a' il simbolo da scrivere nella cella esaminata dalla testina e $x = \pm 1$ lo spostamento della testina: una posizione a sinistra se $x = -1$, a destra se $x = +1$.

Ovviamente ogni singola MdT si riconosce, non tanto dal suo aspetto esteriore di nastri, celle e testine, quanto proprio da queste quintuple, e cioè dal programma che le si richiede di svolgere e dalle istruzioni che esso prevede, tutte ridotte a semplici ordini di cambiamento di stato, di scrittura e di spostamento: così Turing schematizzava il comportamento della mente dell'impiegato diligente, sintetizzandone le capacità operative minime, e secondo questo modello si prevede di riconoscere linguaggi o computare funzioni.

Notiamo poi che un impiegato diligente esegue ordinatamente le sue istruzioni finché ne ha e purché non ne abbia troppe tra loro in concorrenza. Nel primo caso, quando gli ordini mancano, l'impiegato si ferma e ritiene concluso il suo lavoro; nel secondo, invece, resta nell'imbarazzo della scelta: responsabilità che non si può scaricare sulle sue spalle.

È dunque importante che le istruzioni relative a una coppia (q, a), quando esistono, siano univoche. Si può ammettere che manchino, non che si moltiplichino. Il passaggio da (q, a) alla corrispondente terna (q', a', x) deve essere assolutamente deterministico e lontano da ogni ambiguità.

Il programma di una MdT può essere descritto mediante una tabella a righe e colonne, la cosiddetta *matrice di transizione*, o *matrice funzionale*; le righe corrispondono ai possibili stati q della macchina mentre le colonne corrispondono ai possibili simboli a dell'alfabeto (incluso $\star$). All'incrocio di ciascuna coppia q e a si pone l'istruzione che deve essere eseguita dalla macchina quando l'unità di controllo si trova nello stato q e la testina legge il simbolo a: la terna che descrive in che stato entrare, che cosa scrivere, dove spostarsi. È ammesso

che una casella della matrice di transizione sia bianca. In tal caso, si conviene che la macchina non ha azioni da compiere e quindi termina il calcolo. La matrice di transizione della macchina rappresenta una specie di tabella in cui sono concentrate tutte le istruzioni: l'impiegato diligente esegue il suo lavoro applicandola pedissequamente.

Ma è tempo di dare una definizione formalmente rigorosa delle MdT, che ne riassuma i caratteri essenziali, l'alfabeto, gli stati, le istruzioni. Possiamo allora dire:

Definizione 2.1 (Macchina di Turing)**.** Una Macchina di Turing M è una quadrupla (Q, A, δ, q_0), dove:

- Q è un insieme finito e non vuoto di stati;
- A è un alfabeto, cui si aggiunge il simbolo bianco $\star$;
- δ è una funzione da $Q \times (A \cup \{\star\})$ a $Q \times (A \cup \{\star\}) \times \{-1, +1\}$: δ è chiamata *funzione di transizione*, e gli elementi (q, a, q', a', x) di δ sono chiamati *regole di transizione* o *istruzioni* di M;
- $q_0 \in Q$ è lo stato iniziale.

Come già sottolineato, l'informazione cruciale su una MdT è quella fornita dalla funzione di transizione δ, quella che descrive il programma delle istruzioni. È δ che regola il comportamento di M. Così, se M si trova nello stato $q \in Q$, la testina legge il simbolo $a \in A \cup \{\star\}$ e $\delta(q, a) = (q', a', x)$, allora M si sposta nello stato q', il simbolo a' sostituisce a nella cella in esame e la testina si sposta di una posizione a sinistra se $x = -1$, o a destra se $x = +1$. Se invece M si trova nello stato $q \in Q$, la testina legge il simbolo $a \in A \cup \{\star\}$ ma $\delta(q, a)$ non è definito, allora M si arresta. In questo senso, la matrice sopra descritta altro non è che il grafico di δ. Spesso, quando chiaro dal contesto, identificheremo una macchina di Turing M con la sua funzione di transizione δ. L'istruzione di δ che genera la terna (q', a', x) da (q, a) si scriverà talora

$$(q, a) \xrightarrow{\delta} (q', a', x)$$

invece che $\delta(q, a) = (q', a', x)$ o $(q, a, q', a', x) \in \delta$; qualche volta, quando non ci sono rischi di ambiguità, ometteremo δ e scriveremo semplicemente

$$(q, a) \to (q', a', x)$$

Una MdT, così come l'abbiamo appena definita, viene anche detta *deterministica*, a sottolineare che ogni tripla (q', a', x), se esiste, è unica e univocamente determinata dalla coppia (q, a). Tra qualche paragrafo incontreremo anche MdT non deterministiche e ne spiegheremo le differenze rispetto a quanto appena descritto.

Torniamo adesso a una descrizione informale del modo in cui una MdT esegue le sue computazioni a partire da un dato input e restituisce il relativo output. Ci è utile introdurre anche il concetto di configurazione istantanea di una MdT: una sorta di "fotografia" che ritrae la macchina a un dato istante di lavoro, illustrandone lo stato, il contenuto del nastro e la posizione della testina.

Poiché il nastro di una MdT è sempre vuoto, salvo che per un numero finito di celle, è possibile riassumerne il contenuto con una stringa finita di simboli e fornire la descrizione istantanea di una MdT con una quadrupla del tipo (ξ, q, a, η), dove q rappresenta lo stato della macchina all'istante in considerazione, a è il simbolo in lettura, $\xi \in (A \cup \{\star\})^\star$ è la stringa di simboli a sinistra del simbolo in lettura ed $\eta \in (A \cup \{\star\})^\star$ è la stringa di simboli a destra del simbolo in lettura prima che il nastro diventi definitivamente bianco. Per esempio, per $A = \{1\}$, $(11, q_0, 1, \lambda)$ ci dice che la macchina esamina una cella con 1 nello stato q_0, che il nastro è bianco a destra della cella considerata e contiene 11 prima di diventare bianco a sinistra.

Poniamo adesso:

Definizione 2.2. Una *configurazione istantanea* di una Macchina di Turing $M = (Q, A, \delta, q_0)$ è un elemento dell'insieme: $(A \cup \{\star\})^\star \times Q \times (A \cup \{\star\}) \times (A \cup \{\star\})^\star$.

Riepiloghiamo allora come avviene una computazione di una macchina M su una stringa di input $w \in A^\star$.

- Al passo iniziale 0, M esamina una sequenza w scritta da sinistra verso destra sul nastro di input: la testina indica il primo carattere a_0 di w e lo stato interno della macchina è quello iniziale q_0. Questa configurazione è detta *configurazione iniziale* su w. Se $w = a_0 w'$, la 4-upla che la rappresenta è (λ, q_0, a_0, w').

- Ammettiamo che al passo i della computazione la macchina si trovi nella configurazione $C_i = (\xi, q, a, \eta)$. In base alla funzione δ, se esiste una regola di transizione $(q, a) \xrightarrow{\delta} (q', a', x)$, allora la macchina passa nella nuova configurazione C_{i+1} derivata da C_i secondo questa istruzione. Se invece $\delta(q, a)$ non è definita M si arresta. Per esempio, se C_i è come sopra $(11, q_0, 1, \lambda)$ e $(q_0, 1) \xrightarrow{\delta} (q_1, 1, 1)$, si ha $C_{i+1} = (111, q_1, \star, \lambda)$; se invece $(q_0, 1) \xrightarrow{\delta} (q_1, 1, -1)$, allora $C_{i+1} = (1, q_1, 1, 1)$.

- Se M si ferma dopo un numero n finito di passi, allora si dice che M *converge* sull'input w (in notazione, $M \downarrow w$); C_n si chiama *configurazione finale*, la sequenza di configurazioni $C_0, C_1, \ldots, C_n$ è una computazione completa (finita) di M sull'input w e la stringa contenuta sul nastro di input è l' output di tale computazione. Per esempio, se $C_n = (\xi, q, a, \eta)$, allora $\xi a \eta$ è l'output.

- Se M non si arresta mai, allora si dice che M *diverge* sull'input w e si scrive $M \uparrow w$.

Per formalizzare rigorosamente questo comportamento computazionale possiamo introdurre una relazione binaria $\vdash_M$ tra le configurazioni di M. Sostanzialmente $\vdash_M$ associa a ogni configurazione di M quella che la segue dopo l'esecuzione dell'istruzione di M corrispondente. Per esempio, se C è, come sopra, $(11, q_0, 1\lambda)$, C' è $(111, q_1, \star, \lambda)$ e in M si ha l'istruzione $(q_0, 1) \xrightarrow{\delta} (q_1, 1, 1)$, si pone $C \vdash_M C'$. Il lettore potrà scrivere per esercizio, se ha voglia e pazienza, tutti i dettagli della definizione di $\vdash_M$.

Se poi per una data C non esiste alcuna configurazione C' tale che $C \vdash_M C'$, allora scriveremo $C \not\vdash_M$

Sia $\vdash_M^\star$ la chiusura riflessiva e transitiva di $\vdash_M$; vale a dire, per C, C' configurazioni, $C \vdash_M^\star C'$ significa che esistono un naturale n e configurazioni $C_0, C_1, \ldots, C_n$, tali che $C = C_0 \vdash_M C_1 \vdash_M \ldots \vdash_M C_n = C'$.

Chiamiamo poi *computazione* di una MdT M su un input w una sequenza (eventualmente infinita) di configurazioni

$$C_0 \vdash_M C_1 \vdash_M \ldots \vdash_M C_i \vdash_M \ldots$$

tale che C_0 è una configurazione iniziale su w. Chiaramente $M \downarrow w$ se questa computazione è finita

$$C_0 \vdash_M C_1 \vdash_M \ldots \vdash_M C_n$$

e inoltre $C_n \not\vdash_M$; C_n è la *configurazione finale* di M su w. Altrimenti, se $M \uparrow w$, la computazione non ha mai fine.

Esempio 2.1. Si consideri l'alfabeto $A = \star, 1$ composto dal solo simbolo 1 e dal simbolo bianco $\star$. Definiamo una MdT M che, presa come input una successione di 1 consecutivi, restituisce come output tale successione aumentata di un elemento. In dettaglio M dispone di due stati interni, lo stato iniziale q_0 e uno stato di "arresto" q_1, e la funzione di transizione δ è composta dalle istruzioni

$$(q_0, 1) \to (q_0, 1, +1), \ (q_0, \star) \to (q_1, 1, +1)$$

Secondo le convenzioni stabilite, alla partenza M ha lo stato q_0 e la testina indica il primo simbolo a sinistra dell'input. Fintanto che la testina trova celle segnate con 1, allora, in virtù della prima regola di transizione, M scrive 1 sulla cella osservata (cioè il simbolo letto viene lasciato inalterato) e la testina si sposta a destra di una cella, mantenendo lo stato q_0. Quando la testina arriva a una cella vuota, M passa alla seconda regola di transizione, in virtù della quale la macchina segna con 1 la cella osservata, sposta la testina a destra (la scelta della direzione di spostamento è in questo caso ininfluente) e assume lo stato q_1, per il quale non è definita nessuna regola di transizione e, quindi, la macchina si arresta.

Esempio 2.2. Consideriamo adesso una MdT che ha lo stesso alfabeto $A = \star, 1$ dell'esempio precedente, ancora due stati q_0, q_1, ma una sola istruzione $(q_0, \star) \to (q_0, \star, +1)$. Ammettiamo che la macchina abbia come input il nastro bianco, e dunque si trovi a leggere $\star$ nello stato al momento in cui la computazione si avvia. È facile verificare che la macchina scivola verso destra continuando a ristampare $\star$ su ogni nuova cella considerata e a rimanere nello stato q_0. Quindi la computazione diverge.

In conclusione, la macchina di Turing costituisce un soddisfacente modello di calcolo, e comunque corrisponde al decalogo fissato nella Sezione 1.1. Infatti si ha quanto segue.

1. La funzione di transizione di una MdT costituisce un insieme finito di istruzioni.
2. La MdT così descritta è anche l'agente di calcolo che esegue le istruzioni di cui al punto 1).
3. La MdT può utilizzare il nastro per memorizzare i risultati intermedi.
4. La MdT opera in modo discreto.
5. La MdT opera in modo deterministico, con istruzioni univocamente specificate.
6. Non esiste nessuna limitazione sulla lunghezza delle stringhe di ingresso, in quanto si assume che il nastro è illimitato.
7. Il nastro della MdT costituisce, appunto, una memoria di capacità non limitata.
8. Le operazioni che la MdT può eseguire sono molto semplici, quindi di complessità finita.
9. Non esiste nessun limite al numero delle istruzioni eseguite durante una computazione: infatti, le istruzioni sono di numero finito, ma è ammessa la possibilità di usare più volte la stessa istruzione.
10. Esiste la possibilità di computazioni infinite, come l'esempio precedente ci mostra.

2.1.1 Rappresentazione grafica

In questa sezione mostriamo una tecnica per la rappresentazione visuale di macchine di Turing, che consiste nel *grafo di transizione* della macchina di Turing. Il grafo di transizione di una MdT consiste in una collezione di *nodi*, ognuno dei quali corrispondente a uno stato della MdT, e una collezione di *archi* orientati, ognuno dei quali corrisponde a una transizione tra due stati (potenzialmente lo stesso stato) della MdT. Ogni nodo di questo grafo è marcato con lo stato a cui corrisponde, con lo stato iniziale contrassegnato da una freccia entrante. Ogni arco è marcato da un insieme di triple (a, a', x) dove a, a' sono simboli dell'alfabeto e $x \in \{-1, +1\}$. Dati due nodi v_1, v_2, corrispondenti agli stati q_1, q_2, l'arco (v_1, v_2) che ha origine in v_1 e destinazione in v_2 sarà marcato con l'insieme di triple $\{(a_0, a'_0, x_0), (a_1, a'_1, x_1), \ldots\}$ se si ha che:

$$\delta(q_1, a_0) = (q_2, a'_0, x_0), \delta(q_1, a_1) = (q_2, a'_1, x_1), \ldots$$

Per semplificare l'interpretazione di queste triple, le rappresentiamo come

$$a \to a', x$$

a indicare che il simbolo a viene sostituito con a' sul nastro e lo scostamento della testina è pari ad x. Vediamo il grafo di transizione relativo all'esempio precedente.

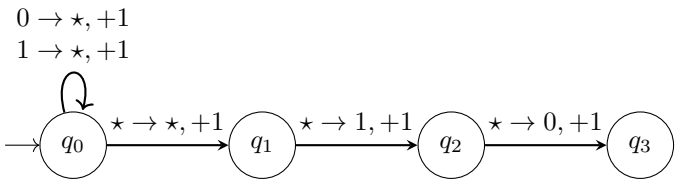
Esempio 2.3. Consideriamo la MdT $M = (Q = \{q_0, q_1\}, A = \{0, 1\}, \delta, q_0)$ con le seguenti istruzioni

$$(q_0, 0) \to (q_0, \star, +1) \quad (q_0, 1) \to (q_0, \star, +1) \quad (q_0, \star) \to (q_1, \star, +1)$$
$$(q_1, \star) \to (q_2, 1, +1) \quad (q_2, \star) \to (q_3, 0, +1)$$

La matrice di transizione è la seguente

	0	1	$\star$
q_0	$(q_0, \star, +1)$	$(q_0, \star, +1)$	$(q_1, \star, +1)$
q_1			$(q_2, 1, +1)$
q_2			$(q_3, 0, +1)$
q_3			

e il corrispondente grafo di transizione è:



2.1.2 Calcolabilità di funzioni

In questa sezione mostriamo come le Macchine di Turing possano calcolare funzioni sulle stringhe di un certo alfabeto A.

Definizione 2.3. Sia f una funzione con dominio e immagine in $A^\star$. Si dice che una MdT $M = (Q, A, \delta, q_0)$ *calcola* o *computa* f se e solo se, $\forall w \in A^\star$,

- se w è nel dominio di f, allora $M \downarrow w$ con output $f(w)$;
- se invece w non è nel dominio di f, allora $M \uparrow w$.

Una funzione f si dice *calcolabile secondo Turing* o *computabile secondo Turing* se esiste una macchina di Turing M che la calcola; altrimenti diciamo che la funzione è *non calcolabile / non computabile secondo Turing*.

La precedente definizione si estende facilmente al caso in cui il dominio o l'immagine di f si compongono di coppie, o terne, o n-uple di parole invece che direttamente di stringhe. Basterà convenire di rappresentare una coppia (w_1, w_2) di parole su A come una stringa $w_1 \star w_2$ in cui w_2 succede a w_1, separata da un simbolo bianco, e procedere analogamente per terne, o sequenze più complicate.

Si noti poi che tra le funzioni computabili secondo Turing sono ammesse anche funzioni parziali, definite solo su porzioni di $A^\star$, e non sulla sua totalità, così come introdotto nella Sezione 1.3.

Esempio 2.4. Presentiamo una MdT M che computa la funzione totale di addizione tra i naturali. Come già in precedenza, lavoriamo sull'alfabeto $A = \{1\}$.
Assumiamo che M abbia quattro stati q_0, q_1, q_2, q_3 e le istruzioni

$$\begin{array}{lll} (q_0, \star) \to (q_1, 1, +1) & (q_0, 1) \to (q_0, 1, +1) & (q_1, \star) \to (q_2, \star, -1) \\ (q_1, 1) \to (q_1, 1, +1) & (q_2, 1) \to (q_3, \star, -1) & (q_3, 1) \to (q_3, \star, +1) \end{array}$$

Mostriamo come, per esempio, M verifichi che $2 + 3$ fa 5, cioè produca l'output 111111 quando il suo input è la coppia $(2, 3)$, cioè $111 \star 1111$. Infatti le istruzioni relative a q_0 fanno ricopiare a M la prima sequenza di 1 senza cambiare stato

$$(\lambda, q_0, 1, 11 \star 1111) \vdash_M^\star (111, q_0, \star, 1111)$$

poi le fanno aggiungere un ulteriore 1 e passare ad esaminare la seconda sequenza di 1 nello stato q_1

$$(111, q_0, \star, 1111) \vdash_M (1111, q_1, 1, 111)$$

L'istruzione su $(q_1, 1)$ ha ancora l'effetto di riprodurre inalterata questa seconda sequenza

$$(1111, q_1, 1, 111) \vdash_M^\star (11111111, q_1, \star, \lambda)$$

a questo punto M cancella gli ultimi due 1, passa allo stato q_3 e si ferma producendo l'output desiderato 111111

$$\begin{array}{c} (11111111, q_1, \star, \lambda) \vdash_M (1111111, q_2, 1, \lambda) \vdash_M \\ (111111, q_3, 1, \lambda) \vdash_M (111111\star, q_3, \star, \lambda) \end{array}$$

Finalmente notiamo che ogni MdT M con alfabeto A computa una funzione, eventualmente parziale, da $A^\star$ a $(A \cup \{\star\})^\star$, e più in generale dall'insieme delle sequenze finite di stringhe di $A^\star$ e $\star$ all'insieme stesso: la funzione associa ad ogni stringa (o sequenza di stringhe) l'output della eventuale computazione convergente di M su di essa.

2.1.3 Decidibilità di linguaggi

Vediamo adesso come una MdT M gestisce la questione di identificare un linguaggio L in un alfabeto A. Ovviamente M avrà lo stesso alfabeto A di L, e quindi sarà $M = (Q, A, \delta, q_0)$ per opportuni Q e δ. Possiamo immaginare per M due possibili strategie per distinguere gli elementi di L.

1. M fissa preventivamente, a prescindere da L, due stringhe di output tra loro distinte, da intendersi una volta per tutte come "SÌ" e "NO". Si chiede poi a M, per ogni input $w \in A^\star$, di:
   - convergere su SÌ se $w \in L$,
   - convergere su NO se $w \notin L$.

Si dice allora che M *decide* L.

2. Alternativamente si ammette che, per ogni $w \in A^\star$, M converge su w se e solo se $w \in L$; dunque

   - $M \downarrow w$ se $w \in L$,
   - $M \uparrow w$ se $w \notin L$.

   Si dice allora che M *accetta* L.

Notiamo che le due strategie non sono tra loro equivalenti e che in realtà solo la prima è adeguata a distinguere effettivamente gli elementi di L da quelli fuori di L. Infatti, se M si limita ad accettare L, M sa riconoscere gli elementi di L, perché converge su di essi; ma non sa escludere gli elementi fuori di L, per i quali segue l'evoluzione della corrispondente computazione, prende atto che essa non si conclude né dopo 10, né dopo 100, né dopo 10^n passi, ma non può su questa base prevedere che essa prosegua indefinitamente (come in realtà accade), o non si arresti proprio al passo $10^n + 1$, o poco dopo. Quando M accetta un linguaggio L e dunque L si compone delle parole $w \in A^\star$ per cui $M \downarrow w$, diremo comunque che (almeno) L è *riconosciuto* da M.

Possiamo quindi introdurre le seguenti categorie di linguaggi:

Definizione 2.4. Un linguaggio L è *decidibile secondo Turing* (o, più semplicemente, *decidibile*) se esiste una macchina di Turing M che decide L, *indecidibile (secondo Turing)* altrimenti. La funzione χ_L calcolata da M viene chiamata *funzione caratteristica* di L ed è definita come

$$\chi_L(w) = \begin{cases} 1 & \text{se } w \in L \\ 0 & \text{se } w \notin L \end{cases}$$

Definizione 2.5. Un linguaggio L è *semidecidibile secondo Turing* (o, più semplicemente, *semidecidibile*) se esiste una macchina di Turing M che accetta L, *non semidecidibile (secondo Turing)* altrimenti. La funzione χ'_L calcolata da M (in alcuni testi chiamata *funzione semi-caratteristica* di L) è definita come

$$\chi'_L(w) = \begin{cases} 1 & \text{se } w \in L \\ \uparrow & \text{se } w \notin L \end{cases}$$

Possiamo notare che le precedenti definizioni di decidibilità e semidecidibilità di un linguaggio sono collegate col concetto introdotto nella Sezione 2.1.2 di calcolabilità di funzioni. Infatti abbiamo che un linguaggio è decidibile se e solo se la sua funzione caratteristica è calcolabile, e allo stesso modo possiamo dire che è semidecidibile se e solo se esiste una funzione che converge su tutte e sole le parole w che appartengono al linguaggio. Inoltre non è difficile convincersi che se un linguaggio è decidibile allora è anche semidecidibile. È facile, infatti, trasformare una MdT M che risponde "SÌ" alle stringhe che appartengono al linguaggio e "NO" alle stringhe che non appartengono al linguaggio in una

macchina di Turing M che converge quando M dichiara "SÌ" e diverge quando M dice "NO". Nella Sezione 3.2 studieremo formalmente le relazioni tra decidibilità e semidecidibilità e mostreremo vari esempi di linguaggi decidibili, semidecidibili e non semidecidibili. Per il momento ci accontentiamo di mostrare un esempio molto semplice.

Esempio 2.5. Sia $A = \{1\}$. Sappiamo che i numeri naturali $0, 1, 2, 3, 4, \ldots$ si possono rappresentare, in verità in modo assai rozzo, come parole su A $1, 11, 111, 1111$, e così via: dunque ogni naturale n diventa una stringa di 1 di lunghezza $n+1$. Sia L l'insieme dei numeri pari. Vogliamo provare che L è decidibile e quindi costruire una MdT M su A che lo decide. Identifichiamo anzitutto le risposte SÌ e NO rispettivamente con i numeri 1 e 0, e dunque con le codifiche 11 e 1. Informalmente M conta la parità di un dato input n e risponde 1 o 11 di conseguenza. Per ottenere questo obiettivo ci servono quattro stati q_0, q_1, q_2, q_3 e una funzione di transizione δ con le seguenti istruzioni

$$\begin{array}{ll} (q_0, \star) \to (q_3, 1, +1) & (q_0, 1) \to (q_1, \star, +1) \\ (q_1, \star) \to (q_2, 1, +1) & (q_1, 1) \to (q_0, \star, +1) \\ (q_2, \star) \to (q_3, 1, +1) & \end{array}$$

Vediamo allora come M verifica che 4, cioè 11111, è pari. Anzitutto M usa cancella tutti gli uni utilizzando la seconda e la quarta istruzione fino a giungere a $\star$ nello stato q_1, secondo la seguente computazione

$$(\lambda, q_0, 1, 1111) \vdash_M (\lambda, q_1, 1, 111) \vdash_M (\lambda, q_0, 1, 11) \vdash_M \\ (\lambda, q_1, 1, 1) \vdash_M (\lambda, q_0, 1, \lambda) \vdash_M (\lambda, q_1, \star, \lambda)$$

a questo punto la terza e la quinta istruzione si occupano di scrivere 11 sul nastro ormai completamente vuoto:

$$(\lambda, q_1, \star, \lambda) \vdash_M (1, q_2, \star, \lambda) \vdash_M (11, q_3, \star, \lambda)$$

dopo di che, in assenza di istruzioni su q_3, M si arresta con l'output atteso 11, cioè SÌ.
Consideriamo ora la computazione di M sull'input 3, cioè 1111. Stavolta gli uni vengono cancellati come mostrato dalla seguente computazione

$$(\lambda, q_0, 1, 111) \vdash_M (\lambda, q_1, 1, 11) \vdash_M (\lambda, q_0, 1, 1) \vdash_M \\ (\lambda, q_1, 1, \lambda) \vdash_M (\lambda, q_0, \star, \lambda)$$

a cui segue la prima istruzione che scrive 1 sul nastro bianco

$$(\lambda, q_0, \star, \lambda) \vdash_M (1, q_3, \star, \lambda)$$

dopo di che si arresta con output 1, cioè NO.
Si può verificare facilmente che una MdT M' con due soli stati q_0, q_1 e le tre istruzioni $(q_0, 1) \to (q_1, \star, +1), (q_1, 1) \to (q_0, \star, +1), (q_0, \star) \to (q_0, \star, +1)$ accetta L, confermando così che L è anche semidecidibile.

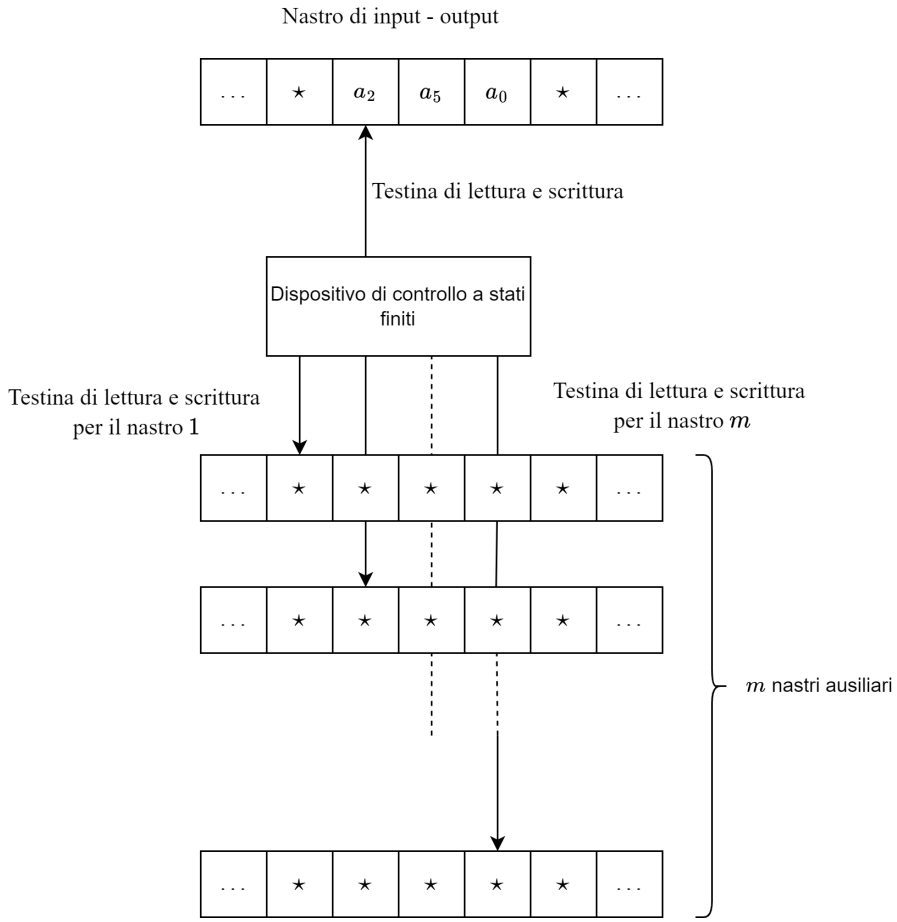


Figura 2.2: Schema di una macchina di Turing a più nastri.

2.1.4 Macchine di Turing con più nastri

La descrizione informale della macchina di Turing data nella Sezione 2.1 in termini di nastro, celle e testina è assai elementare, essenziale e schematica, certamente passibile di adattamenti, modifiche e migliorie. Per esempio, tra le varianti ammissibili, possiamo considerare l'eventualità di disporre di più nastri di lavoro. Ma il meccanismo che ne deriva, anche se apparentemente più duttile e potente della semplice MdT, mantiene in realtà le medesime capacità computazionali: i linguaggi decisi o accettati restano esattamente gli stessi, così come le funzioni calcolate. Vediamo perché.

Consideriamo allora una macchina di Turing M a più nastri (visualizzata in Figura 2.2) immaginata come composta da:

- una unità di controllo a stati finiti,
- un nastro N_0 di input-output di lunghezza infinita con relativa testina,
- m nastri ausiliari $N_1, \ldots, N_m$ di lunghezza infinita ($m \geq 0$), ciascuno dotato di propria testina.

Ogni nastro è suddiviso in celle, e ogni cella è in grado di memorizzare un simbolo di un certo alfabeto $A = \{a_0, a_1, \ldots, a_n\}$, o il simbolo bianco $\star$. Il nastro di input-output N_0 contiene l'input iniziale e l'eventuale output finale di ogni computazione; ma, durante il lavoro, anche gli altri nastri $N_1, \ldots, N_m$ possono essere coinvolti e adoperati. Ogni nastro è scandito dalla sua testina, secondo le convenzioni usuali per MdT semplici. L'unità di controllo, oltre a contenere il programma secondo cui la computazione deve essere eseguita, controlla lo stato della macchina.

All'inizio della computazione, l'input è scritto su N_0 e la testina ne indica il simbolo più a sinistra; gli altri nastri sono vuoti e la testina ne esamina una cella arbitraria; la macchina M è nello stato q_0. Ogni successiva mossa di M è determinata dal suo stato e da i simboli indicati dalle testine sui vari nastri $N_0, N_1, \ldots, N_m$ e consiste in aggiornare lo stato, riscrivere i simboli esaminati su ogni nastro, e infine spostare in ogni nastro la testina di un passo verso destra o verso sinistra.

Come già detto, si conviene che l'eventuale output della computazione sia la stringa scritta in N_0 se e quando M si arresta.

Tutte le definizioni già date per la MdT semplice possono essere adattate al nuovo modello, ovviamente con le opportune modifiche. In particolare, le regole di transizione di una macchina M a m nastri ausiliari sono fatalmente più elaborate, perché devono tenere conto non solo dello stato di M e del simbolo indicato su N_0, ma anche dei simboli considerati sui nastri ausiliari $N_1, \ldots, N_m$; la transizione sarà caratterizzata, come detto, dalla scelta di un nuovo stato e dalle istruzioni di scrittura e spostamento per ciascun nastro, sia di N_0 che di $N_1, \ldots, N_m$. Una regola di transizione sarà allora della forma

$$\delta(q, b_0, b_1, \ldots b_m) = (q', b'_0, x_0, b'_1, x_1, \ldots, b'_m, x_m)$$

dove:

- q, q' rappresentano gli stati di M rispettivamente prima e dopo la transizione,
- per ogni $i \in \{0, 1, \ldots, m\}$, $b_i \in A \cup \{\star\}$ è il simbolo indicato dalla testina sul nastro N_i, $b'_i \in A \cup \{\star\}$ il simbolo che lo sostituisce, $x_i \in \{-1, +1\}$ lo spostamento che la testina deve compiere.

Il modello che si ottiene sembra più potente ed espressivo delle semplici MdT.

Esempio 2.6. Vediamo in particolare come una MdT con un nastro ausiliario N_1 sappia controllare, per ogni parola $w = a_{j_1} \ldots a_{j_k}$ su A, se $w = w^{rev}$ e cioè se

$$a_{j_1} a_{j_2} \ldots a_{j_k} = a_{j_k} a_{j_{k-1}} \ldots a_{j_1}$$

e quindi sappia decidere il linguaggio

$$\{(w, w^{rev}) \mid w \in A^\star, w = w^{rev}\}$$

La computazione di M su un generico w avviene come segue: M inizia il suo lavoro muovendo simultaneamente la testina di N_0 e quella del nastro ausiliario N_1 verso destra, una cella per volta, finché la testina di

N_0 non incontra $\star$, cioè termina di leggere w. Durante questo movimento, M copia su N_1 i simboli letti; quindi M scorre all'indietro N_1 e individua il primo simbolo non bianco. Finalmente, M legge simultaneamente il nastro N_0 da destra a sinistra e N_1 da sinistra a destra e controlla che i simboli esaminati coincidano uno a uno. A seconda dell'esito della verifica scrive SÌ o NO su N_0.

Nonostante le apparenze, le capacità computazionali di una MdT a più nastri sono le stesse di quella a un solo nastro, come adesso proviamo.

Teorema 2.1. Per ogni MdT $M = (Q, A, \delta, q_0)$ a m nastri ausiliari, esiste una MdT $M' = (Q', A', \delta', q_0)$ con $A \supseteq A'$ tale che:

- per ogni linguaggio L su A, L è accettato o deciso da M se e solo se lo è da M';
- per ogni funzione f su stringhe in $A^\star$, f è calcolata da M se e solo se lo è da M'.

Dimostrazione. Dobbiamo simulare il comportamento di M con una MdT "tradizionale" M', nel senso che, per ogni $w \in A^\star$,

- se $M \downarrow w$, allora $M' \downarrow w$ e M, M' hanno lo stesso output su w;
- se $M \uparrow w$, allora $M' \uparrow w$.

Suddividiamo allora l'unico nastro di M in $2m + 2$ tracce consecutive

$$N'_0, N'_1, \ldots, N'_m, N''_0, N''_1, \ldots, N''_m$$

Un nuovo simbolo $\triangle$ nell'alfabeto serve a separarle, segnalando inizio e fine di ognuna; un ulteriore simbolo $\triangledown$ delimita a sinistra e a destra la sequenza delle tracce. Per ogni $i = 0, 1, \ldots, m$, le tracce N'_i, N''_i corrispondono al nastro N_i di M, in particolare

- N'_i ne ricopia il contenuto significativo (cioè non vuoto),
- N''_i provvede a indicare la posizione della testina di N_i.

A quest'ultimo proposito, ammettiamo che A' includa due ulteriori simboli $+, -$ e conveniamo che la cella di N''_i corrispondente a quella indicata dalla testina di N_i contenga $+$, le altre $-$. Per esempio, se N'_i contiene la stringa $a_0a_1a_0a_2$ e la testina indica a_1, N'_i presenta $a_0a_1a_0a_2$ e N''_i presenta $-+--$. Naturalmente il numero delle celle nella traccia N'_i è lo stesso che in N''_i, ma questo comune valore può variare durante la computazione in ragione dei movimenti di M su N_i: i delimitatori $\triangle, \triangledown$ possono però spostarsi e segnalare con chiarezza i confini.
All'inizio della computazione su un dato input w, il nastro di M' si presenta allora nel seguente modo:

- N'_0 contiene w, N''_0 ha $+$ nella prima cella, $-$ nelle altre;

- per $i = 1, \ldots, m$, N'_i è bianca, cioè contiene $\star$, N''_i ha $+$ nella prima cella e $-$ in tutte le altre.

A questo punto M' può iniziare la simulazione della macchina M. Ad ogni regola di transizione di M

$$\delta(q, b_0, b_1, \ldots b_m) = (q', b'_0, x_0, b'_1, x_1, \ldots, b'_m, x_m)$$

M' fa corrispondere le istruzioni adeguate a svolgere il seguente lavoro:

- M' entra nello stato q';
- per ogni $i = 1, \ldots, m$, M' sostituisce b_i con b'_i su N'_i e registra lo spostamento x_i su N''_i con l'uso appropriato di $+$ e $-$.

Quando M converge, M' opera una opportuna pulizia del nastro lasciando solo il contenuto di N'_0.
Ovviamente questa descrizione del comportamento di M è forzatamente imprecisa. Le operazioni sopra descritte necessiterebbero di una implementazione in termini di quintuple secondo quanto richiesto da una MdT tradizionale. Ma, con un minimo di pazienza, il discorso può essere precisato. In conclusione M simula adeguatamente il comportamento di M, e corrisponde alle richieste dell'enunciato del teorema. □

Viceversa, ogni MdT tradizionale può essere facilmente intesa come MdT a più nastri (che in realtà non ha bisogno di ricorrere ai nastri ausiliari). Dunque i due modelli, con o senza nastri ausiliari, si rivelano equivalenti.

2.1.5 Macchine di Turing non deterministiche

In questa sezione mostreremo un'altra possibile estensione delle MdT, quella delle così dette macchine di Turing "non deterministiche"; ne discuteremo brevemente utilità e motivazione, ma dimostreremo che anche questa generalizzazione equivale in definitiva al modello delle macchine di Turing tradizionali.

In una comune MdT $M = (Q, A, \delta, q_0)$, per ogni stato $q \in Q$ e per ogni simbolo $a \in A \cup \{\star\}$, c'è al più un'istruzione da eseguire a proposito di (q, a). In altre parole, δ è una "funzione":

- associa a (q, a) un'unica terna (q', a', x) con $q' \in Q, a' \in A \cup \{\star\}, x \in \{-1, +1\}$,
- oppure esclude (q, a) dal suo dominio (nel qual caso M si ferma non appena incontra (q, a)).

Del resto, queste convenzioni sono stabilite per simulare in modo adeguato il comportamento tipico di un impiegato diligente che, se non ha istruzioni, si ferma; se ne ha una sola, la esegue; se ne ha più d'una, resta nel dubbio e non si prende responsabilità.

Ma a un impiegato diligente e fortunato si può anche concedere la facoltà di scegliere, confidando che la buona sorte gli ispiri l'opzione migliore. Corrispondentemente a una MdT non deterministica si concederà, per certe — o per tutte — le coppie (q, a), la possibilità di scegliere tra più possibili terne (q', a', x), tra loro concorrenziali. In altre parole, si ritirerà a δ la qualifica di funzione da

coppie in $Q \times (A \cup \{\star\})$ a terne in $Q \times (A \cup \{\star\}) \times -1, +1$, e la si ridurrà al rango di una semplice relazione tra coppie e terne, quindi a un insieme di 5-uple in

$$Q \times (A \cup \{\star\}) \times Q \times (A \cup \{\star\}) \times -1, +1$$

Si porrà in conclusione

Definizione 2.6. Una macchina di Turing *non deterministica* M è una 4-upla (Q, A, δ, q_0) come nella definizione delle MdT tradizionali, dove però δ è una relazione in

$$Q \times (A \cup \{\star\}) \times Q \times (A \cup \{\star\}) \times -1, +1$$

In effetti le MdT tradizionali si chiamano talora "deterministiche", a sottolineare l'univocità delle loro istruzioni. Va però osservato che le MdT deterministiche rientrano tra quelle non deterministiche: hanno maggiore rigidità e meno libertà rispetto alle altre — al più una istruzione da eseguire per una data coppia (q, a) —, ma non ne contraddicono la definizione.

In ragione della molteplicità di istruzioni, una MdT non deterministica M può svolgere più possibili computazioni e produrre diversi possibili output per lo stesso input w. Diremo che M *converge* su w se almeno una di queste computazioni ha fine, e dedurremo che M *diverge* su w se tutte queste computazioni divergono.

Esempio 2.7. Sia M la MdT non deterministica che ha due stati q_0, q_1, l'alfabeto $\{1\}$ e due sole istruzioni

$$(q_0, \star) \to (q_1, 1, +1),\ (q_0, \star) \to (q_0, \star, +1)$$

entrambe relative a $(q_0, \star)$. Ammettiamo che M inizi la computazione dal nastro bianco e quindi si trovi a considerare al primo passo proprio la coppia $(q_0, \star)$. Se M esegue la prima istruzione, M entra nello stato q_1 e subito dopo si arresta. Se invece M sceglie la seconda istruzione, M ricopia $\star$, va a destra e resta nello stato q_0, cioè si ritrova nella situazione di partenza, una cella più a destra. Se M continua a eseguire indefinitamente la seconda istruzione su $(q_0, \star)$, M diverge. Comunque M converge sul nastro bianco perché almeno una delle sue computazioni sul nastro bianco ha fine.

È evidente che questa capacità di computazioni plurime dà a una MdT non deterministica M maggiori potenzialità, almeno apparenti: dove una MdT deterministica ha un'unica via da seguire, M ha più alternative e quindi la possibilità di concludere più rapidamente scegliendo la strada migliore. Così, M accetta un linguaggio L se e solo se gli elementi di L coincidono con gli input di M su cui almeno una computazione di M converge, mentre M decide L se e solo se M ha almeno una computazione che converge su SÌ per ogni $x \in L$ e ogni computazione converge su NO per ogni $x \notin L$. Tuttavia, alla resa dei conti, questi presunti vantaggi non aumentano realmente il potere computazionale delle macchine, nel senso del seguente teorema.

Teorema 2.2. Per ogni MdT non deterministica M, c'è una MdT deterministica M' tale che per ogni linguaggio L, L è accettato o deciso da M se e solo se è lo è da M'.

Dimostrazione. (Ci limitiamo a cenni informali). Per ogni input w, M' segue contemporaneamente tutte le computazioni di M su w, e si arresta non appena una di queste si arresta; continua a osservare il comportamento di M altrimenti. □

2.2 Gödelizzazione

Ora che abbiamo definito le macchine di Turing, il prossimo passo consiste nel mostrare che l'insieme di tutte le possibili macchine di Turing è numerabile, ovvero che è in corrispondenza biunivoca con i numeri naturali. Questo passaggio è fondamentale, perché proprio grazie a questa corrispondenza biunivoca saremo in grado di dimostrare che esistono funzioni che non sono calcolabili da nessuna macchina di Turing (e come vedremo a breve, non sono calcolabili neppure da nessun altro approccio alternativo).

La tecnica che utilizzeremo per dimostrare questa corrispondenza biunivoca prende il nome dal matematico Kurt Gödel, che la sviluppò per dimostrare i suoi famosi teoremi di incompletezza. La numerazione di Gödel, nota anche come gödelizzazione, fu il primo tipo di aritmezzazione, e non nacque dunque nel contesto delle macchine di Turing, ma è invece una metodologia generale per associare un numero identificativo ai simboli e le stringhe di un qualsiasi linguaggio formale.

Vedremo innanzitutto come aritmetizzare i simboli di un alfabeto arbitrario e le stringhe che su tale alfabeto si possono costruire. Successivamente vedremo come anche una qualsiasi sequenza di numeri naturali può a sua volta essere ricondotta a un singolo numero naturale. Infine, vedremo come associare ad ogni MdT su un dato alfabeto il suo corrispondente numero di etichetta, il cosiddetto *numero di Gödel*, dal quale è sempre possibile risalire alle istruzioni della macchina e viceversa.

2.2.1 Codifica di stringhe

Abbiamo sin qui considerato stringhe su alfabeti A arbitrari. In realtà è possibile ridurre il nostro ambito ai numeri naturali e a uno degli alfabeti che servono a rappresentarli, come $\{1\}$ o $\{0, 1\}$ o $\{0, 1, 2, \ldots\}$. L'idea che si può seguire è la stessa in base alla quale in un teatro ogni poltrona ha un numero di riconoscimento, o in una biblioteca ogni volume riceve un suo numero di etichetta.

I numeri non sono né poltrone né libri, ma servono a identificarli senza ambiguità. Procedimenti effettivi di codifica relativamente elementari svolgono la stessa funzione per le stringhe su un qualunque alfabeto A. Per A arbitrario, cerchiamo di definire in modo esplicito una funzione di numerazione $\#$ da $A^\star$ a $\mathbb{N}$.

Una prima strategia può fare riferimento alla rappresentazione dei naturali in base 10, o 2, o qualunque valore prefissato > 0. Ammettiamo, ad esempio,

per semplicità, che l'alfabeto A abbia nove elementi. È immediato identificarli con le cifre $1, 2, \ldots, 9$ e magari indicarli con $a_1, a_2, \ldots, a_9$. Così

$$j \mapsto a_j \text{ per ogni } j = 1, \ldots, 9$$

è una corrispondenza biunivoca tra $\{1, 2, \ldots, 9\}$ e A. A questo punto ad ogni parola non vuota $w = a_{j_1} \ldots a_{j_k}$ su A possiamo associare $j_1, \ldots j_k$ in base 10, cioè porre

$$\#(w) = j_1 \cdot 10^{k-1} + j_2 \cdot 10^{k-2} + \cdots + j_{k-1} \cdot 10^1 + j_k \cdot 10^0$$

Gli indici dei simboli di w determinano i coefficienti di questa rappresentazione, la lunghezza k di w ne regola il numero degli addendi (k, appunto, da 10^0 a 10^{k-1}). Si noti che $\#(w) \neq 0$ perché $j_1, \ldots j_k > 0$ e $k > 0$. Alla parola vuota λ, che non ha simboli e ha lunghezza 0, associamo allora 0:

$$\#(\lambda) = 0$$

Parole distinte ricevono così in modo effettivo numeri distinti. Per esempio

$$\#(a_1 a_2 a_3) = 123, \ \#(a_2 a_4) = 24,$$

e via dicendo. Viceversa 2154 è il numero di $a_2 a_1 a_5 a_4$, mentre 1004 non corrisponde a nessuna parola perché contiene 0, ma non è 0. Quindi # non è suriettiva. Tuttavia per ogni naturale n è possibile stabilire in modo effettivo se n è o no nell' immagine di # e, se sì, a quale parola corrisponde.

Il procedimento si estende facilmente al caso in cui l'alfabeto A ha non più nove, ma un numero arbitrario $n > 0$ di simboli. Si scrivono gli elementi di A come $a_1, \ldots, a_n$ e poi si fa riferimento a $n + 1 > 1$ le alla rappresentazione dei naturali in base $n + 1$, al fatto cioè che ogni j si scrive in modo unico come

$$j = j_1 \cdot (n+1)^{k-1} + j_2 \cdot (n+1)^{k-2} + \cdots + j_{k-1} \cdot (n+1)^1 + j_k \cdot (n+1)^0$$

per $k > 0, j_1, \ldots, j_k$ naturali opportuni. Si pone allora, per $w \in A^\star$:

- se $w = \lambda$ è la parola vuota, $\#(w) = 0$,
- se $w = a_{j_1} \ldots a_{j_k}$ non è vuota,

$$\#(w) = j_1 \cdot (n+1)^{k-1} + j_2 \cdot (n+1)^{k-2} + \cdots + j_{k-1} \cdot (n+1)^1 + j_k \cdot (n+1)^0$$

Le proprietà osservate per $n = 9$ si preservano. In particolare ogni parola su A riceve il "suo" numero e, viceversa, da ogni naturale si può recuperare in modo effettivo la parola corrispondente, se esiste.

Questa codifica però ha il difetto di non saper distinguere simboli da parole. Per spiegarci meglio, riferiamoci per un attimo alla lingua italiana, dove a ha ruolo sia di lettera (vocale) che di parola (preposizione: andare "a" Roma). Questa duplice veste meriterebbe un duplice numero di codice, uno come simbolo e uno come stringa. Ma # le assegna soltanto il valore 1, come prima lettera dell'alfabeto.

Cerchiamo di ovviare a questa obiezione proponendo un'altra strategia di numerazione che fa stavolta ricorso al teorema fondamentale dell'Aritmetica,

quello secondo cui ogni naturale ≥ 2 si decompone in modo unico nel prodotto di fattori primi: ad esempio $12 = 2^2 \cdot 3, 15 = 3 \cdot 5$, e così via. Cominciamo allora col numerare i simboli dell'alfabeto A, cui assegniamo valori dispari (per motivi che saranno chiari più tardi). Definiamo dunque una funzione $\#_0$ da A a $\mathbb{N}$ ponendo, per $A = \{a_1, \ldots, a_n\}$ e per $j = 1, \ldots, n$,

$$\#_0(a_j) = 2 \cdot j + 1$$

cioè $\#_0(a_1) = 3, \#_0(a_2) = 5$ e via dicendo.

Passiamo poi a numerare le stringhe di $A^\star$, introducendo la seguente funzione $\#_1$ da $A^\star$ a $\mathbb{N}$. Sia $w \in A^\star$:

- se $w = \lambda$, conveniamo $\#_1(w) = 1$,
- altrimenti sia $w = a_{j_1} \ldots a_{j_k}$; in riferimento ai valori $2 \cdot j_1 + 1, \ldots, 2 \cdot j_k + 1$ già associati da $\#_0$ a $j_1, \ldots, j_k$, poniamo

  $$\#_1(w) = 2^{2 \cdot j_1 + 1} \cdot 3^{2 \cdot j_2 + 1} \cdots p_k^{2 \cdot j_k + 1}$$

  dove p_k denota il k-esimo numero primo.

Assegniamo quindi le codifiche (tramite $\#_0$) dei simboli di w come esponenti $\#_1(w)$ nella sua decomposizione in fattori primi. Per esempio

$$\#_1(a_1 a_4 a_2 a_1 a_1) = 2^3 \cdot 3^9 \cdot 5^5 \cdot 7^3 \cdot 11^3$$

In questo modo, stringhe distinte ricevono in modo effettivo valori distinti. Viceversa, per ogni numero naturale n, è possibile stabilire in modo effettivo se n è o no immagine di $\#_1$ e, se sì, risalire alla parola corrispondente: basta fare riferimento alla decomposizione di n in fattori primi, oppure constatare che $n = 1$. Per esempio, il numero dei fattori primi coinvolti rivela la lunghezza della eventuale stringa corrispondente. Stavolta, poi, gli elementi a_j di A ricevono due numeri distinti:

- l'uno $2 \cdot j + 1$ come simboli,
- l'altro $2^{2 \cdot j + 1}$ come parole.

Né c'è pericolo di confondere questi valori, perché i numeri dei simboli sono dispari > 3, mentre quelli delle parole sono 1 oppure pari, perché esplicitamente divisibili per 2.

Una volta trattati simboli e stringhe, esso permette anche di numerare coppie, o terne, o n-uple ordinate di stringhe. A questo proposito, abbiamo già visto una possibile strategia: tradurre queste sequenze in stringhe in un alfabeto più lungo che utilizzi anche il simbolo bianco $\star$. Così (w_1, w_2) diviene $w_1 \star w_2$, (w_1, w_2, w_3) diviene $w_1 \star w_2 \star w_3$ e via dicendo; in questo modo $(w_1, w_2), (w_1, w_2, w_3), \ldots$ vengono a condividere il numero che $w_1 \star w_2, w_1 \star w_2 \star w_3$ ricevono in $A \cup \{\star\}$.

Il procedimento di Gödel consente tuttavia di evitare il ricorso al simbolo estraneo $\star$. Infatti, di fronte alla n-upla $(w_1, w_2, \ldots, w_n)$ di stringhe di A (con $n > 2$), possiamo

- ricordare che $w_1, w_2, \ldots, w_n$ hanno già un loro numero di codice,

$$\#_1(w_1), \#_1(w_2), \ldots, \#_1(w_n)$$

  rispettivamente,

- e ricorrere allora nuovamente al teorema fondamentale dell'Aritmetica, associando a $(w_1, w_2, \ldots, w_n)$ il numero

$$\#_2(w_1, w_2, \ldots, w_n) = 2^{\#_1(w_1)} \cdot 3^{\#_1(w_2)} \cdots p_k^{\#_1(w_n)}$$

Si ottiene così una funzione effettiva $\#_2$ dall'insieme delle sequenze finite di stringhe su A di ogni possibile lunghezza $n \geq 2$ a $\mathbb{N}$. Sequenze distinte ricevono da $\#_2$ numeri distinti, per il teorema fondamentale dell'Aritmetica e l'unicità della decomposizione in fattori primi. Viceversa, per ogni naturale m, è possibile effettivamente riconoscere se m è o no nell'immagine di $\#_2$ per una qualche sequenza di stringhe e, se sì, di quale: di nuovo basta ricorrere alla decomposizione di m in fattori primi, il numero dei fattori primi distinti coinvolti rivela la lunghezza della sequenza, gli esponenti di questi fattori dicono le stringhe della sequenza. Di più, non c'è pericolo di confondere i valori assegnati alle sequenze di stringhe e quelli associati in precedenza a simboli o stringhe, perché questi ultimi sono dispari oppure pari con esponenti dispari ¿ 3 nella loro decomposizione in fattori primi, i nuovi sono pari e con esponenti pari.

2.2.2 Codifica di naturali

Visto che stiamo trattando coppie, o terne, o n-uple di elementi, vale la pena di citare un'osservazione importante sulle coppie di numeri naturali, che ci sarà presto utile nel seguito. Si tratta di un risultato dimostrato dal matematico Georg Cantor nella seconda metà dell'Ottocento.

Teorema 2.3. C'è una corrispondenza biunivoca r tra $\mathbb{N}^2$ e $\mathbb{N}$; specificamente, per ogni scelta n, m di naturali,

$$r(n, m) = \frac{(n+m)(n+m+1)}{2} + n$$

L'affermazione è per certi versi sorprendente: si può infatti osservare che ogni naturale n compare infinite volte come ascissa di una coppia $(n, m) \in \mathbb{N}^2$ al variare di m. Così "sembra" infinitamente più grande di $\mathbb{N}$. Eppure è in corrispondenza biunivoca con $\mathbb{N}$ tramite r e quindi, in questo senso, ci sono tante coppie ordinate di naturali quanti naturali.

Al di là della complessità della formula che definisce r, cerchiamo di capire l'idea che Cantor seguì per ottenerla.

Supponiamo allora di disporre le coppie ordinate di numeri naturali in una tabella (infinita):

$$\begin{array}{cccccc}
(0,0) & (0,1) & (0,2) & \dots & (0,n) & \dots \\
(1,0) & (1,1) & (1,2) & \dots & (1,n) & \dots \\
(2,0) & (2,1) & (2,2) & \dots & (2,n) & \dots \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
(n,0) & (n,1) & (n,2) & \dots & (n,n) & \dots \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots
\end{array}$$

Le righe corrispondono alle ascisse, le colonne alle ordinate. Per enumerare le coppia di numeri naturali si percorra la tabella sulle diagonali come indicato nella figura che segue:

$$\begin{array}{cccccc}
\swarrow & \swarrow & \swarrow & \swarrow & \dots & \dots \\
\swarrow & \swarrow & \swarrow & \dots & \dots & \dots \\
\swarrow & \swarrow & \dots & \dots & \dots & \dots \\
\swarrow & \dots & \dots & \dots & \dots & \dots \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots
\end{array}$$

La prima diagonale (nell'asse nord-est verso sud-ovest) contiene solo $(0,0)$, la seconda $(0,1)$ e $(1,0)$, la k-esima contiene $(0,k-1),(1,k-2),\dots,(k-2,1)$ e $(k-1,0)$ e così via. La generica diagonale k contiene k coppie, precisamente quelle in cui la somma delle coordinate è $k-1$. In questo modo, le coppie (n,m) di risultano enumerate come segue:

$$(0,0),(0,1),(1,0),(0,2),(1,1),(2,0),(0,3),(1,2),(2,1),(3,0),\dots$$

e così via, dunque prima secondo $m+n$ e poi secondo n. Non è difficile controllare che la funzione r associa alle coppie sopra elencate i corrispondenti numeri naturali

$$0,1,2,3,4,5,6,7,8,9,\dots$$

infatti $r(0,0)=0, r(0,1)=1, r(1,0)=2$ e così via per tutta la sequenza dei numeri naturali. In generale r associa ad ogni coppia (n,m) il numero di passi necessari per raggiungerla meno 1. Dalla costruzione segue allora che la funzione r è una biezione: ogni coppia (n,m) compare una sola volta nella tabella e quindi è univocamente determinato il numero di passi necessario per raggiungerla lungo le varie diagonali. Inoltre r è data effettivamente e ammette, come ogni biiezione, la sua funzione inversa: una corrispondenza biunivoca da $\mathbb{N} a \mathbb{N}^2$, ancora effettiva. Sono quindi anche note le proiezioni di tale inversa su $\mathbb{N}$ cioè le due funzioni (sempre effettive) $\pi_1 : \mathbb{N} \to \mathbb{N}$ e $\pi_2 : \mathbb{N} \to \mathbb{N}$ tali che, per ogni c naturale, $\pi_1(c), \pi_2(c)$ sono rispettivamente quei naturali n, m per cui $r(n,m) = c$. A proposito, un insieme che — come $\mathbb{N}^2$ — è in corrispondenza biunivoca con $\mathbb{N}$ si dice *numerabile*. Il risultato che abbiamo appena visto in questa sezione si può estendere oltre le coppie, quindi è possibile definire analogamente biiezioni tra $\mathbb{N}^m$ e $\mathbb{N}$ per $m > 2$.

2.2.3 Codifica di Macchine di Turing

Adesso mostriamo che, per ogni alfabeto A, anche le Macchine di Turing che corrispondono ad A formano un insieme che è numerabile, e cioè è in corrispondenza biunivoca con $\mathbb{N}$. Anzi, è possibile dare un esempio esplicito di tale biiezione. Se A è fissato, una MdT M su A è determinata dall'insieme Q dei suoi stati e dalla funzione di transizione δ.

Possiamo convenire per semplicità di disporre a priori di un serbatoio infinito (numerabile) di stati

$$q_0, q_1, \ldots, q_n, \ldots$$

da cui ogni M estrae quegli $n+1$

$$q_0, q_1, \ldots, q_n$$

che le sono necessari per formare Q. Notiamo che il numero degli stati di M è sempre finito, ma può essere arbitrariamente grande; quindi non possiamo porre limiti al serbatoio cui M attinge e dobbiamo ammetterlo infinito. Quindi possiamo considerare M come determinata solamente dal numero $n+1$ dei suoi stati oltre che naturalmente dalla sua funzione di transizione.

Possiamo ora provare:

Teorema 2.4. C'è una corrispondenza biunivoca effettiva tra le MdT su un alfabeto A e i numeri naturali.

Dimostrazione. Poniamo anzitutto $A = \{a_1, \ldots, a_m\}$ con $a_1 \neq \cdots \neq a_m$. Procediamo in cinque passi consecutivi, in modo per certi versi analogo a quello seguito per numerare simboli, stringhe e sequenze di stringhe su un alfabeto nella Sezione 2.2.1. Per la precisione numeriamo successivamente

1. i simboli che compaiono nelle istruzioni delle MdT su A,
2. le istruzioni stesse,
3. le funzioni di transizione,
4. le MdT su A

e infine costruiamo la biiezione richiesta tra MdT e numeri.

1. Ricordiamo che i simboli che compaiono nelle istruzioni di una MdT su A sono
   - gli elementi $a_1, \ldots, a_m$ di A e il simbolo bianco $\star$;
   - gli stati $q_0, q_1, \ldots, q_n$;
   - gli indici di spostamento $+1$ e -1.

   Li numeriamo con una funzione $\#_0$ che opera come segue:

   $$\begin{array}{cccccccccc} -1 & +1 & \star & a_1 & a_2 & \ldots & a_n & q_0 & q_1 & \ldots & q_n \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \ldots & \downarrow & \downarrow & \downarrow & \ldots & \downarrow \\ 3 & 5 & 7 & 9 & 11 & \ldots & 2m+7 & 2m+9 & 2m+11 & \ldots & 2m+2n+2 \end{array}$$

   Così i valori di $\#_0$ sono tutti dispari ≥ 3; $\#_0$ è iniettiva ed effettiva.

2. Passiamo adesso alle istruzioni. Ciascuna di esse è una 5-upla $\Omega = (q, a, q', a', x)$ dove q, q' sono tra $q_0, q_1, \ldots, q_n, \ldots$ mentre a, a' appartengono ad $A \cup \{\star\}$ e $x = \pm 1$. Definiamo una funzione $\#_1$ che associa ad ogni Ω il numero naturale $\#_1(\Omega)$ in cui i codici dei cinque simboli componenti di Ω compaiono in ordine come esponenti dei primi $2, 3, 5, 7, 11$

$$\#_1(\Omega) = 2^{\#_0(q)} 3^{\#_0(a)} 5^{\#_0(q')} 7^{\#_0(a')} 11^{\#_0(x)}$$

   Si noti che i codici assegnati da $\#_1$ sono numeri pari che hanno gli esponenti dispari o nulli nella loro decomposizione in fattori primi: dunque, non si confondono con quelli di $\#_0$, che sono dispari. Anche $\#_1$ è una funzione iniettiva, come conseguenza del teorema fondamentale dell'aritmetica e dell' iniettività di $\#_0$; $\#_1$ è anche effettiva.

3. A questo punto siamo in grado di associare un numero naturale anche ad ogni funzione di transizione δ. Supponiamo che δ si componga delle quintuple $\Omega_1, \Omega_2, \ldots, \Omega_k$. Ordiniamo le componenti delle 5-uple convenendo, per esempio,

   - $q_0 < q_1 < q_2 < \cdots < q_n < \cdots$ tra gli stati,
   - $\star < a_1 < \cdots < a_m$ per l'alfabeto,
   - $-1 < +1$ tra gli spostamenti.

   Deduciamo su δ l'ordine lessicografico: così, per esempio, $(q_0, a_1, q_1, a_2, +1)$ precede $(q_1, a_2, q_0, a_1, -1)$ perché $q_0 < q_1$; $(q_1, a_1, q_0, a_1, -1)$ precede $(q_1, a_2, q_0, a_0, +1)$ perché $a_1 < a_2$. Supponiamo che si abbia per tal via $\Omega_1 < \Omega_2 < \cdots < \Omega_k$. Allora poniamo

$$\#_2(\delta) = \prod_{1 \le i \le k} p_i^{\#_1(\Omega_i)}$$

   dove p_i è, al solito, l'i-esimo numero primo. Otteniamo una funzione iniettiva ed effettiva $\#_2$ che non confonde i suoi valori con quelli di $\#_0$ o di $\#_1$ (grazie alle consuete argomentazioni). Per questo motivo $\#_2$ non è suriettiva.

4. A questo punto ogni MdT M su A risulta determinata dal numero $n+1$ dei suoi stati e dal codice $\#_2(\delta)$ della sua funzione di transizione, dunque da una coppia ordinata di naturali; anzi, tramite la biiezione r di su $\mathbb{N}$, risulta definita una funzione iniettiva $\#$ da MdT a naturali, quella che associa a ogni M l'immagine in r della coppia di numeri che determina M.

5. $\#$ è effettiva e iniettiva, ma non suriettiva perché neppure $\#_2$ lo è. Comunque possiamo ordinare gli elementi dell'immagine di $\#$:

$$r_0, r_1, r_2, \ldots, r_j, \ldots$$

ogni r_j corrisponde in $\#$ a un'unica MdT su A, che denotiamo M_j. Quindi $\#(M_j) = r_j$ per ogni $j \in \mathbb{N}$. Così anche le MdT su A risultano enumerate come

$$M_0, M_1, M_2, \ldots, M_j, \ldots$$

la funzione $j \mapsto M_j$ (per $j \in N$) è la biiezione richiesta.

□

L'enumerazione delle MdT appena ottenuta implica immediatamente una enumerazione anche delle funzioni calcolabili da MdT su un alfabeto A

$$\begin{array}{cccccccc} 0 & 1 & 2 & 3 & 4 & \ldots & i & \ldots \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \ldots & \downarrow & \ldots \\ \varphi_0 & \varphi_1 & \varphi_2 & \varphi_3 & \varphi_4 & \ldots & \varphi_i & \ldots \end{array}$$

dove la funzione φ_i denota la funzione calcolata dalla i-esima MdT M_i su A. Stavolta però l'enumerazione delle funzioni calcolabili può non essere una corrispondenza biunivoca poiché più MdT possono computare la stessa funzione.

Nel seguito, quindi, parleremo per ogni alfabeto A della i-esima MdT M_i su A e della i-esima funzione calcolabile φ_j (per intendere la funzione calcolata da M_i). Quando una funzione φ_i è definita su un dato input w (cioè M_i converge sull'input w) allora scriveremo $\varphi_i(w) \downarrow$, altrimenti scriveremo $\varphi_i(w) \uparrow$.

2.3 La tesi di Church-Turing

Come già accennato in precedenza, la macchina di Turing non è l'unico modello astratto di computabilità finora comparso. Anzi, durante gli anni '30 e all'inizio degli anni '40, molte importanti caratterizzazioni della calcolabilità hanno visto la luce: oltre a quella di Turing, anche quella di Kleene, basata sulle equazioni funzionali, quella di Church, fondata sul λ-calcolo, ed altre ancora. In ogni caso, gli autori hanno elaborato modelli elementari di computazione e hanno formulato l'ipotesi che la loro nozione di calcolabilità fosse la più generale possibile e che non potessero esistere procedimenti algoritmici di calcolo non esprimibili nel loro formalismo.

D'altra parte, tutti i tentativi fatti nell'ambito della logica matematica e dell'informatica teorica di definire modelli e paradigmi alternativi alle MdT hanno condotto a caratterizzazioni della calcolabilità di funzioni equivalenti a quelle di Turing. Questa circostanza ha condotto alla formulazione di quella che oggi è nota come *Tesi di Church-Turing* e viene ancora ritenuta universalmente valida. Questa tesi afferma che ogni procedimento algoritmico, espresso in un qualunque modello di calcolo, è realizzabile mediante una macchina di Turing.

Tesi di Church-Turing. *Una funzione è calcolabile se e solo se esiste una macchina di Turing che la calcola (e cioè se e solo se la funzione è calcolabile secondo Turing).*

Che ogni funzione calcolata da una MdT sia effettivamente calcolabile è banale. Ciò che è invece rilevante e problematico è l'implicazione inversa, per la quale ogni procedimento algoritmico è riconducibile a una MdT. "Algoritmo" e

"funzione calcolabile" sono concetti intuitivi, non specificati in modo formale, per cui non è possibile una dimostrazione rigorosa di equivalenza con il concetto di macchina di Turing. La Tesi di Church-Turing non è dunque una congettura che, in linea di principio, potrebbe un giorno diventare un teorema. Tuttavia, la nozione intuitiva di funzione calcolabile è contraddistinta da un insieme di caratteristiche (quali determinismo, finitezza di calcolo, etc), che possono essere considerate in larga misura "oggettive". Questo fa sì che sia praticamente sempre possibile una valutazione concorde nel decidere se un dato procedimento di calcolo possa essere considerato algoritmico o meno. Quindi, almeno in linea di principio, è ammissibile che venga "scoperto" un controesempio alla Tesi di Church-Turing: che si individui cioè una funzione che sia effettivamente calcolabile secondo questi parametri informali, ma che non sia computata da nessuna MdT. Almeno finora, però, nessun controesempio alla tesi di Church-Turing è stato trovato nonostante gli ovvi progressi teorici e pratici che l'Informatica ha avuto dagli anni '30. In effetti, gli argomenti a favore della Tesi di Church-Turing possono essere raccolti in tre gruppi.

- *Evidenza euristica.*
  - Per ogni singola funzione calcolabile che sia stata esaminata, è sempre stato possibile trovare una MdT in grado di computarla. Lo stesso si può dire dei procedimenti che producono effettivamente nuove funzioni a partire da altre. Eppure questa indagine è stata condotta per un gran numero di funzioni, di classi di funzioni e di procedure, con l'intento di renderla la più esaustiva possibile.
  - I metodi per dimostrare che le funzioni computabili sono calcolabili secondo Turing sono stati sviluppati con un grado di generalità tale da far ritenere improbabile che possa essere scoperta una funzione calcolabile cui l'approccio di Turing non possa essere applicato.
  - I vari metodi tentati per costruire funzioni effettivamente calcolabili, ma non computabili da MdT, hanno condotto tutti al fallimento, nel senso che le funzioni ottenute si dimostrano a loro volta calcolabili secondo Turing, oppure in realtà non calcolabili.
- *Equivalenza delle diverse formulazioni proposte.* Tutti i tentativi che sono stadi elaborati per caratterizzare in modo rigoroso la classe di tutte le funzioni effettivamente calcolabili si sono rivelati tra loro equivalenti. Ciò che è particolarmente rilevante è la diversità degli strumenti e dei concetti impiegati nelle diverse formulazioni; in molti casi esse traggono la loro origine da concetti matematici preesistenti. Che punti di vista talmente diversi e vasti convergano concentricamente alla stessa conclusione (e siano comunque equivalenti al modello di Turing) può essere inteso come forte argomento a sostegno di ognuno di loro e in particolare della tesi di Church-Turing.
- *L'impiegato diligente.* In realtà, i precedenti ragionamenti si applicano indifferentemente a tutti i vari approcci equivalenti alla calcolabilìtà. Ma c'è un'ulteriore riflessione che serve in particolare a sostenere l'approccio di Turing. In effetti, le caratteristiche che lo distinguono dagli altri sono la sua naturalezza, la sua minore astrazione, e anzi il suo dichiarato proposito di simulare meccanicamente il comportamento di un essere umano

che esegue un calcolo (l'impiegato diligente). In questo senso il modello di Turing si lascia preferire. Anzi, ci furono matematici illustri, come Kurt Gödel, che, pur perplessi di fronte all'astrazione di questi metodi alternativi, si dichiararono tuttavia pienamente convinti e conquistati dalla naturalezza del modello di Turing.

Queste argomentazioni hanno indotto la teoria della computabilità a sviluppare la tendenza di considerare la Tesi di Church-Turing come una sorta di "legge empirica", piuttosto che come un enunciato a carattere "logico-formale". In altre parole, si è autorizzati a procedere come segue. Se un certo linguaggio ammette un qualche algoritmo (di qualunque natura) che lo decide o accetta, oppure se una certa funzione ha un algoritmo che la calcola, allora, sulla base della tesi di Church-Turing, possiamo assumere che ci sia una macchina di Turing che decide o accetta quel linguaggio, o calcola quella funzione: non c'è bisogno di descrivere in dettaglio la MdT, possiamo affidarci all'autorevolezza della tesi e non attardarci in ulteriori verifiche.

La tesi di Church-Turing permette allora, per ogni algoritmo che sia intuitivamente tale (anche se espresso in maniera informale), di dedurre l'esistenza di una MdT che lavora in modo equivalente. L'uso della tesi di Church-Turing consente di esprimere rapidamente risultati che un approccio più formale costringerebbe a raggiungere più faticosamente. Per esempio, lavoriamo con numeri naturali x, y, z e con un alfabeto che li rappresenta, e consideriamo la funzione così definita:

$$f(x,y,z) = \begin{cases} 1 & \text{se l'}x\text{-esima macchina } M_x \text{ converge in } z \text{ passi sull'input } y \\ 0 & \text{altrimenti} \end{cases}$$

sulla base della tesi di Church-Turing, possiamo dire che essa è calcolabile senza fornire necessariamente una MdT che la computa. È infatti sufficiente fornire un algoritmo che la calcoli anche ricorrendo a procedure di più alto livello. Per esempio: si manda in esecuzione M_x sull'input y per z passi di computazione; a ogni passo si incrementa di 1 un contatore C (che inizialmente viene posto a 0) e si controlla se la macchina di Turing ha terminato la sua esecuzione (cioè ha raggiunto una configurazione terminale). Se si arresta quando il contatore non ha ancora raggiunto z allora l'algoritmo dà in output il valore 1. Se invece il contatore supera z senza che M_x si sia fermata, l'algoritmo termina e restituisce in output 0.

2.4 La macchina di Turing universale

I moderni calcolatori, così come li intendiamo comunemente, sono "macchine" capaci di eseguire svariati programmi. Una macchina di Turing invece, al di là del nome e della sua descrizione esteriore, è determinata dalla funzione δ che ne stabilisce le istruzioni ed è quindi un programma di computazione, delegato a calcolare sempre la stessa funzione. C'è tuttavia modo di costruire una MdT $\mathcal{U}$ che riesce a simulare il lavoro di ogni altra MdT M, nel senso che, per ogni M, e per ogni input naturale y, $\mathcal{U}$ accoglie (M, y) come input e riproduce la computazione di M su y. Per questo $\mathcal{U}$ è chiamata "universale". Vediamo come è possibile definirla.

C'è una ovvia perplessità relativa a questa ipotetica $\mathcal{U}$, e cioè: come si fa a fornire a $\mathcal{U}$ come input un'altra MdT M? Infatti l'alfabeto di $\mathcal{U}$ tratta numeri e non macchine. D'altra parte, abbiamo visto nella Sezione 2.2 come ogni MdT riceva un suo numero di etichetta, secondo una biiezione effettiva tra numeri e macchine.

In altre parole $M = M_x$ per un unico naturale x che può essere esplicitamente calcolato da M. Quindi possiamo proporre a $\mathcal{U}$ come input una coppia di numeri: il numero x come codice di M e il numero y come input di M.

Ci aspettiamo che $\mathcal{U}$ ci fornisca come output della coppia quello di M_x eseguito su input y e cioè $\varphi_x(y)$. Dimostriamo allora l'esistenza di $\mathcal{U}$.

Teorema 2.5. Esiste una MdT universale $\mathcal{U}$. In altre parole, la funzione $g : \mathbb{N}^2 \to \mathbb{N}$ definita ponendo, per ogni $x, y \in \mathbb{N}$,

$$g(x, y) = \varphi_x(y)$$

è calcolabile secondo Turing.

Dimostrazione. Informalmente, ci basta, per ogni scelta di $x, y \in \mathbb{N}$, prima decodificare x per ottenere la MdT x-esima M_x, poi far lavorare M_x su y. Entrambi i passaggi sono possibili, in quando la decodifica è algoritmica e le regole di funzionamento di una macchina di Turing sono anch'esse algoritmiche. Comunque, per non far sembrare questa soluzione un gioco di prestigio, diamo una descrizione più dettagliata della costruzione di $\mathcal{U}$. $\mathcal{U}$ può essere pensata come una macchina con due nastri ausiliari (sappiamo infatti che questo modello è equivalente a quello di una MdT con un solo nastro). Dapprima $\mathcal{U}$ controlla che il suo input sia una coppia (M, y), per qualche macchina di Turing $M = M_x$ e qualche input y per M. Poi $\mathcal{U}$ passa a simulare M sull'input y. In particolare, $\mathcal{U}$ adopera il primo nastro ausiliario per mantenere traccia degli stati e della posizione della testina di M e il secondo nastro per memorizzare le istruzioni di M. Così $\mathcal{U}$ copia, di volta in volta, una configurazione (ξ, q, a, η) di M dal nastro di input/output sul primo nastro ausiliario; poi per determinare la regola di transizione (q, a, q', a', x) di M da usare, $\mathcal{U}$ estrae q e a, quindi determina (q', a', x) cercando nel secondo nastro l'istruzione per (q, a). A quel punto si comporta come dettato dalla terna (q', a', x) per sviluppare sul nastro di input/output la sua computazione. □

2.5 Alcune funzioni calcolabili

Questa sezione è dedicata a presentare e discutere alcuni esempi di funzioni calcolabili secondo Turing. Per cominciare non è difficile vedere che le familiari operazioni dell'aritmetica, come l'addizione, la moltiplicazione, la divisione — intesa come la coppia di funzioni parziali quoziente e resto — sono tutte calcolabili.

Ecco un caso più sofisticato e astratto di funzione calcolabile.

Esempio 2.8. Sia f la funzione da $\mathbb{N}$ a $\mathbb{N}$ definita come segue: per ogni $x \in \mathbb{N}$,

$$f(x) = \begin{cases} 1 & \text{se } \varphi_x(x) \downarrow \\ \uparrow & \text{altrimenti} \end{cases}$$

Dimostriamo che f è una funzione calcolabile. Usiamo la Tesi di Church-Turing proponendo un algoritmo che la calcola a un alto livello di astrazione e deducendo che c'è una MdT che la computa. L'algoritmo procede così.

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Esegui M_x sull'input x;
If M_x termina then
    Output(1)
End
```

Naturalmente nel caso M_x divergesse sull'input x, anche questo algoritmo divergerebbe. Il condizionale alla riga 4 è del tutto facoltativo, infatti nel caso M_x divergesse l'algoritmo sarebbe fermo alla riga 3.

Ci sono poi casi sorprendenti e "anomali" di funzioni che si dimostrano calcolabili senza che si riesca a determinare con precisione una MdT, o anche un algoritmo, che le computi. Eccone un esempio.

Esempio 2.9. Consideriamo la funzione totale $g : \mathbb{N} \to \mathbb{N}$ tale che, per ogni $x \in \mathbb{N}$,

$$g(x) = \begin{cases} 1 & \text{se nell'espansione decimale di } \pi \text{ esistono almeno } x \text{ 5 consecutivi} \\ 0 & \text{altrimenti} \end{cases}$$

La funzione g è calcolabile perché esiste infatti un algoritmo che genera l'espansione decimale di π. Allora per g ci sono due casi possibili. Ammettiamo dapprima che l'espansione decimale di π riesca a includere un numero arbitrariamente grande di occorrenze successive di '5'; allora g coincide con la funzione costante 1, che è chiaramente calcolabile. Altrimenti c'è un massimo numero k di '5' consecutivi nello sviluppo decimale di π, e quindi, per ogni $x \in \mathbb{N}$,

$$g(x) = \begin{cases} 1 & \text{se } x \leq k \\ 0 & \text{altrimenti} \end{cases}$$

Anche questa seconda opzione è calcolabile.
Pertanto g corrisponde ad una delle due possibilità elencate, e in ogni caso g è calcolabile. D'altra parte siccome lo sviluppo decimale di π è

infinito aperiodico, non è dato sapere direttamente da una sua lettura parziale se vale il primo o il secondo caso per g e, semmai, quale è k nel secondo caso, nè si conoscono attualmente altre strategie per chiarire la questione. In conclusione, sappiamo che g ha un algoritmo di calcolo, ma non sappiamo quale è questo algoritmo.

Esistono poi funzioni di cui non è ancora noto se siano calcolabili o non calcolabili.

Esempio 2.10. Consideriamo la funzione totale $h : \mathbb{N} \to \mathbb{N}$ tale che, per ogni $x \in \mathbb{N}$,

$$h(x) = \begin{cases} 1 & \text{se nell'espansione decimale di } \pi \text{ esistono esattamente } x \text{ 5 consecutivi} \\ 0 & \text{altrimenti} \end{cases}$$

Chi potesse leggere per esteso lo sviluppo decimale di π potrebbe anche calcolare h, esaminandovi le occorrenze di 5; ma purtroppo lo sviluppo di π è infinito e sfugge ai nostri occhi. Potrebbero tuttavia esistere altre strategie più mirate per risolvere il problema. Ma ad oggi non è ancora noto alcun algoritmo convincente. Quindi, per quanto ne sappiamo, h può essere calcolabile, ma può anche non esserlo.

2.6 Esercizi

Esercizio 2.1

Programmare una MdT che calcoli la funzione logica NOT su una sequenza binaria.

Input	Output atteso
1	0
11011001	00100110
000	111

Esercizio 2.2

Programmare una Macchina di Turing che preso un numero naturale in notazione binaria ne restituisca il doppio.

Input	Output atteso
101	1010
11011001	110110010
0	0

Esercizio 2.3

Si scriva un programma per Macchina di Turing che, ricevuto sul nastro un numero decimale, lasci sul nastro solo l'intero della divisione del numero di ingresso per 10.

Input	Output atteso
231	23
19	1
7	0

Esercizio 2.4

Programmare una Macchina di Turing che calcoli $h(x, y) = x \dot{-} y$ (la sottrazione troncata) utilizzando il sistema numerico unario (**NB:** utilizzare la notazione per cui l'output corrisponde al numero di simboli 1 sul nastro, mentre l'input è il numero di simboli 1 meno 1, e.g: zero in input corrisponde a "1", ma in output è il nastro vuoto).

Input	Output atteso
111*11	1
111*111111	*
1111*1	111

Esercizio 2.5

Programmare una macchina di Turing che, dato un nastro iniziale contenente due sequenze di A separate da una D, termina la sua esecuzione lasciando sul nastro la sequenza che contiene il maggior numero di A.

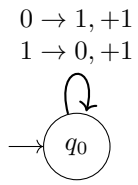
Input	Output atteso
AADA	AA
AADAAA	AAA
AADAA	AA
DA	A

Soluzioni

Soluzione 2.1

Questa MdT deve solamente sostituire ogni 0 con 1 e ogni 1 con 0.

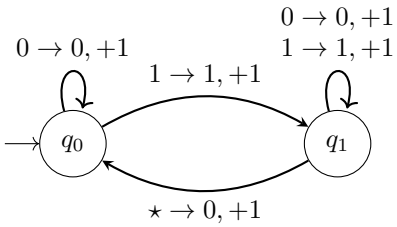
	0	1	$\star$
q_0	$(q_0, 1, +1)$	$(q_0, 0, +1)$	



Soluzione 2.2

Per raddoppiare un numero in notazione binaria è sufficiente aggiungere uno 0 a destra del numero, ad eccezione del caso di 0 che in tal caso rimane invariato.

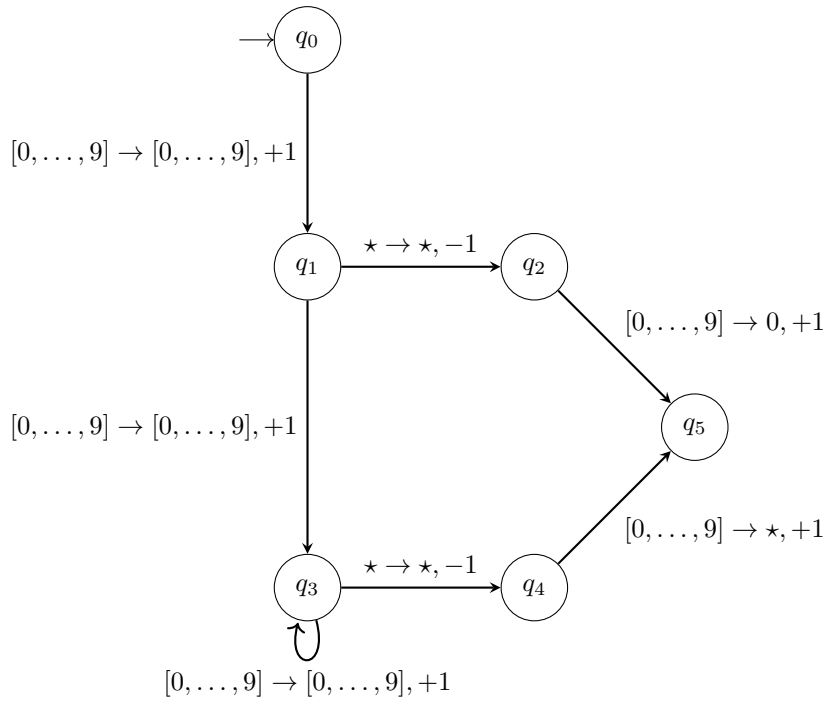
	0	1	$\star$
q_0	$(q_0, 0, +1)$	$(q_1, 1, +1)$	
q_1	$(q_1, 0, +1)$	$(q_1, 1, +1)$	$(q_0, 0, +1)$



Soluzione 2.3

Scriviamo la matrice di transizione e il grafo di transizione adottando una semplificazione visiva. Per indicare che una regola si applica indistintamente ad ognuna delle cifre decimali nella tabella scriviamo una colonna contrassegnata da $[0, \ldots, 9]$. Inoltre per indicare che una certa transizione lascia inalterate ognuna di queste cifre scriviamo $[0, \ldots, 9]$ nella cella che si trova nella colonna di $[0, \ldots, 9]$. L'idea dietro questa MdT è di verificare innanzitutto di quante cifre si compone il numero in input: se è una sola cifra, allora basta cancellarla e scrivere 0, mentre se è più di una cifra è sufficiente cancellare la cifra meno significativa.

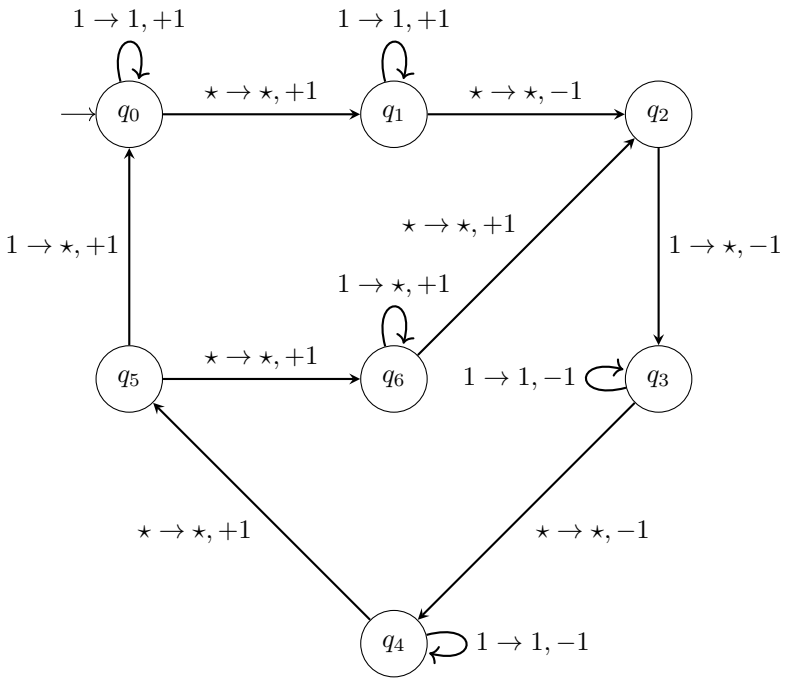
	$[0, \ldots, 9]$	$\star$
q_0	$(q_1, [0, \ldots, 9], +1)$	
q_1	$(q_3, [0, \ldots, 9], +1)$	$(q_2, \star, -1)$
q_2	$(q_5, 0, +1)$	
q_3	$(q_3, [0, \ldots, 9], +1)$	$(q_4, \star, -1)$
q_4	$(q_5, \star, +1)$	
q_5		



Soluzione 2.4

L'idea per svolgere questo esercizio è di ridurre progressivamente l'input, cancellando a due a due i simboli "1" alle estremità dell'input della MdT. Se i simboli "1" a destra dello spazio bianco centrale terminano, il nastro presenta il risultato della sottrazione. Se invece finiscono prima quelli a sinistra, è sufficiente ripulire totalmente il nastro.

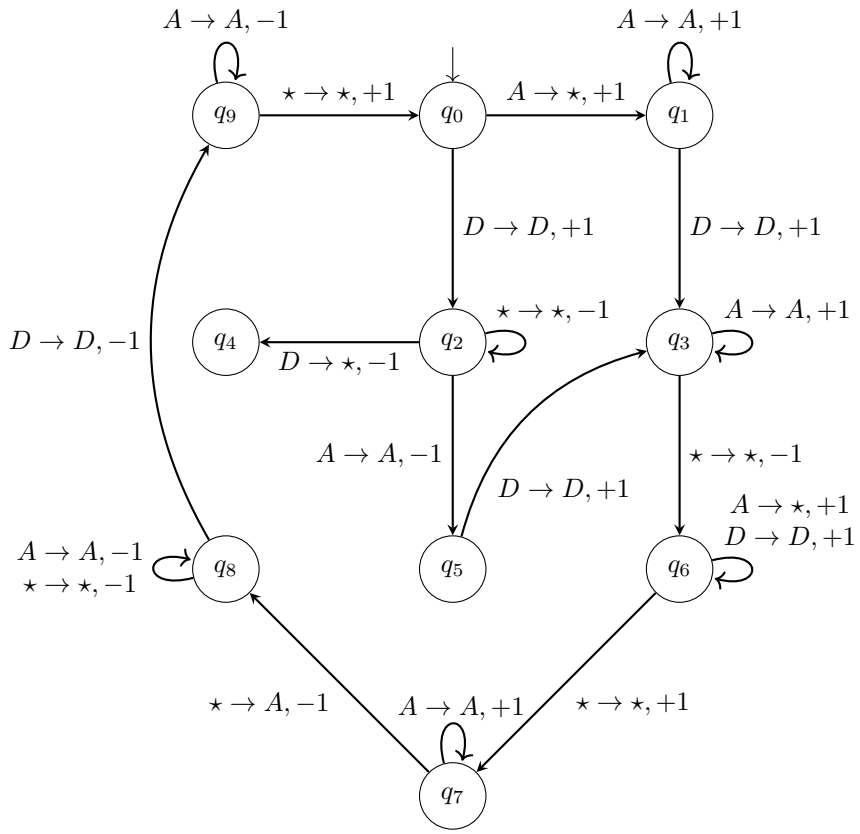
	1	$\star$
q_0	$(q_0, 1, +1)$	$(q_1, \star, +1)$
q_1	$(q_1, 1, +1)$	$(q_2, \star, -1)$
q_2	$(q_3, \star, -1)$	
q_3	$(q_3, 1, -1)$	$(q_4, \star, -1)$
q_4	$(q_4, 1, -1)$	$(q_5, \star, +1)$
q_5	$(q_0, \star, +1)$	$(q_6, \star, +1)$
q_6	$(q_6, \star, +1)$	$(q_2, \star, +1)$



Soluzione 2.5

La MdT deve gestire 3 casi particolari: se c'è solo la D ma nessuna A o se ci sono A solo a sinistra o solo a destra della D deve solamente cancellare la D. Nel caso ci siano A sia a sinistra che a destra si procede cancellando le A a due a due all'inizio e alla fine della stringa di input, ma per ogni coppia di A cancellate si deve anche aggiungere una A ad una sequenza che posizioniamo a destra della stringa di input. Questa sequenza sarà sarà poi l'output della MdT.

	A	D	$\star$
q_0	$(q_1, \star, +1)$	$(q_2, D, +1)$	
q_1	$(q_1, A, +1)$	$(q_3, D, +1)$	
q_2	$(q_5, A, -1)$	$(q_4, \star, -1)$	$(q_2, \star, -1)$
q_3	$(q_3, A, +1)$		$(q_6, \star, -1)$
q_4			
q_5		$(q_3, D, +1)$	
q_6	$(q_6, \star, +1)$	$(q_6, D, +1)$	$(q_7, \star, +1)$
q_7	$(q_7, A, +1)$		$(q_8, A, -1)$
q_8	$(q_8, A, -1)$	$(q_9, D, -1)$	$(q_8, \star, -1)$
q_9	$(q_9, A, -1)$		$(q_0, \star, +1)$


$A \to A, -1$
$A \to A, +1$
$\star \to \star, +1$
$A \to \star, +1$
q_9
q_0
q_1
$D \to D, +1$
$D \to D, +1$
$\star \to \star, -1$
$A \to A, +1$
$D \to D, -1$
q_4
q_2
q_3
$D \to \star, -1$
$A \to A, -1$
$\star \to \star, -1$
$D \to D, +1$
$A \to \star, +1$
$D \to D, +1$
$A \to A, -1$
$\star \to \star, -1$
q_8
q_5
q_6
$\star \to A, -1$
$A \to A, +1$
$\star \to \star, +1$
q_7

Capitolo 3

Problemi non risolvibili algoritmicamente

Nel Capitolo 2 abbiamo introdotto il modello di calcolo della macchina di Turing e abbiamo conseguentemente definito la computabilità secondo Turing (per linguaggi o per funzioni). Nel presente capitolo approfondiamo questa teoria.

Uno dei suoi aspetti più rilevanti è l'esistenza di problemi non computabili: per la precisione, emergono dal modello di Turing

- funzioni che non si possono calcolare,
- linguaggi che non si possono accettare, o che si accettano ma non si decidono.

Limiteremo per semplicità la nostra analisi ai numeri naturali, visti come parole su qualche opportuno alfabeto $\{0, 1, \ldots, 9\}$ o $\{0, 1\}$, o $\{1\}$: questa assunzione è implicita in tutto il capitolo, salvo avviso contrario. Del resto, le strategie di codifica apprese nella Sezione 2.2 ci mostrano come ogni alfabeto A si riduca effettivamente al contesto dei naturali. Ricordiamo poi che le MdT sull'alfabeto fissato si enumerano (in più possibili modi)

$$M_0, M_1, M_2, \ldots, M_n, \ldots (n \in \mathbb{N})$$

così come le funzioni da esse calcolate

$$\varphi_0, \varphi_1, \varphi_2, \ldots, \varphi_n, \ldots (n \in \mathbb{N})$$

(φ_n è in particolare la funzione calcolata da M_n). Nel nostro caso, quando lavoriamo su $\mathbb{N}$, possiamo assumere che ciascuna φ_n sia una funzione, eventualmente parziale, da $\mathbb{N}$ a $\mathbb{N}$. Tanto l'enumerazione delle MdT quanto quella conseguente delle funzioni calcolabili sono da ritenersi fissate per tutto il capitolo. I risultati che otterremo non dipendono comunque dalla loro scelta. Ricordiamo anche la seguente notazione: per f funzione parziale da $\mathbb{N}$ a $\mathbb{N}$ e n naturale, $f(n) \downarrow$ significa che n è nel dominio di f, cioè $f(n)$ è definita, $f(n) \uparrow$ il contrario.

Quanto ai risultati negativi di incomputabilità sopra menzionati, incontreremo due tecniche particolari che permettono di provarli:

1. La prima si chiama diagonalizzazione e riesce a produrre, ad esempio, funzioni non calcolabili manipolando opportunamente la lista di tutte le funzioni calcolabili;

2. La seconda prende il nome di riduzione: essenzialmente deduce l'incapacità di decidere un linguaggio L riferendoli a situazioni negative già note e provando che queste ultime avrebbero soluzione positiva se altrettanto valesse per L.

3.1 Il problema dell'arresto

In questa sezione mostriamo l'esempio più famoso, e forse più importante, di funzione non calcolabile, cioè di una funzione che, se accettiamo la tesi di Church-Turing, non ammette alcun algoritmo risolutivo.

Problema 3.1 (Problema dell'Arresto)**.** Determinare un algoritmo per stabilire, per ogni scelta di una MdT M e di un input $y \in \mathbb{N}$, se $M \downarrow y$ o no.

Allora, se prestiamo fede alla tesi di Church-Turing secondo cui tutto quello che è algoritmicamente risolubile può essere trattato con successo da MdT, il Problema dell'Arresto ci chiede una MdT che, ricevuti in input $x, y \in \mathbb{N}$ (e in particolare, tramite x, la MdT M_x), risponda

- SÌ se $M \downarrow y$,
- NO se $M \uparrow y$,

decida cioè l'insieme

$$K' = \{(x, y) \in \mathbb{N}^2 \mid M_x \downarrow y\}$$

Notiamo che la macchina cercata non può essere quella universale $\mathcal{U}$. Infatti $\mathcal{U}$, dati x, y, simula la computazione di M_x su y, ma non può prevederne l'esito e, se M_x diverge su y, è condannata a divergere su (x, y), a proseguire cioè indefinitamente la sua computazione senza potersi accorgere a nessun passo dell'assenza di una conclusione. Invece la MdT cercata converge per ogni scelta di x, y e anzi calcola la seguente funzione totale h' da $\mathbb{N}^2$ a $\mathbb{N}$:

$$h'(x, y) = \begin{cases} 1 & \text{se } \varphi_x(y) \downarrow \\ 0 & \text{se } \varphi_x(y) \uparrow \end{cases}$$

Ebbene questa macchina non esiste. Per dimostrarlo, iniziamo col dimostrare che nemmeno la seguente funzione, che è un caso particolare di h', è calcolabile.

Teorema 3.1. La funzione h tale che, per ogni input $x \in \mathbb{N}$,

$$h(x) = \begin{cases} 1 & \text{se } \varphi_x(x) \downarrow \\ 0 & \text{se } \varphi_x(x) \uparrow \end{cases}$$

non è calcolabile da nessuna MdT.

Dimostrazione. Supponiamo per assurdo il contrario, che esista cioè una MdT che calcola h. Consideriamo la seguente funzione k per $x \in \mathbb{N}$,

$$k(x) = \begin{cases} \uparrow & \text{se } h(x) = 1 \\ 0 & \text{se se } h(x) = 0 \end{cases}$$

Dalla calcolabilità della funzione h segue facilmente quella di k. Dunque c'è una MdT che calcola k; sia $j \in \mathbb{N}$ il suo codice; in altre parole sia $\varphi_j = k$. Ma allora abbiamo

$$\varphi_j(j) \downarrow \iff k(j) = 0 \iff h(j) = 0 \iff \varphi_j(j) \uparrow$$

il che costituisce una evidente contraddizione. Quindi h non è calcolabile. □

Il metodo usato per trovare una contraddizione nella prova del precedente teorema è un esempio della tecnica della diagonalizzazione, che tratteremo integralmente nella Sezione 3.3. Comunque per avere un idea di questa tecnica, notiamo in questo caso che dall'elenco

$$\varphi_0, \varphi_1, \ldots$$

delle funzioni calcolabili e dall'apparentemente innocua equivalenza, valida per ogni naturale x,

$$k(x) \downarrow \iff \varphi_x(x) \uparrow$$

si deduce, per $k = \varphi_j$ e per $x = j$, la contraddizione

$$\varphi_j(j) \downarrow \iff \varphi_j(j) \uparrow$$

e dunque si conclude che k non può essere calcolabile.

Corollario 3.1. L'insieme $K = \{x \in \mathbb{N} \mid M_x \downarrow x\}$ non è decidibile.

Dimostrazione. La funzione h del Teorema 3.1 non è altro che la funzione caratteristica di K, quindi dalla sua non calcolabilità segue la non decidibilità di K. □

Possiamo concludere ora la nostra dimostrazione della non calcolabilità di h'.

Corollario 3.2. La funzione h' tale che, per ogni scelta di $x, y \in \mathbb{N}$,

$$h'(x, y) = \begin{cases} 1 & \text{se } \varphi_x(y) \downarrow \\ 0 & \text{se } \varphi_x(y) \uparrow \end{cases}$$

non è calcolabile da nessuna MdT.

Dimostrazione. Per assurdo, se h' fosse calcolabile, lo sarebbe anche h: infatti $h(x) = h'(x, x)$ per ogni $x \in \mathbb{N}$ □

Corollario 3.3. L'insieme $K' = \{(x, y) \in \mathbb{N}^2 \mid M_x \downarrow y\}$ non è decidibile.

Dimostrazione. La funzione h' è la funzione caratteristica di K', quindi dalla sua non calcolabilità segue la non decidibilità di K'. □

La prova di questi corollari illustra il metodo della riduzione, che tratteremo in seguito nella Sezione 3.4: in ogni caso l'indecidibilità di K' è dedotta dall'analogo risultato per K tramite la funzione $x \mapsto (x, x)$.

La soluzione negativa del problema dell'arresto ha delle importanti conseguenze anche nell'ambito della teoria della programmazione. In particolare, essa indica chiaramente che non è possibile costruire un perfetto sistema di "debugging", che sia in grado di stabilire se un programma termini o meno sull'input dato. Notiamo poi che, sebbene il problema dell'arresto, nella sua piena generalità, sia stato dimostrato indecidibile, è tuttavia possibile ottenere risultati positivi in casi parziali. Per esempio abbiamo già affermato nella Sezione 2.3 la calcolabilità della funzione f che a ogni terna (x, y, z) di naturali associa

$$f(x, y, z) = \begin{cases} 1 & \text{se } M_x \text{ sull'input } y \text{ termina in } z \text{ passi} \\ 0 & \text{altrimenti} \end{cases}$$

In questo caso, infatti, si limita con z il numero di passi di computazione di M_x, e questa specificazione consente il calcolo effettivo dell'immagine di f in ogni situazione.

Concludiamo questa sezione considerando un'appendice del problema dell'arresto. Abbiamo stabilito che la questione di decidere, per ogni MdT M e per ogni input x, se $M \downarrow x$ o no è priva di algoritmi di soluzione. Si può pensare che, se si fissa M e ci si limita a controllare per quali x $M \downarrow x$ e per quali no, la situazione si semplifichi e ammette risposta positiva. Certe volte è davvero così. Per esempio, se M è una MdT senza istruzioni, allora M converge su ogni input e quindi

$$\{x \in \mathbb{N} \mid M \downarrow x\}$$

coincide con $\mathbb{N}$ ed è ovviamente decidibile. Ma talora la situazione resta così complicata come in generale. Infatti si ha:

Corollario 3.4. Ci sono MdT M tali che $\{x \in \mathbb{N} \mid M \downarrow x\}$ è indecidibile.

Dimostrazione. Basta sfruttare la semidecidibilità di K e prendere una MdT M che accetta K. Allora l'insieme $\{x \in \mathbb{N} \mid M \downarrow x\}$ è indecidibile. Lo stesso ragionamento si applica, ovviamente, a ogni linguaggio semidecidibile e indecidibile. □

3.2 Insiemi decidibili, semidecidibili e non semidecidibili

Nella Sezione 2.1.3 abbiamo introdotto non solo linguaggi decidibili, ma anche semidecidibili. Ricordiamo la definizione di entrambi questi concetti (assumendo semmai per semplicità di trattare linguaggi $L \subseteq \mathbb{N}$). Allora si ha:

- L è decidibile se e solo se c'è una MdT che sa decidere, per ogni $x \in \mathbb{N}$, se $x \in L$ o no (e dunque, in senso lato, se e solo se c'è un algoritmo che sa svolgere questo compito),
- L è semidecidibile se e solo se c'è una MdT che lo accetta e cioè converge su tutti e soli gli elementi di L.

Cerchiamo di cogliere meglio il senso di questa seconda nozione e dimostriamo:

Teorema 3.2. Sia $L \subseteq \mathbb{N}$ Allora sono equivalenti le seguenti proposizioni

(a) L è semidecidibile,

(b) L è il dominio di una funzione calcolabile,

(c) L è vuoto o è l'immagine di una funzione totale calcolabile $f : \mathbb{N} \to \mathbb{N}$ (e dunque gli elementi di L sono enumerati effettivamente come f(0), f(1), f(2) e così via).

Dimostrazione. L'equivalenza tra (a) e (b) è diretta conseguenza della definizione di semidecidibilità. Se M è la MdT che accetta L, L è il dominio di una funzione χ'_L (appunto la funzione calcolata da M), e viceversa. Passiamo allora a confrontare (a) e (c).

- **(a) $\Longrightarrow$ (c)**: Supponiamo dapprima L semidecidibile e fissiamo una MdT M che converge su tutti e soli gli elementi di L. Enumeriamo L seguendo le computazioni di M sui naturali $0, 1, 2, \ldots, m, \ldots$: non appena ci si accorge che la computazione di M su uno di questi elementi m ha termine, si aggiunge m all'elenco di L.

  C'è però un'ovvia obiezione che si può sollevare per questo procedimento, e cioè come controllare contemporaneamente il lavoro di M sugli infiniti input m, tanto più che ogni computazione richiede i suoi passi $0, 1, 2, \ldots, n, \ldots$ e talora, nei casi di divergenza, infiniti passi. Per esempio, se M diverge su 0 come facciamo ad avviare la computazione su $1, 2, \ldots$ e verificarne l'esito? La risposta consiste nell'applicare la tecnica della coda di colomba introdotta in Sezione 3.2.3: si segue il passo 0 di M su 0, poi il passo 1 su 0, il passo 0 su 1, e via dicendo. In questo modo, se $L \neq 0$, si generano i suoi elementi

  $$l_0, l_1, l_2, \ldots, l_j, \ldots$$

  (se $L = \varnothing$, L già soddisfa (c)). La lista così ottenuta è finita per L finito ma, ammettendovi ripetizioni, possiamo supporla infinitamente

lunga, cioè con j che varia tra tutti i naturali. Poniamo adesso, per ogni $j \in \mathbb{N}$,

$$f(j) = l_j$$

Allora f è una funzione totale calcolabile, e la sua immagine coincide proprio con L.

- **(c) $\Longrightarrow$ (a)**: Supponiamo viceversa che ci sia una funzione f totale e calcolabile con queste proprietà. Dimostriamo la semidecidibilità di L costruendo una MdT M che converge su tutti e soli gli elementi di L, procedendo come segue. Per ogni $x \in \mathbb{N}$, M computa finché necessario $f(0), f(1), f(2), \ldots$ e per ognuno controlla contestualmente se coincide o no con x. Se la verifica è positiva, M si arresta, se no prosegue il suo lavoro. Quindi M converge solamente sugli elementi di L come richiesto, dimostrandone la sua semidecidibilità. Lo stesso accade, ovviamente, se $L = \varnothing$: in tal caso L è accettato da una MdT che diverge su ogni input.

□

In conclusione, L (se non vuoto), è semidecidibile se e solo se c'è un programma (e dunque una MdT) che enumera effettivamente i suoi elementi. Proprio per questo motivo, gli insiemi semidecidibili sono anche noti come *ricorsivamente enumerabili*, mentre gli insiemi decidibili sono detti *ricorsivi*.

Data questa nuova caratterizzazione, possiamo ora dimostrare in una nuova maniera il fatto che la decidibilità di un insieme implica anche la sua semidecidibilità, come già fatto notare nella Sezione 2.1.3: una MdT che sa riconoscere gli elementi di un insieme L e scartare gli altri si adatta facilmente a elencare gli elementi di L (se ve ne sono).

Non sembra invece chiaro il contrario, se cioè un insieme semidecidibile non vuoto sia anche decidibile. Ammettiamo infatti di avere un algoritmo che produce successivamente tutti gli elementi di L. Ogni suo output è ovviamente in L. Tuttavia non c'è verso di riconoscere per suo tramite gli elementi x fuori di L: chi osserva gli elementi prodotti dall'algoritmo, prende atto che, dopo 10, o 100 o 10^n passi, x non è ancora presente tra essi, ma non può dedurre che x non comparirà nei passi successivi.

3.2.1 Il teorema di Post

Non è difficile convincersi che un insieme L è decidibile se e solo se anche il complemento $\bar{L}$ lo è. Infatti, per ogni $x \in \mathbb{N}$, $x \in \bar{L}$ se e solo se $x \notin L$. Così una MdT che decide L si può facilmente adattare a decidere $\bar{L}$ (e viceversa): basta scambiare le risposte finali, dire NO (a $x \in \bar{L}$?) quando si diceva SÌ (a $x \in L$?), e viceversa.

Di nuovo, non sembra affatto chiaro che altrettanto valga per la semidecidibilità. Infatti, se L è semidecidibile e dunque sappiamo riconoscere gli elementi di L come quelli su cui un'opportuna MdT M converge, non è detto che sappiamo riconoscere per tal via anche gli altri, quelli su cui M diverge.

In effetti si ha:

Teorema 3.3 (Teorema di Post)**.** Sia $L \subseteq \mathbb{N}$. Allora L è decidibile se e solo se tanto L quanto il suo complemento $\bar{L}$ sono semidecidibili.

Dimostrazione. Sappiamo già che, se L è decidibile, allora L è semidecidibile. Del resto, se L è decidibile, anche $\bar{L}$ lo è, quindi $\bar{L}$ è semidecidibile.
Viceversa, assumiamo $L, \bar{L}$ semidecidibili. Ci sono due MdT $M(L)$ e $M(\bar{L})$ che accettano $L, \bar{L}$ rispettivamente. Per ogni $x \in \mathbb{N}$, seguiamo le computazioni di $M(L)$ e $M(\bar{L})$ su x. Una e una sola delle due converge, perché x è in uno e uno solo degli insiemi $L, \bar{L}$. Se converge $M(L)$, si dichiara $x \in L$; se converge $M(\bar{L})$, si conclude $x \in \bar{L}$. □

3.2.2 L'insieme K e il suo complemento

Nella Sezione 3.1 avevamo già visto un esempio di insieme non decidibile:

$$K = \{x \in \mathbb{N} : M_x \downarrow x\}$$

Ora siamo in grado di affermare il seguente teorema:

Teorema 3.4. K è semidecidibile.

Dimostrazione. Ecco un possibile algoritmo per una MdT M che accetta K:

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Esegui M_x sull'input x;
If M_x termina then
    Output(SÌ)
End
```

Naturalmente dal passo 4 segue che se M_x diverge anche M diverge. □

Corollario 3.5. $\bar{K}$ non è semidecidibile (e dunque esistono linguaggi non semidecidibili).

Dimostrazione. Assumiamo per assurdo che $\bar{K}$ sia semidecidibile. Allora per il teorema di Post possiamo concludere che K è decidibile, che è un assurdo. □

3.2.3 La tecnica della coda di colomba

Ora consideriamo una tecnica molto utile quando si vuole dimostrare la semidecidibilità di alcuni insiemi. Infatti spesso capiterà di voler enumerare il codominio di una funzione f. Questo procedimento consiste nel verificare il risultato di $f(0), f(1), f(2), \ldots$ e così via per ogni numero naturale, fermandosi eventualmente a seconda del risultato della funzione per un certo input. Seguendo questo procedimento, tuttavia, si possono incontrare dei valori che non

fanno parte del dominio della funzione, elementi che, in altre parole, su cui la MdT corrispondente a quella funzione non termina la sua computazione. Quindi per esempio si potrebbe arrivare a un naturale z per cui $f(z) \uparrow$, e in tal caso l'algoritmo rimarrebbe "bloccato" a computare $f(z)$ senza la possibilità di controllare i valori $> z$.

La tecnica a *coda di colomba* permette di evitare questo problema. Come abbiamo visto nella Sezione 2.2.2, l'insieme $\mathbb{N}^2$ può essere messo in corrispondenza biunivoca con l'insieme $\mathbb{N}$. Questa corrispondenza si concretizza tramite una funzione biettiva $r : \mathbb{N}^2 \to \mathbb{N}$ che a ogni coppia $(x, y) \in \mathbb{N}^2$ fa corrispondere un $c \in \mathbb{N}$ e due funzioni calcolabili $\pi_1 : \mathbb{N} \to \mathbb{N}$ e $\pi_2 : \mathbb{N} \to \mathbb{N}$ tali che $\pi_1(c) = x$ e $\pi_2(c) = y$.

Questo ci dà la possibilità di limitarci all'insieme dei numeri naturali per essere in grado di considerare tutte le coppie in $\mathbb{N} \times \mathbb{N}$. In particolare, ci interessa di codificare in questo modo l'input x da dare alla macchina di Turing M_f che calcola f e il numero di passi n per cui vogliamo che la macchina di Turing continui la sua computazione. Quindi la coda di colomba consiste nel considerare i numeri naturali $0, 1, 2, \ldots$ come le coppie $(\pi_1(0), \pi_2(0)), (\pi_1(1), \pi_2(1)), (\pi_1(2), \pi_2(2)), \ldots$, le quali corrispondono a $(0, 0), (1, 0), (0, 1), \ldots$.

Torniamo al nostro problema di enumerare il codominio di una qualsiasi funzione f, e consideriamo la macchina di Turing M_f che la calcola. Sfruttando la tecnica a coda di colomba mandiamo in esecuzione M_f con input 0 per 0 passi, poi su input 1 per 0 passi, poi su input 0 per 1 passo, e così via. Se su un certo input x dopo il numero di passi specificato n la MdT M_f è in una configurazione finale, allora il valore sul nastro è esattamente il valore di $f(x)$, altrimenti è un risultato intermedio che andiamo a ignorare. Il vantaggio di questo procedimento è che se per un certo $z \in \mathbb{N}$ la funzione f non è definita, l'enumerazione del suo codominio non si ferma: eseguire M_f con input z per un numero finito di passi porta sempre a ottenere un risultato intermedio, che andremo a scartare, ma senza risultare nel rimanere bloccati in una computazione senza fine.

Chiarifichiamo quanto detto con un esempio.

Esempio 3.1. Consideriamo il seguente insieme:

$$I = \{i \in \mathbb{N} \mid 5 \in Im(\varphi_i)\}$$

Per dimostrarne la semidecidibilità, tutto ciò che dobbiamo fare è scrivere un algoritmo che letto i, ottiene la MdT corrispondente e controlla se per un qualche valore questa MdT restituisce in output 5.
Utilizzando la tecnica a coda di colomba l'algoritmo è il seguente:

```
Begin
Input(i);
Decodifica i per ottenere la MdT M_i;
Sia c = 0 tale che c = ⟨π_1(c), π_2(c)⟩;
While True do
    Begin
    Siano x = π_1(c); n = π_2(c);
    Esegui M_i su input x per n passi;
    If M_i è in una configurazione finale con output 5 then
        Begin
        Output(SÌ); stop
        End
    Else
        c = c + 1
    End
End
```

Utilizziamo un contatore c che viene inizializzato a 0, e lo scomponiamo tramite π_1 e π_2 nella corrispondente coppia di naturali: il primo valore, x, sarà l'input effettivo che diamo alla MdT, mentre il secondo, n, è il numero di passi per il quale simuliamo la MdT. Se a un certo punto la MdT arriva in una configurazione finale con l'output che cercavamo, allora l'algoritmo accetta i. Altrimenti non abbiamo altra scelta che continuare a cercare, incrementando il contatore e simulando nuovamente la MdT sul nuovo input e numero di passi.

3.3 Diagonalizzazione

La tecnica della *diagonalizzazione* fu introdotta da Georg Cantor nel 1891 come prova della non numerabilità di alcuni insiemi, in particolare la famosa prova della non numerabilità dei numeri reali.

Questa tecnica si può generalmente applicare quando si ha una enumerazione di oggetti (nel nostro caso specifico, questi oggetti per esempio possono essere le funzioni calcolabili) e si vuole costruire un nuovo oggetto dello stesso tipo ma diverso da tutti gli altri. In particolare, quando nel nostro caso si tratta di funzioni si cercherà di definire una funzione f che sia diversa da ogni altra φ_n nel punto n.

	0	1	2	$\cdots$
α_0	d_{11}	d_{12}	d_{13}	$\cdots$
α_1	d_{21}	d_{22}	d_{23}	$\cdots$
α_2	d_{31}	d_{32}	d_{33}	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Figura 3.1: La tabella sulla quale si costruisce una prova per diagonalizzazione.

Il metodo diagonale prende il nome dal fatto che viene sottintesa una tabella con infinite righe e colonne, dove ogni riga corrisponde a un oggetto dell'enumerazione e ogni colonna corrisponde a un numero naturale (Figura 3.1). Da questa tabella si prende in considerazione la diagonale, quindi tutte le coppie (α_n, n). I contenuti della tabella dipendono dal caso in questione, e la prova consiste nel costruire un oggetto i cui contenuti sono diversi per ognuno dei valori su questa diagonale. Mostriamo questo procedimento con un esempio.

Esempio 3.2. Vediamo un esempio di questa tecnica mostrando un'altra dimostrazione dell'indecidibilità di $K = \{x \in \mathbb{N} \mid \varphi_x(x) \downarrow\}$, e quindi nella non calcolabilità della sua funzione caratteristica

$$\chi_K(x) = \begin{cases} 1 & \text{se } \varphi_x(x) \downarrow \\ 0 & \text{altrimenti} \end{cases}$$

In questo caso possiamo immaginare che la tabella abbia una riga per ogni funzione calcolabile φ_n e una colonna per ogni numero naturale n. Vogliamo quindi costruire, tramite la tecnica di diagonalizzazione, una funzione che sia diversa da ogni altra nell'enumerazione facendo in modo che differisca da φ_n nel punto n.
In che modo dovrebbe differire? Ovvero, cosa contengono le celle della tabella? Questo dipende da quale sia il nostro scopo, e in questo caso è di dimostrare la non decidibilità di K. Quindi possiamo stabilire che la cella di riga i e colonna j ci dica se φ_i sia definita nel punto j oppure no.
A questo punto abbiamo varie opzioni per costruire la nostra funzione. Una di queste, che abbiamo mostrato in Sezione 3.1, consiste nel costruire una funzione k che è definita sul punto x se e solo se φ_x non lo è. In questo esempio però vogliamo mostrare un'altra possibilità. Chiamiamo la nostra funzione f. Se φ_x non è definita su x, allora proprio come nel caso di k avremo che la nostra funzione f è invece definita con risultato 0 (o qualsiasi altro numero), e quindi sicuramente $f(x) \neq \varphi_x(x)$ perché f converge mentre φ_x diverge. Invece, se φ_x è definita su x, allora sicuramente avrà un qualche risultato $z = \varphi_x(x)$. Per rendere la nostra funzione diversa, poniamo che f sia definita, ma con risultato $z+1$. Così, anche in questo caso $f(x) \neq \varphi_x(x)$ perché $z + 1 \neq z$. Formalmente f è definita come segue:

$$f(x) = \begin{cases} \varphi_x(x) + 1 & \text{se } \varphi_x(x) \downarrow \\ 0 & \text{altrimenti} \end{cases}$$

Notiamo che, come nella costruzione di k, nella costruzione di f siamo stati interessati solo agli elementi sulla diagonale (φ_x e x).
Come ci aiuta questa funzione a dimostrare che K non è decidibile? Se assumiamo per assurdo che K sia decidibile, questa funzione sarebbe calcolabile. Infatti basterebbe controllare se $x \in K$, e poi in caso affermativo calcolare $\varphi_x(x) + 1$, o in caso negativo semplicemente restituire 0. Quindi per la tesi di Church-Turing, e assumendo la decidibilità di K, f è calcolabile.

Ora, se mostrassimo che f porta a un risultato assurdo, il ché potremmo già fortemente sospettarlo dato che è stata definita tramite diagonalizzazione, saremmo costretti a rigettare l'ipotesi della decidibilità di K, deducendo K non decidibile che è proprio ciò che volevamo dimostrare all'inizio.
Mostriamo come f porti a un assurdo. Dato che per le nostre assunzioni f è calcolabile, allora sicuramente esiste un indice m tale che $f = \varphi_m$. Inoltre dalla definizione di f si nota che la funzione è totale. Quindi

$$\varphi_m(m) = f(m) = \varphi_m(m) + 1$$

dal fatto che $\varphi_m(m) \downarrow$. Questo risultato è chiaramente un assurdo perché un numero non può essere uguale al suo successore. Quindi la nostra funzione f non può essere calcolabile, e di conseguenza K non è decidibile.

3.4 Riduzione

La tecnica della riduzione è molto utile per dimostrare l'indecidibilità o la non semidecidibilità di insiemi.

Definizione 3.1. Dati due insiemi $A, B \subseteq \mathbb{N}$, diciamo che A è *riducibile "molti-uno"* a B, e si scrive $A \leq_m B$, se esiste una funzione $f : \mathbb{N} \to \mathbb{N}$ totale e calcolabile tale che per ogni $x \in \mathbb{N}$ si abbia

$$x \in A \iff f(x) \in B$$

che in altre parole significa

$$\begin{aligned} x \in A &\implies f(x) \in B \\ x \notin A &\implies f(x) \notin B \end{aligned}$$

Se la funzione f è anche iniettiva, diciamo invece che A è *riducibile "uno-uno"* a B e lo scriviamo $A \leq_1 B$.

La riduzione si può interpretare come una misura di complessità relativa tra insiemi. Cioè, se $A \leq_m B$ allora possiamo dire, equivalentemente:

- B è complicato almeno quanto A
- A non è più complicato di B

Formalizziamo questo concetto dimostrando il seguente teorema.

Teorema 3.5. Siano $A, B \subseteq \mathbb{N}$ tali che $A \leq_m B$ tramite una funzione totale e calcolabile f. Allora possiamo dire:

1. Se A non è decidibile, allora non lo è neppure B.
2. Se A non è semidecidibile, allora non lo è neppure B.

Dimostrazione. Dimostriamo entrambi i punti:

1. Supponiamo per assurdo che B sia decidibile, e quindi che la sua funzione caratteristica χ_B sia calcolabile. Questa funzione è quindi totale e in grado di riconoscere, per ogni $y \in \mathbb{N}$, se $y \in B$ oppure no. Allora possiamo scrivere il seguente algoritmo:

```
Begin
Input(x);
Sia y = f(x);
Output(χ_B(y))
End
```

   Questo algoritmo, per la tesi di Church-Turing, sarebbe in grado di decidere A: infatti se $f(x)$ ci restituisse un valore $y \in B$ vorrebbe dire che $x \in A$, e analogamente se $y \notin B$ sapremmo che necessariamente $x \notin A$. Ma per ipotesi abbiamo detto che A non è decidibile, e quindi è un assurdo.

2. Supponiamo per assurdo che B sia semidecidibile, e quindi che esista una funzione calcolabile χ'_B che lo accetti. Questa funzione converge solo e soltanto su ogni $y \in \mathbb{N}$ che appartiene a B. Allora possiamo scrivere il seguente algoritmo:

```
Begin
Input(x);
Sia y = f(x);
Output(χ'_B(y))
End
```

   Questo algoritmo, per la tesi di Church-Turing, sarebbe in grado di accettare anche A: infatti se $f(x)$ ci restituisse un valore y sul quale χ'_B converge vorrebbe dire che $x \in A$ e il nostro algoritmo correttamente converge, e analogamente se su y χ'_B diverge vorrebbe dire che $x \notin A$ e l'algoritmo divergerebbe a sua volta. Tuttavia per ipotesi abbiamo detto che A non è semidecidibile, e quindi è un assurdo.

□

Prima di mostrare un esempio, vogliamo menzionare un importante teorema che ci sarà utile nel seguito quando useremo la riduzione. Ne riportiamo una versione semplificata per il caso di una funzione in due parametri x, y di cui il primo è fissato, tuttavia il risultato è valido in generale per funzioni di $m + n$ parametri di cui i primi m sono fissati.

Teorema 3.6 (Teorema del parametro (con $m = 1$ e $n = 1$))**.** Sia $f : \mathbb{N}^2 \to \mathbb{N}$ una funzione calcolabile. Allora esiste una funzione totale e

calcolabile $s : \mathbb{N} \to \mathbb{N}$ tale che $\forall x, y \in \mathbb{N}$

$$\varphi_{s(x)}(y) = f(x, y)$$

Dimostrazione. Sia M_e la MdT che calcola la funzione f, e cioè sia $\varphi_e(x, y) = f(x, y)$.
Descriviamo ora il comportamento della MdT che computa $\varphi_{s(x)}(y)$: la MdT inizia che ha solo y sul nastro, quindi inizia la sua computazione eseguendo un sottoprogramma $M(x)$ che ha il compito di scrivere il valore di x alla sinistra del valore di y. Per fare ciò sposta la sua testina a sinistra di $n(x) + 1$ celle, quanto basta per scrivere x e lasciare esattamente uno spazio bianco tra x e y. Quindi la testina viene posizionata sul primo simbolo di x e abbiamo sia x che y sul nastro. A questo punto la MdT simula M'_e, che è la stessa MdT di M_e ma con l'accortezza di aver rinominato i suoi stati per far sì che non ci sia confusione con le istruzioni di $M(x)$.
Ora sia $s(x)$ l'indice della MdT che abbiamo appena descritto. Per convincerci che s sia totale e calcolabile, notiamo che il programma che abbiamo specificato sopra può essere generato a partire dal solo valore di x. La codifica di $M(x)$ è facilmente generabile a partire da x, mentre la codifica di M'_e si riesce a ottenere a partire da e e conoscendo il numero di stati di $M(x)$: basta ricalcolare la codifica di M_e dopo averne rinominato gli stati. Infine concatenando la codifica di $M(x)$ e quella di M'_e, otteniamo la codifica della MdT che calcola $\varphi_{s(x)}(y)$. L'indice di questa MdT è la posizione di questa MdT nella enumerazione di tutte le MdT. □

Esempio 3.3. Dimostriamo che l'insieme $I = \{x \in \mathbb{N} \mid \varphi_x(x) = x^2\}$ non è decidibile utilizzando la tecnica della riduzione. In particolare, riduciamo l'insieme non decidibile $K = \{x \mid \varphi_x(x) \downarrow\}$ all'insieme I in modo che $x \in K \iff f(x) \in I$ per una qualche funzione totale calcolabile f.
Definiamo una funzione parziale calcolabile $g : \mathbb{N}^2 \to \mathbb{N}$:

$$g(x, y) = \begin{cases} y^2 & \text{se } \varphi_x(x) \downarrow \\ \uparrow & \text{se } \varphi_x(x) \uparrow \end{cases}$$

Inoltre per ogni x si può definire una funzione $h_x : \mathbb{N} \to \mathbb{N}$ calcolabile tale che:

$$h_x(y) = \begin{cases} y^2 & \text{se } \varphi_x(x) \downarrow \\ \uparrow & \text{se } \varphi_x(x) \uparrow \end{cases}$$

in altre parole, $h_x(y)$ calcola la funzione $g(x, y)$ dove il primo argomento x è fissato.
Dato che le (infinite) funzioni h_x sono calcolabili, hanno un proprio indice nell'enumerazione di tutte le funzioni calcolabili, e tale indice si può ottenere tramite una funzione totale calcolabile $s : \mathbb{N} \to \mathbb{N}$ proprio come mostrato dal teorema del parametro. Quindi per ogni x si ha che $h_x = \varphi_{s(x)}$.
A questo punto possiamo osservare che per ogni x:

$$x \in K \iff \varphi_x(x)\downarrow \implies \forall y, \varphi_{s(x)}(y) = y^2 \implies \varphi_{s(x)}(s(x)) = s(x)^2 \iff s(x) \in I$$
$$x \notin K \iff \varphi_x(x)\uparrow \implies \forall y, \varphi_{s(x)}(y)\uparrow \implies s(x) \notin Im(s(x)) \iff s(x) \notin I$$

In questo modo abbiamo mostrato che $K \leq_m I$ tramite la funzione $f = s$, e dall'indecidibilità di K segue l'indecidibilità di I. Infatti, se ci fosse un algoritmo che decidesse I, sarebbe possibile definirne uno che decide anche K: per ogni x, si calcola $s(x)$ e si verifica se appartiene ad I; in caso affermativo $x \in K$, altrimenti $x \notin K$.

3.5 Altri problemi non decidibili

Il problema dell'arresto non è l'unico esempio rilevante di problema indecidibile in Informatica Teorica. Altre questioni di particolare interesse teorico condividono la sua assenza di soluzioni algoritmiche. Citiamo qui il caso del *Problema dell'equivalenza tra MdT* che adesso enunciamo.

Problema 3.2 (Problema dell'Equivalenza tra Macchine di Turing)**.** Determinare un algoritmo per decidere, per ogni scelta di due naturali x, y, se M_x e M_y calcolano o no la stessa funzione, cioè se $\varphi_x = \varphi_y$ oppure no.

In altre parole, sulla base della tesi di Church-Turing, si domanda se $E = \{(x, y) \in \mathbb{N}^2 \mid \varphi_x = \varphi_y\}$ è o no decidibile. Si intende con $\varphi_x = \varphi_y$ che φ_x e φ_y hanno lo stesso dominio e agiscono allo stesso modo sui suoi elementi. Ma si prova che il problema non è decidibile secondo Turing e dunque, per la tesi di Church-Turing, non è risolubile da nessun modello di calcolo.

Un caso particolarmente critico in questo ambito più "generale" è quello dei linguaggi di programmazione cui dedicheremo il Capitolo 6. Qui la soluzione negativa del problema dell'equivalenza implica che non sarà mai possibile decidere se due programmi qualsiasi calcolano la stessa funzione o, in altri termini, se hanno la stessa semantica. Ciò preclude la speranza di trovare un algoritmo per l'analisi e la verifica di correttezza automatica dei programmi: le tecniche di *testing* e il *model checking* rimangono così le uniche strategie per scoprire errori in un dato programma.

La nostra dimostrazione dell'indecidibilità del problema dell'equivalenza per MdT procede in tre passi.

1. Dapprima mostriamo che non è possibile decidere se una funzione calcolabile è totale (cioè converge su tutti gli input) o meno.

2. Proviamo poi che non è possibile riconoscere quali funzioni calcolabili coincidono con l'identità *id* di $\mathbb{N}$, cioè con la funzione di $\mathbb{N}$ in $\mathbb{N}$ che lascia fisso ogni naturale.

3. Dimostreremo infine l'indecidibilità dell'equivalenza tra MdT.

Si noti che anche i passi 1 e 2 costituiscono per proprio conto risultati di indecidibilità. Procediamo con la dimostrazione del primo punto usando un argomento di diagonalizzazione.

Teorema 3.7. L'insieme

$$T = \{x \in \mathbb{N} \mid \varphi_x \text{ è totale}\}$$

non è decidibile.

Dimostrazione. Ammettiamo per assurdo che T sia decidibile. Allora siamo capaci di riconoscere effettivamente nell'enumerazione $\varphi_0, \varphi_1, \varphi_2, \ldots$ di tutte le funzioni calcolabili quelle che sono anche totali. C'è dunque una funzione totale e calcolabile f tale che, per ogni $x \in \mathbb{N}$,

$$f(x) = \text{indice della } (x+1)\text{-esima funzione totale nell'enumerazione}$$

Così ogni funzione calcolabile totale è della forma $\varphi_{f(x)}$ per un unico numero naturale x. Definiamo adesso una nuova funzione g, tale che per ogni $x \in \mathbb{N}$

$$g(x) = \varphi_{f(x)}(x) + 1$$

Allora, per ogni x, g aggiunge 1 all'output di $\varphi_{f(x)}$ calcolata su x, che convergerà sicuramente perché f è totale e restituisce sempre indici di funzioni totali. Segue quindi che anche g è calcolabile totale.
Ma se g è calcolabile e totale, allora esiste sicuramente un naturale x_0 tale che $\varphi_{f(x_0)} = g$. Quindi abbiamo che

$$\varphi_{f(x_0)}(x_0) = g(x_0) = \varphi_{f(x_0)}(x_0) + 1$$

e questo è assurdo. Segue che T è indecidibile. □

Passiamo alla prova del secondo punto. Usiamo qui una tecnica di riduzione dall'insieme T che abbiamo appena dimostrato indecidibile.

Teorema 3.8. L'insieme $I = \{i \in \mathbb{N} \mid \varphi_i = id\}$ non è decidibile.

Dimostrazione. Riduciamo l'insieme T ad I, così dall'indecidibilità di T possiamo dedurne quella di I. Consideriamo la funzione parziale f da $\mathbb{N}^2$ a $\mathbb{N}$ tale che, per ogni scelta di x, y naturali,

$$f(x,y) = \begin{cases} y & \text{se } \varphi_x(y) \downarrow \\ \uparrow & \text{altrimenti} \end{cases}$$

Non è difficile vedere che f è calcolabile. Supponiamo di fissare il primo argomento x della funzione f. Otteniamo allora per ogni x una funzione calcolabile g_x da $\mathbb{N}$ a $\mathbb{N}$ così definita:

$$g_x(y) = \begin{cases} y & \text{se } \varphi_x(y) \downarrow \\ \uparrow & \text{altrimenti} \end{cases}$$

Tutte le funzioni g_x sono calcolabili e quindi compaiono nell'enumerazione delle funzioni calcolabili. Così per il teorema del parametro c'è una funzione calcolabile s da $\mathbb{N}$ a $\mathbb{N}$ tale che, per ogni $x \in \mathbb{N}$,

$$\varphi_{s(x)} = g_x$$

Possiamo osservare ora che φ_x è totale $\iff \varphi_{s(x)} = id$, questo perché

$$x \in T \iff \forall y, \varphi_x(y) \downarrow \implies \forall y, \varphi_{s(x)}(y) = y \iff \varphi_{s(x)} = id \implies s(x) \in I$$
$$x \notin T \iff \exists y, \varphi_x(y) \uparrow \implies \exists y, \varphi_{s(x)}(y) \uparrow \implies \varphi_{s(x)} \neq id \implies s(x) \notin I$$

Quindi la funzione s è la funzione totale e calcolabile che riduce l'insieme T a I. Da questa riduzione possiamo vedere che da un algoritmo che decide I si può costruirne uno che decide T: basta che, per ogni $x \in \mathbb{N}$, prima si calcoli $s(x)$ e poi si applichi a $s(x)$ l'algoritmo di I; se esso ci dice $s(x) \in I$, deduciamo $x \in T$, e viceversa. Questo è un assurdo, quindi I è necessariamente non decidibile. □

Possiamo infine dimostrare il terzo punto.

Teorema 3.9. L'insieme $E = \{(x, y) \in \mathbb{N}^2 \mid \varphi_x = \varphi_y\}$ non è decidibile.

Dimostrazione. Di nuovo usiamo una tecnica di riduzione, stavolta riducendo I ad E. Osserviamo che la funzione id è certamente calcolabile e dunque $id = \varphi_j$ per qualche j. Ma allora, per ogni i

$$i \in I \iff (i, j) \in E$$

Quindi E non è decidibile perché non lo è neppure I. □

3.6 Il teorema di Kleene

Il teorema di Kleene, anche noto come teorema del punto fisso di Kleene, contiene uno dei risultati più affascinanti della teoria della calcolabilità. Utilizza e sottolinea, infatti, una proprietà per certi verso paradossale che la codifica delle macchine di Turing conferisce alle MdT stesse, e cioè la capacità di riprodursi e determinare nuovi algoritmi. In effetti ogni funzione calcolabile totale t da $\mathbb{N}$ a $\mathbb{N}$ permette, per ogni $x \in \mathbb{N}$, alla MdT M_x e alla corrispondente funzione φ_x di "generare" rispettivamente la MdT $M_{t(x)}$ e la funzione $\varphi_{t(x)}$. Anzi, se prestiamo fede alla tesi di Church-Turing, ogni procedimento effettivo che a partire da vecchi algoritmi ne produce di nuovi corrisponde a una tale funzione t.

Il contesto suggerisce in particolare la possibilità di definire una funzione calcolabile non solo esplicitamente, ma anche implicitamente, tramite condizioni quali

$$\varphi_x = \varphi_{x^2}$$

o, più in generale,

$$\varphi_x = \varphi_{t(x)}$$

per t totale calcolabile, identificando quindi φ_x come una delle funzioni che operano allo stesso modo di quelle determinate tramite $x \mapsto x^2$, o una generica trasformazione t.

Il meccanismo di codifica conduce poi al notevole fenomeno dell' autoriferimento di un algoritmo: infatti ogni funzione φ_x può applicarsi a se stessa, più precisamente al proprio indice x, e usare x al proprio interno. Possiamo allora immaginare di identificare le funzioni calcolabili φ_x tramite condizioni di autoriferimento, per esempio chiedendo

$$\varphi_x(y) = x, \text{ per ogni } y \in \mathbb{N}$$

e cioè che la MdT M_x produca come output di ogni input il proprio indice x. In realtà, a proposito di quest'ultimo esempio, l'intuizione provoca qualche fondato dubbio sulla sua attuabilità: infatti, una MdT M che include x nelle sue istruzioni riceve una codifica maggiore di x. Ma possiamo comunque ammettere che una MdT M possa calcolare un certo indice x e poi coinvolgere φ_x nelle sue computazioni, come suo sottoprogramma. Del resto, la dimostrazione del teorema di Kleene si basa essenzialmente su questa idea.

Il teorema di Kleene si riferisce ai meccanismi di produzione di programmi appena descritti e afferma che, per ogni procedura t con cui ogni MdT M_x genera una MdT $M_{t(x)}$ c'è sempre un programma che resta invariato, e cioè un $e \in \mathbb{N}$ tale che

$$\varphi_e = \varphi_{t(e)}$$

dunque $M_e, M_{t(e)}$ calcolano la stessa funzione. Per questo motivo il risultato di Kleene è anche chiamato Teorema del punto fisso: infatti t lascia invariato, se non il numero e, almeno il programma φ_e che gli corrisponde.

Teorema 3.10. Per ogni funzione calcolabile totale t da $\mathbb{N}$ in $\mathbb{N}$ esiste un $e \in \mathbb{N}$ tale che

$$\varphi_e = \varphi_{t(e)}$$

Dimostrazione. Per ogni naturale u, si consideri la MdT $M(u)$ che, dato un input x, calcola dapprima $\varphi_u(u)$ e poi, se $\varphi_u(u)\downarrow$, applica la funzione $\varphi_{\varphi_u(u)}$ a x con relativo output $\varphi_{\varphi_u(u)}(x)$; se invece $\varphi_u(u)\uparrow$ anche $M(u)$ diverge su x. L'algoritmo di $M(u)$ consiste quindi in:

```
Begin
Input(x);
Decodifica u per ottenere la MdT M_u che calcola φ_u;
Sia y = φ_u(u);
Decodifica y per ottenere la MdT M_y che calcola φ_y = φ_{φ_u(u)};
Output(φ_y(x))
End
```

È facile convincersi che per ogni u una MdT $M(u)$ come descritta dall'algoritmo si può effettivamente costruire. Anzi la funzione g che associa a ogni u il codice di $M(u)$, cioè $M(u) = M_{g(u)}$ è totale e calcolabile.
Quindi riassumendo, per ogni u la MdT $M(u)$ calcola una funzione $\varphi_{g(u)}$ definita per ogni $x \in \mathbb{N}$ come segue:

$$\varphi_{g(u)}(x) = \begin{cases} \varphi_{\varphi_u(u)}(x) & \text{se } \varphi_u(u) \downarrow \\ \uparrow & \text{altrimenti} \end{cases}$$

(ovviamente $\varphi_{g(u)}(x)$ può essere divergente anche nel primo caso, se x non è nel dominio di $\varphi_{\varphi_u(u)}$). Sia ora t una funzione calcolabile totale. Allora anche la computazione $t \circ g$ è calcolabile totale, e dunque $t \circ g = \varphi_v$ per qualche naturale v. Inoltre, per ogni $x \in \mathbb{N}$,

$$\varphi_{g(v)}(x) = \varphi_{\varphi_v(v)}(x) = \varphi_{t(g(v))}(x)$$

Quindi $e = g(v)$ soddisfa la tesi del teorema. □

3.7 Il teorema di Rice

Il teorema di Kleene che abbiamo visto nella Sezione 3.6 permette di dimostrare uno dei più forti risultati negativi della teoria della computabilità: il *teorema di Rice*. Il teorema di Rice asserisce che qualunque proprietà non banale delle funzioni calcolabili (e quindi degli algoritmi) è indecidibile.

Definizione 3.2. Sia F una famiglia di funzioni calcolabili. Un insieme $S \subseteq \mathbb{N}$ si dice *insieme degli indici* di F se

$$S = \{x \in \mathbb{N} \mid \varphi_x \in F\}$$

In generale, un qualsiasi sottoinsieme S dei naturali è un insieme di indici se per ogni $i, j \in \mathbb{N}$ con $i \in S$ e $\varphi_i = \varphi_j$ si ha che anche $j \in S$.

Esempio 3.4. Un esempio di insieme di indici è l'insieme $A = \{x \in \mathbb{N} \mid \varphi_x = 0\}$, che indicizza la famiglia F delle funzioni calcolabili che sono costanti nel valore 0. Verifichiamo che rispetta la definizione, cioè che per ogni $i, j \in \mathbb{N}$ con $i \in A$ e $\varphi_i = \varphi_j$ si ha che anche $j \in A$. Dato che $i \in A$ si ha che φ_i è una funzione costante tale che $\varphi_i(y) = 0$ per ogni $y \in \mathbb{N}$. Allora $Im(\varphi_i) = \{0\}$ e quindi anche $Im(\varphi_j) = \{0\}$ (dato che sono la stessa funzione) e dunque anche $j \in I$.
Un esempio di insieme che invece non è un insieme di indici, perché non rispetta le funzioni, è l'insieme $B = \{x \in \mathbb{N} \mid \varphi_x = x\}$. L'insieme B *dovrebbe* essere l'insieme che indicizza la famiglia F' delle funzioni calcolabili che sono costanti nel valore del proprio indice nell'enumerazione delle funzioni. Anche se a prima vista può sembrare un insieme di indici, perché dopotutto è un insieme dei naturali ognuno dei quali è l'indice di una funzione φ, in realtà non rispetta la definizione data sopra.
Per mostrare ciò, consideriamo la funzione

$$\varphi_i(x) = \begin{cases} x & \text{se } x = i \\ \uparrow & \text{altrimenti} \end{cases}$$

Chiaramente si ha che $i \in B$, perché nel punto i la funzione restituisce esattamente il proprio indice i. Ora prendiamo j tale che $\varphi_j = \varphi_i$ e

$j \neq i$. Si ha che $\varphi_i(i) = \varphi_j(i) = i$, proprio in virtù del fatto che la funzione i-esima e j-esima sono proprio la stessa funzione. Ma $\varphi_i(j) \uparrow$ e quindi anche $\varphi_j(j) \uparrow$, e dunque $j \notin B$. L'insieme B per questo motivo non è un insieme di indici per la definizione che abbiamo dato sopra.

Teorema 3.11. Sia F una famiglia di funzioni calcolabili. L'insieme S degli indici di F è decidibile se e solo se $F = \varnothing$ oppure F coincide con l'intera classe delle funzioni calcolabili.

Dimostrazione. Se $F = 0$, $S = \varnothing$ è decidibile; se F coincide con la classe di tutte le funzioni calcolabili, $S = \mathbb{N}$ è ancora decidibile. Supponiamo allora che F non sia vuoto né contenga tutte le funzioni calcolabili, e mostriamo che S è indecidibile. La dimostrazione è una applicazione della tecnica di diagonalizzazione.
Ammettiamo per assurdo S decidibile. Le ipotesi su F ci assicurano che ci sono funzioni calcolabili dentro e fuori di F: siano i e j rispettivamente il primo indice per cui fa $\varphi_i \in F$, cioè $i \in S$ e il primo indice per cui $\varphi_j \notin F$, cioè $j \notin S$. Siccome S è decidibile, possiamo calcolare esplicitamente i e j, ricorrendo a una MdT M che decide S. Basta che facciamo operare M su 0, poi su 1, e così via finché necessario. Se M stabilisce che 0 è in S, si pone $i = 0$, altrimenti si deduce $j = 0$; dopo di che si passa a 1. Se si è ottenuto $i = 0$,e M dichiara che $1 \notin S$ si pone $j = 1$; altrimenti per $i = 0$ e $1 \in S$, si passa a 2, certi di incontrare prima o poi nella sequenza dei naturali il nostro j. Analogamente si procede per $j = 0$.
Di conseguenza la funzione g tale che, per ogni $x \in \mathbb{N}$,

$$g(x) = \begin{cases} i & \text{se } x \notin S \\ j & \text{se } x \in S \end{cases}$$

risulta calcolabile, e anche ovviamente totale. Inoltre si ha, per ogni $x \in \mathbb{N}$, che:

$$g(x) \in S \iff x \notin S$$

cioè

$$\varphi_{g(x)} \in F \iff \varphi_x \notin F$$

Ma, siccome g è totale calcolabile, il teorema di Kleene ci fornisce un naturale e tale che $\varphi_{g(e)} = \varphi_e$, e questo conduce alla contraddizione

$$\varphi_e \in F \iff \varphi_e \notin F$$

Dunque S non può essere decidibile. □

Vediamo alcuni esempi di applicazione del teorema di Rice

Esempio 3.5. L'insieme $T = \{x \in \mathbb{N} \mid \varphi_x \text{ è totale}\}$ è l'insieme degli indici corrispondenti alla famiglia F delle funzioni calcolabili totali, che non è vuota ma neppure esaurisce la classe delle funzioni calcolabili. Infatti esistono funzioni calcolabili totali (per esempio, la funzione identità è totale) ed esistono funzioni calcolabili non totali (per fare un esempio molto semplice, la funzione indefinita non è totale). Così il teorema di Rice si applica e garantisce che $T = \{x \in \mathbb{N} \mid \varphi_x \in F\}$ non è decidibile. Lo stesso si può affermare di $E = \{x \in \mathbb{N} \mid \varphi_x = id\}$, per il quale F si riduce alla sola funzione identità. Di nuovo, possiamo mostrare che la famiglia F non è vuota (perché naturalmente contiene la funzione identità) ma non corrisponde all'intera classe delle funzioni calcolabili (perché esistono anche altre funzioni che non sono l'identità, come per esempio la funzione costante zero).

Le conseguenze del teorema di Rice in termini di teoria della programmazione sono particolarmente significative. Infatti, in tale contesto, si deve ammettere l'impossibilità di provare esplicitamente specifiche proprietà, come la costanza, la crescenza, la monotonia, e così via, delle funzioni calcolate da programmi. Infatti la classe delle funzioni che verificano una di queste proprietà corrisponde alle ipotesi del teorema di Rice.

In particolare, un'interessante applicazione riguarda l'impossibilità di decidere la correttezza di un programma. Infatti la questione si traduce nel considerare l'insieme:

$$S = \{x \in \mathbb{N} \mid \varphi_x = f\}$$

dove f è la specifica del programma e le varie φ_x rappresentano le implementazioni di f. Dal teorema di Rice segue che S è indecidibile.

3.8 Il problema di decisione

La famosa dimostrazione di Turing dell'assenza di un algoritmo risolutivo per il problema dell'arresto fu originariamente sviluppata per dimostrare un altro importante risultato negativo, quello dell'indecidibilità dell'*Entscheidungsproblem*, il cosiddetto *problema di decisione* che abbiamo menzionato nell'introduzione. Riportiamo nuovamente l'enunciato di questo problema:

Problema 3.3 (Entscheidungsproblem)**.** Determinare un procedimento capace di stabilire, per ogni proposizione della logica del primo ordine, se essa sia valida o meno, ovvero se essa sia universalmente vera in ogni possibile interpretazione oppure no.

La dimostrazione di Turing del 1936, avvenuta in maniera indipendente dall'analoga dimostrazione di Alonzo Church pochi mesi prima, si basa sulla dimostrazione di due teoremi preliminari. Il primo di questi teoremi è una formulazione diversa, ma equivalente, di quello che poi diventerà noto nel 1952 come il problema dell'arresto. Riportiamo questo teorema nella sua formulazione originaria.

Teorema 3.12. Non esiste una macchina di Turing che, preso l'indice x di una MdT, possa stabilire se M_x sia *non circolare*, cioè se M_x continui a scrivere un numero infinito di simboli 0 o 1 sul nastro senza mai arrestarsi.

Dimostrazione. Supponiamo che esista una MdT che possa decidere per ogni x se M_x sia circolare o meno, e dunque che la seguente funzione sia calcolabile.

$$f(x) = \begin{cases} 1 & \text{se } M_x \text{ è circolare} \\ 0 & \text{altrimenti} \end{cases}$$

Allora dalla calcolabilità di f segue che sia possibile costruire una MdT che svolga i seguenti passi:

```
Begin
Siano i = 0; j = 0;
While True do
    Begin
    If M_i non è circolare then
        Begin
        Sia y il j-esimo 0/1 che M_i scrive sul nastro;
        Scrivi y sul nastro;
        j = j + 1
        End
    i = i + 1
    End
End
```

La macchina di Turing che questo algoritmo descrive è a sua volta non circolare, infatti la computazione del punto 7 avviene sempre in un numero finito di passi e per ipotesi siamo in grado di determinare la circolarità di una MdT tramite f.
Quindi questa MdT M_k avrà un certo indice k e $f(k) = 1$. Quando la MdT deve valutare il proprio indice, tenterà di simulare se stessa per ottenere il j-esimo simbolo 0 o 1 che viene scritto sul nastro. Ma M_k scrive 0 o 1 solo quando incontra una M_i non circolare e ogni volta incrementa il valore di j, quindi il suo j-esimo valore 0/1 è proprio il valore che scriverebbe quando incontra il proprio indice k. M_k quindi simula se stessa, ripetendo le simulazioni delle MdT precedenti e scrivendo i $j - 1$ simboli 0/1, fino a raggiungere nuovamente k, e a quel punto simulare nuovamente se stessa, senza raggiungere mai il suo j-esimo simbolo 0/1. Ma questo è un assurdo, perché per ipotesi si era affermato che la MdT non è circolare (e quindi scrive un numero infinito di zeri e di uni sul nastro). □

La dimostrazione di questo teorema, proprio come la dimostrazione di irrisolvibilità del problema dell'arresto, si basa sulla tecnica di diagonalizzazione, e il lettore può riconoscere un certo senso di familiarità tra le due dimostrazioni.

Il secondo teorema che Turing dimostra lo si può considerare come un pre-

cursore del teorema di Rice, ed enuncia l'impossibilità di decidere l'insieme di indici di una particolare classe di macchine di Turing.

Teorema 3.13. Non esiste un algoritmo che, preso l'indice x di una MdT, possa stabilire se M_x scriverà mai il simbolo 0 sul proprio nastro.

Dimostrazione. Supponiamo per assurdo che esista tale algoritmo. Allora è possibile stabilire un algoritmo che determini se una certa MdT M scriva un infinito numero di simboli 0 sul proprio nastro. Per fare ciò, indichiamo con M_1 la MdT che esegue le stesse computazioni di M ma sostituisce il primo 0 che M scrive sul nastro con $\bar{0}$. Alla stessa maniera M_2 sostituisce i primi due simboli 0 scritti da M con $\bar{0}$, e così via con M_3, M_4, etc. Sia inoltre Z la MdT che scrive sul nastro la sequenza infinita degli indici di $M, M_1, M_2, \ldots$. Costruiamo una nuova MdT G che svolge i seguenti passi:

1. Usa Z per scrivere M
2. Verifica se M scrive almeno uno zero tramite l'algoritmo in ipotesi. In caso negativo G scrive uno 0 sul proprio nastro.
3. Usa Z per scrivere M_1
4. Verifica se M_1 scrive almeno uno zero tramite l'algoritmo in ipotesi. In caso negativo G scrive un 0 sul proprio nastro.

e così via per $M_2, M_3, \ldots$. Ora se diamo in input G alla nostra procedura, siamo in grado di determinare se M scrive un numero infinito di simboli 0: se G ha scritto almeno uno 0 durante le sue computazioni, M non scrive un numero infinito di simboli 0, e viceversa, se G non ha scritto nessuno 0 vuol dire che M scrive infiniti simboli 0 sul nastro.
La dimostrazione si conclude notando che usando la stessa tecnica si può stabilire un algoritmo che determini se una certa MdT scrive un numero infinito di simboli 1. Utilizzando questi due algoritmi si potrebbe determinare se una MdT è circolare o meno, e questo è assurdo per il Teorema 3.8 □

Dimostrati questi due teoremi, ora possiamo dimostrare l'indecidibilità del problema di decisione.

Teorema 3.14. Non esiste un algoritmo che possa stabilire, per ogni proposizione della logica del primo ordine, se essa sia valida o meno.

Dimostrazione. Sia $Un(M)$ una formula della logica del primo ordine che sia valida se e solo se la MdT M scrive almeno una volta 0 sul proprio nastro. Turing fa uso di due lemmi, che riportiamo senza dimostrazione, che mostrano come costruire $Un(M)$ e asseriscono:

- Se M scrive almeno una volta 0 sul proprio nastro, allora $Un(M)$ è valida.
- Se $Un(M)$ è valida, allora M scrive almeno una volta 0 sul proprio nastro.

Assumiamo che esista un algoritmo di risoluzione per il problema di decisione e sia M una macchina di Turing. Se diamo in input all'algoritmo la formula $Un(M)$ si vengono a creare due possibili situazioni:

1. Se $Un(M)$ è valida, allora sappiamo che M scrive uno 0 sul proprio nastro.
2. Se $Un(M)$ non è valida, allora sappiamo che M non scrive mai uno 0 sul proprio nastro.

Ma allora abbiamo una procedura per stabilire se una qualsiasi MdT scrive uno zero sul proprio nastro, in diretta contraddizione a quanto dimostrato dal Teorema 3.13 □

Così Alan Turing diede la propria risposta negativa al problema di decisione, e 34 anni dopo, nel 1970, fu possibile dimostrare anche l'indecidibilità del Decimo Problema di Hilbert tramite una prova per riduzione dall'insieme K. L'insieme che deve il suo nome proprio al fatto che, nel suo articolo del 1936, Turing indicava con K il *calcolo funzionale*, quello che noi oggi chiameremmo *logica* (e.g. del prim'ordine), e a cui Turing si riferiva in quest'ultima dimostrazione.

3.9 Esercizi

Esercizio 3.1
Dimostrare tramite una prova per diagonalizzazione che l'insieme $I = \{n \mid n \in Im(\varphi_n)\}$ non sia decidibile.

Esercizio 3.2
Dire quali tra le seguenti funzioni sono calcolabili

$$f_1(x) = \begin{cases} \varphi_x(x) + 1 & \text{se } \varphi_x(x) \downarrow \\ \uparrow & \text{altrimenti} \end{cases}$$

$$f_2(x) = \begin{cases} \varphi_x(x) + 1 & \text{se } \varphi_x \text{ è totale} \\ 0 & \text{altrimenti} \end{cases}$$

$$f_3(x) = \begin{cases} 1 & \text{se domani piove} \\ 0 & \text{altrimenti} \end{cases}$$

$$f_4(x) = \begin{cases} \varphi_x(x) + 1 & \text{se } \varphi_x(x) \downarrow \text{ in } x \text{ passi} \\ 0 & \text{altrimenti} \end{cases}$$

Esercizio 3.3
Si dica quale tra i seguenti insiemi è un insieme di indici (e quindi che rispetta le funzioni).

$$\begin{aligned} A &= \{x \mid \varphi_x(x) = 0 \text{ e converge entro } x \text{ passi}\} \\ B &= \{x \mid \varphi_x(0) \downarrow\} \\ C &= \{x \mid \exists y \text{ tale che } \varphi_x(y) = x\} \\ D &= \{x \mid \exists y \text{ tale che } \varphi_x(y) = y\} \end{aligned}$$

Esercizio 3.4

Stabilire, per ognuno degli insiemi dell'Esercizio 3.3, se essi siano decidibili o meno.

Esercizio 3.5

Stabilire, per ognuno dei seguenti insiemi, se essi siano semidecidibili o meno.

$$\begin{aligned} A &= \{x \mid Im(\varphi_x) = \varnothing\} \\ B &= \{x \mid 3 \in Im(\varphi_x)\} \\ C &= \{x \mid \varphi_i \text{ è una funzione costante}\} \\ D &= \{x \mid \varphi_x(0) \downarrow\} \end{aligned}$$

Soluzioni

Soluzione 3.1

Supponiamo per assurdo che I sia decidibile. Allora possiamo definire una funzione $f : \mathbb{N} \to \mathbb{N}$ tale che, per ogni $n \in \mathbb{N}$:

$$f(n) = \begin{cases} \uparrow & \text{se } n \in Im(\varphi_n) \\ n & \text{se } n \notin Im(\varphi_n) \end{cases}$$

Naturalmente, se assumiamo I decidibile, questa funzione è calcolabile, e dunque esiste un indice m tale che $f = \varphi_m$ e quindi $Im(f) = Im(\varphi_m)$. Ma allora si ha la contraddizione

$$m \in Im(\varphi_m) \iff m \in Im(f) \iff m \notin Im(\varphi_m)$$

Segue quindi che I non è decidibile.

Soluzione 3.2

- La funzione f_1 è calcolabile. Per dimostrare ciò, invochiamo la tesi di Church-Turing stabilendo il seguente algoritmo per calcolare f_1:

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Esegui M_x su input x;
If M_x termina con output y then
    Output(y + 1)
End
```

  Si noti che, se x non fa parte del dominio di φ_x, la MdT che esegue il precedente algoritmo diverge come stabilito da $f_1(x)$ nel caso che $\varphi_x(x) \uparrow$.

- La funzione f_2 non è calcolabile. Assumiamo per assurdo che lo sia ed esista quindi un indice m tale che $f_2 = \varphi_m$. Allora, considerando che f_2 è totale, si ha la contraddizione:

$$\varphi_m(m) = f(m) = \varphi_m(m) + 1$$

- La funzione f_3 è calcolabile. Per convincersi di ciò, basta considerare che, a seconda del meteo di domani, si ha che $f_3 = c_1$ o $f_3 = c_0$, ossia f_3 corrisponde alla funzione costante 1 o alla funzione costante 0. Sia c_1 che c_0 sono funzioni calcolabili. Quindi la MdT che computa f_3 è un elemento di un insieme composto da due elementi: una MdT M_1 che restituisce 1 per ogni input e una MdT M_0 che restituisce 0 per ogni input, tuttavia non è noto quale delle due MdT sia quella che computi f_3.

- La funzione f_4 è calcolabile, perché è sufficiente simulare φ_x con input x per x passi e quindi generare il risultato richiesto da f_4 a seconda se la MdT x-esima ha terminato o meno. Quindi dimostriamo la calcolabilità di f_4 per la tesi di Church-Turing mostrando il suo algoritmo.

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Esegui M_x su input x per x passi;
If M_x è in una configurazione finale con output y then
    Output(y + 1)
Else
    Output(0)
End
```

Soluzione 3.3

Verifichiamo quali insiemi rispettano le funzioni e quali no:

- Dimostriamo che A non rispetta le funzioni mostrando controesempio. Consideriamo la funzione

$$\varphi_i(x) = \begin{cases} 0 & \text{se } x = i \\ 1 & \text{altrimenti} \end{cases}$$

  Chiaramente si ha che $i \in I$, perché nel punto x la funzione restituisce 0. Ora prendiamo j tale che $\varphi_j = \varphi_i$. Si ha che $\varphi_i(i) = \varphi_j(i) = 0$ ma $\varphi_i(j) = \varphi_j(j) = 1$, quindi $j \notin I$ anche se φ_i e φ_j sono la stessa funzione.

- Dimostriamo che B rispetta le funzioni. Sia $i \in I$, e quindi sia φ_i una funzione che è definita nel punto 0. Ora prendiamo j tale che $\varphi_j = \varphi_i$. Si ha che $0 \in Dom(\varphi_i) = Dom(\varphi_j)$ e quindi anche $j \in I$.

- Dimostriamo che C non rispetta le funzioni mostrando controesempio. Consideriamo la funzione

$$\varphi_i(x) = \begin{cases} i & \text{se } x = 3 \\ \uparrow & \text{altrimenti} \end{cases}$$

  Chiaramente si ha che $i \in I$, perché esiste un y tale per cui la funzione restituisce il proprio indice (che in questo caso è per $y = 3$). Ora prendiamo j tale che $\varphi_j = \varphi_i$. Si ha che $\varphi_i(3) = \varphi_j(3) = i$ ma $\varphi_i(j) \uparrow$ e $\varphi_j(j) \uparrow$, quindi $j \notin I$ anche se φ_i e φ_j sono la stessa funzione.

- Dimostriamo che B rispetta le funzioni. Sia $i \in I$, e quindi sia φ_i una funzione che ha almeno un punto fisso (cioè per un qualche y restituisce proprio quel y); indichiamo con z uno di questi punti fissi. Ora prendiamo j tale che $\varphi_j = \varphi_i$. Si ha che $z = \varphi_i(z) = \varphi_j(z)$ e quindi anche $j \in I$.

Soluzione 3.4

Per dimostrare la non decidibilità di un insieme possiamo applicare il teorema di Rice nel caso sia un insieme di indici che rispetta le funzioni, mentre in caso contrario svolgeremo una prova per riduzione. Per dimostrare che invece un insieme sia decidibile è sufficiente fornire un algoritmo che lo decida.

- L'insieme A è decidibile perché si può decidere tramite il seguente algoritmo:

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Esegui M_x su input x per x passi e sia y il suo risultato;
If y = 0 then
    Output(SÌ)
Else
    Output(NO)
End
```

  Per la tesi di Church-Turing questo algoritmo specifica una funzione calcolabile che decide l'insieme A.

- L'insieme B è l'insieme di indici che rappresenta la famiglia F delle funzioni calcolabili che sono definite nel punto 0. L'insieme F:

  - non corrisponde all'insieme vuoto perché esistono funzioni calcolabili definite nel punto 0, per esempio la funzione (totale) identità id. Quindi $F \neq \varnothing$;
  - non corrisponde all'insieme di tutte le funzioni calcolabili C, perché esistono funzioni unarie calcolabili non definite su 0, per esempio la funzione indefinita $\emptyset$. Quindi $F \neq C$.

  Quindi per il teorema di Rice l'insieme B non è decidibile.

- Riduciamo l'insieme $K = \{x \mid \varphi_x(x) \downarrow\}$ all'insieme C in modo che $x \in K \iff s(x) \in C$ per una qualche funzione totale calcolabile s. A questo scopo, definiamo una funzione parziale calcolabile $f : \mathbb{N}^2 \to \mathbb{N}$:

$$f(x, y) = \begin{cases} y & \text{se } \varphi_x(x) \downarrow \\ \uparrow & \text{se } \varphi_x(x) \uparrow \end{cases}$$

  Inoltre per ogni x si può definire una funzione $g_x : \mathbb{N} \to \mathbb{N}$ calcolabile tale che:

$$g_x(y) = \begin{cases} y & \text{se } \varphi_x(x)\downarrow \\ \uparrow & \text{se } \varphi_x(x)\uparrow \end{cases}$$

in altre parole, $g_x(y)$ calcola la funzione $f(x, y)$ dove il primo argomento x è fissato.

Dato che le (infinite) funzioni $g_x(y)$ sono calcolabili, hanno un proprio indice nell'enumerazione di tutte le funzioni calcolabili, e tale indice si può ottenere tramite una funzione totale calcolabile $s : \mathbb{N} \to \mathbb{N}$ per il teorema del parametro. Quindi per ogni x si ha che $g_x = \varphi_{s(x)}$

A questo punto possiamo osservare che per ogni x:

$$x \in K \iff \varphi_x(x)\downarrow \implies \forall y, \varphi_{s(x)}(y) = y \implies \varphi_{s(x)}(s(x)) = s(x) \iff s(x) \in C$$

$$x \notin K \iff \varphi_x(x)\uparrow \implies \forall y, \varphi_{s(x)}(y)\uparrow \implies s(x) \notin Im(s(x)) \iff s(x) \notin C$$

Quindi un algoritmo che decide C ne produrrebbe uno che decide anche K: per ogni x, si calcola $s(x)$ e si verifica se appartiene ad C; in caso affermativo $x \in K$ e in caso negativo $x \notin K$. Concludiamo che C non è decidibile, altrimenti anche K lo sarebbe a sua volta.

- L'insieme D è l'insieme di indici che rappresenta la famiglia F delle funzioni calcolabili che hanno almeno un punto fisso. L'insieme F:

  - non corrisponde all'insieme vuoto perché esistono funzioni calcolabili che hanno almeno un punto fisso, per esempio la funzione identità id che ne ha infiniti, ma anche la funzione costante zero c_0 che ha un punto fisso in $y = 0$. Quindi $F \neq \varnothing$;
  - non corrisponde all'insieme di tutte le funzioni calcolabili C, perché esistono funzioni calcolabili che non hanno punti fissi, per esempio la funzione successore $f(x) = x + 1$. Quindi $F \neq C$.

  Quindi per il teorema di Rice l'insieme D non è decidibile.

Soluzione 3.5

Per dimostrare che un insieme sia semidecidibile è sufficiente mostrare l'algoritmo che calcoli una funzione che converga su tutti e soli gli elementi che appartengono all'insieme. Invece, per dimostrarne la non semidecidibilità, si può ricorrere al teorema di Post studiandone il complemento oppure tramite una prova per riduzione.

- Dimostriamo la non semidecidibilità dell'insieme A valutando il suo complemento $\bar{A} = \{x \mid Im(\varphi_x) \neq \varnothing\}$. Il complemento di A è semidecidibile, perché possiamo definire il seguente algoritmo:

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Sia c = 0 tale che c = ⟨π_1(c), π_2(c)⟩;
While True do
    Begin
    Siano y = π_1(c); n = π_2(c);
    Esegui M_x su input y per n passi;
    If M_x è in una configurazione finale then
        Begin
        Output(SÌ); stop
        End
    Else
        c = c + 1
    End
End
```

che descrive una funzione calcolabile per la tesi di Church-Turing, e certifica che $\bar{A}$ è semidecidibile. Inoltre possiamo verificare tramite il teorema di Rice che A non è decidibile: A è l'insieme di indici che rappresenta la famiglia F delle funzioni calcolabili che non sono definite in nessun punto (l'immagine di una funzione è vuota se e solo se lo è anche il dominio). L'insieme F:

- non corrisponde all'insieme vuoto perché esistono funzioni calcolabili che non sono definite in nessun punto, questo è esattamente il caso della funzione indefinita $\emptyset$. Quindi $F \neq \varnothing$;

- non corrisponde all'insieme di tutte le funzioni calcolabili C, perché esistono funzioni calcolabili che sono definite in almeno un punto, per esempio la funzione identità id, che è pure totale. Quindi $F \neq C$.

Ora siamo in grado di applicare il teorema di Post: dato che A non è decidibile e $\bar{A}$ è semidecidibile, deve necessariamente essere il caso che A non sia semidecidibile, perché altrimenti si avrebbe per assurdo che A è decidibile.

- L'insieme B è semidecidibile, perché possiamo definire il seguente algoritmo:

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Sia c = 0 tale che c = ⟨π_1(c), π_2(c)⟩;
While True do
    Begin
    Siano y = π_1(c); n = π_2(c);
    Esegui M_x su input y per n passi;
    If M_x è in una configurazione finale con output 3 then
        Begin
        Output(SÌ); stop
        End
    Else
        c = c + 1
    End
End
```

Per la tesi di Church-Turing questo algoritmo descrive una funzione calcolabile, e dato che questa funzione converge solo sugli elementi di B (perché se $3 \notin Im(\varphi_x)$ l'algoritmo continuerà all'infinito), concludiamo che B è semidecidibile.

- Per dimostrare la non semidecidibilità di C purtroppo non possiamo studiare il suo complemento: tra le funzioni che non sono costanti c'è anche la funzione sempre indefinita, e non esiste un algoritmo che possa accettare il suo indice. Quindi quello che possiamo fare è ridurre un insieme non semidecidibile, per esempio l'insieme T delle funzioni totali, a C in modo che $x \in T \iff s(x) \in C$ per una qualche funzione totale calcolabile s.

  Definiamo una funzione parziale calcolabile $f : \mathbb{N}^2 \to \mathbb{N}$:

$$f(x,y) = \begin{cases} 0 & \text{se } \varphi_x(y) \downarrow \\ \uparrow & \text{se } \varphi_x(y) \uparrow \end{cases}$$

  Inoltre per ogni x si può definire una funzione $g_x : \mathbb{N} \to \mathbb{N}$ calcolabile tale che:

$$g_x(y) = \begin{cases} 0 & \text{se } \varphi_x(y) \downarrow \\ \uparrow & \text{se } \varphi_x(y) \uparrow \end{cases}$$

  in altre parole, $g_x(y)$ calcola la funzione $f(x,y)$ dove il primo argomento x è fissato.

  Dato che le (infinite) funzioni $g_x(y)$ sono calcolabili, hanno un proprio indice nell'enumerazione di tutte le funzioni calcolabili, e tale indice si può ottenere tramite una funzione totale calcolabile $s : \mathbb{N} \to \mathbb{N}$ per il teorema del parametro. Quindi per ogni x si ha che $g_x = \varphi_{t(x)}$

  A questo punto possiamo osservare che per ogni x:

$$x \in T \iff \forall y, \varphi_x(y) \downarrow \implies \forall y, \varphi_{s(x)}(y) = 0 \implies s(x) \in C$$

$$x \notin T \iff \exists y, \varphi_x(y) \uparrow \implies \exists y, \varphi_{s(x)}(y) \uparrow \implies s(x) \notin C$$

Quindi un algoritmo che accetti C ne produrrebbe uno che accetti anche T: per ogni x, si calcola $s(x)$ e si esegue l'algoritmo che accetta C; se $s(x)$ viene accettato, accettiamo anche x. Quindi C non è semidecidibile, altrimenti anche T lo sarebbe a sua volta e ciò sarebbe un assurdo.

- L'insieme D è semidecidibile, perché possiamo definire il seguente algoritmo:

```
Begin
Input(x);
Decodifica x per ottenere la MdT M_x;
Esegui M_x su input x;
If M_x termina then
    Output(SÌ)
End
```

Per la tesi di Church-Turing questo algoritmo descrive una funzione calcolabile, e dato che questa funzione converge solo sugli elementi di D (perché se M_x diverge al punto 3, anche la MdT che computa questo algoritmo divergerà), concludiamo che D è semidecidibile.

Capitolo 4

Grammatiche e Automi

Lo studio formale dei linguaggi è una parte fondamentale dell'informatica teorica e trova applicazione in varie direzioni. Tra di esse va certamente compresa l'analisi delle proprietà sintattiche dei programmi, e dunque lo studio dei metodi di definizione della sintassi di un linguaggio di programmazione. Un'altra direzione riguarda i metodi di traduzione da un linguaggio a un altro (tipicamente da un linguaggio di programmazione ad alto livello a uno di più basso livello come il linguaggio macchina). Non da meno sono le tecniche di verifica formale di proprietà comportamentali dei programmi. La familiarità con i concetti di generazione e riconoscimento dei linguaggi è anche importante nell'elaborazione di dati: si pensi, infatti, alla sequenza di operazioni eseguite da un programma, alla sequenza di invocazioni di metodi in programmazione orientata agli oggetti, alle interazioni con l'ambiente esterno di un sistema, alla descrizione di protocolli di comunicazione, e così via.

Ma quel che ci preme maggiormente sottolineare in queste pagine è il collegamento tra studio dei linguaggi e computabilità. Del resto abbiamo già accennato al fatto che problemi di varia natura (come il calcolo di funzioni, o questioni di ottimizzazione) si riconducano spesso alla situazione standard di decidere l'appartenenza di una stringa a un determinato linguaggio: ad esempio, la computazione di una funzione f da $\mathbb{N}$ a $\mathbb{N}$ si riduce alla decisione del grafico $\{(n, f(n)) \mid n \in \mathbb{N}\}$ di f e dunque, tramite opportune codifiche, a quello di un linguaggio quale $\{a^n b^{f(n)} \mid n \in \mathbb{N}\}$ sull'alfabeto formato da due numeri primi $a < b$.

I metodi che permettono di definire un linguaggio sono diversi. Tra essi possiamo distinguere:

- un approccio di tipo *generativo*, nel quale le stringhe del linguaggio sono costruite secondo regole prefissate all'interno di "grammatiche", così come in genere capita per le frasi della lingua comune;
- un approccio di tipo *riconoscitivo*, che consiste nell'utilizzare macchine astratte, dette *automi riconoscitori* e atte a esaminare le stringhe di un alfabeto e certificarne l'appartenenza al linguaggio considerato.

In questo capitolo concentreremo l'attenzione sulle grammatiche generative e sugli automi riconoscitori. Definiremo in dettaglio che cosa è una grammatica e come essa generi un linguaggio, e allo stesso modo definiremo il concetto di automa e come un automa può riconoscere un linguaggio.

4.1 Grammatiche generative

Nella lingua comune la grammatica sovrintende alla composizione delle frasi, fissando le regole con cui costruirle. Per esempio, ammette come frase corretta una sequenza *soggetto*, *verbo*, *complemento oggetto*, accetta poi la possibilità che il *soggetto* e il *complemento oggetto* siano nomi propri (come *maria*), ma esclude questa opzione per la categoria *verbo*. Con queste regole si riescono già a comporre frasi quali "*maria canta una romanza*". Tuttavia, in assenza di ulteriori norme, anche frasi come "*maria canta maria*" diventano ammissibili e dunque altre regole andranno stabilite per evitare queste anomalie. Frasi semplici come le precedenti permettono poi di costruirne altre più complesse, come "*se maria canta una romanza, allora io tappo i miei orecchi*", secondo nuove ulteriori convenzioni.

Cerchiamo comunque di capire con maggior precisione come una grammatica regola il modo di formare una frase. Per schematizzare la situazione appena descritta (quella di Maria e della romanza),

(1) conviene usare simboli A, V, B rispettivamente per *soggetto*, *verbo*, *complemento oggetto* e, ancora, un simbolo di inizio frase S,

(2) d'altro lato, ci servono le comuni lettere dell'alfabeto $a, b, c, \ldots$ per costruire un dato oggetto, o un dato verbo, o un dato complemento oggetto, e magari un ultimo simbolo "_" a indicare uno spazio vuoto.

Siano N, T gli alfabeti (disgiunti) dei simboli rispettivamente in (1) e in (2). Consideriamo poi le seguenti regole per produrre le frasi a partire dal simbolo di avvio S:

$$
\begin{aligned}
&S \longrightarrow AVB \\
&A \longrightarrow P_,\ A \longrightarrow un_M_,\ A \longrightarrow una_F_ \\
&B \longrightarrow un_M,\ B \longrightarrow una_F \\
&P \longrightarrow maria \\
&M \longrightarrow orecchio \\
&F \longrightarrow romanza \\
&V \longrightarrow canta
\end{aligned}
$$

Le regole nelle prime tre righe ci consentono di costruire frasi composte da *soggetto* + *verbo* + *complemento oggetto* e di scegliere il soggetto tra nomi propri, oppure nomi di genere maschile o di genere femminile, mentre la prima opzione (i nomi propri) non è disponibile per il complemento oggetto. Le ultime quattro regole delimitano la scelta di nomi e verbi. Il complesso di queste norme determina la grammatica; infatti, possiamo chiamare *frase* ogni stringa di T (cioè dei simboli in (2)) che la grammatica produce a partire da S tramite un uso ripetuto delle regole appena stabilite. Si ottengono allora frasi come

$$maria_canta_una_romanza$$

o anche

$$maria_canta_un_orecchio$$

$$un_orecchio_canta_una_romanza$$

ma si vieta, per esempio,

$$maria_canta_maria$$

Un simile approccio *grammaticale* consente di generare i nostri linguaggi formali e di specificare le stringhe che li compongono. In questo capitolo considereremo in particolare le così dette *grammatiche a struttura di frase*, che richiamano già dal nome il precedente paragone linguistico. Le introdurremo e discuteremo, vedremo in particolare come anch'esse possono essere pensate come modelli di calcolo ma che, sotto questo punto di vista, si dimostrano computazionalmente equivalenti alle MdT. Eccone intanto la definizione formale.

Definizione 4.1. Una *grammatica a struttura di frase* (o *grammatica di tipo 0*) è una quadrupla $G = (N, T, \mathcal{P}, S)$ dove:

- T e N sono alfabeti disgiunti, composti rispettivamente da quelli che sono chiamati *simboli terminali* e *simboli non terminali* di G;
- $\mathcal{P}$ è un insieme *finito* di coppie ordinate (γ, γ') (che in questo ambito denoteremo come $\gamma \longrightarrow \gamma'$ o $\gamma ::= \gamma'$) con γ, γ' parole su $N \cup T$ tali che $\gamma \neq \lambda$ e γ contiene almeno un simbolo non terminale; le coppie di $\mathcal{P}$ sono chiamate *regole di produzione* o, più semplicemente, *produzioni* della grammatica G;
- S è un simbolo di N, e viene chiamato *simbolo iniziale* di G.

In genere useremo le lettere maiuscole $A, B, C, \ldots$ per indicare i simboli non terminali di G (che comprendono anche il simbolo iniziale S) e quelle minuscole $a, b, c, \ldots$ per i simboli terminali; le stringhe composte soltanto da simboli terminali sono denotate da $u, v, w, x, y, z, \ldots$, le altre, eventualmente aperte anche a simboli non terminali, dalle lettere greche α, β, γ, etc. Per semplicità di notazione, quando più produzioni di una grammatica G condividono la stessa parte sinistra, come in $\gamma \longrightarrow \gamma_1, \ldots, \gamma \longrightarrow \gamma_n$, scriveremo per sintetizzarle $\gamma \longrightarrow \gamma_1 \mid \ldots \mid \gamma_n$.

Esempio 4.1. Siano $N = \{S\}$, $T = \{a, b\}$. Per $\mathcal{P} = \{S \longrightarrow aSb, S \longrightarrow \lambda\}$, $G = (N, T, \mathcal{P}, S)$ è una grammatica di tipo 0. Invece, se $\mathcal{P}$ è l'insieme delle coppie $\{S \longrightarrow aSb, S \longrightarrow ab, \lambda \longrightarrow S\}$ allora $G = (N, T, \mathcal{P}, S)$ non è una grammatica di tipo 0 perché la parte sinistra della terza produzione è la stringa vuota. Un esempio di grammatica di tipo 0 è, ancora, quello precedente di maria e delle romanze, come il lettore può facilmente verificare.

Ogni grammatica G genera un linguaggio fissando con le sue regole di produzione il modo di formarne le stringhe, come adesso descriviamo in dettaglio.

Definizione 4.2. Sia $G = (N, T, \mathcal{P}, S)$ una grammatica a struttura di frase. Consideriamo stringhe con simboli terminali e non, dunque parole sull'alfabeto $N \cup T$. Diciamo allora che la stringa δ' è *derivabile*

direttamente in G dalla stringa δ, e scriviamo $\delta \longrightarrow_G \delta'$, se $\delta = \alpha\gamma\beta$ e $\delta' = \alpha\gamma'\beta$ per un'opportuna scelta di stringhe α, β, γ e γ' tali che $\gamma \longrightarrow \gamma'$ è una produzione in $\mathcal{P}$. Dunque $\delta' = \alpha\gamma'\beta$ è ottenuta da $\delta = \alpha\gamma\beta$ sostituendovi γ con γ'.
Diciamo poi che δ' è *derivabile* in G da δ, e scriviamo $\delta \longrightarrow_G^\star \delta'$ se esistono un naturale n e stringhe $\delta_0, \delta_1, \ldots, \delta_n$ tali che $\delta = \delta_0, \delta' = \delta_n$ e, per ogni $i = 1, \ldots, n$, δ_i è derivabile direttamente in G da δ_{i-1}. La sequenza $\delta_0, \delta_1, \ldots, \delta_n$ viene chiamata una *derivazione* di δ' da δ in G, e n la sua *lunghezza*.
Si chiama finalmente *linguaggio generato da* G, e si indica con $L(G)$, l'insieme di quelle stringhe w di simboli terminali di G che sono derivabili dal simbolo iniziale S in G. Quindi $L(G) = \{w \in T^\star \mid S \longrightarrow_G^\star w\}$ è un linguaggio sull'alfabeto T. Questo tipo di linguaggio si dice anche essere *a struttura di frase*, o *di tipo* 0.

Così, se G è la grammatica $(N = \{S\}, T = \{a, b\}, \mathcal{P} = \{S \longrightarrow aSb, S \longrightarrow \lambda\}, S)$, $L(G)$ è formato, oltre che dalla parola vuota λ, dalle stringhe ab, a^2b^2, a^3b^3 e così via (tutte generate dal ripetuto uso della produzione $S \longrightarrow aSb$ e infine usando $S \longrightarrow \lambda$); si noti che $L(G)$ è generato anche dalla grammatica G' che aggiunge la produzione $S \longrightarrow ab$ a quelle di G, e da altre grammatiche ancora. Un ulteriore esempio: se G è la grammatica di *maria* e delle *romanze*,

$$maria_canta_una_romanza$$

fa parte di $L(G)$. Nel seguito, scriveremo talora semplicemente $\longrightarrow$ e $\longrightarrow^\star$ invece di $\longrightarrow_G$ e $\longrightarrow_G^\star$ quando sarà evidente a quale grammatica G ci si riferisce.

Definizione 4.3. Un linguaggio L in un alfabeto T si dice *di tipo* 0, o a *struttura di frase*, se $L = L(G)$ per qualche grammatica G di tipo 0 per la quale T è l'insieme dei simboli terminali.

Esempio 4.2. Ogni linguaggio finito L su un alfabeto T è di tipo 0 perché è generato da una qualche grammatica G di tipo 0 (in genere non unica): basta infatti assumere $G = (N, T, \mathcal{P}, S)$ dove N si riduce al solo simbolo iniziale S e $\mathcal{P}$ si compone di tutte e sole le produzioni $S \longrightarrow_G w$ con $w \in L$. Siccome L è finito, anche l'insieme $\mathcal{P}$ è finito. La stessa costruzione non vale, evidentemente, per linguaggi infiniti. In effetti vedremo a fine capitolo che ci sono linguaggi che non sono di tipo 0.

Prima di proseguire, vale la pena di ribadire che può accadere che due diverse grammatiche G_1 e G_2 generino lo stesso linguaggio $L(G_1) = L(G_2)$.

Esempio 4.3. La grammatica $G_1 = (N = \{S_1\}, T = \{a, b\}, \mathcal{P}_1, S_1)$ con le seguenti produzioni

$$S_1 \longrightarrow aS_1b \mid \lambda$$

e la grammatica $G_2 = (N = \{S_2, A\}, T = \{a, b\}, \mathcal{P}_2, S_2)$ con le produzioni

$$S_2 \longrightarrow aAb \mid ab \mid \lambda$$
$$A \longrightarrow aAb \mid ab$$

generano lo stesso linguaggio $L(G_1) = L(G_2) = \{a^n b^n \mid n \geq 0\}$.

4.1.1 La gerarchia di Chomsky

Le grammatiche di tipo 0 sono anche note come *Grammatiche di Chomsky* perché furono introdotte dal linguista americano Noam Chomsky, con lo scopo di rappresentare i procedimenti sintattici elementari che sono alla base della costruzione delle frasi nella lingua inglese. Pur essendosi presto rivelate uno strumento inadeguato per lo studio del linguaggio naturale, le grammatiche formali hanno un ruolo fondamentale nello studio delle proprietà sintattiche dei programmi e dei linguaggi di programmazione.

La classe di tutte le grammatiche di tipo 0 contiene alcune sottoclassi notevoli, che adesso descriviamo. Le motivazioni che portano a considerarle sono varie e fanno spesso riferimento a esigenze di programmazione. Per esempio quelle che chiameremo grammatiche regolari sono importanti per la specifica dei token di un linguaggio di programmazione e quindi utili per l'analisi lessicale di un programma; invece quelle che definiremo grammatiche libere dal contesto si applicano all'analisi sintattica dei programmi. Ma è tempo di procedere con l'introduzione di queste sottoclassi.

Definizione 4.4. Una *grammatica monotona*, o *non contraente*, è una grammatica a struttura di frase $G = (N, T, \mathcal{P}, S)$ tale che ogni produzione in $\mathcal{P}$ è della forma $\alpha \longrightarrow \beta$ con ha $l(\alpha) \leq l(\beta)$.

Esempio 4.4. La grammatica $G = (N = \{S, B, C\}, T = \{a, b, c\}, \mathcal{P}, S)$ dove $\mathcal{P}$ è l'insieme di produzioni dato da:

$$S \longrightarrow aSBC \mid aBC$$
$$CB \longrightarrow BC$$
$$bB \longrightarrow bb$$
$$bC \longrightarrow bc$$
$$cC \longrightarrow cc$$
$$aB \longrightarrow ab$$

è una grammatica monotona. Il linguaggio generato da G è $L(G) = \{a^n b^n c^n \mid n \geq 1\}$

Definizione 4.5. Una *grammatica di tipo 1*, o *dipendente dal contesto*, è una grammatica a struttura di frase $G = (N, T, \mathcal{P}, S)$ tale che ogni

produzione in $\mathcal{P}$ è della forma $\gamma A\delta \longrightarrow \gamma\beta\delta$ con $\gamma, \delta \in (N \cup T)^\star$, $A \in N$, $\beta \in (N \cup T)^+$ (quindi un simbolo non terminale A può essere sostituito da una stringa β purché si trovi all'interno di un contesto della forma γ_δ).

Esempio 4.5. La grammatica $G = (N = \{S, B, C, X, Y\}, T = \{a, b, c\}, \mathcal{P}, S)$ dove $\mathcal{P}$ è l'insieme di produzioni dato da:

$$\begin{aligned}
&S \longrightarrow aSBC \mid aBC\\
&CB \longrightarrow CX\\
&CX \longrightarrow YX\\
&YX \longrightarrow YC\\
&YC \longrightarrow BC\\
&bB \longrightarrow bb\\
&bC \longrightarrow bc\\
&cC \longrightarrow cc\\
&aB \longrightarrow ab
\end{aligned}$$

è una grammatica dipendente dal contesto. Il linguaggio generato da G è $L(G) = \{a^n b^n c^n \mid n \geq 1\}$

È evidente che, all'interno della gerarchia di Chomsky, tutte le grammatiche dipendenti dal contesto sono monotone, ma che viceversa si possono costruire grammatiche monotone che non sono dipendenti dal contesto. Così la classe delle grammatiche monotone estende propriamente quella della grammatiche dipendenti dal contesto. Tuttavia c'è un risultato fondamentale che collega queste due sottoclassi, o meglio i linguaggi generati dalle rispettive grammatiche:

Teorema 4.1. I linguaggi generati da grammatiche monotone coincidono con quelli dipendenti dal contesto.

Dimostrazione. Per prima cosa assumiamo che la grammatica monotona è in forma normale di Kuroda. In questo caso le sue produzioni possono solo essere della forma $AB \longrightarrow CD, A \longrightarrow BC, A \longrightarrow B$ o $A \longrightarrow a$, con $A, B, C, D \in N$ e $a \in T$. Notiamo che l'unica produzione problematica è la prima: $AB \longrightarrow CD$ non è ammissibile secondo la definizione di grammatica dipendente dal contesto. Possiamo però osservare che questa produzione può essere rimpiazzata dalla seguente sequenza di regole:

$$\begin{aligned}
&AB \longrightarrow AX\\
&AX \longrightarrow YX\\
&YX \longrightarrow YD\\
&YD \longrightarrow CD
\end{aligned}$$

dove $X, Y \in N$ sono nuovi simboli non terminali usati soltanto in queste

nuove regole di produzione. La grammatica che si ottiene si dice essere in forma normale di Révész, perché fu György Révész il primo a fare questa osservazione [3]. Una grammatica in forma normale di Révész rispetta tutte le condizioni di grammatica dipendente dal contesto. □

Le grammatiche monotone e le grammatiche dipendenti dal contesto hanno quindi la stessa capacità espressiva dal punto di vista del linguaggio generato.

Definizione 4.6. Un linguaggio L si dice di *tipo* 1, o *dipendente dal contesto*, se $L = L(G)$ per qualche grammatica G dipendente dal contesto o monotona.

Proseguiamo nella gerarchia con le seguenti classi di grammatiche:

Definizione 4.7. Una *grammatica di tipo* 2, o *libera dal contesto*, è una grammatica a struttura di frase $G = (N, T, \mathcal{P}, S)$ tale che ogni produzione in $\mathcal{P}$ è della forma $A \longrightarrow \beta$ con $A \in N$, $\beta \in (N \cup T)^+$ (cioè un simbolo non terminale A può essere sostituito da una stringa β indipendentemente dal contesto in cui si trova).
Ogni grammatica libera dal contesto può essere convertita in una delle seguenti forme normali senza che venga modificato il linguaggio generato:

- Una grammatica libera dal contesto si dice essere in *Forma Normale di Chomsky* se tutte le sue produzioni sono della forma $A \longrightarrow BC$ oppure $A \longrightarrow a$ con $A, B, C \in N$ e $a \in T$.
- Una grammatica libera dal contesto si dice essere in *Forma Normale di Greibach* se tutte le sue produzioni sono della forma $A \longrightarrow a\alpha$, dove $A \in N, a \in T$ e $\alpha \in N^\star$.

Definizione 4.8. Un linguaggio L si dice di *tipo* 2, o *libero dal contesto*, se $L = L(G)$ per qualche grammatica G di tipo 2.

Esempio 4.6. La grammatica $G = (N = \{S\}, T = \{a, b\}, \mathcal{P}, S)$ dove $\mathcal{P}$ è l'insieme di produzioni dato da:

$$S \longrightarrow aSb \mid ab$$

è una grammatica libera dal contesto. Il linguaggio generato da G è $L(G) = \{a^n b^n \mid n \geq 1\}$

Definizione 4.9. Una *grammatica di tipo* 3, o *regolare*, è una grammatica a struttura di frase $G = (N, T, \mathcal{P}, S)$ tale che ogni produzione in $\mathcal{P}$ è della forma $A \longrightarrow aB$ oppure $A \longrightarrow a$ con $A, B \in N$ e $a \in T$ (cioè, un simbolo non terminale A può essere sostituito da un simbolo termi-

nale seguito da uno non terminale oppure direttamente da un simbolo terminale).

Definizione 4.10. Un linguaggio L si dice di *tipo* 3, o *regolare*, se $L = L(G)$ per qualche grammatica G di tipo 3.

Esempio 4.7. La grammatica $G = (N = \{S, A\}, T = \{a, b\}, \mathcal{P}, S)$ dove $\mathcal{P}$ è l'insieme di produzioni dato da:

$$S \longrightarrow aS \mid aA$$
$$A \longrightarrow bA \mid b$$

è una grammatica regolare. Il linguaggio generato da G è $L(G) = \{a^n b^m \mid n, m \geq 1\}$

È facile verificare che le grammatiche regolari sono anche libere dal contesto e che quelle libere dal contesto sono anche dipendenti dal contesto (basta porre $\gamma = \delta = \lambda$); inoltre quelle dipendenti dal contesto sono anche grammatiche a struttura di frase. È altrettanto semplice convincersi che esistono grammatiche a struttura di frase che non sono dipendenti dal contesto, grammatiche dipendenti dal contesto che non sono libere e, infine, grammatiche libere dal contesto che non sono regolari. Si determina così all'interno della classe di tutte le grammatiche a struttura di frase, rispetto alla relazione di inclusione, una gerarchia di grammatiche (usualmente chiamata *gerarchia di Chomsky*), in cui si succedono progressivamente quelle regolari, poi quelle libere dal contesto e, ancora, quelle dipendenti dal contesto. Una rappresentazione schematica di quanto appena detto è mostrata in Figura 4.1. Il lettore attento può notare che le grammatiche monotone non sono rappresentate: questo è dovuto al fatto che Chomsky non le prese in considerazione nella sua iniziale formulazione della gerarchia. Tuttavia, come abbiamo visto, queste grammatiche formano una classe intermedia tra il tipo 0 e il tipo 1, e sono in ogni caso equivalenti dal punto di vista dei linguaggi generati alle grammatiche dipendenti dal contesto.

4.1.2 Le lambda produzioni

Una produzione $A \longrightarrow \lambda$ viene tipicamente chiamata *λ-produzione* o *ϵ-produzione* (nei testi dove la stringa vuota viene denotata con ϵ). Le grammatiche G introdotte nella Sezione 4.1.1 (quelle dipendenti dal contesto e, di conseguenza, quelle libere dal contesto e quelle regolari) non generano mai con le loro produzioni la stringa vuota λ e dunque la escludono dal linguaggio $L(G)$. Si tratta, ovviamente, di convenzione formale. Comunque, ove si preferisse coinvolgere λ nelle proprie considerazioni, si potrebbe convenire di estendere l'insieme delle produzioni di G ammettendovi anche $S \longrightarrow \lambda$ con l'unica precauzione che il simbolo iniziale S non compaia a destra di nessun'altra produzione. Questo adattamento non modifica sostanzialmente il quadro generale delle precedenti definizioni.

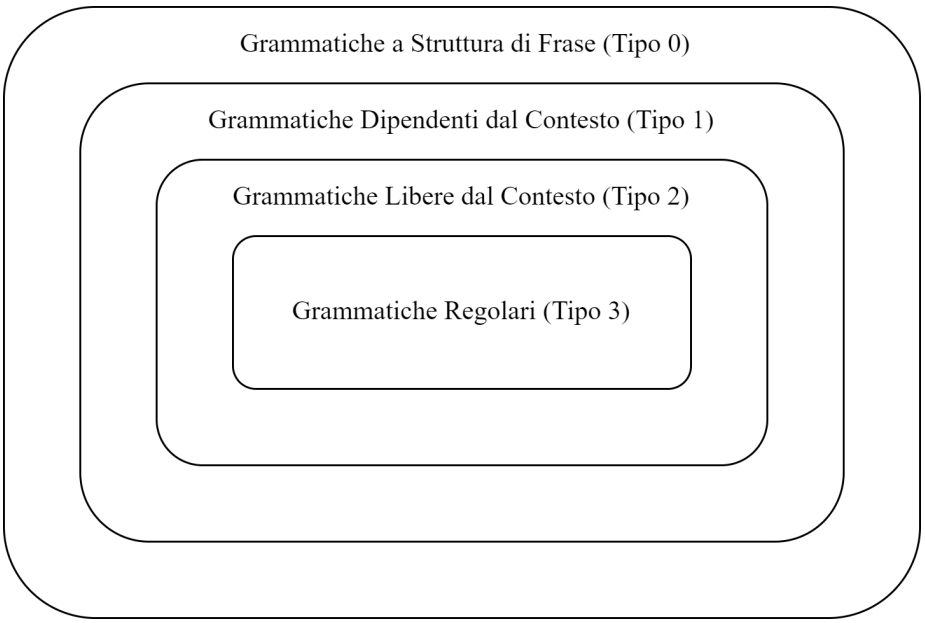


Figura 4.1: Gerarchia di Chomsky

In realtà, finché si tratta di grammatiche libere dal contesto (e quindi anche di grammatiche regolari) possiamo fare liberamente uso di λ-produzioni, senza in nessun modo influenzare la capacità espressiva di queste grammatiche. Lo scopo di questa sezione allora è di dimostrare la seguente proposizione

Proposizione 4.1. Sia G grammatica libera dal contesto che fa uso di λ-produzioni. Allora esiste una grammatica G' libera dal contesto tale che:

- $L(G') = L(G) - \{\lambda\}$;
- G' non contiene alcuna λ-produzione.

Prima di dimostrare questa proposizione, facciamo notare che la grammatica che si ottiene non può generare la stringa vuota. Tuttavia, come evidenziato all'inizio di questa sezione, sarà sufficiente avere premura che il simbolo iniziale non compaia a destra di nessuna produzione e aggiungere alla grammatica la produzione $S \longrightarrow \lambda$. In questo modo la grammatica che otteniamo avrà una e una sola λ-produzione che non può essere usata in alcun altro modo se non per generare la parola vuota.

Dimostrazione. Sia $G = (N, T, \mathcal{P}, S)$ una grammatica libera dal contesto. La grammatica $G' = (N, T, \mathcal{P}', S)$ si costruisce mantenendo gli stessi simboli terminali, non terminali e il simbolo iniziale di G, mentre le produzioni di G' si ottengono da quelle di G tramite la procedura che adesso descriviamo. Bisogna innanzitutto identificare quali simboli non terminali sono *annullabili*, ovvero quali simboli $A \in N$ figurano in produzioni del tipo $A \longrightarrow \lambda$ oppure

$A \longrightarrow X_1 \cdots X_n$ dove ogni simbolo non terminale X_i è annullabile. In altre parole, identifichiamo tutti i non terminali A per cui $A \longrightarrow_G^\star \lambda$. Fatto ciò, per ogni produzione della forma $A \longrightarrow X_1 X_2 \cdots X_k$ con $k \geq 1$ in $\mathcal{P}$, se m dei k simboli non terminali nella parte destra sono annullabili allora aggiungiamo a $\mathcal{P}'$ 2^m versioni di questa produzione, dove gli X_i che sono annullabili sono presenti o assenti in ognuna delle possibili combinazioni. Nel caso $m = k$, cioè tutti i simboli sono annullabili, evitiamo di aggiungere la versione dove tutte le X_i sono omesse. Svolto questo passaggio, aggiungiamo a $\mathcal{P}_1$ tutte le altre produzioni di $\mathcal{P}$ che non siano λ-produzioni.
Ora ci resta da dimostrare che $L(G') = L(G) - \{\lambda\}$. Per fare ciò è sufficiente mostrare che se $w \neq \lambda$, allora $w \in L(G') \iff w \in L(G)$, e cioè che $S \longrightarrow_{G'}^\star w \iff S \longrightarrow_G^\star w \wedge w \neq \lambda$. Nel caso di questa prova, risulta più semplice considerare un caso più generale, e quindi dimostreremo per induzione sulla lunghezza della derivazione che questa proprietà è in realtà verificata per ogni simbolo non terminale di G e G', cioè $A \longrightarrow_{G'}^\star w \iff A \longrightarrow_G^\star w \wedge w \neq \lambda$.

- ($\Longrightarrow$) Supponiamo che $A \longrightarrow_{G'}^\star w$. Allora sicuramente $w \neq \lambda$ dato che G' non ha λ-produzioni. Quindi ci resta da dimostrare che $A \longrightarrow_G^\star w$.
  - Il caso base è che la derivazione di w avviene in un passo. Allora significa che G' contiene una produzione $A \longrightarrow w$. Quindi in G deve esserci una produzione $A \longrightarrow \alpha$ dove α corrisponde a w includendo zero o più simboli non terminali annullabili. Allora $A \longrightarrow_G \alpha \longrightarrow_G^\star w$, dove tutti gli eventuali passi dopo il primo derivano λ da qualsiasi simbolo annullabile che α contenga.
  - Per il passo induttivo consideriamo una derivazione che richieda $n > 1$ passi, e quindi che abbia forma $A \longrightarrow_{G'} X_1 X_2 \cdots X_k \longrightarrow_{G'}^\star w$. La prima produzione che viene usata deve corrispondere in G a una produzione $A \longrightarrow Y_1 Y_2 \cdots Y_m$ che includa nello stesso ordine tutte le X assieme a zero o più simboli non terminali annullabili. Inoltre $w = w_1 w_2 \cdots w_k$, dove $X_i \longrightarrow_{G'}^\star w_i$ per $i = 1, 2, \ldots, k$. Se X_i è un simbolo terminale, allora $X_i = w_i$, altrimenti $X_i \longrightarrow_{G'}^\star w_i$ in meno di n passi. Inoltre per l'ipotesi induttiva $X_i \longrightarrow_G^\star w_i$. Ora possiamo costruire una derivazione in G come segue:

    $$A \longrightarrow_G Y_1 Y_2 \cdots Y_m \longrightarrow_G^\star X_1 X_2 \cdots X_k \longrightarrow_G^\star w_1 w_2 \cdots w_k = w$$

    Il primo passaggio è l'applicazione della produzione $A \longrightarrow Y_1 Y_2 \cdots Y_m$, poi i prossimi passi rappresentano la derivazione di λ da tutti gli Y_j che non corrispondono a uno degli X_i. L'ultimo passaggio mostra la derivazione delle stringhe w_i dagli X_i che sappiamo esistano per l'ipotesi induttiva.

- ($\Longleftarrow$) Supponiamo che $A \longrightarrow_G^\star w$ e $w \neq \lambda$. Dimostriamo che $A \longrightarrow_{G'}^\star w$.
  - Il caso base è che la derivazione di w avviene in un passo. Allora significa che G contiene una produzione $A \longrightarrow w$. Dato che $w \neq \lambda$ questa produzione è anche una produzione di G' e $A \longrightarrow_{G'}^\star w$.
  - Per il passo induttivo consideriamo una derivazione che richieda $n > 1$ passi, e quindi che abbia forma $A \longrightarrow_G Y_1 Y_2 \cdots Y_m \longrightarrow_G^\star w$.

Possiamo suddividere w in modo che $w = w_1 w_2 \cdots w_m$ e $Y_i \longrightarrow_G^\star w_i$ per $i = 1, 2, \ldots, m$. Siano $X_1, X_2, \ldots, X_k$ simboli non terminali di G' che corrispondano in ordine agli Y_j tali che $w_j \neq \lambda$. Inoltre deve essere il caso che $k \geq 1$ perché abbiamo detto che $w \neq \lambda$. Quindi $A \longrightarrow X_1 X_2 \cdots X_k$ deve essere una produzione di G'. Possiamo dichiarare che $X_1 X_2 \cdots X_k \longrightarrow_G^\star w$, dato che ognuno degli X_i corrisponde a un Y_j che non è stato usato per derivare λ, e quindi corrisponde solo a simboli non terminali di G che hanno contribuito a generare w. Dato che ogni derivazione $Y_j \longrightarrow_G^\star w_j$ richiede meno di n passi, possiamo applicare l'ipotesi induttiva e concludere che se $w_j \neq \lambda$, allora $Y_j \longrightarrow_{G'}^\star w_j$. Quindi $A \longrightarrow_{G'} X_1 X_2 \cdots X_k \longrightarrow_{G'}^\star w$.

Possiamo concludere la prova notando che per $A = S$ otteniamo ciò che volevamo dimostrare. □

Chiarifichiamo il procedimento che abbiamo appena descritto mostrando alcuni esempi.

Esempio 4.8. Consideriamo la seguente grammatica libera dal contesto $G = (N = \{S, A, B, C\}, T = \{a, b\}, \mathcal{P}, S)$ con le seguenti produzioni:

$$\begin{aligned}
S &\longrightarrow AbB \mid C \mid cSc \mid \lambda \\
A &\longrightarrow a \mid \lambda \\
B &\longrightarrow AA \mid bA \\
C &\longrightarrow bCb \mid a
\end{aligned}$$

Procediamo in due passaggi:

- Costruiamo una grammatica G' senza λ-produzioni.
- Adattiamo G' per generare la parola vuota senza che il suo simbolo iniziale appaia a destra di nessuna produzione.

Definiamo $G' = (N, T, \mathcal{P}', S)$ con gli stessi simboli terminali e non terminali di G e, per ora, lo stesso simbolo iniziale. I simboli non terminali annullabili sono A, B e S. Nel caso di A e C nessuna delle loro produzioni genera simboli non terminali annullabili, quindi ci basta aggiungere a $\mathcal{P}'$ tutte le loro produzioni che non sono λ-produzioni. Nel caso di B, aggiungiamo a $\mathcal{P}'$ le seguenti produzioni:

$$B \longrightarrow AA \mid A\cancel{A} \mid \cancel{A}A \mid bA \mid b\cancel{A}$$

ovvero

$$B \longrightarrow AA \mid A \mid bA \mid b$$

Nel caso di S, aggiungiamo le seguenti produzioni:

$$S \longrightarrow AbB \mid \cancel{A}bB \mid Ab\cancel{B} \mid \cancel{A}b\cancel{B} \mid cSc \mid c\cancel{S}c$$

ovvero

$$S \longrightarrow AbB \mid bB \mid Ab \mid b \mid cSc \mid cc$$

Quindi le produzioni di G' sono:

$$\begin{aligned}
&S \longrightarrow AbB \mid bB \mid Ab \mid b \mid cSc \mid cc\\
&A \longrightarrow a\\
&B \longrightarrow AA \mid A \mid bA \mid b\\
&C \longrightarrow bCb \mid a
\end{aligned}$$

Adesso ci resta solo da adattare G' in modo che sia in grado di generare la parola vuota tramite il suo simbolo iniziale senza che questi appaia sulla destra di nessuna produzione. Nel caso di una grammatica libera dal contesto questo è molto semplice e ci basta creare un nuovo simbolo iniziale S_0 e aggiungere due produzioni $S_0 \longrightarrow S \mid \lambda$.
Quindi la grammatica che otteniamo alla fine è $G' = (N, T, \mathcal{P}_1, S_0)$ con le seguenti produzioni:

$$\begin{aligned}
&S_0 \longrightarrow S \mid \lambda\\
&S \longrightarrow AbB \mid bB \mid Ab \mid b \mid cSc \mid cc\\
&A \longrightarrow a\\
&B \longrightarrow AA \mid A \mid bA \mid b\\
&C \longrightarrow bCb \mid a
\end{aligned}$$

Esempio 4.9. Consideriamo la seguente grammatica regolare $G = (N = \{S\}, T = \{a, b\}, \mathcal{P}, S)$ con le seguenti produzioni:

$$S \longrightarrow aS \mid bS \mid \lambda$$

Procediamo anche in questo caso in due passaggi:

- Costruiamo una grammatica G' senza λ-produzioni.
- Adattiamo G' per generare la parola vuota senza che il suo simbolo iniziale appaia a destra di nessuna produzione.

Definiamo $G' = (N, T, \mathcal{P}', S)$ con gli stessi simboli terminali e non terminali di G e, per ora, lo stesso simbolo iniziale. L'unico simbolo non terminale annullabile è S. Quindi aggiungiamo a $\mathcal{P}_1$ le seguenti produzioni:

$$S \longrightarrow aS \mid a\cancel{S} \mid aB \mid b\cancel{S}$$

ovvero

$$S \longrightarrow aS \mid a \mid aB \mid b$$

ed evitiamo di aggiungere $S \longrightarrow \lambda$.
Quindi in $\mathcal{P}_1$ abbiamo le seguenti produzioni:

$$S \longrightarrow aS \mid bS \mid a \mid b$$

Adesso dobbiamo adattare G' in modo che possa generare la parola vuota tramite il suo simbolo iniziale senza che esso appaia nella parte destra

di qualche produzione. Questo caso è un po' più complicato del caso libero dal contesto, perché non possiamo usare produzioni $S_0 \longrightarrow S$. In ogni caso ci basta creare un nuovo simbolo iniziale S_0 e aggiungervi tutte le produzioni del precedente simbolo iniziale S, più naturalmente la λ-produzione $S_0 \longrightarrow \lambda$. Quindi la grammatica che otteniamo dopo anche questo passaggio è $G' = (N, T, \mathcal{P}_1, S_0)$ con le seguenti produzioni:

$$S_0 \longrightarrow aS \mid bS \mid a \mid b \mid \lambda$$
$$S \longrightarrow aS \mid bS \mid a \mid b$$

Abbiamo quindi visto come, per le grammatiche libere dal contesto e dunque anche quelle regolari, la presenza di λ-produzioni non solleva alcun problema, ed è sempre possibile costruire una grammatica equivalente che non ne fa uso a eccezione che per la generazione della stringa vuota. Per questo motivo talvolta si potranno vedere grammatiche di tipo 2 o 3 che fanno un uso libero di λ-produzioni.

Tuttavia ciò non vale in generale per le grammatiche di tipo 1: difatti se le grammatiche dipendenti dal contesto ammettessero λ-produzioni, allora sarebbero in grado di generare tutti i linguaggi di tipo 0, anche quelli che di norma le grammatiche di tipo 1 non sono in grado di generare. Questo è evidente dal fatto che se una grammatica di tipo 1 ammettesse lambda produzioni non sarebbe più monotona, e come abbiamo visto nella Sezione 4.1.1 dall'equivalenza delle grammatiche monotone con quelle dipendenti dal contesto, la monotonia delle regole di produzione è l'unica restrizione che questa classe di grammatiche ha rispetto alle grammatiche di tipo 0.

4.1.3 Alberi di derivazione

Ogni grammatica a struttura di frase G genera un suo linguaggio $L(G)$ e quindi pone in modo naturale il problema di decidere, per ogni stringa w di simboli terminali, se w appartiene o no a $L(G)$, e cioè se w è o no derivabile in G dal simbolo iniziale S di G. Nel caso di grammatiche G libere dal contesto (quindi anche per grammatiche regolari) e per stringhe $w \in L(G)$ è possibile rappresentare il processo che genera w da S in G tramite un albero, nel modo che adesso descriviamo.

Definizione 4.11. Siano $G = (N, T, \mathcal{P}, S)$ la nostra grammatica libera dal contesto e sia $w \in L(G)$. L'*albero di derivazione* di w in G è un albero finito (in cui per semplicità è fissato un ordinamento dei nodi) in cui:

- a ogni nodo si assegna come etichetta un simbolo in $N \cup T$,
- in particolare alla radice si assegna il simbolo iniziale S, ai nodi che non sono foglie si associano simboli non terminali (di N) e alle foglie simboli terminali (di T),
- si richiede poi che, se un nodo di etichetta A ha come figli nell'ordine stabilito nell'albero i nodi di etichette $A_1, \ldots, A_n$ (terminali

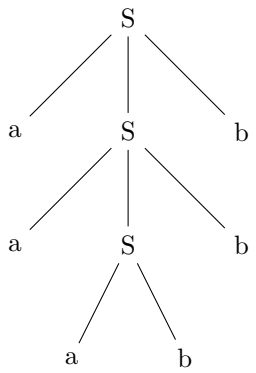
o no), allora $A \longrightarrow A_1 \dots A_n$ sia una produzione di $\mathcal{P}$,

- finalmente, la stringa delle etichette delle foglie (elencate secondo l'ordine fissato nell'albero) è proprio w.

Esempio 4.10. Si consideri la grammatica libera dal contesto $G = (N, T, \mathcal{P}, S)$, dove $N = \{S\}$, $T = \{a, b\}$ e $\mathcal{P}$ è l'insieme di produzioni

$$S \longrightarrow aSb \mid ab$$

Prendiamo come stringa $a^3b^3 \in L(G)$. L'albero di derivazione in G è il seguente:



Nel seguito ci capiterà di incontrare talora anche alberi di derivazione costruiti nel modo sopra descritto, ma liberi di ammettere come radice un qualunque simbolo non terminale, e non soltanto obbligatoriamente S.

Ci si può poi domandare come mai gli alberi di derivazione sono definiti soltanto per grammatiche libere dal contesto e non anche per grammatiche dipendenti dal contesto. D'altra parte, si può ricordare che, nel più vasto ambito dipendente dal contesto, le produzioni ammettono a sinistra non solo singoli elementi di N, e cioè simboli non terminali di G, ma anche stringhe che li contengono, il che rende ovviamente problematica l'assegnazione delle etichette ai vari nodi dell'albero.

4.2 Automi di riconoscimento

Nella Sezione 4.1 abbiamo discusso la possibilità di "generare" un linguaggio come l'insieme delle stringhe derivabili da una grammatica a struttura di frase. Un secondo approccio consiste nello stabilire una procedura che sia invece in grado di "riconoscere" (laddove possibile, infatti come sappiamo alcuni linguaggi non sono decidibili) quali stringhe appartengono a un linguaggio e quali invece non appartengono.

In termini generali, questa procedura dovrebbe scorrere ed esaminare a uno a uno i caratteri della stringa in ingresso, modificando a ogni passo il suo *stato*, così che al termine della lettura della stringa, esaminando lo stato, si possa stabilire

l'appartenenza o meno al linguaggio della parola appena letta. Questa procedura si concretizza nella forma di una macchina astratta che chiamiamo *automa*. Da un punto di vista esteriore, ogni automa si compone di un dispositivo di controllo con un numero finito di stati, di una testina di lettura e scrittura e di un nastro suddiviso in celle. La stringa in input viene scritta su questo nastro da sinistra verso destra, un carattere per ogni cella. I quadri a sinistra e a destra dell'input sono riempiti dal simbolo bianco $\star$. All'inizio del riconoscimento, la testina di lettura e scrittura esamina il primo simbolo dell'input e il controllo è nello stato iniziale. All'eventuale termine dell'esecuzione dell'automa si controlla il suo stato: se si trova in uno degli stati cosiddetti *finali*, allora la parola è stata riconosciuta come appartenente al linguaggio, altrimenti è stato stabilito che non appartiene al linguaggio.

Il lettore attento potrebbe notare che questa descrizione ricorda molto la definizione delle Macchine di Turing, e non è un caso: l'automa riconoscitore per i linguaggi di tipo 0 è proprio la Macchina di Turing, che in questo contesto chiamiamo *Automa di Turing*. L'unica differenza è la definizione degli stati finali dell'Automa di Turing, che tuttavia non cambia affatto le capacità espressive di quest'ultimo rispetto a una MdT.

Definizione 4.12. Un *automa di Turing non deterministico* (AdTND) è una macchina di Turing non deterministica $M = (Q, A, \delta, q_0, F)$, con $F \subseteq Q$ l'insieme di stati finali, anche detti *accettanti*.
Se la funzione di transizione δ associa al più una terna (q', a', x) a ogni coppia (q, a) allora si ha un *automa di Turing deterministico* (AdTD).

La classe di linguaggi riconosciuti da automi di Turing deterministici e non deterministici è la stessa, e segue direttamente dalla dimostrazione dell'equivalenza delle macchine di Turing deterministiche con quelle non deterministiche.

Definizione 4.13. Il linguaggio *riconosciuto* da un automa di Turing non deterministico M, che indichiamo con $L(M)$, è l'insieme di quelle stringhe w su un dato alfabeto A per le quali esiste una computazione finita dell'automa al termine della quale l'automa si trova in uno stato finale. Quindi $L(M) = \{w \in A^\star \mid M \downarrow w \text{ con } M \text{ in uno stato finale}\}$

Gli automi giocano un ruolo di fondamentale importanza e, in particolare, possono essere adoperati per determinare i linguaggi L in modo alternativo alle grammatiche, volto non a generare, ma a riconoscere e accettare le stringhe di L. In realtà, si possono formare varie classi di automi, anzi ogni classe di grammatiche nella gerarchia di Chomsky si può collegare a una corrispondente classe di automi in modo tale che differenti poteri espressivi delle classi di grammatiche si rispecchino in differenti poteri riconoscitivi delle relative classi di automi. Del resto abbiamo già visto nello scorso paragrafo che alla classe di tutte le grammatiche a struttura di frase corrisponde quella delle Macchine di Turing, nel senso che un linguaggio è accettato da un automa di Turing se e solo se è generato da una grammatica di tipo 0. Allo stesso modo si vede che:

- alla classe delle grammatiche dipendenti dal contesto corrisponde la classe

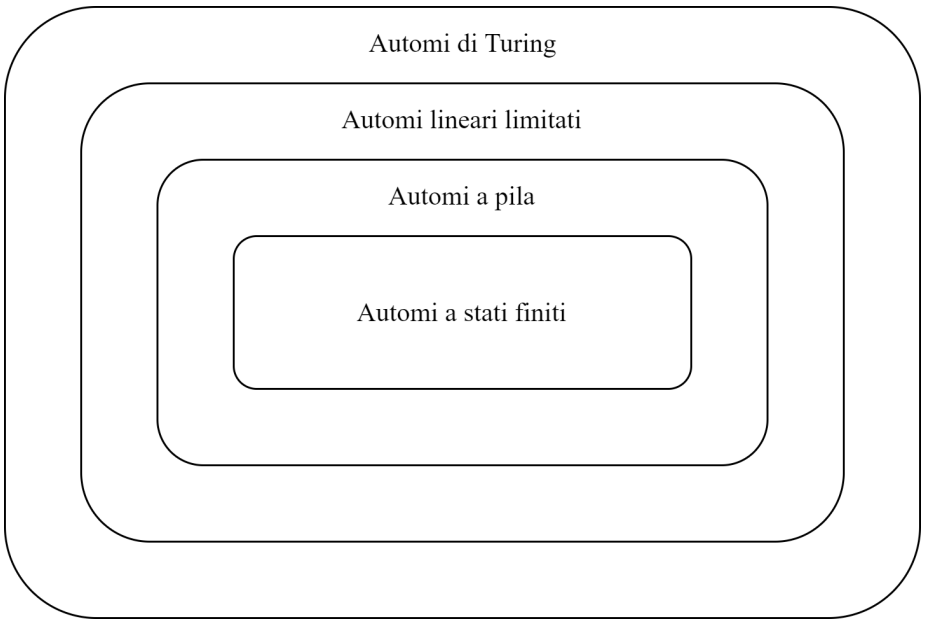


Figura 4.2: Classi di automi

degli *automi lineari limitati non deterministici*,

- alla classe delle grammatiche libere dal contesto corrisponde la classe degli *automi a pila non deterministici*,

- alla classe delle grammatiche regolari corrisponde la classe degli *automi a stati finiti deterministici* e *non deterministici*.

La gerarchia delle classi di automi che risulta da questa caratterizzazione è mostrata in Figura 4.2. Nelle prossime sezioni descriveremo tutte queste tipologie di automi, a partire dalla classe più restrittiva degli automi a stati finiti e concludendo con quella più permissiva, gli automi lineari limitati.

4.2.1 Automi a stati finiti

Un *automa a stati finiti* scorre la stringa in input da sinistra verso destra, un simbolo alla volta: a ogni passo l'automa legge il carattere sul nastro, cambia il suo stato secondo quanto gli ordina una fissata funzione di transizione e sposta finalmente la sua testina di un passo verso destra. Esaminata tutta la stringa in ingresso, l'automa si arresta, ed è proprio lo stato con cui si ferma a stabilire se l'input appartiene o no al linguaggio. Un automa a stati finiti si può dunque pensare come una MdT con capacità ridotte: licenza di leggere ma non di scrivere, permesso di spostarsi solo a destra. Come detto, si mostra che la classe degli automi a stati finiti corrisponde alla classe dei linguaggi di tipo 3: vale a dire, un linguaggio è riconosciuto da un qualche automa a stati finiti se e solo se è regolare.

Definizione 4.14. Un *automa a stati finiti non deterministico* (ASFND, o NFA in inglese) è un automa di Turing non deterministico $M = (Q, A, \delta, q_0, F)$, dove:

- Q è un insieme finito e non vuoto di stati;
- A è un alfabeto;
- δ è la funzione di transizione totale che associa a ogni coppia $(q, a) \in Q \times A$ un insieme, potenzialmente vuoto, di stati $q' \in Q$;
- q_0 è lo stato iniziale;
- $F \subseteq Q$ è l'insieme degli stati finali.

Se la funzione di transizione δ è una funzione parziale che associa al più uno stato q' a ogni coppia (q, a) allora si ha un *automa a stati finiti deterministico* (ASFD o anche DFA in inglese).

Quindi nel caso di un ASFND se $q' \in \delta(q, a)$ vuol dire che l'automa nello stato q che legge a dal nastro può passare allo stato q', dopodiché automaticamente la testina si muove al carattere successivo. Prima di definire il linguaggio riconosciuto da un automa a stati finiti ci sarà utile estendere la definizione della funzione di transizione δ come segue.

Definizione 4.15. La funzione di transizione δ si estende al dominio $Q \times A^\star$ definendo, per $a \in A, w \in A^\star$

- Nel caso di un ASFND,

$$\delta'(q, a) = \delta(q, a)$$

$$\delta'(q, wa) = \bigcup_{p \in \delta'(q,w)} \delta(p, a)$$

  Ovvero la funzione δ' dato uno stato q e una parola ritorna l'insieme, potenzialmente vuoto, di tutti gli stati in cui l'automa si può trovare al termine della lettura completa di quella parola a partire dallo stato q.

- Nel caso di un ASFD,

$$\delta'(q, a) = \delta(q, a)$$

$$\delta'(q, wa) = \delta(\delta'(q, w), a)$$

  Ovvero la funzione δ' dato uno stato q e una parola ritorna, se esiste, lo stato in cui l'automa si può trovare al termine della lettura completa di quella parola a partire dallo stato q.

Esempio 4.11. Consideriamo l'automa a stati finiti deterministico M_1 dell'Esempio 4.12. In questo caso

$$\begin{aligned}\delta'(q_0, abb) &= \delta(\delta'(q_0, ab), b)\\ &= \delta(\delta(\delta'(q_0, a), b), b)\\ &= \delta(\delta(\delta(q_0, a), b), b) = q_2\end{aligned}$$

Per il caso non deterministico, consideriamo l'ASFND dell'Esempio 4.13 sempre sulla stringa abb.

$$\delta'(q_0, abb) = \bigcup_{p \in \delta'(q_0, ab)} \delta(p, b)$$

Valutiamo innanzitutto la chiamata ricorsiva a δ':

$$\begin{aligned}\delta'(q_0, ab) &= \bigcup_{p \in \delta'(q_0, a)} \delta(p, b) = \bigcup_{p \in \delta(q_0, a)} \delta(p, b) = \bigcup_{p \in \{q_1, q_2\}} \delta(p, b)\\ &= \delta(q_1, b) \cup \delta(q_2, b) = \{q_1\} \cup \{q_2, q_3\} = \{q_1, q_2, q_3\}\end{aligned}$$

Ora sostituiamo nell'equazione iniziale:

$$\begin{aligned}\delta'(q_0, abb) &= \bigcup_{p \in \{q_1, q_2, q_3\}} \delta(p, b) = \delta(q_1, b) \cup \delta(q_2, b) \cup \delta(q_3, b)\\ &= \{q_1\} \cup \{q_2, q_3\} \cup \{\} = \{q_1, q_2, q_3\}\end{aligned}$$

Il linguaggio riconosciuto da un automa a stati finiti si può adesso definire in termini di δ'.

Definizione 4.16. Il linguaggio riconosciuto da un automa a stati finiti non deterministico $M = (Q, A, \delta, q_0, F)$, che indichiamo con $L(M)$ è l'insieme $\{w \in A^\star \mid \delta'(q_0, w) \cap F \neq \varnothing\}$, cioè l'insieme di tutte quelle parole sull'alfabeto A tali per cui esiste una sequenza di transizioni che portano l'automa da q_0 a uno stato finale.

Definizione 4.17. Il linguaggio riconosciuto da un automa a stati finiti deterministico $M = (Q, A, \delta, q_0, F)$, che indichiamo con $L(M)$ è l'insieme $\{w \in A^\star \mid \delta'(q_0, w) \in F\}$, cioè l'insieme di tutte quelle parole sull'alfabeto A tali per cui l'automa a partire da q_0 raggiunge uno stato finale.

Prima di vedere un esempio di automa a stati finiti è opportuno stabilire come rappresentarli graficamente. Sia gli ASFND che gli ASFD si possono rappresentare graficamente tramite un *grafo di transizione*, dove i nodi corrispondono agli stati e gli archi orientati rappresentano le transizioni da uno stato a un altro stato causate dalla lettura di un certo simbolo. Ogni arco viene marcato col simbolo che ha provocato la transizione. Se ci sono più archi che da uno stesso nodo raggiungono un altro nodo, questi si possono combinare in un

unico arco con i corrispondenti simboli separati da una virgola. Lo stato iniziale viene rappresentato con una freccia entrante da "fuori" mentre lo stato finale si disegna contenente un secondo cerchio concentrico. Nel grafo di transizione, possiamo caratterizzare l'insieme delle parole riconosciute dall'automa come l'insieme di tutti i possibili cammini dallo stato iniziale allo stato finale.

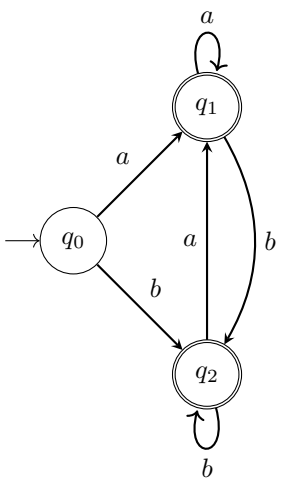
Esempio 4.12. Definiamo un automa a stati finiti deterministico che riconosca il linguaggio $L = \{a^n b^m \mid n, m \geq 1\}$.
Una possibilità è l'automa

$$M_1 = (Q = \{q_0, q_1, q_2\}, A = \{a, b\}, \delta, q_0, F = \{q_1, q_2\})$$

con le seguenti transizioni:

	a	b
q_0	q_1	q_2
q_1	q_1	q_2
q_2	q_1	q_2

La seguente figura mostra il corrispondente grafo di transizione:



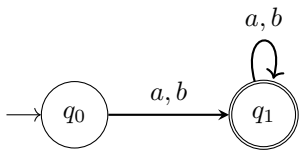
Questo non è l'unico automa a stati finiti deterministico possibile per L.
Un automa ancora più semplice è

$$M_2 = (Q = \{q_0, q_1\}, A = \{a, b\}, \delta, q_0, F = \{q_1\})$$

con le seguenti transizioni:

	a	b
q_0	q_1	q_1
q_1	q_1	q_1

e grafo di transizione:



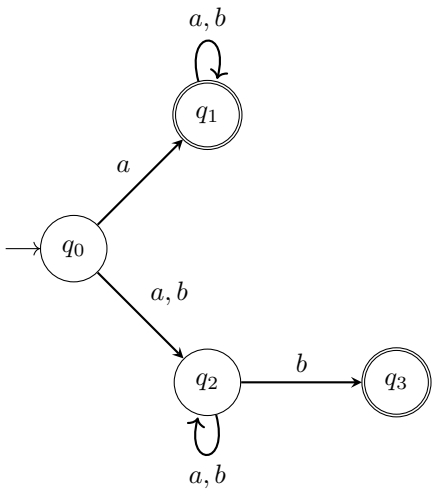
Esempio 4.13. Un esempio di automa a stati finiti non deterministico è

$$M = (Q = \{q_0, q_1, q_2, q_3\}, A = \{a, b\}, \delta, q_0, F = \{q_1, q_3\})$$

con le seguenti transizioni:

	a	b
q_0	$\{q_1, q_2\}$	$\{q_2\}$
q_1	$\{q_1\}$	$\{q_2\}$
q_2	$\{q_2\}$	$\{q_2, q_3\}$
q_3	$\{\}$	$\{\}$

La seguente figura mostra il corrispondente grafo di transizione:



Il linguaggio riconosciuto da questo automa è $L(M) = \{aw \mid w \in \{a, b\}^\star\} \cup \{wb \mid w \in \{a, b\}^\star\}$, in altre parole il linguaggio composto da tutte le parole su $\{a, b\}$ che iniziano per a oppure finiscono per b.

Grazie a questo tipo di rappresentazione grafica, è possibile omettere da adesso in poi la tabella di transizione dell'automa (e di fatto anche la sua definizione, dato che il grafo di transizione include tutte le informazioni necessarie).

Proseguiamo la nostra trattazione degli automi a stati finiti dimostrando una serie di importanti risultati che mostrano l'equivalenza espressiva tra automi a stati finiti non deterministici e quelli deterministici, e tra questi ultimi e le grammatiche regolari.

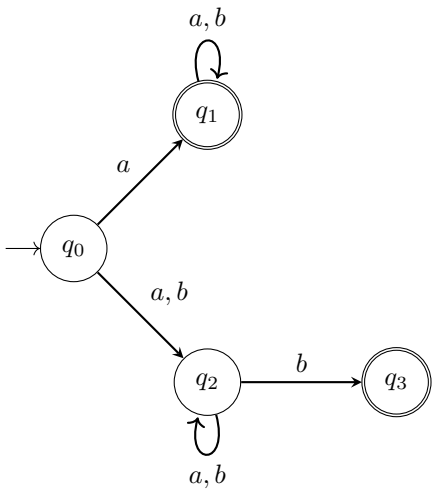
Teorema 4.2. Per ogni automa a stati finiti non deterministico N esiste un automa a stati finiti deterministico D equivalente, cioè tale per cui $L(D) = L(N)$.

Dimostrazione. Sia $N = (Q_N, A_N, \delta_N, q_0, F_N)$ un ASFND. Mostriamo ora come costruire il corrispondente ASFD $D = (Q_D, A_D, \delta_D, q'_0, F_D)$.

1. Gli stati Q_D rappresentano tutti i possibili sottoinsiemi degli di Q_N. A questo proposito indichiamo con $[q_0, q_1, \ldots, q_k]$ lo stato che corrisponde all'insieme $\{q_0, q_1, \ldots q_k\}$.
2. L'alfabeto A_D è lo stesso di N, quindi $A_D = A_N$.
3. La funzione di transizione è data da:
$$\delta_D([q_0, \ldots q_i], a) = [\delta_N(q_0, a) \cup \delta_N(q_1, a) \cup \ldots \cup \delta_N(q_i, a)], \forall a \in A_D$$
4. Lo stato iniziale q'_0 è quello che corrisponde al sottoinsieme $\{q_0\}$.
5. Gli stati finali F_D rappresentano tutti i sottoinsiemi di Q_N che contengono almeno un elemento che appartiene a F_N, cioè i sottoinsiemi che contengono almeno uno stato finale di N.

Mostriamo ora che D riconosce tutte le parole riconosciute da N e viceversa. Dalla definizione di δ_D, a ogni passo l'automa D tiene traccia di tutte le possibili transizioni di N. Inoltre D raggiunge uno stato finale ogni qualvolta esiste almeno una sequenza di transizioni in N che porta a uno stato finale. Quindi D riconosce tutte le parole riconosciute da N.
Viceversa assumiamo che $w \in L(D)$. Allora esiste una sequenza di stati che inizia con lo stato $[q_0]$ e termina con lo stato $[\ldots q_m \ldots]$ con $q_m \in F_N$. Ogni altro stato $[q_i \ldots q_k]$ nella sequenza, dalla definizione di δ_D, rappresenta un insieme che contiene un q_h tale che esiste una sequenza di transizioni di N che inizia con q_0 e termina in q_h nello stesso numero di passi occorrenti a D per raggiungere $[q_i \ldots q_k]$ a partire da $[q_0]$. Per induzione sulla sequenza degli stati, se D riconosce w allora esiste una sequenza di passi di N che porta al riconoscimento di w. □

Esempio 4.14. Trasformiamo l'automa a stati finiti non deterministico M dell'Esempio 4.13, che riportiamo qui, in un automa a stati finiti deterministico M'.



Descriviamo la procedura per costruire la matrice di transizione di M':

1. Iniziamo costruendo una tabella con una colonna per ogni simbolo dell'alfabeto e, inizialmente, una sola riga, quella per lo stato $[q_0]$ di M' che corrisponde all'insieme $\{q_0\}$ che contiene solo lo stato iniziale di M.

2. Si completa questa riga con l'insieme degli stati raggiungibili eseguendo una singola transizione da ognuno degli stati dell'insieme di stati che corrisponde alla riga. Per ogni insieme di stati successori che si ottiene, se non è già presente una corrispondente riga nella tabella, viene creata una nuova riga della tabella per lo stato che corrisponde a questo insieme di stati.

3. Si ripete il passo 2 per ogni nuova riga che viene aggiunta alla tabella, fino a che non vengono più aggiunte nuove righe.

Mostriamo i passaggi appena descritti, iniziando dalla tabella come da primo passaggio:

	a	b
$[q_0]$		

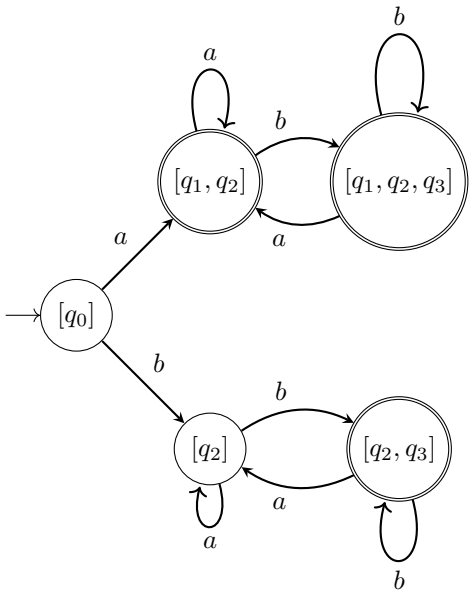
Completiamo la prima riga come descritto dal secondo passaggio:

	a	b
$[q_0]$	$\{q_1, q_2\}$	$\{q_2\}$
$[q_1, q_2]$		
$[q_2]$		

Ora come da terzo passaggio completiamo le nuove righe fino a che non ci sono più righe da aggiungere:

	a	b
$[q_0]$	$\{q_1, q_2\}$	$\{q_2\}$
$[q_1, q_2]$	$\{q_1, q_2\}$	$\{q_1, q_2, q_3\}$
$[q_2]$	$\{q_2\}$	$\{q_2, q_3\}$
$[q_1, q_2, q_3]$	$\{q_1, q_2\}$	$\{q_1, q_2, q_3\}$
$[q_2, q_3]$	$\{q_2\}$	$\{q_2, q_3\}$

Ora possiamo disegnare l'automa a stati finiti deterministico corrispondente a questa matrice di transizione. Lo stato iniziale sarà $[q_0]$ mentre gli stati finali sono $[q_1, q_2], [q_1, q_2, q_3]$ e $[q_2, q_3]$, perché questi stati corrispondono a insiemi di stati che contengono almeno uno stato finale di M.



Teorema 4.3. Dato un automa a stati finiti deterministico M, esiste una grammatica regolare G tale che $L(G) = L(M)$.

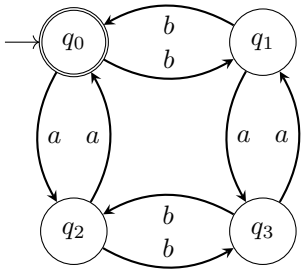
Dimostrazione. Fissato un ASFD $M = (Q, A, \delta, q_0, F)$ si costruisce una grammatica regolare tramite la seguente procedura:

1. L'insieme dei simboli terminali T coincide con l'alfabeto A;
2. L'insieme dei simboli non terminali N coincide o è in corrispondenza con gli stati Q;
3. Il simbolo iniziale di G è il simbolo non terminale che corrisponde allo stato q_0;
4. L'insieme delle produzioni è dato da:

- Per ogni transizione da uno stato q_i a uno stato q_j per effetto del simbolo a_h si crea una produzione $N_i \longrightarrow a_h N_j$ con N_i, N_j simboli non terminali corrispondenti a q_i e q_j. Se $q_j \in F$ si aggiunge anche la produzione $N_i \longrightarrow a_h$.
- Se $q_0 \in F$ allora l'automa riconosce anche la parola vuota, e quindi si aggiunge la produzione $S \longrightarrow \lambda$.

Mostriamo ora come per ogni parola $w = a_1 \dots a_n$ riconosciuta da M esiste una derivazione dal simbolo iniziale di G che la produce. Se $w \in L(M)$ allora esiste una sequenza di stati $q_0, q_1, \dots q_{n-1}$ tale che $\delta(q_0, a_1) = q_1, \delta(q_1, a_2) = q_2, \dots, \delta(q_{n-1}, a_n) \in F$. Per come è stata costruita G allora esisterà una sequenza di produzioni $N_0 \longrightarrow aN_1, N_1 \longrightarrow a_2N_2, \dots, N_{n-1} \longrightarrow a_n$ che costituiscono la derivazione di w in G.
In modo analogo si mostra che per ogni derivazione $N_0 \longrightarrow^{\star}_{G} w$ in G esiste una sequenza di transizioni dell'automa tale che $\delta'(q_0, w) \in F$. □

Esempio 4.15. Ricaviamo la grammatica regolare equivalente al seguente automa



La grammatica che si ottiene è

$$G = (N = \{N_0, N_1, N_2, N_3\}, T = \{a, b\}, \mathcal{P}, N_0)$$

con le seguenti produzioni:

$$\begin{aligned}
N_0 &\longrightarrow aN_2 \mid bN_1 \mid \lambda \\
N_1 &\longrightarrow bN_0 \mid aN_3 \mid b \\
N_2 &\longrightarrow bN_3 \mid aN_0 \mid a \\
N_3 &\longrightarrow aN_1 \mid bN_2
\end{aligned}$$

Notiamo che finora non abbiamo minimamente considerato la possibilità di eseguire transizioni senza leggere dal nastro, ovvero istruzioni della forma $\delta(q, \lambda)$. Possiamo estendere la definizione di automa a stati finiti non deterministico per includere anche le λ-transizioni, che chiamiamo *λ-ASFND* o *λ-NFA*, e tramite il seguente teorema mostriamo che la classe dei λ-ASFND è equivalente alla classe degli ASFND.

Teorema 4.4. Per ogni automa a stati finiti non deterministico con λ-transizioni M esiste un automa a stati finiti non deterministico N equivalente, cioè tale per cui $L(N) = L(M)$.

Dimostrazione. L'automa N si può costruire applicando ripetutamente la seguente regola:

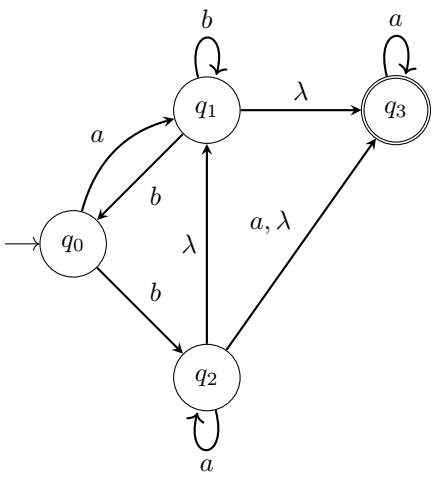
- Se c'è una λ-transizione da uno stato q_i a un insieme di stati $\{q_j, q_k, \ldots\}$, allora questa transizione si può rimuovere dopo aver *aggiunto* a q_i tutte le transizioni che partono da ognuno degli stati $q_j, q_k, \ldots$. Inoltre se almeno uno degli stati $q_j, q_k, \ldots$ è uno stato finale, allora anche q_i lo diventa.

Aggiungere le transizioni di uno stato q_j a uno stato q_i significa che per ogni istruzione $\delta(q_j, a) = \{q_k, q_l, \ldots\}$ con $a \in A$ si aggiungono gli stati $q_k, q_l, \ldots$ all'insieme $\delta(q_i, a)$.
Possiamo giustificare questa regola notando che

$$\delta'(q_i, \lambda a) = \bigcup_{p \in \delta(q_i, \lambda)} \delta(p, a) = \delta(q_j, a) \cup \ldots$$

□

Esempio 4.16. Consideriamo il seguente λ-ASFND:



Per ottenere il corrispondente ASFND senza λ-transizioni, applichiamo la regola del Teorema 4.4 alle transizioni $\delta(q_1, \lambda) = \{q_3\}$ e $\delta(q_2, \lambda) = \{q_1, q_3\}$:

- Per rimuovere $\delta(q_1, \lambda) = \{q_3\}$ dobbiamo aggiungere a q_1 tutte le transizioni di q_3. L'unica transizione di q_3 è $\delta(q_3, a) = \{q_3\}$, quindi $\delta(q_1, a) = \{\} \cup \{q_3\} = \{q_3\}$. Dato che q_3 è uno stato finale, q_1 diventa uno stato finale.

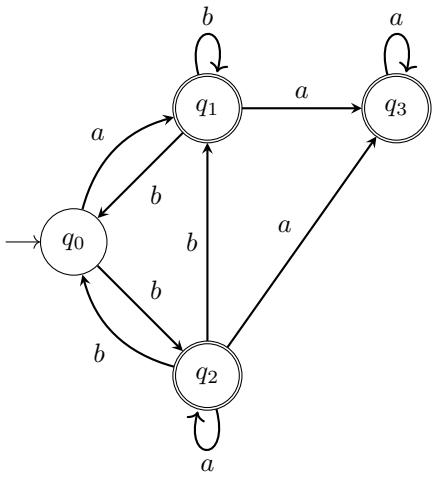
- Per rimuovere $\delta(q_2, \lambda) = \{q_1, q_3\}$ dobbiamo aggiungere a q_2 tutte le transizioni di q_1 e q_3 (tenendo in considerazione anche le modifiche che abbiamo fatto al punto precedente), quindi otteniamo:

$$\delta(q_2, a) = \{q_2, q_3\} \cup \{q_3\} \cup \{q_3\} = \{q_2, q_3\}$$

$$\delta(q_2, b) = \{\} \cup \{q_0, q_1\} \cup \{\} = \{q_0, q_1\}$$

  Dato che q_1 e q_3 sono stati finali, anche q_2 diventa uno stato finale.

L'automa a stati finiti non deterministico, ora senza λ-transizioni, che si ottiene è il seguente:



La limitazione delle capacità di riconoscimento degli automi a stati finiti si riconduce al fatto che essi dispongono soltanto di una memoria finita (quella che è loro assicurata dai loro stati). Per questo motivo essi non sono in grado di riconoscere linguaggi che richiedono invece di ricordare una quantità di informazione illimitata. Si pensi per esempio al linguaggio

$$L = \{a^n b^n \mid n \geq 1\}$$

Per riconoscerne una stringa $a^n b^n$ bisogna ricordare il numero n delle a lette prima di incontrare la prima b, in modo da poter poi controllare che le b che completano la stringa siano altrettante. Nessun automa a stati finiti ha questa potenzialità di memoria (e del resto L non è regolare). Nella Sezione 4.2.2 vedremo come estendere gli automi a stati finiti per superare questa limitazione.

4.2.2 Automi a pila

Per migliorare le capacità di riconoscimento degli automi a stati finiti, possiamo pensare a nuovi modelli che dispongano di una memoria ausiliare illimitata e svolgano ogni mossa della loro computazione in dipendenza non solo del carattere letto e dello stato del dispositivo di controllo, ma anche della informazione precedentemente accumulata nella memoria. Un *automa a pila* corrisponde a questi requisiti: ha il suo dispositivo di controllo a stati finiti, la sua testina

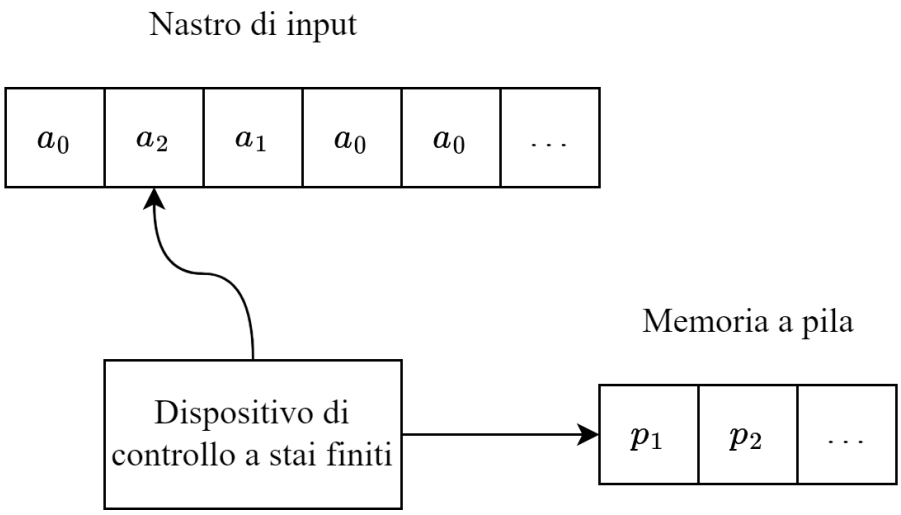


Figura 4.3: Schema di un automa a pila

di lettura e scrittura, il suo nastro e, in più, una memoria ausiliaria a pila di lunghezza infinita cui però può accedere solo dalla testa (vedi Figura 4.3).

A ogni passo di computazione, l'automa può svolgere una delle seguenti mosse:

1. Leggere il contenuto di una cella dal nastro e il simbolo in cima alla pila, e, considerando lo stato in cui si trova, passare a un nuovo stato e sostituire il simbolo letto dalla pila con una nuova stringa (possibilmente anche λ). La testina quindi si sposta sul nastro alla cella successiva a destra, e in cima alla pila ora c'è il primo simbolo della stringa che vi è stata appena scritta.

2. Eseguire gli stessi passaggi al punto 1, ma senza leggere dal nastro (leggendo quindi λ) e senza spostare la testina.

Il secondo tipo di mossa permette all'automa di manipolare la pila senza proseguire con la lettura del nastro di input; tuttavia scorrere la pila per leggere i simboli più in profondità porta l'automa a perdere completamente traccia di quanto è stato già prelevato dalla pila.

Definizione 4.18. Un *automa a pila non deterministico* (APND, o in inglese PDA) è un automa di Turing non deterministico $M = (Q, A, R, \delta, q_0, Z_0, F)$, dove:

- Q è un insieme finito e non vuoto di stati;
- A è un alfabeto, detto *alfabeto del nastro*;
- R è un alfabeto, detto *alfabeto della pila*;

- δ è la funzione di transizione totale che associa a ogni tripla $(q, a, r) \in Q \times (A \cup \{\lambda\}) \times R$ un insieme, potenzialmente vuoto, di coppie $(q', \gamma) \in Q \times R^\star$;
- q_0 è lo stato iniziale;
- Z_0 è il simbolo iniziale, che appare all'inizio della pila;
- $F \subseteq Q$ è l'insieme degli stati finali.

Se la funzione di transizione δ è una funzione parziale che associa al più una coppia (q', γ) a ogni tripla (q, a, r) allora si ha un *automa a pila deterministico* (APD, o in inglese DPDA). Nel caso di un APD le istruzioni della forma $\delta(q, \lambda, Z) = (q', \gamma)$ sono eseguibili solo se non esiste nessuna istruzione $\delta(q, a, Z)$ dove a è il simbolo su cui è posizionata la testina.

La funzione di transizione in questo caso non solo può cambiare lo stato dell'automa, ma può anche scrivere sulla pila, tuttavia il nastro di input è comunque di sola lettura e viene scorso un simbolo alla volta da sinistra verso destra senza possibilità per la testina di tornare indietro, come nel caso degli automi a stati finiti. Più precisamente se $(q', \gamma) \in \delta(q, a, Z)$ vuol dire che l'automa nello stato q che legge a dal nastro e Z dalla pila può passare allo stato q' e sovrascrivere Z con γ nella pila. La lettura del simbolo implica inoltre che la testina si sposta al simbolo successivo alla sua destra. Un caso particolare è $(q', \gamma) \in \delta(q, \lambda, Z)$, dove l'automa nello stato q legge Z dalla pila ma non legge nulla dal nastro e dunque non sposta la testina, ma comunque può eseguire un passaggio di stato a q' e sovrascrivere Z con γ.

Prima di definire il linguaggio riconosciuto da un automa a pila abbiamo bisogno di definire alcune nozioni ausiliarie.

Definizione 4.19. Un automa a pila è in una *configurazione* (q, x, γ) con $q \in Q, x \in A^\star$ e $\gamma \in R^\star$ se si trova nello stato q, la parte del nastro che la testina deve ancora leggere consiste in x e γ è il contenuto della pila.
Un *passo* dell'automa a pila è indicato con $(q, aw, Z\alpha) \vdash_M (p, w, \gamma\alpha)$. L'automa può compiere questo passo se ciò è previsto dalla sua funzione di transizione, cioè se $\delta(q, a, Z)$ contiene (p, γ).
Una *computazione* dell'automa a pila è indicata con

$$(q, a_1 \dots a_n w, Z\alpha) \vdash_M^\star (p, w, \gamma\alpha)$$

dove $a_1, \dots a_n \in A \cup \{\lambda\}$ e la configurazione che otteniamo a destra è derivata in n passi da quella di sinistra.

Nel definire il linguaggio riconosciuto da un automa a pila abbiamo due possibilità equivalenti:

- Come per gli automi a stati finiti, diciamo che una parola è riconosciuta da un automa a pila se esiste una computazione dell'automa che legga

l'intera stringa e lasci l'automa in uno stato finale.

- In alternativa, possiamo considerare come riconosciute tutte quelle parole per le quali esiste una computazione dell'automa che legga l'intera stringa e lasci l'automa con la pila vuota.

Definizione 4.20. Il linguaggio riconosciuto da un automa a pila (deterministico o non deterministico) M, che indichiamo con $L(M)$ può essere definito equivalentemente come:

- $L(M) = \{w \in A^\star \mid (q_0, w, Z_0) \vdash_M^\star (q, \lambda, \gamma) \text{ con } \gamma \in R^\star, q \in F\}$ (le parole sono riconosciute per *stato finale*).
- $L(M) = \{w \in A^\star \mid (q_0, w, Z_0) \vdash_M^\star (q, \lambda, \lambda) \text{ con } q \in Q\}$ (le parole sono riconosciute per *pila vuota*).

In entrambi i casi la parola in input deve essere stata letta completamente, quindi sia se l'automa si arresta in uno stato finale nel primo caso o si arresta con la pila vuota nel secondo caso, se l'input non è stato processato completamente la parola non è considerata come riconosciuta.

Vediamo subito un esempio di automa a pila. Come tecnica di rappresentazione grafica, ci ispiriamo agli automi a stati finiti, con l'unica differenza che ora ogni arco del grafo di transizione dovrà essere contrassegnato non solo dal simbolo che viene letto dall'automa, ma anche da quale sostituzione viene fatta sulla pila. Formalmente, contrassegneremo un arco da uno stato q_i a uno stato q_j con $a; N \to \gamma$ con $a \in A, N \in R$ e $\gamma \in R^\star$ per indicare che dallo stato q_i leggendo il simbolo a dal nastro e il simbolo N dalla pila l'automa raggiunge lo stato q_j e sovrascrive N nella pila sostituendolo con γ.

Esempio 4.17. Definiamo un automa a pila deterministico che riconosca il linguaggio $L = \{a^n b^n \mid n \geq 1\}$ per stato finale. Una possibilità è l'automa

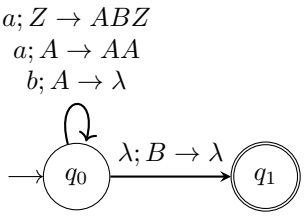
$$M = (Q = \{q_0, q_1\}, A = \{a, b\}, R = \{Z, A\}, \delta, q_0, Z, \{q_1\})$$

Mostriamo la tabella delle transizioni dell'automa relativa allo stato q_0, mentre omettiamo la corrispondente tabella per q_1 dato che è vuota:

	λ	a	b
Z		q_0, ABZ	
A		q_0, AA	q_0, λ
B	q_1, λ		

q_0

Mostriamo ora l'automa a pila graficamente:



L'idea dietro a questo automa è di copiare la sequenza iniziale di a sulla pila come simboli A, e poi eliminarli uno per uno dalla pila per ogni b che si incontra successivamente. Il simbolo B viene aggiunto inizialmente alla pila per assicurarsi che l'automa non riconosca la stringa vuota.
Per esempio, la parola $aabb$ viene valutata come:

$$(q_0, aabb, Z) \vdash (q_0, abb, ABZ) \vdash (q_0, bb, AABZ)$$
$$\vdash (q_0, b, ABZ) \vdash (q_0, \lambda, BZ) \vdash (q_1, \lambda, Z)$$

Invece nel caso di abb abbiamo:

$$(q_0, abb, Z) \vdash (q_0, bb, ABZ) \vdash (q_0, b, BZ) \vdash (q_1, b, Z)$$

dove la stringa non è riconosciuta malgrado q_1 sia uno stato finale perché b è ancora sul nastro.

Anche nel caso degli automi a pila la rappresentazione grafica contiene tutte le informazioni necessarie, e ci permette perlomeno di omettere la matrice di transizione. Tuttavia se l'automa in questione ha per esempio un solo stato, come spesso potrebbe essere il caso, la rappresentazione grafica è di poca utilità e utilizzare la matrice di transizione è più appropriato.

A questo punto è naturale chiedersi se gli automi a pila deterministici e non deterministici siano equivalenti, così come è il caso per gli automi a stati finiti. La risposta è negativa, e più precisamente la classe dei linguaggi riconosciuti dagli automi a pila deterministici è più ristretta rispetto a quella riconosciuta dagli automi a pila non deterministici. Dimostreremo infatti che questi ultimi sono gli automi riconoscitori per i linguaggi liberi dal contesto, mentre la loro controparte deterministica è limitata a un particolare sottoinsieme dei linguaggi liberi, i cosiddetti linguaggi $LR(k)$, che sebbene non trattiamo in questo libro sono estremamente importanti dal punto di vista pratico nello sviluppo di linguaggi per i computer, per via della loro migliore propensione all' analisi sintattica.

Concludiamo questa sezione mostrando che la classe degli automi a pila non deterministici corrisponde esattamente alla classe dei linguaggi di tipo 2. Prima di dimostrare questo teorema, dimostriamo un risultato ausiliario che ci sarà utile nella prova.

Lemma 4.1. Se $L = L(M)$ per pila vuota per un qualche APND M, allora $L = L(M_1)$ per pila vuota per un qualche APND M_1 con un solo stato.

Dimostrazione. La dimostrazione di questo lemma si ottiene costruendo M_1 come segue:

- con un solo stato q_0;
- con alfabeto della pila costituito da un simbolo per ogni possibile coppia (q, Z) di stati e simboli dell'alfabeto della pila di M;
- con funzione di transizione tale che se Z' è il simbolo che corrisponde alla coppia q, Z, allora l'istruzione $\delta(q, a, Z)$ di M diventa $\delta'(q_0, a, Z')$ in M'.

□

Teorema 4.5. L è un linguaggio libero dal contesto se e solo se è riconosciuto da un automa a pila non deterministico M.

Dimostrazione. Consideriamo solo APND che riconoscono linguaggi per pila vuota. La prova si svolge in due parti: prima mostriamo che per ogni grammatica libera dal contesto G esiste un APND M tale che $L(M) = L(G)$, poi dimostriamo il viceversa, cioè che per ogni automa a pila non deterministico con un solo stato M esiste una grammatica libera dal contesto G tale che $L(G) = L(M)$.

- Sia $G = (N, T, \mathcal{P}, S)$ una grammatica libera dal contesto in forma normale di Greibach. Costruiamo un automa a pila non deterministico $M = (Q = \{q\}, A = T, R = N, \delta, q, S, \varnothing)$ la cui funzione di transizione δ è definita in modo che $\delta(q, a, A)$ contiene (q, α) se e solo se $A \longrightarrow a\alpha$ è una produzione di G.

  Mostriamo ora che il linguaggio generato da M, $L(M)$, coincide con $L(G)$. Notiamo che se esiste una derivazione in G del tipo

  $$xA\beta \longrightarrow_G xa\alpha\beta \text{ con } x \in T^\star, A \in N, a \in T, \alpha, \beta \in N^\star$$

  allora esisterà una mossa di M tale che:

  $$(q, a, A\beta) \vdash_M (q, \lambda, \alpha\beta)$$

  e viceversa.

  Per induzione sul numero di passi della derivazione ne segue che

  $$xA \longrightarrow_G^\star xy\alpha\beta, \text{ dove } x, y \in T^\star, A \in N \text{ e } \alpha, \beta \in N^\star$$

  se e solo se esiste una sequenza di mosse di M tali che

  $$(q, y, A\beta) \vdash_M^\star (q, \lambda, \alpha\beta)$$

  Perciò $S \longrightarrow_G^\star y$ se e solo se $(q, y, S) \vdash_M^\star (q, \lambda, \lambda)$

- Sia $M = (Q = \{q\}, A, R, \delta, q, Z_0, \varnothing)$ un automa a pila non deterministico (con un solo stato). Costruiamo una grammatica $G = (N = R, T = A, \mathcal{P}, Z_0)$ equivalente ad M. Le produzioni di G saranno:

  $$Z \longrightarrow aZ_1Z_2 \ldots Z_n \text{ se } \delta(q, a, Z) \text{ contiene } (q, Z_1Z_2 \ldots Z_n)$$

con $a \in A$ e $Z, Z_1, Z_2, \ldots Z_n \in R$. Risulta immediatamente che, per $\alpha \in R^\star$

$$(q, a, Z\alpha) \vdash_M (q, \lambda, Z_1 Z_2 \ldots Z_n \alpha) \text{ se e solo se } Z\alpha \longrightarrow_G aZ_1 Z_2 \ldots Z_n \alpha$$

e quindi per induzione che per ogni $x \in A^\star$

$$(q, x, Z_0) \vdash_M^\star (q, \lambda, \lambda) \text{ se e solo se } Z_0 \longrightarrow_G^\star x$$

□

Esempio 4.18. Il linguaggio generato dalla grammatica libera dal contesto in forma normale di Greibach $G = (N = \{S, A, B\}, T = \{a, b\}, \mathcal{P}, S)$ con le seguenti produzioni:

$$\begin{aligned} S &\longrightarrow aB \mid bA \\ A &\longrightarrow aS \mid bAA \mid a \\ B &\longrightarrow bS \mid aBB \mid b \end{aligned}$$

è lo stesso linguaggio riconosciuto per pila vuota dall'automa a pila non deterministico $M = (Q = \{q\}, A = \{a, b\}, R = \{S, A, B\}, \delta, q, S, \varnothing)$ con la seguente funzione di transizione (relativa all'unico stato q):

	λ	a	b
S		q, B	q, A
A		q, λ q, S	q, AA
B		q, BB	q, λ q, S

q

4.2.3 Automi lineari limitati

Nella Sezione 4.2.2 abbiamo visto che la disponibilità di una memoria ausiliaria potenzialmente illimitata non è sufficiente a riconoscere ogni linguaggio a struttura di frase: ci sono infatti linguaggi dipendenti dal contesto che nessun automa a pila sa riconoscere. In effetti, quello che è essenziale in questo ambito più generale non è tanto una memoria ausiliaria illimitata, quanto la libertà di poterne disporre senza eccessivi vincoli. Nel caso dei linguaggi dipendenti dal contesto, allora, basta contare su di una memoria linearmente limitata dalla lunghezza dell'input e accessibile senza restrizioni. L'automa che ne risulta viene chiamato *automa lineare limitato*.

Definizione 4.21. Un *automa lineare limitato non deterministico* (ALND, o in inglese LBA) è un automa di Turing non deterministico $M = (Q, A, B, \delta, q_0, F)$, dove:

- Q è un insieme finito e non vuoto di stati;
- $A = B \cup \{\star, \Delta, \nabla\}$ è l'alfabeto del nastro, che include il simbolo bianco $\star$ e due simboli speciali Δ, ∇ per marcare l'inizio e la fine della parola in ingresso;
- B è l'alfabeto di input;
- δ è la funzione di transizione che associa a ogni coppia $(q, a) \in Q \times A$ un insieme, potenzialmente vuoto, di terne $(q', a', x) \in Q \times A \times \{-1, +1\}$;
- q_0 è lo stato iniziale;
- $F \subseteq Q$ è l'insieme degli stati finali.

La funzione di transizione δ è inoltre soggetta ai seguenti vincoli:

1. se $(q', a', x) \in \delta(q, \Delta)$ allora $a' = \Delta$ e $x = +1$,
2. se $(q', a', x) \in \delta(q, \nabla)$ allora $a' = \nabla$ e $x = -1$,

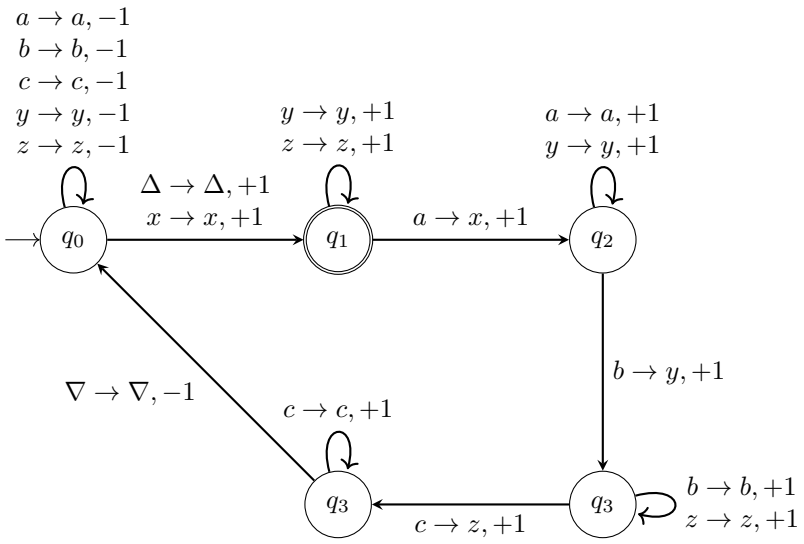
Se la funzione di transizione δ associa al più una terna (q', a', x) a ogni coppia (q, a) allora si tratta di un *automa lineare limitato deterministico* (ALD, o in inglese DLBA).

In altre parole, la testina di un automa lineare limitato non può sovrascrivere i simboli marcatori di inizio e fine parola, e non può spostarsi nelle parti del nastro all'infuori di questi marcatori. Con queste restrizioni si limita il funzionamento di questi automi a un nastro che, al contrario degli automi di Turing, non è illimitato ma confinato alla lunghezza della parola di input più le due celle con i simboli di inizio e fine parola. A parte per questa differenza, il loro funzionamento è del tutto analogo a quello dell'automa di Turing, ed hanno totale libertà di spostare la testina per leggere e rileggere a piacimento la porzione di nastro a cui hanno accesso.

Definizione 4.22. Il linguaggio riconosciuto da un automa lineare limitato non deterministico M, che indichiamo con $L(M)$ è l'insieme di tutte le parole sull'alfabeto B tali che, una volta trascritte sul nastro delimitate a sinistra da Δ e a destra da ∇, esiste una sequenza di passi che porta l'automa dallo stato iniziale q_0 e la testina posizionata su Δ a uno stato finale con la testina posizionata su ∇.

Vediamo un esempio di automa lineare limitato. Per rappresentarli graficamente possiamo usare lo stesso grafo di transizione utilizzato per rappresentare le macchine di Turing, con l'unica differenza che contrassegneremo alcuni stati come finali, come del resto è anche il caso per gli automi di Turing.

Esempio 4.19. Definiamo un automa lineare limitato deterministico che riconosca il linguaggio $L = \{a^n b^n c^n \mid n \in \mathbb{N}, n \geq 1\}$. Per esempio l'automa $M = (Q, A, B, \delta, q_0, F)$ dove $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $A = \{a, b, c, x, y, z, \star, \Delta, \nabla\}$, $B = \{a, b, c\}$, $F = \{q_1\}$ e le cui transizioni sono evidenziate dal seguente grafo di transizione:



Mentre gli automi a stati finiti deterministici e non deterministici sono equivalenti e gli automi a pila non deterministici sono più espressivi della controparte deterministica, nel caso degli automi lineari limitati la questione dell'equivalenza tra la versione non deterministica e quella deterministica è ancora priva di risposta. Ciò che è noto finora è che gli automi lineari limitati deterministici sono quantomeno in grado di riconoscere tutti i linguaggi liberi dal contesto e che ci sono linguaggi dipendenti dal contesto che essi sono in grado di riconoscere. Per questo motivo, il prossimo teorema riguarda esclusivamente gli automi lineari limitati non deterministici.

Teorema 4.6. L è un linguaggio dipendente dal contesto se e solo se è riconosciuto da un automa lineare limitato non deterministico.

Dimostrazione. Mostriamo solo, a grandi linee, come da una grammatica G di tipo 1 sia possibile costruire un automa lineare non deterministico M tale che $L(M) = L(G)$. Supponiamo che M abbia a disposizione come nastro di ingresso un nastro a 2 tracce: la prima traccia contiene la parola da riconoscere delimitata dai simboli Δ, ∇, mentre la seconda traccia si ottiene dalla prima mantenendo solo i delimitatori e cancellando tutto il resto. Quindi per esempio il nastro di ingresso per la parola *aabbcc* potrebbe essere rappresentato come:

Δ	a	a	b	b	c	c	∇
Δ	$\star$	$\star$	$\star$	$\star$	$\star$	$\star$	∇

Ad ogni passo facciamo in modo che l'automa legga contemporaneamente i simboli della prima e della seconda traccia, quindi nell'esempio precedente la prima lettura dal nastro consiste nella coppia di simboli $(a, \star)$.
Costruiamo l'automa M in modo da compiere le seguenti operazioni:

1. Copia il simbolo iniziale di G sulla prima cella libera della seconda traccia;
2. Seleziona una sottostringa α di simboli dalla seconda traccia tale che $\alpha \to \beta$ sia una produzione di G;
3. Sostituisce α con β, spostando se necessario tutti i simboli alla destra di α. Se questo passaggio comporta il superamento del delimitatore di destra, l'automa si arresta;
4. M decide non deterministicamente se tornare al punto 2 o proseguire al punto 5;
5. A questo punto la seconda traccia contiene una parola γ tale che $S \longrightarrow_G^\star \gamma$. Ora M confronta γ con la parola sulla prima traccia e si arresta in uno stato finale se le due stringhe coincidono. Altrimenti si arresta in uno stato non finale.

Utilizzando questo procedimento, se $w \in L(G)$ allora esiste almeno una sequenza di passi che porta M a costruire sulla seconda traccia la stessa parola sulla prima traccia, e viceversa, se M riconosce una parola w allora esiste una derivazione di w da S in G. □

4.3 Esercizi

Esercizio 4.1
Stabilire il linguaggio generato dalle seguenti grammatiche:

1. $G_1 = (N = \{S, X\}, T = \{a, b, c\}, \mathcal{P}, S)$ con produzioni
$$\begin{aligned} S &\longrightarrow X \mid \lambda \\ X &\longrightarrow aXa \mid bXb \mid c \end{aligned}$$
2. $G_2 = (N = \{S, X\}, T = \{a, b, c\}, \mathcal{P}, S)$ con produzioni
$$\begin{aligned} S &\longrightarrow X \mid \lambda \\ X &\longrightarrow aXa \mid Y \\ Y &\longrightarrow bYb \mid c \end{aligned}$$
3. $G_3 = (N = \{S, A, B\}, T = \{a, b\}, \mathcal{P}, S)$ con produzioni
$$\begin{aligned} S &\longrightarrow A \mid bB \\ X &\longrightarrow bA \mid aB \mid a \\ Y &\longrightarrow bB \mid b \mid a \end{aligned}$$

Esercizio 4.2

Definire una grammatica regolare e un automa a stati finiti che, rispettivamente, generino e riconoscano il linguaggio sulle cifre binarie di tutte le stringhe di zeri e uni alternati.

Esercizio 4.3

Definire una grammatica libera dal contesto e un automa a pila che, rispettivamente, generino e riconoscano il linguaggio

$$L = \{0^n 1^m 0^{m+n} \mid m, n \geq 0\}$$

Soluzioni

Soluzione 4.1

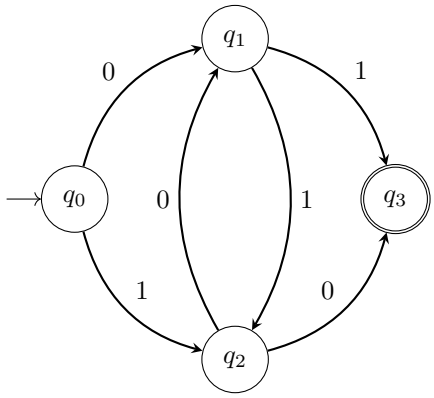
1. $L(G_1) = \{\alpha \cdot c \cdot \alpha^{rev} \mid \alpha \in \{a, b\}^\star\} \cup \{\lambda\}$
2. $L(G_2) = \{a^n b^m c b^m a^n \mid n \geq 0,\ m \geq 0\} \cup \{\lambda\}$
3. $L(G_3) = \{b^n a^i b^m a^j \mid n, m \geq 0, i, j \in \{0, 1\}\} \setminus \{\lambda\}$

Soluzione 4.2

Consideriamo che una stringa di cifre binarie abbia gli zeri e gli uni alternati se contiene almeno uno zero e un uno, quindi considereremo solo stringhe di lunghezza ≥ 2. Una grammatica per questo linguaggio è $G = (N = \{S, A, B\}, T = \{0, 1\}, \mathcal{P}, S)$ con produzioni

$$\begin{aligned} S &\longrightarrow 0A \mid 1B \\ A &\longrightarrow 1B \mid 1 \\ B &\longrightarrow 0A \mid 0 \end{aligned}$$

Un automa a stati finiti (in questo caso non deterministico) che riconosca il linguaggio è

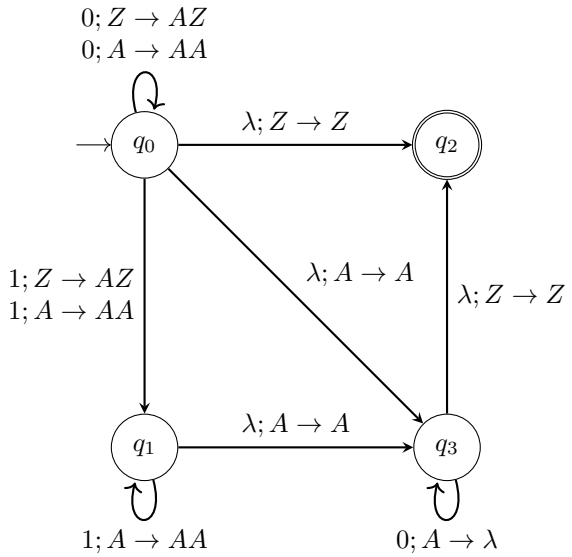


Soluzione 4.3

Una grammatica per questo linguaggio è $G = (N = \{S, A, B\}, T = \{0, 1\}, \mathcal{P}, S)$ con produzioni

$$\begin{aligned} S &\longrightarrow A \mid B \mid \lambda \\ A &\longrightarrow 0A0 \mid B \mid 00 \\ B &\longrightarrow 1B0 \mid 10 \end{aligned}$$

Un automa a pila (in questo caso non deterministico) che riconosca il linguaggio per stato finale è



Capitolo 5

Proprietà dei linguaggi formali

Abbiamo visto nel Capitolo 4 due modi per definire un linguaggio formale: attraverso le grammatiche generative e tramite gli automi riconoscitori. In questo capitolo continuiamo la nostra trattazione dei linguaggi formali descrivendo quali proprietà sono soddisfatte dalle varie classi di linguaggi che abbiamo introdotto. Così come è stata definita la gerarchia delle grammatiche di Chomsky e la

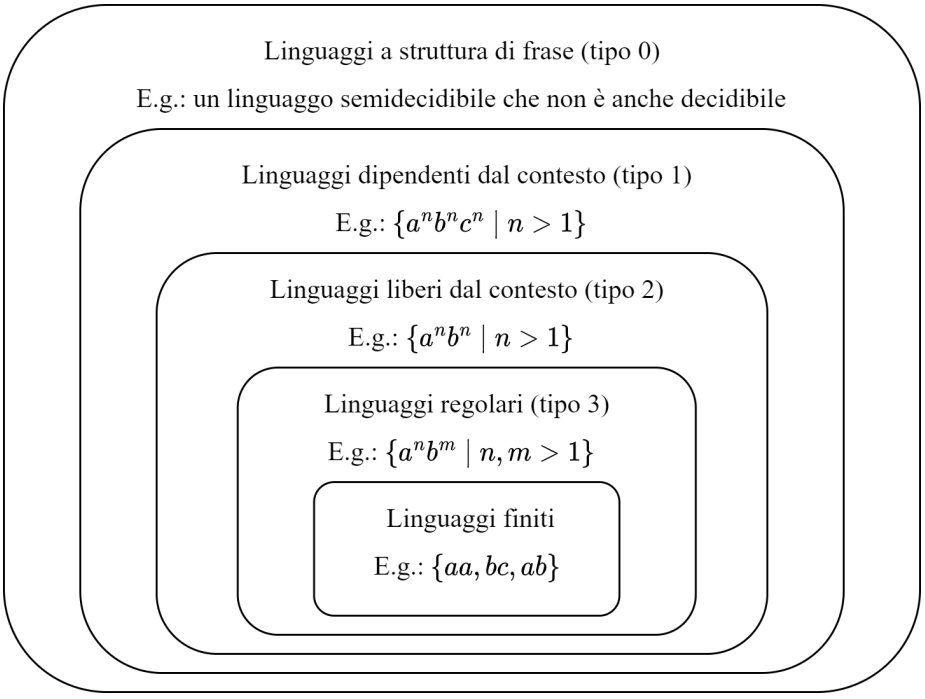


Figura 5.1: Classi di linguaggi formali

gerarchia delle classi di automi riconoscitori, esiste una analoga gerarchia anche per i linguaggi formali, come mostrato in Figura 5.1. In modo del tutto analogo

al caso delle grammatiche abbiamo che un linguaggio regolare è anche libero dal contesto, e un linguaggio libero dal contesto è anche dipendente dal contesto che a sua volta è anche un linguaggio a struttura di frase. Viceversa, esistono linguaggi a struttura di frase che non sono dipendenti dal contesto, linguaggi dipendenti dal contesto che non sono liberi, e linguaggi liberi dal contesto che non sono regolari. Infine notiamo che una sottoclasse particolare dei linguaggi regolari è quella dei *linguaggi finiti*, o *non ricorsivi*, ovvero quei linguaggi che si compongono di un numero finito di parole. Questa classe di linguaggi è generata da una cosiddetta *grammatica non ricorsiva*, cioè da una grammatica regolare dove lo stesso simbolo non terminale non occorre mai sia nella parte sinistra che la parte destra di una produzione. Questa categoria di linguaggi non è particolarmente interessante e dunque verranno considerati nel più ampio contesto dei linguaggi regolari.

5.1 Linguaggi regolari

I linguaggi regolari sono tutti quei linguaggi che sono generabili da una grammatica di tipo 3, o riconosciuti da un automa a stati finiti. Mostriamo innanzitutto un esempio di linguaggio regolare.

Esempio 5.1. Il linguaggio

$$L = \{a^n b^m \mid n, m \geq 1\}$$

è regolare. Ciò si può verificare facendo riferimento all'Esempio 4.7 per la grammatica regolare che lo genera o all'Esempio 4.12 per l'automa a stati finiti che lo riconosce.

5.1.1 Pumping lemma per i linguaggi regolari

Un risultato fondamentale noto come *Teorema uvw*, o *Pumping lemma* (quest'ultima denominazione inglese è ormai invalsa anche nell'uso italiano) stabilisce una condizione che un linguaggio regolare deve necessariamente soddisfare, e dunque esclude dai linguaggi regolari tutti quelli incapaci di rispettarla.

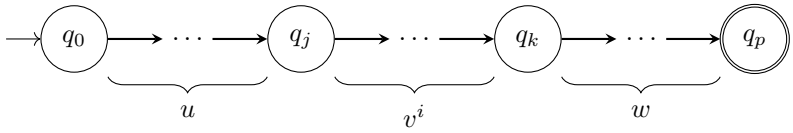
Teorema 5.1 (Pumping lemma per i linguaggi regolari)**.** Sia L un linguaggio regolare su un alfabeto T. Esiste una costante $p \in \mathbb{N}$ (detta *pumping length*) che dipende solo da L e tale che, per ogni $z \in L$, se $l(z) \geq p$, allora esistono stringhe $u, v, w \in T^\star$ tali per cui $z = uvw$ e valgono le seguenti proprietà:

(a) $l(v) \geq 1$;

(b) $l(uv) \leq p$;

(c) $\forall i \in \mathbb{N}, uv^i w \in L$

Dimostrazione. Sia L un linguaggio regolare e sia $M = (Q, A, \delta, q_0, F)$ l'automa a stati finiti deterministico che lo riconosce. Fissiamo p come il numero degli stati di M, $p = |Q|$. Per una qualunque stringa z lunga almeno p, sia q_0 lo stato iniziale dell'automa e $q_1, \ldots, q_p$ la sequenza dei successivi p stati visitati mentre la stringa viene letta dall'automa. Dato che M ha solo p stati, nella sequenza degli stati visitati deve esserci necessariamente uno stato q_s che si ripete più di una volta (quindi l'automa contiene un ciclo). Siano j, k con $j \neq k$ l'indice della prima e della seconda occorrenza di q_s nella sequenza degli stati visitati durante la lettura di z. Le transizioni che l'automa esegue tra il passo j e il passo k riconoscono una certa sottostringa di z, che nel Pumping lemma indichiamo con v. La sottostringa che viene letta prima del passo j corrisponde a u e la sottostringa letta dopo il passo k corrisponde a w. Quindi $z = uvw$.



Mostriamo che le tre condizioni del teorema sono soddisfatte:

(a) Dato che $j \neq k$, abbiamo $l(v) \geq 1$;

(b) Sappiamo che $k \leq p + 1$ dato che $l(z) \geq p$. Inoltre $l(uv) = k - 1$ per come abbiamo scelto u e v. Dunque $l(uv) = k - 1 \leq p$;

(c) Se prendiamo $i = 0$ otteniamo la stringa $uv^0w = uw$ che viene accettata dall'automa ignorando il ciclo che ha generato v. Per $i > 1$ invece la stringa che si ottiene viene accettata dall'automa ripetendo il ciclo altrettante volte.

□

Ribadiamo che il Pumping lemma fornisce soltanto una condizione necessaria perché un linguaggio sia regolare. Esso non può essere quindi utilizzato per mostrare la regolarità di un linguaggio, ma semmai per escludere che un linguaggio sia regolare, come il seguente esempio ci mostra.

Esempio 5.2. Il linguaggio

$$L = \{a^n b^n \mid n \geq 1\}$$

non è regolare.
Applichiamo il Pumping lemma assumendo che L sia regolare. Sia $p \in \mathbb{N}$ la pumping length di L. Prendiamo dunque la stringa $z = a^p b^p$. Si ha che $z \in L$ e $l(z) = 2p \geq p$, quindi si può scrivere $z = uvw$ per qualche u, v, w come descritto dal teorema.
Dato che $l(v) \geq 1$, v non può essere una stringa vuota, e quindi consiste di almeno un simbolo. Inoltre, dato che $l(uv) \leq p$, v può consistere solamente di a, questo perché i primi p simboli in z sono proprio simboli a. Fatte queste premesse, che corrispondono ai punti (a) e (b) del teorema, notiamo al punto (c) che la stringa uv^2w avrebbe necessariamente più

simboli a che b: v^2 infatti è la concatenazione di v con se stessa e quindi v^2 ha il doppio delle a di v. Quindi la stringa uv^2w non appartiene al linguaggio (perché ha più a che b), violando la condizione (c) del Pumping lemma. Quindi concludiamo che L non è un linguaggio regolare.
Notiamo che abbiamo scelto 2 come esponente nel passaggio finale della dimostrazione, ma avremmo potuto, almeno in questo caso, utilizzare anche altri valori: si sarebbe potuto giungere alla stessa conclusione anche "pompando" più a tramite $v^3, v^4, \ldots$. Inoltre sarebbe stato anche possibile rimuovere a tramite v^0, arrivando alla conclusione che uv^0w non appartiene al linguaggio perché contiene meno a che b.

Si può illustrare l'applicazione del Pumping lemma per la dimostrazione della non regolarità di un linguaggio L come un gioco tra se stessi e un "demone" che fa di tutto per ostacolarci. Questo gioco lo si può vedere in quattro passi:

1. Il demone sceglie un valore per p;
2. Noi scegliamo una stringa z di L che sia lunga almeno p;
3. Il demone divide z in tre sottostringhe u, v, w tali che $l(v) \geq 1$ e $l(uv) \leq p$;
4. Noi scegliamo un naturale i. Se $uv^iw \notin L$ allora abbiamo vinto, altrimenti se $uv^iw \in L$ vuol dire che abbiamo perso.

Seguendo le regole di questo gioco, un linguaggio non è regolare se esiste una strategia vincente per il giocatore. Se tale strategia non esiste, il linguaggio potrebbe o non potrebbe essere regolare. Questo gioco mette in evidenza due questioni importanti:

- Il demone è libero di suddividere la stringa z a proprio piacimento nel punto 3, e lo farà per rendere il nostro compito al punto 4 il più difficile possibile. Crucialmente, non abbiamo nessun controllo sulla suddivisione che il demone sceglierà.
- Una scelta appropriata di z al punto 2 può limitare le mosse del demone al punto 3 e semplificare il nostro compito al punto 4.

Come si evince da quanto detto finora, esistono linguaggi non regolari che soddisfano le condizioni del Pumping lemma. Vediamone un esempio.

Esempio 5.3. Consideriamo il linguaggio

$$L = \{ab^nc^n \mid n \geq 0\} \cup \{a^ks \mid k \neq 1 \wedge s \in \{a,b,c\}^\star \text{ e } w \text{ non inizia per } a\}$$

Questo linguaggio soddisfa le condizioni del Pumping lemma, infatti per ogni stringa lunga almeno p:

- Stringhe della forma a^0s, quindi che non hanno nessuna a nel loro prefisso, possono essere pompate in qualsiasi maniera.
- Stringhe della forma ab^kc^k possono essere pompate impostando $u = \lambda, v = a, w = b^kc^k$.

- Stringhe della forma a^2s possono essere pompate impostando $u = \lambda, v = aa, w = s$.
- Stringhe della forma a^3s o con più a iniziali possono essere pompate impostando $u = \lambda, v = a, w = aas$.

Per dimostrare la non regolarità di L si deve quindi ricorrere ad altre tecniche di dimostrazione.

5.1.2 Proprietà di chiusura

Una proprietà notevole dei linguaggi regolari è la loro capacità di preservarsi rispetto alle usuali operazioni su linguaggi.

Proposizione 5.1. La classe dei linguaggi di tipo 3 è chiusa rispetto alle operazioni di unione, concatenazione e chiusura di Kleene.

Dimostrazione. Ci limitiamo a considerare il caso dell'unione. Le altre operazioni di concatenazione e chiusura di Kleene si trattano in modo analogo e sono lasciate al lettore come esercizio. Consideriamo dunque il caso dell'unione.
Siano $G_1 = (N_1, T_1, \mathcal{P}_1, S_1)$ e $G_2 = (N_2, T_2, \mathcal{P}_2, S_2)$ due grammatiche regolari, L_1 e L_2 i linguaggi che esse rispettivamente generano. Possiamo assumere per semplicità $N_1 \cap N_2 = \varnothing$, altrimenti si provvede a rinominare i simboli non terminali in comune tra le due grammatiche in modo da disgiungerne gli insiemi; l'operazione è possibile senza che i simboli terminali di G_1 e G_2 (quelli che interessano L_1 e L_2) abbiano a risentirne sostanzialmente. Vogliamo trovare una grammatica regolare $G_3 = (N_3, T_3, \mathcal{P}_3, S_3)$ tale che $L(G_3) = L_1 \cup L_2$. A tale scopo, possiamo anzitutto porre $N_3 = N_1 \cup N_2 \cup \{S_3\}$, $T_3 = T_1 \cup T_2$. A questo punto, la strategia più immediata per generare le stringhe di L_1 o di L_2 pare quella di aggiungere in $\mathcal{P}_3$ alle produzioni di $\mathcal{P}_1 \cup \mathcal{P}_2$

$$S \longrightarrow S_1 \mid S_2$$

In tal modo, infatti, S_3 dà origine, tramite S_1 o S_2, a tutte le stringhe generate da G_1 o da G_2. Ma $S3 \longrightarrow S_1 \mid S_2$ non è ammessa nelle grammatiche regolari, che richiedono espressamente un simbolo terminale a destra di ogni produzione. Così dobbiamo procedere in modo lievemente più lungo, accogliendo in $\mathcal{P}_3$ le produzioni in $\mathcal{P}_1$, quelle in $\mathcal{P}_2$ e in più $S_3 \longrightarrow \gamma$ se $S_1 \longrightarrow \gamma \in \mathcal{P}_1$ oppure $S_2 \longrightarrow \gamma \in \mathcal{P}_2$. Non è difficile convincersi che la grammatica G_3 così ottenuta è regolare e che il linguaggio generato da G_3 è l'unione di L_1 e L_2. □

Teorema 5.2. La classe dei linguaggi di tipo 3 è chiusa rispetto all'operazione di complementazione.

Dimostrazione. Sia $M = (Q, A, \delta, q_0, F)$ l'automa a stati finiti che riconosce il linguaggio regolare L. Il linguaggio complementare $\bar{L}$ è l'insieme di tutte

quelle parole per le quali l'automa si arresta in uno stato non finale. Dunque l'automa riconoscitore del linguaggio complementare di L è l'automa $M' = (Q, A, \delta, q_0, Q - F)$ i cui stati finali sono gli stati non finali di M. Quindi $\bar{L}$ è regolare. □

Teorema 5.3. La classe dei linguaggi di tipo 3 è chiusa rispetto all'operazione di intersezione.

Dimostrazione. La chiusura rispetto all'intersezione segue invece direttamente dalla chiusura rispetto all'unione e alla complementazione: infatti è ben noto che l'intersezione di due insiemi è il complementare dell'unione dei loro complementari □

Esempio 5.4. Tramite i risultati di questa sezione siamo in grado di dimostrare la non regolarità del linguaggio dell'Esempio 5.3, cioè

$$L = \{ab^n c^n \mid n \geq 0\} \cup \{a^k s \mid k \neq 1 \wedge s \in \{a, b, c\}^\star \text{ e } w \text{ non inizia per } a\}$$

Se assumiamo per assurdo che questo linguaggio sia regolare, per il Teorema 5.3 abbiamo che anche il linguaggio $L' = L \cap ab\{a, b, c\}^\star = \{ab^n c^n \mid n \geq 1\}$ sia a sua volta regolare. Tuttavia si può facilmente dimostrare, proprio usando il Pumping lemma e seguendo un ragionamento del tutto analogo a quello dell'Esempio 5.2, che questo linguaggio non è regolare. Quindi anche L non è regolare.

5.1.3 Proprietà decidibili e indecidibili

Altre applicazioni del Pumping lemma rivelano importanti proprietà di decidibilità dei linguaggi regolari. Discutiamone una.

Proposizione 5.2. Siano G una grammatica di tipo 3, $L = L(G)$ il corrispondente linguaggio regolare. Sia poi p il numero dei simboli non terminali di G e cioè la costante associata dal Teorema uvw a L. Allora

(a) L non è vuoto se e solo se G genera una stringa z di lunghezza $< p$.

(b) L è infinito se e solo se G genera una stringa z tale che $p \leq l(z) < 2p$.

Come vedremo nel prossimo paragrafo, la proposizione si può estendere opportunamente anche alle grammatiche e ai linguaggi liberi dal contesto. Ne presentiamo comunque la dimostrazione nel caso regolare, per illustrarvi il ruolo del Teorema uvw.

Dimostrazione. Valutiamo entrambi i punti (a) e (b):

(a) È chiaro che, se G genera qualche stringa z, allora, a prescindere dalla lunghezza di z, $L = L(G)$ non è vuoto. Viceversa assumiamo che L non sia vuoto ma che tutte le stringhe di L abbiano lunghezza $\geq p$. Sia poi z

una stringa di lunghezza minima in L. Poiché $l(z) \geq p$, per il Teorema uvw esistono stringhe u, v e w tale che $z = uvw$, $l(v) \geq 1$, $l(uv) \leq p$ e, per ogni naturale i, $uv^iw \in L$. In particolare $uv^0w = uw \in L$; ma uw ha lunghezza inferiore a $z = uvw$, e questo contraddice la scelta di z. Allora L, quando non è vuoto, deve contenere stringhe di lunghezza $< p$.

(b) Supponiamo dapprima che G generi una stringa z tale che $p \leq l(z) < 2p$. Per il Teorema uvw ci sono u, v, w nell'alfabeto T dei simboli terminali di G tali che $z = uvw$, $v \neq \lambda$ e ogni stringa uv^iw (per i naturale) è in L. Ovviamente $uv^iw \neq uv^jw$ per $i \neq j$. Dunque L ha infiniti elementi. Viceversa, assumiamo L infinito. Siccome il numero delle stringhe di fissata lunghezza su T è comunque finito, L contiene elementi di lunghezza arbitrariamente grande. Sia in particolare z una stringa in L di lunghezza minima tra quelle di lunghezza $\geq p$. Se $l(z) < 2p$ allora abbiamo la stringa che stiamo cercando. Altrimenti applichiamo il Teorema uvw a z come nel caso precedente e troviamo u, v e w in $T^\star$ con $l(v) \geq 1$, $l(uv) \leq p$, $z = uvw$ e $uv^iw \in L$ per ogni naturale i. In particolare $uv^0w = uw \in L$; inoltre $l(uw) \geq p$ perché $l(z) \geq 2p$ e $l(v) \leq p$; finalmente $l(uw) < l(z)$ perché $l(v) \geq 1$. Ma così si contraddice la scelta di z.

□

Come già osservato nel corso della precedente dimostrazione, le stringhe di $T^\star$ di una prefissata lunghezza l sono in numero finito, e dunque si possono enumerare e decidere effettivamente. Lo stesso vale ovviamente anche per stringhe di lunghezza $\leq l$, o compresa tra l e $2l$, per ogni l. Consideriamo allora i problemi di decidere, per ogni grammatica regolare G,

(a) se $L(G)$ è vuoto o no,

(b) se $L(G)$ è finito o no.

In entrambi i casi, una strategia di soluzione consiste nel calcolare anzitutto il numero p dei simboli non terminali di G e poi, se T denota l'alfabeto dei simboli terminali di G, considerare una per una

(a) le stringhe $z \in T^\star$ con $l(z) < p$,

(b) le stringhe $z \in T^\star$ con $p \leq l(z) < 2p$

cercando di stabilire, per ognuna di esse, se z è o non è in $L(G)$. Se l'esame dà esito positivo almeno una volta, allora

(a) $L(G) \neq \varnothing$,

(b) $L(G)$ è infinito,

altrimenti

(a) $L(G) = \varnothing$,

(b) $L(G)$ è finito.

Dunque i due problemi hanno algoritmo di soluzione **purché** ci sia un procedimento effettivo che sappia decidere, per ogni $z \in T^\star$, se $z \in L(G)$ oppure no, in altre parole purché $L(G)$ sia decidibile. Vedremo nella Sezione 5.3 che in realtà questo è vero non solo per le grammatiche regolari, ma anche per le grammatiche di tipo 1. Se dunque il lettore ha sufficiente fiducia per accettare sin d'ora questo risultato e sufficiente pazienza per aspettarne la dimostrazione, potrà ammettere senza problemi:

Teorema 5.4. Ci sono algoritmi effettivi che sanno decidere, per ogni linguaggio regolare L,

(a) se L è vuoto oppure no,

(a) se L è finito oppure no,

Dalle proprietà di chiusura dei linguaggi regolari segue direttamente la decidibilità del *problema dell'equivalenza tra grammatiche regolari*, ovvero della questione se due grammatiche regolari generano o no lo stesso linguaggio. Si ha infatti:

Corollario 5.1 (Problema dell'Equivalenza di Grammatiche Regolari)**.** C'è un algoritmo che sa decidere, per ogni scelta di grammatiche regolari G_1 e G_2 se $L(G_1) = L(G_2)$ oppure no.

Dimostrazione. La teoria degli insiemi ci dice che $L(G_1) = L(G_2)$ se e solo se $L' = (L(G_1) \cap \overline{L(G_2)}) \cup (\overline{L(G_1)} \cap L(G_2))$ è vuoto. Dalle proprietà di chiusura dei linguaggi regolari possiamo concludere che anche L' è regolare, e dunque per il Teorema 5.4 possiamo stabilire se L' è vuoto oppure no. □

5.1.4 Complessità dell'analisi sintattica

L'analisi sintattica, meglio nota col termine inglese *parsing*, consiste nel riconoscere se la parola di input appartiene o meno a un certo linguaggio. L'analisi dei linguaggi regolari consiste nella simulazione del corrispondente automa a stati finiti deterministico, e quindi il numero di passi richiesto è proporzionale alla lunghezza della parola in ingresso. Infatti il numero di transizioni effettuate dall'automa è al più pari al numero di simboli della stringa in questione.

5.1.5 Espressioni regolari

Le espressioni regolari sono un formalismo alternativo per la rappresentazione dei linguaggi regolari allo stesso modo delle grammatiche regolari e degli automi a stati finiti. Se le grammatiche sono l'approccio generativo e gli automi quello riconoscitivo, le espressioni regolari forniscono una definizione dei linguaggi regolari dal punto di vista *algebrico*. Dedichiamo questa sezione a questa ulteriore caratterizzazione dei linguaggi regolari, concludendo con la dimostrazione della loro equivalenza con le grammatiche di tipo 3 e gli ASFND.

Definiamo la *sintassi* delle espressioni regolari su un certo alfabeto $A = \{a_0, a_1, \ldots, a_n\}$ tramite una grammatica libera dal contesto $G = (N = \{E\}, T = \{\varnothing, \lambda, +, \star, \cdot,), (\} \cup A, \mathcal{P}, E)$ con le seguenti produzioni:

$$E \longrightarrow \varnothing \mid \lambda \mid (E + E) \mid (E \cdot E) \mid (E)^\star$$
$$E \longrightarrow a_0 \mid a_1 \mid \cdots \mid a_n$$

Il linguaggio generato da questa grammatiche costituisce l'insieme delle espressioni regolari sull'alfabeto A.

Esempio 5.5. Sono espressioni regolari sull'alfabeto $A = \{a, b\}$:

- a
- $(a \cdot b)$
- $((a \cdot a) + b)$
- $((\lambda + a) \cdot (b)^\star)$

Adottiamo alcune convenzioni per semplificare la scrittura delle espressioni regolari:

- Definiamo un ordine di priorità in modo che $\star$ abbia la precedenza su $\cdot$ che a sua volta ha la precedenza su $+$, in questo modo possiamo omettere le parentesi quando sono superflue;
- Omettiamo il simbolo $\cdot$;
- Scriviamo a^n per indicare $a \cdot a \cdots a$ dove il simbolo a viene ripetuto n volte.

Esempio 5.6. Le espressioni regolari del precedente esempio possono essere riscritte come:

- a
- ab
- $a^2 + b$
- $(\lambda + a)b^\star$

Definiamo ora la semantica delle espressioni regolari. Il significato di una espressione regolare e è il linguaggio associato $L[e]$ definito ricorsivamente applicando le seguenti regole:

$$L[\varnothing] = \varnothing$$
$$L[\lambda] = \{\lambda\}$$
$$L[a] = \{a\}, \forall a \in A$$
$$L[e + e'] = L[e] \cup L[e']$$
$$L[e \cdot e'] = L[e]L[e']$$
$$L[e^\star] = L[e]^\star$$

Esempio 5.7. Le espressioni regolari del precedente esempio corrispondono ai seguenti linguaggi:

- $L[a] = \{a\}$
- $L[ab] = L[a]L[b] = \{a\}\{b\} = \{ab\}$
- $L[a^2 + b] = L[a^2] \cup L[b] = \{aa\} \cup \{b\} = \{aa, b\}$
- $L[(\lambda + a)b^\star] = L[\lambda + a]L[b^\star] = \{\lambda, a\}\{b\}^\star = \{\lambda, a, b, ab, b^2, ab^2, \ldots\}$

Sulle espressioni regolari, interpretate come linguaggi, si possono dimostrare alcune equivalenze algebriche che saranno utili nel seguito:

$$
\begin{array}{ll}
e_1(e_2 + e_3) = e_1e_2 + e_1e_3 & (e_1 + e_2)e_3 = e_1e_3 + e_2e_3 \\
e_1 + (e_2 + e_3) = (e_1 + e_2) + e_3 & e_1(e_2e_3) = (e_1e_2)e_3 \\
e\lambda = \lambda e = e & e\varnothing = \varnothing e = \varnothing \\
e + e = e & e + \varnothing = e \\
e^\star + e = e^\star & (e^\star)^\star = e^\star \\
(e_1 + e_2)^\star = (e_1^\star e_2^\star)^\star & \varnothing^\star = \lambda
\end{array}
$$

Teorema 5.5. I linguaggi generati da grammatiche regolari sono rappresentabili tramite espressioni regolari.

Dimostrazione. Costruiamo a partire da una grammatica regolare $G = (N, T, \mathcal{P}, S)$ un sistema lineare di equazioni in cui a ogni simbolo non terminale $A_i \in N$ viene assegnata una variabile X_i. Mostriamo poi un metodo per risolvere questo tipo di sistemi. La soluzione di un sistema di questo tipo assegnerà una espressione regolare r_i a ogni variabile X_i, e l'espressione regolare che rappresenta il linguaggio generato da G sarà quella associata al simbolo iniziale di G.
Per ogni $A_i \in N$, siano:

- $A_{i_1}, \ldots A_{i_h}$ i simboli non terminali (eventualmente anche proprio A_i) tali per cui esiste una produzione $A_i \longrightarrow a_{i_j} A_{i_j}$ per $a_{i_j} \in T$.
- $c_{i_1}, \ldots, c_{i_k}$ i simboli terminali, oppure λ, tali per cui esiste una produzione $A_i \longrightarrow c_{i_j}$

Dunque ad ogni simbolo non terminale A_i di G assegniamo l'equazione:

$$X_i = a_{i_1} X_{i_1} + \ldots + a_{i_h} X_{i_h} + c_{i_1} + \ldots + c_{i_k}$$

Il sistema di equazioni che si ottiene si può risolvere con l'aiuto delle manipolazioni algebriche possibili sulle espressioni regolari e tramite il metodo della *sostituzione*, tipicamente utilizzato per risolvere sistemi lineari, nel caso in cui X_i non occorra nella parte destra dell'equazione che lo definisce. In caso contrario, ci si può ricondurre tramite opportune manipolazioni algebriche, a una equazione della forma

$$X_i = s_1 X_i + s_2$$

per opportuni s_1 ed s_2 dove X_i non occorre. La soluzione di una equazione del genere è $X_i = s_1^\star s_2$, poiché:

$$s_1(s_1^\star s_2) + s_2 = s_1^+ s_2 + s_2 = (s_1^+ + \lambda)s_2 = s_1^\star s_2$$

A questo punto il sistema si può risolvere sostituendo X_i con $s_1^\star s_2$. □

Esempio 5.8. Consideriamo la grammatica $G = (N = \{S, A\}, T = \{a, b\}, \mathcal{P}, S)$ con le seguenti produzioni:

$$S \longrightarrow aS \mid bA \mid a$$
$$A \longrightarrow bS \mid bA \mid b$$

Da questa grammatica si ottiene il sistema

$$X_S = aX_S + bX_A + a$$
$$X_A = bX_S + bX_A + b$$

Dato che possiamo considerare la seconda equazione come $X_A = bX_A + (bX_S + b)$, la sostituiamo nel sistema con $X_A = b^\star(bX_S + b)$, che diventa

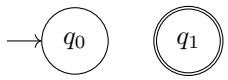
$$X_S = aX_S + bb^\star(bX_S + b) + a$$
$$X_A = b^\star(bX_S + b)$$

con le opportune riscritture abbiamo $X_S = aX_S + bb^\star bX_S + bb^\star b + a = (a + bb^\star b)X_S + bb^\star b + a$. Ripetiamo lo stesso procedimento con X_S e otteniamo $X_S = (a+bb^\star b)^\star(bb^\star b+a)$. Dato che X_S è la variabile che corrisponde al simbolo iniziale di G, questa espressione regolare rappresenta il linguaggio generato da G.

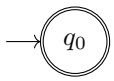
Teorema 5.6. Se α è una espressione regolare allora esiste un automa a stati finiti non deterministico M tale che $L(M)$ è l'insieme delle parole rappresentate da α

Dimostrazione. Dimostriamo il teorema mostrando come si costruisce M in maniera induttiva sulla struttura di α. Faremo in modo che ognuno degli automi viene definito per avere un solo stato iniziale e un solo stato finale. Consideriamo i tre casi base:

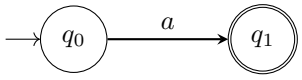
- Per $\alpha = \varnothing$ l'automa è:



- Per $\alpha = \lambda$ l'automa è:

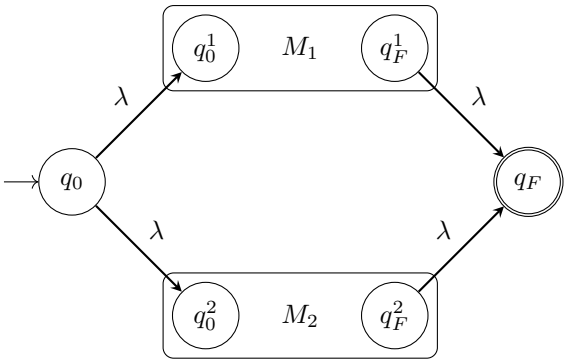


- Per $\alpha = a$ con a simbolo dell'alfabeto A l'automa è:

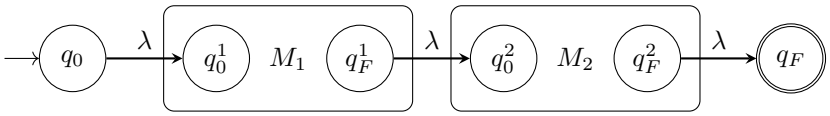


Per il passo induttivo dobbiamo considerare tre casi:

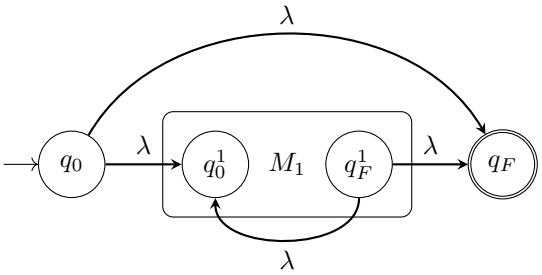
- Per $\alpha = \alpha_1 + \alpha_2$ siano M_1 ed M_2 gli automi che riconoscono il linguaggio di α_1 e α_2. Inoltre siano q_0^1, q_0^2 i rispettivi stati iniziali e q_F^1, q_F^2 gli stati finali. Il seguente automa con stato iniziale q_0 e stato finale q_F riconosce il linguaggio di $L[\alpha] = L[\alpha_1] \cup L[\alpha_2]$:



- Per $\alpha = \alpha_1 \cdot \alpha_2$ siano M_1 ed M_2 gli automi che riconoscono il linguaggio di α_1 e α_2. Inoltre siano q_0^1, q_0^2 i rispettivi stati iniziali e q_F^1, q_F^2 gli stati finali. Il seguente automa con stato iniziale q_0 e stato finale q_F riconosce il linguaggio di $L[\alpha] = L[\alpha_1]L[\alpha_2]$:

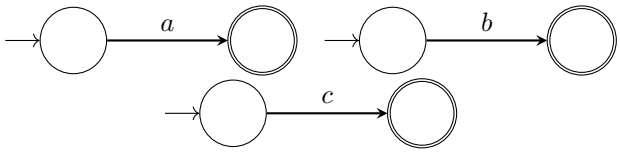


- Per $\alpha = \alpha_1^\star$ sia M_1 l'automa che riconosce il linguaggio di α_1 con stato iniziale q_0^1 e stato finale q_F^1. Il seguente automa con stato iniziale q_0 e stato finale q_F riconosce il linguaggio di $L[\alpha] = L[\alpha_1]^\star$:

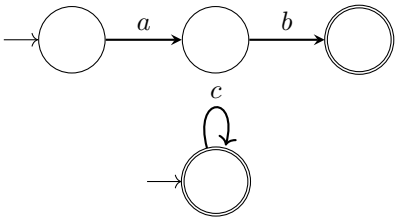


L'ASFND che otteniamo ha lambda produzioni che possono essere rimosse usando il procedimento mostrato dal Teorema 4.4. □

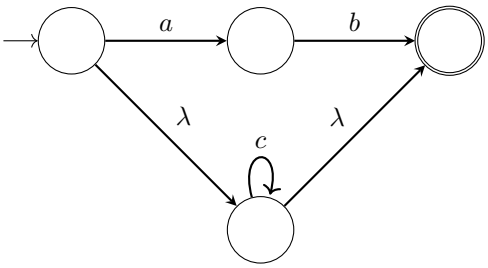
Esempio 5.9. Costruiamo l'ASFND che riconosce il linguaggio definito dall'espressione regolare $\alpha = ((ab + c^\star)b + a)$.
Iniziamo con gli automi relativi ad a, b e c:

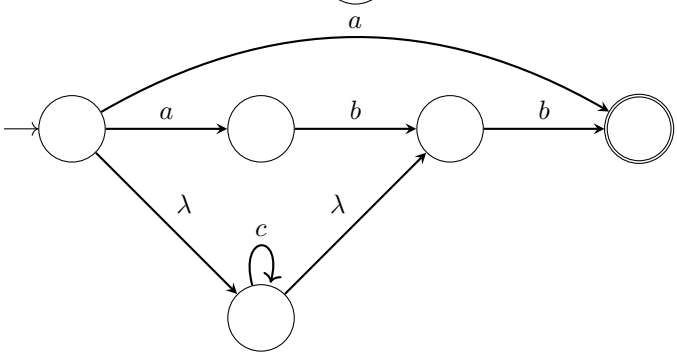


Poi costruiamo gli automi relativi ad ab e $c^\star$ (rimuovendo le λ-transizioni in maniera appropriata, cioè facendo in modo che ci sia comunque un solo stato iniziale e un solo stato finale):

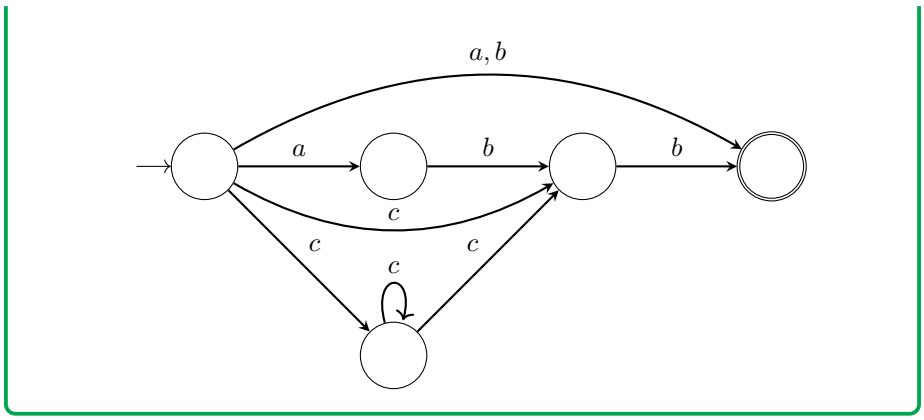


Passiamo poi ad $ab + c^\star$ e infine $((ab + c^\star)b + a)$ (rimuovendo alcune λ-transizioni per migliorare la visualizzazione):





A questo punto possiamo rimuovere le λ-transizioni rimanenti ottenendo:



Con quest'ultima dimostrazione abbiamo terminato di mostrare l'equivalenza, e al contempo gli algoritmi di conversione, tra le quattro tecniche per rappresentare i linguaggi regolari: le grammatiche regolari, gli ASFD, gli ASFND e le espressioni regolari (vedi i Teoremi 4.2, 4.3, 5.5 e 5.6). La Figura 5.2 mostra in maniera schematica questo importante risultato.

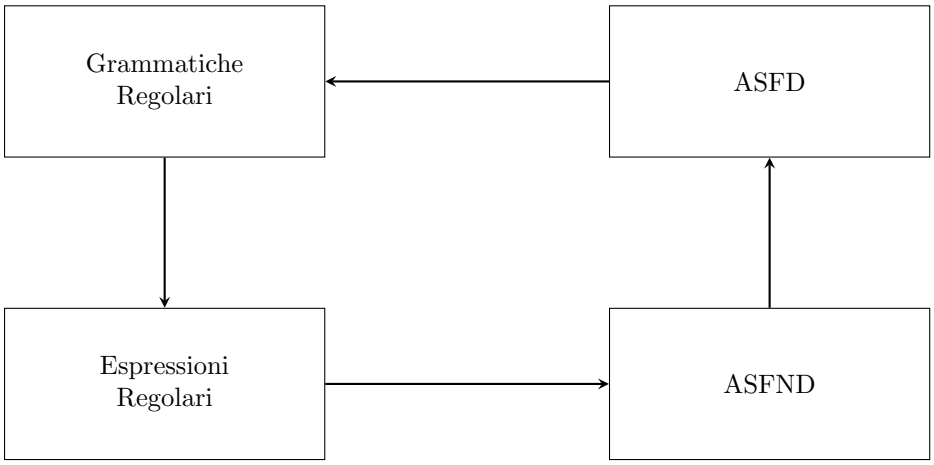


Figura 5.2: Le grammatiche regolari, le espressioni regolari e gli automi a stati finiti deterministici e non deterministici definiscono in maniera equivalente la classe dei linguaggi regolari.

5.2 Linguaggi liberi dal contesto

I linguaggi liberi dal contesto sono tutti quei linguaggi che sono generabili da una grammatica di tipo 2, o riconosciuti da un automa a pila non deterministico. Molte delle proprietà appena viste per i linguaggi regolari sono soddisfatte anche da quelli liberi dal contesto (magari con gli opportuni adattamenti); altre invece si perdono. Iniziamo col mostrare un esempio di linguaggio libero dal contesto che non sia anche regolare.

Esempio 5.10. Il linguaggio

$$L = \{a^n b^n \mid n \geq 1\}$$

è libero dal contesto. Ciò si può verificare facendo riferimento all'Esempio 4.6 per la grammatica libera dal contesto che lo genera o all'Esempio 4.17 per l'automa a pila che lo riconosce.

5.2.1 Pumping lemma per i linguaggi liberi dal contesto

Una versione del Pumping lemma per linguaggi liberi dal contesto, anche noto come *Teorema uvwxy*, è la seguente.

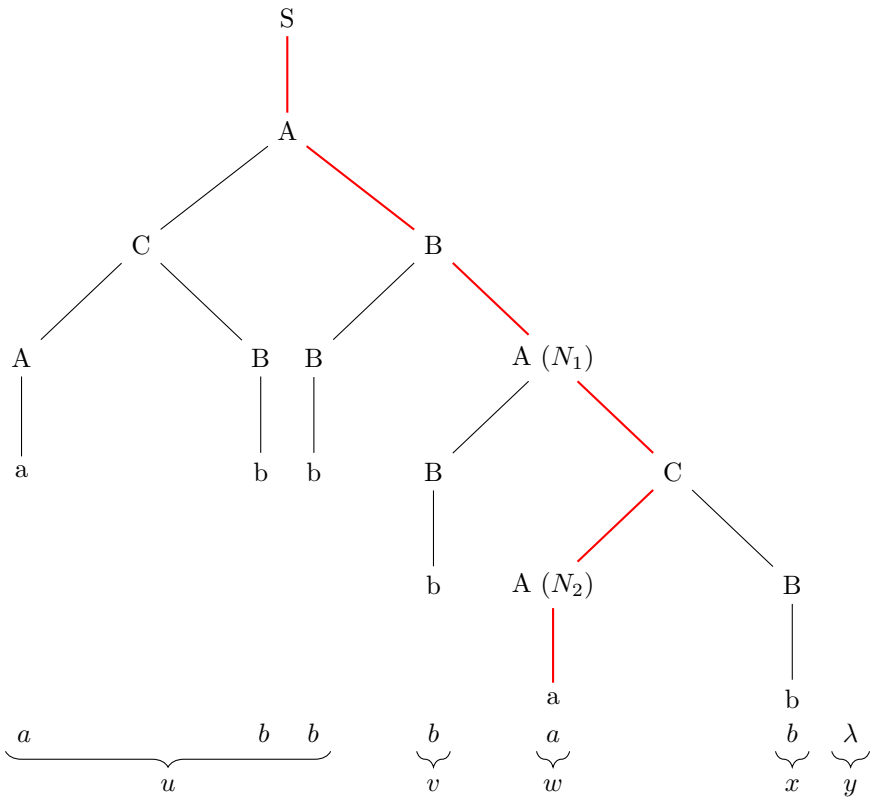
Teorema 5.7 (Pumping lemma per i linguaggi liberi dal contesto)**.** Sia L un linguaggio di tipo 2 su un alfabeto T. Allora esiste un intero positivo p dipendente da L tale che, per ogni $z \in L$, se $l(z) \geq p$, allora esistono stringhe $u, v, w, x, y \in T^\star$ tali per cui $z = uvwxy$ e valgono le seguenti proprietà:

(a) $l(vx) \geq 1$;

(b) $l(vwx) \leq p$

(c) $\forall i \in \mathbb{N}, uv^i wx^i y \in L$

Dimostrazione. Sia $G = (N, T, \mathcal{P}, S)$ una grammatica libera dal contesto per cui $L = L(G)$ e sia n il numero dei simboli non terminali di G. Consideriamo tutti gli alberi di derivazione di G che hanno l'etichetta di un simbolo non terminale come radice e ammettono profondità $\leq n + 1$ (ricordiamo che la profondità di un albero è la lunghezza massima di un suo cammino — cioè di una sua sequenza di lati — dalla radice a una foglia). Otteniamo così un numero finito di alberi: sia k la lunghezza massima delle stringhe generate dai nodi foglia di questi alberi nel modo descritto in Sezione 4.1.3. Fissiamo $p = k + 1$.
Consideriamo adesso una stringa $z \in L$ tale che $l(z) \geq p$: z può essere generata solo da un albero di derivazione (con radice S) di profondità maggiore di $n+1$. Tra tutti gli alberi di derivazione con radice S per la stringa z consideriamone uno con il numero minimo di nodi e scegliamo in questo albero un cammino (nell'esempio della figura seguente, questo cammino è evidenziato in rosso) di lunghezza maggiore di $n + 1$ da S a una foglia.



Esempio di albero di derivazione per *abbbab* con la grammatica $G = (N = \{A, B, C\}, T = \{a, b\}, \mathcal{P} = \{S \to A, A \to CB, A \to a, B \to BA, B \to b, C \to AB\}, S)$.

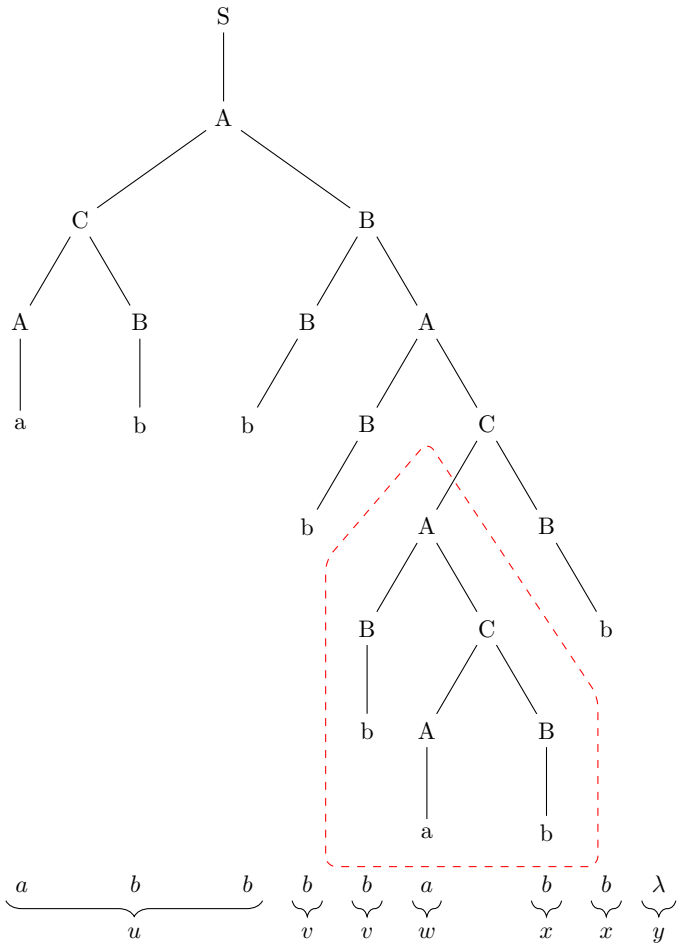
Sottoalbero di radice N_1 Sottoalbero di radice N_2

Poiché n è il numero dei simboli non terminali di G, questo cammino deve contenere almeno un simbolo non terminale che si ripete tra le etichette dei suoi nodi; in altre parole, esistono due nodi N_1 e N_2 del cammino tali che:

- N_1 e N_2 hanno la stessa etichetta non terminale, diciamo A;

- N_2 è discendente di N_1 (cioè è più vicino alla foglia);
- il cammino da N_1 alla foglia è di lunghezza al più $n+1$.

Per trovare N_1 e N_2 basta percorrere a ritroso il cammino dalla foglia alla radice alla ricerca della prima ripetizione di una etichetta A.
Il sottoalbero di radice N_2 genera una stringa $w \in T^\star$ e quindi rappresenta una derivazione $A \longrightarrow_G^\star w$. Da parte sua il sottoalbero di radice N_1, che comprende N_2 tra i suoi nodi, genera una parola che contiene w come sottostringa e la racchiude come vwx per un'opportuna scelta di v e x in $T^\star$. Allo stesso modo l'intero albero di computazione di z, che include N_1 tra i suoi nodi, genera una stringa che allarga vwx e quindi ha la forma $uvwxy$ per opportune parole u e y in $T^\star$; quindi $z = uvwxy$.
Se adesso sfruttiamo il fatto che N_1 e N_2 ammettono la stessa etichetta A e sostituiamo nell'albero scelto per la derivazione di z il sottoalbero di radice N_2 con quello di radice N_1, generiamo con le foglie la stringa che si ottiene rimpiazzando w con vwx in $z = uvwxy$, e cioè uv^2wx^2y. Si prova così che uv^2wx^2y è generata da G e quindi appartiene a L.



Albero di derivazione di uv^2wx^2y per $z = abbbab$.

Allo stesso modo, si dimostra che, per ogni $i > 0$, $uv^iwx^iy \in L$. Se invece nell'albero di z sostituiamo il sottoalbero di radice N_1 con quello di radice N_2 (di nuovo sfruttando il fatto che N_1 e N_2 hanno la stessa etichetta) proviamo che anche uwy sta in L, cioè $uv^iwx^iy \in L$ anche per $i = 0$. Questo dimostra (c).
Per provare (b), si noti anzitutto che il sottoalbero di radice N_1 ha profondità $\leq n+1$; quindi la stringa che il sottoalbero genera, e cioè vwx, ha lunghezza al più p per la definizione stessa di p.
Finalmente dimostriamo (a). Se $l(vx) = 0$, e cioè vx è vuota, z coincide con uwy e quindi continua a essere generata anche se sostituiamo nel suo albero di derivazione il sottoalbero di radice N_1 con quello di radice N_2, e dunque diminuiamo il numero complessivo dei nodi: ma questo contraddice la scelta dell'albero stesso. □

Anche per linguaggi liberi dal contesto, come già per linguaggi regolari, il Pumping lemma può essere usato per costruire esempi negativi, come il seguente.

Esempio 5.11. Il linguaggio

$$L = \{a^nb^nc^n \mid n \in \mathbb{N}, n \geq 1\}$$

non è libero dal contesto. Supponiamo infatti che lo sia. Applicando il Pumping lemma troviamo $p \in \mathbb{N}$ tale che, se $z \in L$ e $l(z) \geq p$, allora ci sono $u, v, w, x, y \in T^\star$ per cui $z = uvwxy$, $l(vx) > 0$, $l(vwx) \leq p$, e $\forall i \in \mathbb{N}, uv^iwx^ixy \in L$.
Consideriamo la stringa $z = a^pb^pc^p$. Ovviamente $z \in L$ e $l(z) = 3p \geq p$. Dato che $l(vwx) \leq p$, vwx non può contenere sia a che c. Quindi è necessariamente il caso che u includa completamente a^p oppure che y includa completamente c^p. Inoltre sappiamo che vx non può essere la stringa vuota.
Per quanto riguarda il punto (c) del teorema possiamo notare che la stringa $uv^0wx^0y = uvw$ non appartiene al linguaggio, perché contiene a^p o c^p, ma sicuramente non contiene p copie di ognuno dei tre simboli a, b, c, dato che $uvw < 3p$.

Naturalmente anche in questo caso esistono linguaggi non liberi dal contesto che soddisfano le condizioni del Pumping lemma. Vediamone un esempio.

Esempio 5.12. Consideriamo il linguaggio

$$L = \{b^jc^kd^l \mid j, k, l \geq 0\} \cup \{a^ib^jc^jd^j \mid i \geq 1, j \geq 0\}$$

Questo linguaggio soddisfa le condizioni del Pumping lemma, infatti per ogni stringa lunga almeno p:

- Stringhe della forma $b^jc^kd^l$ con $j \geq 1$, possono essere pompate impostando $u = \lambda, vwx = b^j, y = c^kd^l$.
- Stringhe della forma $a^ib^jc^jd^j$ possono essere pompate impostando $u = \lambda, vwx = a^i, y = b^jc^jd^j$.

Per dimostrare che L non è libero dal contesto si deve quindi ricorrere ad altre tecniche di dimostrazione.

5.2.2 Proprietà di chiusura

Esaminiamo adesso il comportamento dei linguaggi liberi dal contesto rispetto alla usuali operazioni tra linguaggi.

Proposizione 5.3. I linguaggi di tipo 2 sono chiusi rispetto alle operazioni di unione, concatenazione e chiusura di Kleene.

Dimostrazione. La dimostrazione è simile a quella per i linguaggi regolari, e anche più semplice. Siano L_1, L_2 due linguaggi liberi dal contesto, $G_1 = (N_1, T_1, \mathcal{P}_1, S_1)$ e $G_2 = (N_2, T_2, \mathcal{P}_2, S_2)$ le grammatiche tipo 2 che li generano. Di nuovo possiamo assumere che N_1 e N_2 sono disgiunti. Per provare che, per esempio, $L_1 \cup L_2$ è di tipo 2, prendiamo un nuovo simbolo iniziale S_3 e formiamo

$$G_3 = (N_1 \cup N_2 \cup \{S_3\}, T_1 \cup T_2, \mathcal{P}_1 \cup \mathcal{P}_2 \cup \{S_3 \longrightarrow S_1 \mid S_2\}, S_3)$$

accettiamo dunque $S_3 \longrightarrow S_1 \mid S_2$ nelle produzioni di G_3, come concesso dalla definizione di grammatica libera dal contesto anche se proibito nelle grammatiche regolari. Si verifica allora che G_3 è davvero una grammatica libera dal contesto e che genera $L_1 \cup L_2$. In modo analogo si procede per concatenazione e chiusura di Kleene. □

Ma i linguaggi di tipo 2, a differenza di quelli regolari, non si preservano per intersezione. Vediamo perché.

Proposizione 5.4. La classe dei linguaggi di tipo 2 non è chiusa rispetto all'operazione di intersezione.

Dimostrazione. Il linguaggio $L_1 = \{a^n b^n c^m \mid n, m \geq 1\}$ é generato dalla grammatica G_1 con produzioni

$$\begin{aligned} S &\longrightarrow AB \\ A &\longrightarrow ab \mid aAb \\ B &\longrightarrow c \mid cB \end{aligned}$$

e il linguaggio $L_2 = \{a^n b^m c^m \mid n, m \geq 1\}$ è generato da una grammatica G_2 con produzioni

$$\begin{aligned} S &\longrightarrow AB \\ A &\longrightarrow a \mid aA \\ B &\longrightarrow bc \mid bBc \end{aligned}$$

Si vede facilmente che le grammatiche G_1 e G_2 sono libere dal contesto, e quindi tali sono anche L_1 e L_2. Ma la loro intersezione è il linguaggio $\{a^n b^n c^n \mid n \geq 1\}$, che abbiamo visto non essere libero dal contesto. □

Ne segue:

Corollario 5.2. La classe dei linguaggi di tipo 2 non è chiusa rispetto alla complementazione.

Dimostrazione. Una classe chiusa per unione e complementazione lo è anche per intersezione, visto che l'intersezione di due insiemi è il complemento dell'unione dei loro complementi. □

5.2.3 Proprietà decidibili e indecidibili

Il Pumping lemma per i linguaggi liberi dal contesto ci permette di estendere i risultati già provati per linguaggi regolari nella Sezione 5.1.3. Anche la dimostrazione è analoga ma, al posto del Teorema uvw, sfrutta il Teorema $uvwxy$.

Proposizione 5.5. Siano G una grammatica di tipo 2, $L(G)$ il linguaggio che G genera. Sia poi p la costante associata a G dal Teorema $uvwxy$. Allora:

(a) $L(G)$ non è vuoto se e solo se G genera una stringa di lunghezza $\leq p$,

(b) $L(G)$ è infinito se e solo se G genera una stringa z tale che $p < l(z) \leq 2p$.

Dimostrazione. Valutiamo entrambi i punti (a) e (b):

(a) L'implicazione da destra a sinistra è ovvia. Viceversa supponiamo che $L(G)$ non sia vuoto, ma contenga solo stringhe di lunghezza $> p$. Sia $z \in L(G)$ la stringa di lunghezza minima (comunque $> p$). Applichiamo il Teorema $uvwxy$ ed esprimiamo appunto z come $uvwxy$ per opportune stringhe u, v, w, x, y tali che, in particolare, $l(vx) \geq 1$ e per ogni naturale i si ha $uv^iwx^iy \in L(G)$. In particolare, per $i = 0$ anche $uwy \in L(G)$. Ma $l(uwy) < l(z)$ perché $l(vx) > 0$, così uwy contraddice la scelta di z. Segue che, se $L(G) \neq \varnothing$, $L(G)$ contiene stringhe di lunghezza $\leq p$.

(b) Se $L(G)$ contiene una stringa z di lunghezza maggiore di p, allora $L(G)$ è infinito: basta applicare il Teorema $uvwxy$ per osservare che z si scrive $uvwxy$ per opportune stringhe u, v, w, x e y tali che, in particolare, $L(G)$ comprende tutte le infinite parole uv^iwx^iy per i che varia tra i numeri naturali. Viceversa, supponiamo che $L(G)$ sia infinito. Allora $L(G)$ contiene stringhe di ogni possibile lunghezza, in particolare anche stringhe z di lunghezza $> p$. Scegliamo z di lunghezza minima (comunque $> p$). Se $l(z) \leq 2p$, siamo a posto. Altrimenti applichiamo di nuovo il Teorema $uvwxy$ a z e la esprimiamo come $uvwxy$ ricavando u, v, w, x e y dalle parole sull'alfabeto T dei simboli terminali di G: in particolare, $uwy \in L(G)$, $l(uwy) \leq n$ e $l(vx) > 0$. Notiamo $l(uwy) < l(z)$ proprio perché $l(vx) > 0$; inoltre $l(uwy) \geq l(uy) > p$

perchè $l(uvwxy) = l(z) > 2p$ e $l(vwx) \leq p$. Ma allora uwy contraddice la scelta di z.

□

Come già nel caso dei linguaggi regolari, possiamo allora ridurre il problema di stabilire se un linguaggio L libero dal contesto è o no vuoto, oppure è o no finito, alla questione della decidibilità di L. Come già preannunciato, dimostreremo nella Sezione 5.3 che, in realtà, tutti i linguaggi dipendenti dal contesto (e in particolare quelli liberi dal contesto) sono decidibili. Se accettiamo sin da ora questa conclusione, possiamo enunciare il seguente risultato, che estende il Teorema 5.4.

Teorema 5.8. Ci sono algoritmi effettivi che sanno decidere, per ogni linguaggio libero dal contesto L,

(a) se L è vuoto oppure no,

(b) se L è finito oppure no.

Si ricordi che la soluzione positiva del problema dell'equivalenza di grammatiche regolari, così come dimostrata dal Corollario 5.1, dipende proprio dal fatto che la classe dei linguaggi regolari si preserva per intersezione e complemento. Come visto nella Sezione 5.2.2, non altrettanto vale per arbitrari linguaggi liberi dal contesto, e in effetti si può provare che il *problema dell'equivalenza per grammatiche libere dal contesto* rimane indecidibile.

Teorema 5.9. Non c 'è algoritmo capace di stabilire, date due grammatiche libere dal contesto G_1 e G_2, se $L(G_1) = L(G_2)$ oppure no.

Dimostrazione. La dimostrazione è elaborata; così rimandiamo ai testi citati in bibliografia per i dettagli [1]. □

5.2.4 Complessità dell'analisi sintattica

La procedura di riconoscimento per i linguaggi liberi corrisponde alla procedura eseguita dagli automi a pila non deterministici. Una procedura di riconoscimento deterministica che può essere implementata nella pratica sarebbe obbligata a seguire tutte le possibili computazioni dell'APND e quindi la complessità dell'algoritmo crescerebbe esponenzialmente con la dimensione della stringa da riconoscere. Tuttavia è possibile definire algoritmi di riconoscimento più efficienti, con complessità polinomiale: l'algoritmo Earley e l'algoritmo CYK nel caso generale hanno complessità proporzionale al cubo della lunghezza dell'input. Un algoritmo con complessità quadratica fu proposto da Snelling nel 1973 [4], ma, sebbene non sia mai stato confutato, l'algoritmo non viene descritto completamente e non è stato mai menzionato negli anni seguenti.

5.3 Linguaggi dipendenti dal contesto

I linguaggi dipendenti dal contesto sono tutti quei linguaggi che sono generabili da una grammatica di tipo 1, o riconosciuti da un automa lineare limitato non deterministico. È anzitutto opportuno fornirne un esempio che non sia anche libero dal contesto.

Esempio 5.13. Il linguaggio

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

è dipendente dal contesto. Ciò si può verificare facendo riferimento all'Esempio 4.5 per la grammatica dipendente dal contesto che lo genera, l'Esempio 4.4 per la grammatica monotona che lo genera, o all'Esempio 4.19 per l'automa lineare limitato che lo riconosce.

Per quanto abbiamo visto nella Sezione 4.1.1 sappiamo che le grammatiche monotone sono equivalenti a quelle dipendenti dal contesto, e ovviamente anche le grammatiche di tipo 2 e 3 sono monotone, in quanto rientrano tra le grammatiche di tipo 1. Il seguente risultato, già preannunciato nelle precedenti sezioni, riguarda allora non solo i linguaggi generati da grammatiche monotone, per le quali viene enunciato, ma anche i linguaggi dipendenti dal contesto, liberi dal contesto e regolari; ribadisce in questi ultimi due casi che i problemi di stabilire se un linguaggio libero dal contesto (in particolare regolare) è o no vuoto, oppure è o no finito, hanno un algoritmo di decisione (si vedano i precedenti Teoremi 5.4 e 5.8).

Teorema 5.10. Tutti i linguaggi generati da grammatiche monotone sono decidibili.

Dimostrazione. Sia $G = (N, T, \mathcal{P}, S)$ una grammatica monotona. Cerchiamo un algoritmo per decidere, per ogni stringa w su T, se $w \in L(G)$ oppure no. Sia n la lunghezza di w. Consideriamo la seguente successione di insiemi T_i (per $i \in \mathbb{N}$):

- $T_0 = \{S\}$,
- per $i \in \mathbb{N}$, $T_{i+1} = T_i \cup \{\alpha \in (N \cup T)^\star \mid \exists \beta \in T_i$ tale che $\beta \longrightarrow_G \alpha, l(\alpha) \leq n\}$.

Dunque, per ogni $i \in \mathbb{N}$, si costruisce un insieme T_i che estende quelli dei passi precedenti e contiene tutte le stringhe α su $N \cup T$ (in particolare su T) che hanno lunghezza $\leq n$ e sono derivabili da S in G in al più i passi. Si osservi poi che, se $T = T_{i+1}$, allora $T_i = T_{i+t}$ per ogni indice $t \geq 0$. D'altra parte, siccome Gè monotona, per ogni derivazione $\beta \longrightarrow_G \alpha$ deve essere $l(\beta) \leq l(\alpha)$. Questa osservazione, unita al fatto che ogni stringa α di un qualsiasi T_i ha lunghezza $\leq n$, implica che tutte le stringhe usate per derivare α devono avere anch'esse lunghezza $\leq n$. Ne segue che la sequenza di insiemi T_i è limitata superiormente, ovvero esiste un indice m tale che $T_m = T_{m+t}$ per ogni $t \geq 0$. Come limite superiore m possiamo infatti

prendere il numero totale delle stringhe in $(N \cup T)^\star$ di lunghezza $\leq n$, che è ovviamente finito. L'algoritmo di decisione per l'appartenenza a $L(G)$ di stringhe $w \in T^\star$ consiste dunque nel controllare, per ogni $i \leq m$,se $w \in T_i$ e $T_i \neq T_{i+1}$. □

Tuttavia i linguaggi generati da grammatiche monotone (dunque quelli dipendenti dal contesto) non esauriscono quelli decidibili.

Proposizione 5.6. Esistono linguaggi decidibili che non sono generati da grammatiche dipendenti dal contesto.

Dimostrazione. Usiamo un argomento astratto di diagonalizzazione, che non fornisce esempi espliciti dei linguaggi richiesti, ma ne assicura comunque l'esistenza. Ne accenniamo i punti essenziali. Anzitutto, fissato un alfabeto T, enumeriamo in modo effettivo le grammatiche dipendenti dal contesto che hanno T come insieme di simboli terminali e attingono quelli non terminali da un insieme numerabile prefissato (si procede qui in modo simile a quello seguito per le macchine di Turing). Anche le stringhe di $T^\star$ possono essere enumerate effettivamente sulla base di un prefissato ordinamento di T secondo il solito principio lessicografico. Siano

$$G_0, G_1, \ldots, G_i, \ldots$$

$$w_0, w_1, \ldots, w_i, \ldots$$

queste enumerazioni (per i che varia tra i naturali). A questo punto formiamo una matrice di dimensioni infinite, in cui le righe corrispondono alle grammatiche dipendenti dal contesto e le colonne alle stringhe su T. Così la riga i etichetta la grammatica G_i e la colonna i-esima rappresenta la stringa w_i. Si consideri il linguaggio

$$L = \{w_i \in T^\star \mid i \in \mathbb{N}, w_i \notin L(G_i)\}$$

L è decidibile perché per ogni w_i è possibile anzitutto recuperare effettivamente il corrispondente indice i, determinare conseguentemente la grammatica G_i e verificare finalmente se $w_i \in L(G_i)$ oppure no, sfruttando il fatto che G_i è di tipo 1 e quindi $L(G_i)$ è decidibile. Comunque L non può essere generato da nessuna grammatica dipendente dal contesto. Altrimenti $L = L(G_j)$ per qualche j e di conseguenza

$$w_j \in L \iff w_j \notin L(G_j) \iff w_j \notin L$$

si ottiene dunque una contraddizione. □

5.3.1 Proprietà di chiusura

I linguaggi di tipo 1 si preservano per le usuali operazioni. Per unione, concatenazione e chiusura di Kleene, la prova non è difficile e riprende quella del caso libero dal contesto. Invece il caso della complementazione, trattato da Neil Immerman e Róbert Szelepcsényi [2, 5], non è banale. Comunque si ha:

Teorema 5.11. La classe dei linguaggi dipendenti dal contesto è chiusa per complementazione.

Siccome allora i linguaggi di tipo 1 si preservano per unione e complementazione, segue immediatamente:

Corollario 5.3. La classe dei linguaggi dipendenti dal contesto è chiusa per intersezione.

5.3.2 Proprietà decidibili e indecidibili

I linguaggi dipendenti dal contesto estendono propriamente quelli liberi dal contesto e quelli regolari, formando in questo senso una classe meno interessante e, del resto, godono minori proprietà. Per esempio si osserva quanto segue.

- Non esiste un algoritmo che sa stabilire se due grammatiche di tipo 1 generano o no lo stesso linguaggio (del resto questa incapacità si rileva già tra le grammatiche di tipo 2, e un procedimento che funzionasse per grammatiche dipendenti dal contesto si applicherebbe anche a quelle libere dal contesto)

- Non esiste un algoritmo che sa stabilire, per ogni grammatica G di tipo 1, se $L(G)$ è vuoto o no, oppure se è finito o no (invece analoghi procedimenti esistono nel caso libero dal contesto; entrambi i problemi sono trattati in [1]).

5.3.3 Complessità dell'analisi sintattica

Come abbiamo visto nella Sezione 4.2.3, gli automi lineari limitati non deterministici forniscono una procedura di riconoscimento per i linguaggi dipendenti dal contesto. Non è noto se esiste una procedura deterministica che riconosca questi linguaggi in un numero di passi che non sia una funzione esponenziale della lunghezza della parola di input. Questo problema è PSPACE-completo.

5.4 Linguaggi a struttura di frase

In quali casi il linguaggio generato da una grammatica è decidibile, o semidecidibile? Nella Sezione 5.3 abbiamo visto che i linguaggi di tipo 1 (e quindi anche quelli di tipo 2 e 3) sono tutti decidibili, ma non esauriscono la classe di tutti i linguaggi decidibili: ci sono linguaggi decidibili che non sono generati da nessuna grammatica di tipo 1. In questa sezione, invece, consideriamo i linguaggi a struttura di frase, quelli generati da grammatiche di tipo 0 e riconosciuti dagli automi Turing. Dimostreremo che essi coincidono esattamente con quelli semidecidibili: forniscono così una ulteriore caratterizzazione della semidecidibilità. In questa maniera dimostriamo anche l'equivalenza del concetto di Automa di Turing, e quindi di macchina di Turing, con quello delle grammatiche a struttura di frase.

Teorema 5.12. Un linguaggio è di tipo 0 se e solo se è semidecidibile.

Dimostrazione. Proviamo dapprima che, per ogni grammatica $G = (N, T, \mathcal{P}, S)$ di tipo 0, $L(G)$ è semidecidibile. Cerchiamo dunque una MdT M che converge su tutte e sole le stringhe di $L(G)$, e quindi, sostanzialmente, un algoritmo di enumerazione per $L(G)$ come visto nella Sezione 3.2.
Consideriamo allora la sequenza di insiemi T_i definita come segue:

- $T_0 = \{S\}$,
- per $i \in \mathbb{N}$, $T_{i+1} = T_i \cup \{\alpha \in (N \cup T)^\star \mid \exists \beta \in T_i \text{ tale che } \beta \longrightarrow_G \alpha\}$.

Allora ogni insieme T_i contiene tutte le stringhe derivabili in G da S in al più i passi e può essere costruito applicando le produzioni di G in tutti i modi possibili alle stringhe degli insiemi che lo precedono nella sequenza. T_i è comunque finito, così come l'insieme delle produzioni di G. Le stringhe dei vari T_i si possono allora enumerare prima in base all'indice i e poi in riferimento alle produzioni che le generano. In particolare le parole del linguaggio $L(G)$ sono quelle stringhe di $T^\star$ che sono generate per questa via. Dunque $L(G)$ è semidecidibile.
Dimostriamo adesso che un linguaggio semidecidibile è di tipo 0. Sia L un linguaggio semidecidibile su un alfabeto A. Cerchiamo una grammatica a struttura di frase G_L tale che $L(G_L) = L$. Sia $M = (Q, A, \delta, q_0)$ una MdT che accetta L. La grammatica G_L che adesso definiamo cerca di descrivere e accompagnare le computazioni di M sugli input $w \in A^\star$ in modo tale che G_L genera w se e solo se M converge su w. Per questo obiettivo G_L utilizza un alfabeto di simboli terminali $T = A \cup \{\Delta, \nabla, \star\}$ (dove $\star$ è il simbolo bianco di M) e un alfabeto N di simboli non terminali che si compone, oltre che del simbolo iniziale S, di un simbolo $\bar{q}$ per ogni stato $q \in Q$ di M e di tre ulteriori simboli P, q_F e q'_F.
Descriviamo adesso le produzioni di G_L (quelle che simulano il comportamento di M). Ricordiamo allora che, all'inizio di una computazione, M riceve un input di $A^\star$ e ne esamina il simbolo più a sinistra nello stato q_0. Corrispondentemente si pongono in G_L le produzioni

$$\begin{aligned} &S \longrightarrow \Delta\overline{q_0}P \\ &P \longrightarrow bP \qquad \forall b \in A \\ &P \longrightarrow \nabla \end{aligned}$$

che servono a generare ogni possibile configurazione iniziale di M; in particolare Δ, ∇ hanno funzioni di delimitatori, rispettivamente a sinistra e a destra, $\overline{q_0}$ ricorda lo stato iniziale e P permette di coinvolgere successivamente ogni elemento di A fino al completamento dell'input, dopo di che P lascia il suo posto a ∇ e scompare. Passiamo adesso a definire le produzioni di G_L che descrivono il comportamento di M durante i successivi passi delle sue computazioni. Per ogni istruzione di M della forma $\delta(q, a) = (q', a', +1)$ con $q, q' \in Q$ e $a, a' \in A \cup \{\star\}$, aggiungiamo le produzioni

$$\begin{aligned} &\bar{q}ab \longrightarrow a'\bar{q}'b \\ &\bar{q}a\nabla \longrightarrow a'\bar{q}' \star \nabla \end{aligned}$$

Invece a ogni istruzione $\delta(q, a) = (q', a', -1)$ associamo le produzioni

$$b\bar{q}a \longrightarrow \bar{q'}ba'$$
$$\Delta\bar{q}a \longrightarrow \Delta\bar{q'} \star a'$$

Finalmente, per ogni coppia (q, a) fuori dal dominio della funzione di transizione δ, aggiungiamo la produzione

$$\bar{q}a \longrightarrow q_F a$$

La comparsa di q_F in una stringa generata da G_L ha dunque il ruolo di segnalare che M sta convergendo (l'indice F sta qui a ricordare proprio la fermata di M). Aggiungiamo infine delle produzioni che, non appena M, appunto, si arresta, cancellano i due delimitatori Δ e ∇ e q_F e restituiscono l'esatto output di M:

$$bq_F \longrightarrow q_F b$$
$$\Delta q_F \longrightarrow q'_F$$
$$q'_F b \longrightarrow bq'_F$$
$$q'_F \nabla \longrightarrow \lambda$$

dove b varia in $A \cup \{\star\}$. In base a queste produzioni, q_F si sposta a sinistra lungo l'output e, raggiunto il delimitatore Δ, lo cancella e lascia campo a q'_F che svolge lo stesso ruolo a destra, mantenendo ancora invariato l'output ed eliminando alla fine tanto il delimitatore destro ∇ quanto se stesso.
Si vede facilmente che G_L è una grammatica di tipo 0. Con un po' di pazienza, si verifica anche che, per ogni stringa $w \in A^\star$, M converge su w (e cioè $w \in L$) se e solo se $S \longrightarrow^\star_{G_L} w$ (ovvero G_L genera w). □

Da quest'ultimo teorema e dal fatto che esistono insiemi non semidecidibili, concludiamo osservando quanto segue:

Corollario 5.4. Esistono linguaggi che non sono generabili da nessuna grammatica.

5.4.1 Proprietà di chiusura

I linguaggi a struttura di frase si preservano per le usuali operazioni di unione, concatenazione, intersezione e chiusura di Kleene, e la dimostrazione consiste nel costruire l'automa di Turing appropriato tenendo conto del fatto che questi linguaggi sono semidecidibili. Invece come già abbiamo visto nella Sezione 3.2.1 i linguaggi semidecidibili non sono in generale chiusi per complementazione: infatti per il teorema di Post il complemento di un insieme semidecidibile L è semidecidibile se e solo se L è anche decidibile.

5.4.2 Proprietà decidibili e indecidibili

Naturalmente non sono decidibili i problemi di stabilire se due grammatiche di tipo 0 generino lo stesso linguaggio, o se il linguaggio generato da una gramma-

tica di tipo 0 sia vuoto, finito o infinito. D'altronde questi problemi erano già irrisolvibili per quanto riguarda i linguaggi di tipo 1.

5.4.3 Complessità dell'analisi sintattica

Il problema di stabilire se una stringa appartiene a un linguaggio di tipo 0 non è decidibile quindi non esiste una procedura di analisi.

5.5 Sommario dei risultati

Riassumiamo i risultati di questo capitolo nelle Tabelle 5.1 e 5.2.

	Metodo generativo	Metodo riconoscitivo	Esempio di linguaggio	Complessità
L. tipo 0	G. a str. di frase	AdTND	$\{x \mid \varphi_x(x) \downarrow\}$	—
L. tipo 1	G. dipendenti o monotone	ALND	$\{a^n b^n c^n\}$	$O(2^n)$
L. tipo 2	G. libere	APND	$\{a^n b^n\}$	$O(n^2)$
L. tipo 3	G. regolari	ASFND	$\{a^n b^m\}$	$O(n)$
L. finiti	G. non ric.	ASFND	$\{a, bb, c\}$	$O(1)$

Tabella 5.1: Riepilogo delle classi di linguaggi, grammatiche e automi.

	$L(G)$ vuoto? finito? infinito?	$L(G_1) = L(G_2)$?
L. tipo 0	non decidibile	non decidibile
L. tipo 1	non decidibile	non decidibile
L. tipo 2	decidibile	non decidibile
L. tipo 3	decidibile	decidibile
L. finiti	decidibile	decidibile

Tabella 5.2: Riepilogo delle proprietà di ogni classe di linguaggi.

5.6 Esercizi

Esercizio 5.1

Stabilire, per ognuno dei seguenti linguaggi, se esso sia regolare o meno. Nel primo caso si mostri la grammatica regolare che lo generi, mentre nel secondo si applichi il Pumping lemma per i linguaggi regolari.

$$A = \{a^n a^n \mid n > 0\} \qquad B = \{x \in \{0, 1, \ldots, 9\}^\star \mid x \bmod 5 = 0\}$$
$$C = \{0^n 1^m \mid 0 \leq n < m\} \qquad D = \{0^n 1^n \mid n \leq 2\}$$

Esercizio 5.2

Stabilire, per ognuno dei seguenti linguaggi, se esso sia libero dal contesto o meno. Nel primo caso si mostri la grammatica libera dal contesto che lo generi, mentre nel secondo si applichi il Pumping lemma per i linguaggi liberi dal contesto.

$$A = \{a^n \mid n \text{ è primo}\} \qquad B = \{x \in \{0,1\}^\star \mid x \text{ è palindroma}\}$$
$$C = \{a^n c^m b^n \mid n > m \geq 0\} \quad D = \{a^n b^m c^n \mid n, m > 0\}$$

Esercizio 5.3

Scrivere, per ognuna delle seguenti specifiche, l'espressione regolare corrispondente.

1. L'insieme di stringhe $\{0, 11, 101\}$.
2. L'insieme delle stringhe composte solo da zeri.
3. L'insieme di tutte le stringhe binarie.
4. L'insieme delle stringhe binarie che cominciano e finiscono per 1.
5. L'insieme delle stringhe binarie che contengono almeno tre 1 consecutivi.
6. L'insieme delle stringhe binarie che contengono almeno tre 1.
7. L'insieme delle stringhe binarie di lunghezza dispari.

Soluzioni

Soluzione 5.1

A) Il linguaggio è regolare perché è generabile tramite una grammatica regolare $G = (N = \{S, A\}, T = \{a\}, \mathcal{P}, S)$ con le seguenti produzioni

$$S \longrightarrow aA$$
$$A \longrightarrow aS \mid a$$

B) Il linguaggio è regolare perché è generabile tramite una grammatica regolare $G = (N = \{S\}, T = \{0, 1, \ldots, 9\}, \mathcal{P}, S)$ con le seguenti produzioni

$$S \longrightarrow 0S \mid 1S \mid \ldots \mid 9S \mid 0 \mid 5$$

C) Il linguaggio non è regolare, e possiamo mostrarlo applicando il Pumping lemma per i linguaggi regolari.

Assumiamo C regolare e fissiamo un $p \in \mathbb{N}$ come richiesto dal Pumping lemma. Prendiamo dunque la stringa $z = 0^p 1^{p+1}$. Si ha che $z \in C$ e $l(z) = 2p + 1 \geq p$, quindi si può scrivere $z = uvw$ per qualche u, v, w.

Dato che $l(uv) \leq p$, v può consistere solamente di zeri, e dato che $l(v) > 0$, v non può essere una stringa vuota. Quindi pompando nuovi zeri in v, per esempio con uv^2w, il numero di zeri diventa necessariamente pari o superiore al numero di uni; nel caso peggiore che v sia un singolo zero, con

v^2 la nuova stringa ha $n = m$ che quindi non può appartenere al linguaggio C. Quindi C non è un linguaggio regolare.

In effetti C è un linguaggio libero dal contesto perché generato da una grammatica libera dal contesto $G = \{N = \{S\}, T = \{0, 1\}, \mathcal{P}, S\}$

$$S \longrightarrow 0S1 \mid S1 \mid 1$$

D) Il linguaggio è regolare perché è finito, e quindi generabile tramite una grammatica non ricorsiva (e dunque regolare) $G = (N = \{S, A, B, C\}, T = \{0, 1\}, \mathcal{P}, S)$ con le seguenti produzioni

$$\begin{aligned} S &\longrightarrow 0A \mid \lambda \\ A &\longrightarrow 0B \mid 1 \\ B &\longrightarrow 1C \\ C &\longrightarrow 1 \end{aligned}$$

Soluzione 5.2

A) Il linguaggio non è libero, e possiamo mostrarlo applicando il Pumping lemma per i linguaggi liberi dal contesto.

Assumiamo A libero dal contesto e fissiamo un $p \in \mathbb{N}$ come richiesto dal Pumping lemma. Scegliamo un numero primo m tale che $m \geq p$ e prendiamo la stringa $z = a^m$. Si ha che $z \in C$ e $l(z) = m \geq p$, quindi si può scrivere $z = uvwxy$ per qualche u, v, w, x, y.

Dato che $l(vx) \geq 1$ abbiamo che $v = a^j$ e $x = a^k$ con $j + k \geq 1$. Per il punto (c) del teorema scegliamo di pompare z con $i = 1 + m$, quindi otteniamo $uv^{1+m}wx^{1+m}y$, che corrisponde a $a^{m+j\cdot m+k\cdot m}$. Questa stringa tuttavia non appartiene ad A: $m + j \cdot m + k \cdot m = m(1 + j + m)$ non è un numero primo. Quindi A non è un linguaggio libero dal contesto.

B) Il linguaggio è libero dal contesto perché è generabile tramite una grammatica libera dal contesto $G = (N = \{S\}, T = \{0, 1\}, \mathcal{P}, S)$ con le seguenti produzioni

$$S \longrightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid 00 \mid 11 \mid \lambda$$

C) Il linguaggio non è libero, e possiamo mostrarlo applicando il Pumping lemma per i linguaggi liberi dal contesto.

Assumiamo C libero dal contesto e fissiamo un $p \in \mathbb{N}$ come richiesto dal Pumping lemma. Prendiamo la stringa $z = a^{p+1}c^pb^{p+1}$. Si ha che $z \in C$ e $l(z) = 3p+2 \geq p$, quindi si può scrivere $z = uvwxy$ per qualche u, v, w, x, y con $l(vx) \geq 1$ e $l(vwx) \leq p$.

Abbiamo tre casi da considerare:

1. v o x contiene delle a. Dato che $l(vwx) \leq p$, allora nè v nè x può contenere delle b.
2. v o x contiene delle b. Dato che $l(vwx) \leq p$, allora nè v nè x può contenere delle a.
3. Sia v che x consiste solamente di c.

Per il punto (c) del teorema scegliamo di pompare z con $i = 2$, quindi otteniamo $uvvwxxy$. Se siamo nel caso (1) avremmo più a che b, nel caso (2) avremmo più b che a mentre infine nel caso (3) avremmo almeno un simbolo c in più, e quindi il numero delle c non sarebbe più strettamente inferiore a quello delle a e delle b. In ogni caso, la stringa $uvvwxxy$ non appartiene al linguaggio C, che concludiamo non essere libero dal contesto.

D) Il linguaggio è libero dal contesto perché è generabile tramite una grammatica libera dal contesto $G = (N = \{S, A\}, T = \{a, b, c\}, \mathcal{P}, S)$ con le seguenti produzioni

$$S \longrightarrow aSc \mid aAc$$
$$A \longrightarrow bA \mid b$$

Soluzione 5.3

1. $0 + 11 + 101$
2. $0 \cdot 0^\star$
3. $(0 + 1)^\star \cdot (0 + 1)$
4. $1 \cdot (0 + 1)^\star \cdot 1$
5. $(0 + 1)^\star \cdot 111 \cdot (0 + 1)^\star$
6. $(0 + 1)^\star \cdot 1 \cdot (0 + 1)^\star \cdot 1 \cdot (0 + 1)^\star \cdot 1 \cdot (0 + 1)^\star$
7. $(0 + 1) \cdot (00 + 01 + 10 + 11)^\star$ oppure $(0 + 1) \cdot ((0 + 1) \cdot (0 + 1))^\star$

Capitolo 6

Linguaggi di programmazione

In questo capitolo presenteremo un nuovo possibile modo di accostare la computabilità: stavolta, infatti, la introdurremo attraverso i moderni linguaggi di programmazione. In effetti, la ricerca di linguaggi *ad alto livello*, capaci cioè di esprimere tutte le funzioni effettivamente computabili, risale già agli anni sessanta e ha prodotto una vasta famiglia di linguaggi come ALGOL e PASCAL, progenitori dei più moderni C, C++ e Java.

6.1 Il linguaggio WHILE

Concentriamo la nostra attenzione su un semplice linguaggio di programmazione imperativo chiamato *WHILE*. Si tratta di un linguaggio adeguato a rappresentare tutte le funzioni calcolabili, povero in confronto ad altri linguaggi di alto livello come il PASCAL o il FORTRAN, ma, proprio in virtù della sua semplicità descrittiva, capace di sottolineare in maniera più diretta e immediata il collegamento con la computabilità.

6.1.1 Sintassi del linguaggio WHILE

L'insieme dei caratteri di base del linguaggio WHILE contiene nomi per le variabili, simboli per gli operatori di base che modificano il valore delle variabili, un operatore per confrontare valori di coppie di variabili e un simbolo di programma. La sintassi del linguaggio WHILE è specificata in accordo alle regole di produzione di una grammatica libera dal contesto ed è definita in Figura 6.1.

Si noti che le variabili sono formate da stringhe finite arbitrarie di lettere maiuscole $A, B, \ldots, Z$ e di cifre decimali $0, 1, \ldots, 9$ che hanno l'unico obbligo di iniziare con una lettera maiuscola come accade, per esempio, a X e A ma anche $PIPPO1$, $CASA3$ e $P55$.

Definizione 6.1. Un *programma WHILE* è una sequenza ordinata finita (eventualmente vuota) di comandi, separati da punti e virgole, preceduta da `begin` e seguita da `end`. I comandi sono definiti induttivamente come

PROGRAM	::= `begin end` | `begin` SEQCOMMANDS `end`
SEQCOMMANDS	::= COMMAND | SEQCOMMANDS;COMMAND
COMMAND	::= ASSIGNMENT | `while` TEST `do` COMMAND | PROGRAM
ASSIGNMENT	::= VAR `:= 0` | VAR `:= s`(VAR) | VAR `:= pd`(VAR)
TEST	::= VAR $\neq$ VAR
VAR	::= LETTERA | VAR LETTERA | VAR CIFRA
LETTERA	::= A | B | ... | Z
CIFRA	::= 0 | 1 | ... | 9

Figura 6.1: Sintassi del linguaggio WHILE

segue:

- Al livello più elementare abbiamo i comandi di assegnamento, quelli della forma

$$X \texttt{ := } 0$$
$$X \texttt{ := s}(Y)$$
$$X \texttt{ := pd}(Y)$$

  dove 0 è il numero naturale zero, `s` è la funzione *successore* $n \mapsto n+1$ per ogni naturale n e `pd` è la funzione *predecessore* definita come $\texttt{pd}(n) = n-1$ se $n > 0$ e $\texttt{pd}(0) = 0$. Quindi 0, `s` e `pd` costituiscono gli operatori base del linguaggio. Si noti che nei programmi WHILE i valori assegnati alle variabili saranno sempre numeri naturali.

Si definiscono poi per induzione gli altri comandi, suddivisi tra *comandi composti* e *comandi* `while`.

- Un *comando composto* è un'istruzione della forma

$$\texttt{begin } \mathrm{C}_1; \ldots ; \mathrm{C}_n \texttt{ end}$$

  con n naturale e indica che le istruzioni $\mathrm{C}_1, \ldots, \mathrm{C}_n$ devono essere eseguite in successione, secondo l'ordine assegnato. Ammettiamo anche il caso $n = 0$, che si riduce sostanzialmente a un'istruzione che lascia inalterato il valore di tutte le variabili.

- Finalmente un *comando* `while` ha la forma

$$\texttt{while } X \neq Y \texttt{ do } \textsc{Command};$$

  la sua semantica intuitiva consiste nel ripetere l'esecuzione del comando descritto in COMMAND per tutto il tempo in cui la variabile X assume un valore diverso dal valore della variabile Y.

In particolare, l'unico operatore di confronto tra variabili nel linguaggio WHILE è, almeno inizialmente, $\neq$.

Quindi il linguaggio WHILE mette a disposizione del programmatore solo pochissimi operatori e comandi. Tuttavia mostreremo nella Sezione 6.2 come definire varie *macroistruzioni* che possono agevolare la stesura di un programma; cercheremo così di convincere il lettore che, proprio grazie a queste macro, il potere espressivo dei programmi WHILE non è inferiore a quello di altri usuali linguaggi di programmazione, almeno per quanto concerne l'aspetto computazionale. In particolare i linguaggi WHILE riescono a rappresentare tutte le familiari operazioni aritmetiche. Ma prima di fare ciò, nella Sezione 6.1.2 definiremo rigorosamente la semantica del linguaggio.

6.1.2 Semantica del linguaggio WHILE

La sintassi dei linguaggi WHILE ha già insinuato il loro collegamento con le funzioni dell'aritmetica. Ma per apprezzare compiutamente come i programmi WHILE possano computare queste funzioni, abbiamo bisogno di conoscere anche la semantica WHILE: un insieme di convenzioni che ci consentiranno, appunto, di associare una funzione parziale a ogni programma del linguaggio, di formare conseguentemente la classe delle funzioni che sono calcolabili con programmi WHILE e di confrontare finalmente questa classe con le altre che già conosciamo e derivano da approcci alternativi alla computabilità.

Tutti questi sviluppi saranno il tema dei futuri paragrafi. L'obiettivo di quello presente è semplicemente quello di descrivere la semantica WHILE. Cominciamo col fissare con maggiore precisione il nostro contesto.

Consideriamo programmi WHILE P a k variabili, cioè programmi abilitati a utilizzare k variabili `X1`, ..., `Xk`, o eventualmente un loro sottoinsieme. Conveniamo poi che le variabili `X1`, ..., `Xk` siano ordinate in base ai loro indici $i \in \{1, \ldots, k\}$. Chiamiamo *stato di computazione* di P una qualunque k-upla $a = (a_1, \ldots, a_k)$ di naturali: si intende allora che una variabile `X1` in P assume il valore a_1, `X2` assume il valore a_2 e così via per le k variabili. Uno stato di computazione è anche detto *vettore di stato*.

Le definizioni che seguono permettono di introdurre il concetto di computazione di un programma WHILE P a k variabili. Useremo le lettere C, C' e così via per denotare termini della categoria sintattica COMMAND (cioè singoli comandi per P) e $\mathcal{C}, \mathcal{C}', \ldots$ per indicare termini della categoria sintattica SEQ-COMMAND (cioè sequenze di comandi per P). Si ricordi che P stesso può essere visto come una sequenza di comandi.

Una coppia ordinata $\langle \mathcal{C}, a \rangle$ dove $\mathcal{C}$ è una sequenza di comandi di P e a è un vettore di stato di P è chiamata *configurazione* di P. La sequenza $\mathcal{C}$ descrive, intuitivamente, quali comandi sono da svolgere a un certo punto della sua esecuzione mentre il vettore di stato a precisa il valore delle variabili a cui $\mathcal{C}$ si applica.

La *relazione di transizione* $\rightarrow$ tra queste coppie $\langle \mathcal{C}, a \rangle$ (sequenza di comandi, vettore di stato), rappresentata in Figura 6.2, mostra come vengono eseguiti i singoli comandi di un programma e come vengono modificate le variabili del programma stesso. La terminazione di una sequenza di comandi viene denotata con `end`.

$$[\mathrm{SC}_{\mathrm{zero}}]\ \frac{a = (a_1, \ldots, a_k) \qquad a' = (a_1, \ldots, a_{u-1}, 0, a_{u+1}, \ldots, a_k)}{\langle \texttt{Xu := 0}, a\rangle \to \langle \texttt{end}, a'\rangle}$$

$$[\mathrm{SC}_{\mathrm{s}}]\ \frac{a = (a_1, \ldots, a_k) \qquad a' = (a_1, \ldots, a_{u-1}, \texttt{s}(a_v), a_{u+1}, \ldots, a_k)}{\langle \texttt{Xu := s(Xv)}, a\rangle \to \langle \texttt{end}, a'\rangle}$$

$$[\mathrm{SC}_{\mathrm{pd}}]\ \frac{a = (a_1, \ldots, a_k) \qquad a' = (a_1, \ldots, a_{u-1}, \texttt{pd}(a_v), a_{u+1}, \ldots, a_k)}{\langle \texttt{Xu := pd(Xv)}, a\rangle \to \langle \texttt{end}, a'\rangle}$$

$$[\mathrm{SC}_{\mathrm{while}_1}]\ \frac{a = (a_1, \ldots, a_k) \qquad a_u \neq a_v}{\langle \texttt{while Xu} \neq \texttt{Xv do C}, a\rangle \to \langle \texttt{C; while Xu} \neq \texttt{Xv do C}, a\rangle}$$

$$[\mathrm{SC}_{\mathrm{while}_2}]\ \frac{a = (a_1, \ldots, a_k) \qquad a_u = a_v}{\langle \texttt{while Xu} \neq \texttt{Xv do C}, a\rangle \to \langle \texttt{end}, a\rangle}$$

$$[\mathrm{SC}_{\mathrm{program}_1}]\ \frac{}{\langle \texttt{begin end}, a\rangle \to \langle \texttt{end}, a\rangle}$$

$$[\mathrm{SC}_{\mathrm{program}_2}]\ \frac{}{\langle \texttt{begin } \mathcal{C} \texttt{ end}, a\rangle \to \langle \mathcal{C}, a\rangle}$$

$$[\mathrm{SC}_{\mathrm{seq}_1}]\ \frac{\langle \mathcal{C}_1, a\rangle \to \langle \mathcal{C}_1', a'\rangle \qquad \mathcal{C}_1' \neq \texttt{end}}{\langle \mathcal{C}_1\texttt{;C}_2, a\rangle \to \langle \mathcal{C}_1'\texttt{;C}_2, a'\rangle}$$

$$[\mathrm{SC}_{\mathrm{seq}_2}]\ \frac{\langle \mathcal{C}_1, a\rangle \to \langle \texttt{end}, a'\rangle}{\langle \mathcal{C}_1\texttt{;C}_2, a\rangle \to \langle \texttt{C}_2, a'\rangle}$$

Figura 6.2: Semantica operazionale strutturale dei programmi WHILE

Le regole $\mathrm{SC}_{\mathrm{zero}}$, SC_{s} e $\mathrm{SC}_{\mathrm{pd}}$ descrivono come vengono modificati i valori del vettore di stato quando un comando di assegnamento viene eseguito. Le regole $\mathrm{SC}_{\mathrm{while}_1}$ e $\mathrm{SC}_{\mathrm{while}_2}$ gestiscono invece i comandi di iterazione `while`. In particolare, la regola $\mathrm{SC}_{\mathrm{while}_1}$ mostra come permettere l'esecuzione del corpo del `while` nel caso in cui il test è soddisfatto; la regola $\mathrm{SC}_{\mathrm{while}_2}$ ammette, invece, la terminazione del comando di iterazione quando il test non è soddisfatto. Le regole $\mathrm{SC}_{\mathrm{program}_1}$ e $\mathrm{SC}_{\mathrm{program}_2}$ gestiscono comandi composti (sequenze di comandi tra un `begin` e un `end`). In questo caso, naturalmente, si procede alla valutazione del loro corpo. Infine, le regole $\mathrm{SC}_{\mathrm{seq}_1}$ e $\mathrm{SC}_{\mathrm{seq}_2}$ trattano sequenze di comandi e, in particolare, implementano la sequenzializzazione dei comandi specificati.

Definizione 6.2. Siano P un programma WHILE a k variabili e $a(0) = (a_1, \ldots, a_k)$ un vettore di stato. Una computazione di P è una sequenza, possibilmente infinita, della forma

$$\langle P, a(0)\rangle \to \langle \mathcal{C}_1, a(1)\rangle \to \langle \mathcal{C}_2, a(2)\rangle \to \cdots$$

dove per ogni $i \geq 1, a(i)$ è un vettore stato di lunghezza k e $\mathcal{C}_i$ una sequenza di comandi (che corrisponde alla situazione del programma P al passo i-esimo della computazione). Se la sequenza di computazione è finita allora ha la forma

$$\langle P, a(0)\rangle \to \langle \mathcal{C}_1, a(1)\rangle \to \langle \mathcal{C}_2, a(2)\rangle \to \cdots \to \langle end, a(n)\rangle$$

per un qualche $n \in \mathbb{N}$.

La proposizione che segue è diretta conseguenza delle regole operazionali della relazione di transizione $\to$ definita nella Figura 6.2.

Proposizione 6.1. Siano P un programma WHILE a k variabili e $a = (a_1, \ldots, a_k)$ un vettore di stato. Allora esiste esattamente una computazione da P che ammette a come vettore di stato iniziale.

In questo caso a prende il nome di vettore di input. Si conviene poi quanto segue:

- Se la computazione di P su a è infinita, si dice che P diverge sull'input a e che il suo output è indefinito.
- Se invece la computazione è finita e della forma

  $$\langle P, a(0)\rangle \to \langle \mathcal{C}_1, a(1)\rangle \to \langle \mathcal{C}_2, a(2)\rangle \to \cdots \to \langle end, a(n)\rangle$$

  con $a = a(0)$, allora si dice che P converge o termina sull'input a e che il suo output è $a(n)$

6.1.3 Funzioni WHILE-calcolabili

Avendo dato semantica ai programmi WHILE, e avendo appurato che la computazione di un programma WHILE su un dato input è unica, siamo ora in grado

di parlare della *funzione calcolata* da un programma WHILE P. Sia dunque P un programma a k variabili `X1`, ..., `Xk` per qualche $k \geq 1$. Si dice funzione calcolata da P la funzione parziale f_P da $\mathbb{N}^k$ a $\mathbb{N}$ tale che, per ogni $a \in \mathbb{N}^k$,

- $f_P(a)$ è il valore `X1` nell'output della computazione di P su a se questa computazione ha fine,
- $f_P(a)$ non è definita altrimenti

In realtà possiamo convenire che un programma P a k variabili calcoli anche funzioni parziali h-arie per h diverso da k: basta che concordiamo che, per ogni h-upla $a = (a_1, \ldots, a_h)$ di naturali, P consideri come input

- per $h > k$, la k-upla troncata $(a_1, \ldots, a_k)$ (dunque la sequenza delle prime k componenti di a),
- per $h < k$, la k-upla $(a_1, \ldots, a_h, 0, \ldots, 0)$ (quella che aggiunge $k - h$ zeri finali ad a)

e poi proceda come appena descritto.

Definizione 6.3. Una funzione parziale f da $\mathbb{N}^k$ a $\mathbb{N}$ si dice WHILE-*calcoiabile* se f è la funzione calcolata da qualche programma WHILE P.

Dunque la definizione di una funzione WHILE-calcolabile f non richiede espressamente che P abbia esattamente tante variabili quante sono le variabili indipendenti di f.

Esempio 6.1. Vediamo alcuni semplici esempi di funzioni WHILE-calcolabili. Per semplicità nei seguenti programmi omettiamo le istruzioni che si occupano di associare a ogni variabile $X1, X2, \ldots$ i corrispettivi valori del vettore di stato iniziale $a_1, a_2, \ldots$.

- La funzione costante zero è calcolata dal programma:

```
begin
X1 := 0
end
```

- La funzione successore è calcolata dal programma:

```
begin
X1 := s(X1)
end
```

- La funzione identità è calcolata dal programma:

```
begin end
```

- La funzione k-aria di proiezione $(a_1, a_2, \ldots, a_k) \mapsto a_2$ che restituisce il suo secondo argomento è calcolata dal programma:

```
begin
X1 := s(X2);
X1 := pd(X1);
end
```

- La funzione indefinita è calcolata dal programma:

```
begin
X := 0;
Y := s(X);
while X ≠ Y do begin end
end
```

6.2 Estensioni del linguaggio WHILE

In questa sezione mostreremo come, a partire dalle semplici e basilari istruzioni messe a disposizione dal linguaggio WHILE, quest'ultimo possa essere esteso fino ad avere tutte le tipologie di espressioni tipiche dei linguaggi di programmazione moderni. Queste estensioni prendono la forma di *macroistruzioni*, anche più semplicemente note come *macro*: una istruzione che il compilatore del linguaggio va a convertire in una specifica serie di istruzioni prestabilite, tipicamente in un altro linguaggio. Nel nostro caso le macro che andremo a definire saranno messe in corrispondenza con una serie di istruzioni WHILE e/o altre macro che abbiamo definito in precedenza.

Con l'utilizzo di queste macroistruzioni, il linguaggio WHILE raggiunge sostanzialmente l'espressività del PASCAL. Le macro sono quindi di grande aiuto nella stesura di programmi per computazioni complesse; ma quando il nostro obiettivo si sposta a dimostrare proprietà astratte dei programmi, allora la semplicità dell'originario contesto WHILE torna utile per abbreviare e snellire la trattazione.

6.2.1 Assegnamento di variabili

La prima macro che consideriamo riguarda il problema di assegnare il valore di una variabile Y a un'altra variabile X. Il nostro linguaggio di programmazione non ha, al momento, alcun comando primitivo a questo proposito. È facile tuttavia convincersi che il programma:

```
begin
X := s(Y);
X := pd(X)
end
```

permette il passaggio del valore della variabile Y alla variabile X. Nel seguito, quindi, potremo usare il comando X `:=` Y per denotare sinteticamente la precedente macrodefinizione.

6.2.2 Assegnamento di costanti

Un'altra operazione usuale in programmazione è quella di assegnare il valore di una costante $n > 0$ a una variabile U, cioè una istruzione del tipo U `:=` n. La macro che la effettua consiste nella seguente lista di comandi elementari:

```
begin
U := 0;
U := s(U);
⋮
U := s(U)
end
```

dove il comando U `:=` `s(`U`)` è ripetuto n volte.

6.2.3 Operazioni aritmetiche

Se si vuole aggiungere il valore della variabile Y alla variabile X, Z `:=` X `+` Y, si può invece usare la macro:

```
begin
Z := X;
U := 0;
while U ≠ Y do
    begin
    Z := s(Z);
    U := s(U)
    end
end
```

Usando le macro già definite si può poi scrivere il programma:

```
begin
Z := X + Y;
X := Z;
end
```

che corrisponde all'istruzione X `:=` X `+` Y, e dunque alla somma.

Anche l'altra familiare operazione numerica della moltiplicazione Z `:=` $X \cdot Y$ si esprime nel linguaggio WHILE tramite un'opportuna macro:

```
begin
Z := 0;
U := 0
while U ≠ Y do
    begin
        Z := Z + X;
        U := s(U)
    end
end
```

Le funzioni descrivibili in WHILE tramite opportune macro includono anche la sottrazione troncata, il quoziente, il resto e così via. Consideriamo per esempio il caso della *sottrazione troncata* Z `:=` $X \dot{-} Y$ di X e Y; ricordiamo che si conviene che $X \dot{-} Y$ è $X - Y$ se $X > Y$, 0 altrimenti. La macro che la esprime è la seguente:

```
begin
Z := X;
U := 0;
while U ≠ Y do
    begin
        Z := pd(Z);
        U := s(U)
    end
end
```

6.2.4 Operatori di confronto

Di uso frequente in programmazione sono anche gli operatori di confronto. Tuttavia l'unico di essi disponibile almeno inizialmente nel linguaggio WHILE è $X \neq Y$. Ma è spesso necessario considerare anche test del tipo: $X < Y$, $X > Y$, $X \geq Y$, $X = Y$ e via dicendo. Il linguaggio WHILE garantisce queste opportunità se non direttamente almeno tramite macro.

Per esempio $X < Y$ può essere coinvolto mediante $(Y \dot{-} X) \dot{-} \texttt{pd}(Y \dot{-} X)$, nel senso che $X < Y$ è vera se e soltanto se $(Y \dot{-} X) \dot{-} \texttt{pd}(Y \dot{-} X) = 1$, mentre $X < Y$ è falsa se e soltanto se $(Y \dot{-} X) \dot{-} \texttt{pd}(Y \dot{-} X) = 0$. Analogamente si possono definire a questa maniera anche gli altri operatori di confronto, tuttavia procederemo a definirli sfruttando un'altra possibilità che ci è resa possibile da questo esempio: tramite l'aritmetica Booleana. Infatti a partire dai connettivi della logica Booleana $\neg$ (NOT), $\wedge$ (AND), $\vee$ (OR) e da i due operatori ($X < Y$ e $X \neq Y$) che abbiamo già a disposizione possiamo definire tutti gli altri operatori di confronto. Innanzitutto notiamo che possiamo definire $X = Y$ utilizzando la negazione logica di $X \neq Y$: $\neg(X \neq Y)$. Dopodiché abbiamo che il confronto $X \leq Y$ corrisponde a $(X < Y) \vee (X = Y)$, $X > Y$ si può esprimere come

$(X \neq Y) \wedge \neg(X < Y)$ e infine $X > Y$ si ottiene tramite $\neg(X < Y)$. Nella prossima sezione vedremo come definire questi operatori nel linguaggio WHILE.

6.2.5 Operatori Booleani

Per definire gli usuali connettivi della logica elementare $\neg$ (NOT), $\wedge$ (AND), $\vee$ (OR) ci basta associare a ciascuna espressione ε una condizione di test T_ε tra variabili in modo tale che T_ε ha 0 e 1 come unici valori ammissibili e si ha $T_\varepsilon = 1$ se e soltanto se ε è vera, e $T_\varepsilon = 0$ se e soltanto se ε è falsa.

Le regole per costruire questi termini sono facili da stabilire; per esempio, usando l'operatore `pd` e le macro $+$ e $\dot{-}$, possiamo scriverle come segue.

- $T_{\neg\varepsilon} = 1 \dot{-} T_\varepsilon$ (dunque 1 se $T_\varepsilon = 0$ e viceversa),
- $T_{\varepsilon_1 \wedge \varepsilon_2} = \mathtt{pd}(T_{\varepsilon_1} + T_{\varepsilon_2})$ (dunque 1 se $T_{\varepsilon_1} = T_{\varepsilon_2} = 1$ e 0 altrimenti),
- $T_{\varepsilon_1 \vee \varepsilon_2} = T_{\varepsilon_1} + T_{\varepsilon_2} \dot{-} \mathtt{pd}(T_{\varepsilon_1} + T_{\varepsilon_2})$ (dunque 1 se $T_{\varepsilon_1} = 1$ o $T_{\varepsilon_2} = 1$ e 0 altrimenti).

È anche utile definire in questo senso le due costanti booleane, quindi l'espressione costante `True` corrisponde al valore 1, e l'espressione costante `False` corrisponde al valore 0.

6.2.6 Cicli con condizioni Booleane

In modo simile possiamo catturare nel nostro linguaggio estensioni del comando `while` come

`while` ExprBool `do` Command

dove ExprBool è una generica espressione Booleana ε, e non soltanto $X \neq Y$. Per costruire la macroistruzione che corrisponde a questo comando, assumiamo per semplicità che in ExprBool compaiano solo variabili e non costanti (del resto espressioni Booleane con costanti si possono ottenere da quelle che hanno solo variabili assegnando le costanti alle variabili nel modo sopra descritto). Allora la macro che ci serve è:

```
begin
U := T_ExprBool;
V := 0;
while U ≠ V do
    begin
    Command;
    U := T_ExprBool
    end
end
```

dove T_{ExprBool} indica, come sappiamo, la condizione di test associata all'espressione Booleana ExprBool.

6.2.7 Esecuzione condizionale

Un'altra istruzione non direttamente disponibile nel linguaggio dei programmi WHILE, ma assai comune in altri linguaggi di programmazione, è il comando

`if` ExprBool `then` Command$_1$`else` Command$_2$

che fa eseguire la sequenza di istruzioni che segue `then`, e cioè Command$_1$, se l'espressione Booleana ExprBool è vera, e la sequenza di comandi dopo `else`, e cioè Command$_2$, se l'espressione Booleana ExprBool è falsa. La macro per questo operatore condizionale è la seguente:

```
begin
U := T_ExprBool;
V := 0;
while (U = 1) ∧ (V = 0) do begin Command_1; V := 1 end
while (U = 0) ∧ (V = 0) do begin Command_2; V := 1 end
end
```

Naturalmente, omettendo la seconda istruzione `while` si ottiene il più semplice comando `if` ExprBool `then` Command, che corrisponde a una istruzione `if-then-else` dove Command$_2$ è il programma vuoto `begin end`.

6.2.8 Numeri interi e razionali

Malgrado abbiamo definito il linguaggio WHILE sui numeri naturali, non è complicato esprimere anche i numeri interi (quindi anche valori negativi) o anche numeri razionali: infatti, come abbiamo visto nella Sezione 2.2.2, è possibile definire una biiezione tra coppie di numeri naturali coi naturali stessi. In questo modo, possiamo esprimere un numero intero come una coppia modulo e segno e un numero razionale come una coppia numeratore e denominatore. Quindi, per esempio, possiamo esprimere $+2$ come $(0, 2)$ e -1 come $(1, 1)$, dove il segno negativo corrisponde al valore 1 e il segno positivo al valore 0. Analogamente la frazione $\frac{3}{4}$ si può esprimere come la coppia $(3, 4)$. Nel linguaggio WHILE questo si concretizza nell'utilizzare, per questi due casi appena descritti, due variabili per rappresentare ogni numero, ognuna delle quali memorizza un valore della coppia. Tutte le altre macro che abbiamo appena visto possono essere adattate per fare uso opportuno di questa rappresentazione. Per questo motivo si potranno scrivere istruzioni come X `:=` -1 o X `:=` $\frac{1}{2}$ e utilizzare tutte le altre macro nella stessa maniera in cui le abbiamo usate sui numeri naturali.

6.2.9 Input e output

L'ultima tipologia di macro che andremo a presentare ci permette di specificare l'input e l'output dei programmi WHILE. Una istruzione `INPUT`(X) va a indicare che il prossimo valore di input del programma verrà salvato sulla variabile X. Similmente, una istruzione `OUTPUT`(X) indica che il prossimo output del programma è il valore contenuto nella variabile X.

6.3 Equivalenza con le macchine di Turing

In questa sezione dimostriamo che le funzioni WHILE-calcolabili coincidono esattamente con quelle calcolabili nel senso di Turing. Dal punto di vista della computabilità, questo risultato fornisce un ulteriore sostegno alla Tesi di Church-Turing e conferma la generalità del modello di Turing. Dal punto di vista della programmazione, invece, ci assicura che le MdT, pur costituendo un formalismo di calcolo a bassissimo livello, formano tuttavia un sistema di programmazione accettabile, visto che la loro espressività è la stessa del linguaggio WHILE.

Teorema 6.1. Una funzione parziale f da a $\mathbb{N}^h$ a $\mathbb{N}$ è WHILE-calcolabile se e solo è calcolabile (secondo Turing).

Dimostrazione. Dimostriamo dapprima che la WHILE-calcolabilità di una funzione f ne implica la computabilità nel senso di Turing. Ricordiamo che un programma WHILE P che calcola f consiste in una sequenza di comandi WHILE. Ci basta allora mostrare che ogni comando di questo genere può essere eseguito da un'opportuna MdT. Infatti, se questo è vero, possiamo anche costruire una MdT che simula l'intero programma P, combinando opportunamente e successivamente le MdT che eseguono i suoi comandi. Ripercorriamo allora la definizione induttiva dei comandi dei programmi WHILE. Nel seguito, faremo riferimento a MdT che come alfabeto hanno $A = \{1\}$.

- Iniziamo con i *comandi di assegnamento*, quelli che coinvolgono gli operatori 0, `s` e `pd`:
  - L'operatore 0 è implementato da una MdT che prima svuota il proprio nastro e poi scrive un 1 (Ricordiamo che un singolo 1 corrisponde al numero naturale 0).
  - La funzione successore `s` è implementata da una MdT che scorre il proprio input e poi aggiunge un 1 in fondo.
  - La funzione predecessore `pd` è implementata da una MdT che, nel caso ci sia più di un 1 sul nastro, scorre il proprio input e poi cancella l'ultimo 1 in fondo.

I passi induttivi della definizione di comando riguardano *comandi composti* come `begin` $\mathrm{C}_1; \ldots ; \mathrm{C}_n$ `end` oppure *comandi* `while` della forma `while Xi` $\neq$ `Xj do` C'.

- Concentriamo prima l'attenzione su `begin` $\mathrm{C}_1; \ldots ; \mathrm{C}_n$ `end` e supponiamo che, per ogni $i = 1, \ldots, n$, esista una MdT M_i capace di calcolare il programma WHILE C_i. Possiamo assumere che gli insiemi di stati delle macchine $M_i, \ldots, M_n$ siano a tra loro disgiunti, altrimenti si procede a una rinominazione degli stati. Allora una MdT M che esegue il programma `begin` $\mathrm{C}_1; \ldots ; \mathrm{C}_n$ `end` è quella che coinvolge in serie $M_1, M_, \ldots, M_n$, ovvero comincia la sua computazione simulando M_1 a partire dalla sua configurazione iniziale, poi propone la configurazione finale di M_1 come configurazione iniziale di M_2, e via di seguito.

- Consideriamo ora il caso `while Xi` $\neq$ `Xj do` C$'$. Per induzione, possiamo supporre che il programma C' sia eseguito da una MdT M'. Sia poi k il numero delle variabili di questo programma. Costruiamo preliminarmente una MdT ausiliaria $M''_{(i,j)}$ che, in ogni vettore di $\mathbb{N}^k$, controlla se l'i-esima componente coincide con la j-esima. Con l'aiuto di M' e $M''_{(i,j)}$ è allora possibile definire una MdT M che esegue il ciclo iterativo richiesto: ogni volta la macchina M controlla il test `Xi` $\neq$ `Xj` mediante $M''_{(i,j)}$ e, quando l'esito del test è positivo (e solo in questo caso), simula M'; altrimenti M termina la sua esecuzione.

Questo conclude la dimostrazione che la WHILE-calcolabilità implica la computabilità (nel senso di Turing). Viceversa, dobbiamo provare che ogni funzione calcolabile secondo Turing è anche WHILE-calcolabile. Per fare ciò, ci basta mostrare come programma WHILE può simulare una qualsiasi MdT.
Sia M una MdT che computa f. Siano poi $q_0, \ldots, q_n$ gli stati di M. Possiamo assumere che M si arresti se e solo se M entra nello stato q_n: se questo non è il caso, ci basta aggiungere a M un nuovo stato (senza istruzioni che lo riguardino) e alla funzione di transizione di M le istruzioni che conducano ogni configurazione di arresto di M al nuovo stato. Utilizziamo una variabile `Q` per indicare lo stato corrente della MdT: se `Q := 0` la MdT si trova nello stato q_0, se se `Q := 1` la MdT si trova nello stato q_1, e così via. Poi useremo un'altra variabile `P` per indicare la posizione corrente della testina; conveniamo che `P := 0` corrisponde al primo simbolo sul nastro di M. Fissiamo poi una variabile `F`, che rimarrà costante durante l'esecuzione del programma, al valore n che corrisponde allo stato finale.
A questo punto necessitiamo di opportune rappresentazioni per il contenuto del nastro e per la funzione di transizione di M. Il nastro possiamo codificarlo come visto nella Sezione 2.2.1 e salvarlo su una variabile `T`. La funzione di transizione la codifichiamo come visto nella Sezione 2.2 e la salviamo su una variabile `D`. Rappresentiamo la codifica del nastro con $\#_2(N)$ e la codifica della funzione di transizione con $\#_2(\delta)$.
Prima di passare all'algoritmo, abbiamo bisogno di definire alcune procedure:

- `READ(T, P, A)` decodifica il nastro `T` e scrive il simbolo di indice `P` su `A`.

- `EVAL(D, Q, A, Q1, A1, I)` decodifica la funzione di transizione `D` e cerca una regola, che sicuramente esiste per motivi che vedremo poi, che ha come input uno stato `Q` e un simbolo `A`. Una volta trovata, scrive in `Q1`, `A1` e `I` il nuovo stato, il nuovo simbolo e l'indice di scostamento corrispondenti. In particolare `I` $= 0$ indica una scostamento negativo e `I` $= 1$ uno scostamento positivo.

- `WRITE(T, P, A)` decodifica il nastro `T` e scrive alla posizione `P`, se esiste, il valore della variabile `A`. Se la posizione `P` non esiste, aggiunge un nuovo simbolo col valore di `A` in testa al nastro. Se `P` $= 0$, aggiunge un nuovo simbolo bianco al nastro da mettere a sinistra del simbolo appena scritto e `P` viene impostato a 1. Quindi il nastro viene nuovamente codificato e sovrascritto su `T`.

L'algoritmo è il seguente:

```
begin
T := #2(N);
D := #2(δ);
Q := 0;
F := n;
P := 0;
while Q ≠ F do
    begin
    READ(T, P, A);
    EVAL(D, Q, A, Q1, A1, I);
    WRITE(T, P, A1);
    Q := Q1;
    if I = 0 then P := pd(P) else P := s(P)
    end
end
```

dove la funzione `EVAL(D, Q, A, Q1, A1, I)` troverà sempre una regola da applicare perché l'unico stato dove la computazione può terminare è quello salvato in `F`. Se la computazione della MdT termina, l'output è codificato nel valore della variabile `T`; se la computazione non termina, il programma while continuerà a eseguire il ciclo `while` indefinitamente. □

6.4 II teorema di Kleene per i programmi

Nella Sezione 3.6, abbiamo presentato, discusso e dimostrato il Teorema di Kleene nell'ambito delle MdT. Il risultato si basa essenzialmente sulla possibilità di enumerare effettivamente le MdT su un fissato alfabeto A e anzi di costruire una biiezione effettiva tra MdT su A e numeri naturali. Ma anche i programmi WHILE ammettono una analoga enumerazione effettiva (il lettore può cimentarsi a trovarla, partendo da una codifica effettiva delle istruzioni elementari su cui è definita la sintassi WHILE). Così un analogo del Teorema di Kleene si può formulare e, soprattutto, dimostrare anche per il linguaggio WHILE. Ci riferiamo, come detto, a una fissata enumerazione effettiva dei programmi WHILE

$$P_0, P_1, \ldots, P_i, \ldots \ (i \in \mathbb{N})$$

accompagnata, per ogni intero positivo h, da una enumerazione delle funzioni parziali h-arie calcolate da questi programmi

$$f_0^{(h)}, f_1^{(h)}, \ldots, f_i^{(h)}, \ldots$$

(quindi, per ogni i, $f_i^{(h)}$ denota qui la funzione parziale h-aria calcolata da P_i).

Teorema 6.2 (di Kleene per Programmi WHILE)**.** Sia t una funzione calcolabile totale da $\mathbb{N}$ a $\mathbb{N}$. Allora, per ogni intero positivo h, esiste un naturale e tale che i programmi P_e e $P_{t(e)}$ calcolano la stessa funzione h-aria:

$$f_e^{(h)} = f_{t(e)}^{(h)}$$

Possiamo qui intendere indifferentemente che t sia calcolabile nel senso classico di Turing, oppure tramite programmi WHILE; infatti, abbiamo appena dimostrato che le due nozioni sono equivalenti. Ma quel che è notevole se si assume il nuovo punto di vista è il fatto che t può essere adesso introdotta come trasformazione di programmi, con esplicito riferimento all'ambito WHILE. Il seguente esempio illustra questa potenzialità.

Esempio 6.2. Consideriamo la funzione t da $\mathbb{N}$ a $\mathbb{N}$. Per ogni naturale n, t prende il programma P_n di indice n e vi opera le seguenti sostituzioni:

1. rimpiazza in P_n ogni occorrenza di `s` con `pd` e, viceversa, ogni occorrenza di `pd` con `s`;
2. prende poi la prima istruzione I del programma:
   - se I è della forma `Xi := s(Xj)`, allora t la sostituisce con

     `begin Xi := s(Xj); Xi := s(Xj) end;`

   - se I è della forma `Xi := pd(Xj)`, allora t la sostituisce con

     `begin Xi := pd(Xj); Xi := pd(Xj) end;`

   - se I non è né dell'una né dell'altra forma, t inserisce `begin X1 := 0 end` immediatamente dopo il primo `begin` del programma;
3. finalmente prende il programma ottenuto dopo i primi due passaggi (dunque una sequenza ordinata finita di istruzioni $\mathrm{C}_1, \mathrm{C}_2, \ldots, \mathrm{C}_n$), lo intende come un comando composto

   `begin` C_1`;` C_2`; ...;` C_n `end`

   e lo scrive a rovescio, trasformandolo in

   `begin` C_n`;` C_{n-1}`; ...;` C_1 `end`

Allora $t(n)$ è l'indice del programma ottenuto in questo modo. La funzione t così definita in termini di riscrittura di programmi va da $\mathbb{N}$ a $\mathbb{N}$ ed è calcolabile.

Anche alle funzioni t come quella costruita nell'ultimo esempio si applica il Teorema di Kleene, stabilendo per ogni intero positivo h l'esistenza di un punto fisso e per cui P_e e $P_{t(e)}$ finiscono col calcolare la stessa funzione parziale h-aria. Se ne deducono notevoli conseguenze, tra cui:

Proposizione 6.2. Esiste un programma WHILE che fornisce in output il proprio indice. In altre parole per ogni h esistono un naturale e (e quindi un programma P_e e una funzione parziale h-aria $f_e^{(h)}$ tali che, per ogni $x \in \mathbb{N}^h$, $f_e^{(h)}(x) = e$.

Dimostrazione. Definiamo la funzione di riscrittura di programmi $t : \mathbb{N} \to \mathbb{N}$ che trasforma un arbitrario programma P_n in un programma $P_{t(n)}$ così fatto:

$$P_{t(n)} = \texttt{begin X1 := } n \texttt{ end}$$

Anche t ha un punto fisso, diciamo e, rispetto a h. Quindi per ogni $x \in \mathbb{N}^h$

$$f_e^{(h)}(x) = f_{t(e)}^{(h)}(x) = e$$

□

6.5 Esercizi

Esercizio 6.1
Scrivere un programma WHILE che calcoli la funzione *segno*, definita come segue:

$$sign(x) = \begin{cases} 0 & \text{se } x = 0 \\ 1 & \text{altrimenti} \end{cases}$$

Esercizio 6.2
Scrivere un programma WHILE che presi due numeri restituisca il risultato della divisione intera del primo per il secondo.

Esercizio 6.3
Definire una macro per il linguaggio WHILE **repeat** C **until** E dove C è un comando e E è una espressione booleana. Informalmente, il comando **repeat** C **until** E esegue C e poi verifica E, se E falsa ripete C, altrimenti esce dal ciclo.

Esercizio 6.4
Definire una macro per il linguaggio WHILE

$$loopmax\ E_1, E_2\ do\ C\ end$$

la cui semantica intuitiva consiste nell'eseguire il comando C tante volte quanto vale il massimo tra E_1 ed E_2 fissato all'inizio.

Esercizio 6.5
Definire una macro per il linguaggio WHILE

$$case\ E\ do\ C_1, C_2, C_3\ end$$

dove E è una espressone aritmetica sui numeri naturali e C_1, C_2 e C_3 sono comandi. La semantica intuitiva del comando *case* è quella di valutare E e se il valore di E è 1 si esegua C_1, se il valore di E è 2 si esegua C_2, se il valore di E è 3 si esegua C_3. Per tutti gli altri valori si esca dal *case*.

Soluzioni

Soluzione 6.1

Il programma richiesto dall'esercizio si può esprimere con facilità utilizzando la macro if-then-else.

```
begin
INPUT(X);
if X = 0 then
    OUTPUT(X)
else
    Y := 1;
    OUTPUT(Y)
end
```

Soluzione 6.2

Scriviamo il programma per la divisione prendendo in input due numeri A e B e restituendo il risultato della divisione calcolata come sottrazione (troncata) ripetuta.

```
begin
INPUT(A);
INPUT(B);
Q := 0;
while A ≥ B do
    begin
    A := A ∸ B;
    Q := s(Q)
    end
OUTPUT(Q)
end
```

Soluzione 6.3

Definiamo questa macro facendo solamente uso della costante Booleana `True` che altro non è un nome di variabile che corrisponde al valore 1.

```
begin
C;
while E ≠ True do
    begin
    C
    end
end
```

Soluzione 6.4

Per definire questa macro è sufficiente controllare quale valore sia il massimo e quindi eseguire un ciclo `while` decrementando di volta in volta il valore che si è trovato all'inizio.

```
begin
if E1 > E2 then
    begin
    I := E1
    end
else
    begin
    I := E2
    end
while I ≠ 0 do
    begin
    C;
    I := pd(I)
    end
end
```

Soluzione 6.5

Per definire questa macro ci ispiriamo al fatto che nei linguaggi di programmazione più diffusi espressioni di questo tipo, ovvero gli *switch*, sono facilmente esprimibili come una serie di if-then-else annidati.

```
begin
if E = 1 then
    begin
    C1
    end
else
    begin
    if E = 2 then
        begin
        C2
        end
    else
        begin
        if E = 3 then
            begin
            C3
            end
        else
            begin
            end
        end
    end
end
```

Bibliografia

[1] John E. Hopcroft e Jeff D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley Publishing Company, 1979.

[2] Neil Immerman. ≪Nondeterministic Space is Closed under Complementation≫. In: *SIAM J. Comput.* 17.5 (ott. 1988), pp. 935–938. ISSN: 0097-5397. DOI: 10.1137/0217058.

[3] György E. Révész. *Introduction to Formal Languages*. Family Law Series. McGraw-Hill, 1983. ISBN: 9780070519169.

[4] Michel Snelling. ≪General context-free parsing in time n^2≫. In: *Proceedings of the International Computing Symposium 1973* (set. 1973), pp. 19–24.

[5] R. Szelepcsényi. ≪The Method of Forced Enumeration for Nondeterministic Automata≫. In: *Acta Inf.* 26.3 (nov. 1988), pp. 279–284. ISSN: 0001-5903. DOI: 10.1007/BF00299636.

[6] Carlo Toffalori et al. *Teoria della computabilità e della complessità*. A cura di Chiara Tartara. Via Ripamonti 89 - 20139 Milano: McGraw-Hill, 2005. ISBN: 978-88-386-7260-6.

Indice analitico

$A^\star$, 22
A^+, 22
K, 65, 69
$M \downarrow w$, 33
$M \uparrow w$, 33
$\longrightarrow_G$, 96
$\longrightarrow_G^\star$, 96
λ, 21
λ-ASFND, 116
λ-produzione, 100
$l(w)$, 22
$\leq_1$, 73
$\leq_m$, 73
$\vdash_M$, 33
$\vdash_M^\star$, 34

accettare, 38, 44
albero di derivazione, 105
alfabeto, 21
algoritmo, 14
automa
  a pila, 118
    con un singolo stato, 122
    deterministico, 120
    non deterministico, 119
  a stati finiti, 108
    deterministico, 109
    non deterministico, 109
  di Turing, 107
    deterministico, 107
    non deterministico, 107
  lineare limitato, 124
    deterministico, 125
    non deterministico, 124

calcolabile, 36
Cantor, Georg, 48
cardinalità, 18
cella, 30
chiusura
  di Kleene, 24
  positiva, 24
Chomsky, Noam, 97
codominio, 19
complemento
  di un insieme, 19
  di un linguaggio, 24
composizione di un linguaggio, 24
computazione
  di un automa a pila, 120
  di una MdT, 34
concatenazione di una stringa, 22
configurazione
  finale, 33
  iniziale, 33
  istantanea, 33
    di un automa a pila, 120
convergere, 33, 44

decidere, 38, 44
decidibile, 38
Decimo problema di Hilbert, 12
derivazione, 96
diagonalizzazione, prova per, 71
differenza
  di insiemi, 19
  di linguaggi, 24
divergere, 33, 44
dominio, 19
  di arrivo, 19
  di definizione, 19
  di partenza, 19

Entscheidungsproblem, 12, 82
estensionale, 17

funzione, 19

calcolabile secondo Turing, 36
caratteristica, 38
computabile secondo Turing, 36
costante, 20
di proiezione, 167
di transizione, 32, 109, 120, 125
identità, 20
indefinita, 20
non calcolabile secondo Turing, 36
parziale, 19
predecessore, 162
successore, 162
totale, 19
WHILE-calcolabile, 166

gerarchia
di automi, 108
di Chomsky, 100
di linguaggi formali, 131
grafo di transizione
per automi a pila, 121
per automi a stati finiti, 110
per automi lineari limitati, 125
per MdT, 35
grammatica
a struttura di frase, 95
di Chomsky, 97
di tipo 0, 95
di tipo 1, 97
di tipo 2, 99
di tipo 3, 99
dipendente dal contesto, 97
libera dal contesto, 99
monotona, 97
non contraente, 97
non ricorsiva, 132
regolare, 99
Gödel, Kurt, 12, 45, 54

Hilbert, David, 12

immagine, 20
indecidibile, 38
insieme, 17
decidibile, 38
finito, 17
indecidibile, 38
infinito, 17
non semidecidibile, 38
numerabile, 49
ricorsivamente enumerabile, 68
ricorsivo, 68
semidecidibile, 38
vuoto, 18
intensionale, 17
intersezione
di insiemi, 19
di linguaggi, 24
inversa di una stringa, 22
istruzione di una MdT, 32

Kleene, Teorema di, 78

linguaggio, 24
a struttura di frase, 96
ad alto livello, 161
decidibile, 38
di tipo 0, 96
di tipo 1, 99
di tipo 2, 99
di tipo 3, 100
dipendente dal contesto, 99
finito, 132
formale, 24
generato, 96
indecidibile, 38
libero dal contesto, 99
non ricorsivo, 132
non semidecidibile, 38
regolare, 100
riconosciuto da MdT, 38
ricorsivamente enumerabile, 68
ricorsivo, 68
semidecidibile, 38
WHILE, 161
lunghezza di una stringa, 22

macchina di Turing, 29
a più nastri, 40
deterministica, 32
non deterministica, 43
universale, 54
matrice
di transizione, 31
funzionale, 31
MdT, Macchina di Turing, 29

non calcolabile, 36

non semidecidibile, 38
numero di Gödel, 45

ordinamento lessicografico, 23

parola, 21
Post, Teorema di, 69
potenza di un linguaggio, 24
prefisso, 22
    proprio, 22
problema, 24
    dell'arresto, 64
    dell'equivalenza tra MdT, 76
    di decisione, 12, 82
prodotto
    di insiemi, 19
produzione di una grammatica, 95
programma WHILE, 161
proiezione, 167
Pumping lemma
    per linguaggi liberi dal
        contesto, 145
    per linguaggi regolari, 132

quadro, 30

regola
    di produzione, 95
    di transizione, 32
Rice, Teorema di, 80
riconoscere, 38, 107
ricorsivamente enumerabile, 68
ricorsivo, 68

semidecidibile, 38
simbolo
    iniziale, 95
    non terminale, 95
    terminale, 95
sottoinsieme, 18
    proprio, 18
sottostringa, 22
stringa, 21
    derivabile, 96
    derivabile direttamente, 96
    vuota, 21
suffisso, 22
    proprio, 22

Teorema
    uvw, 132
    $uvwxy$, 145
Teorema di
    Kleene, 78
        per linguaggi WHILE, 174
    Post, 69
    Rice, 80
Tesi di Church-Turing, 52
Turing, Alan, 29

unione
    di insiemi, 19
    di linguaggi, 24